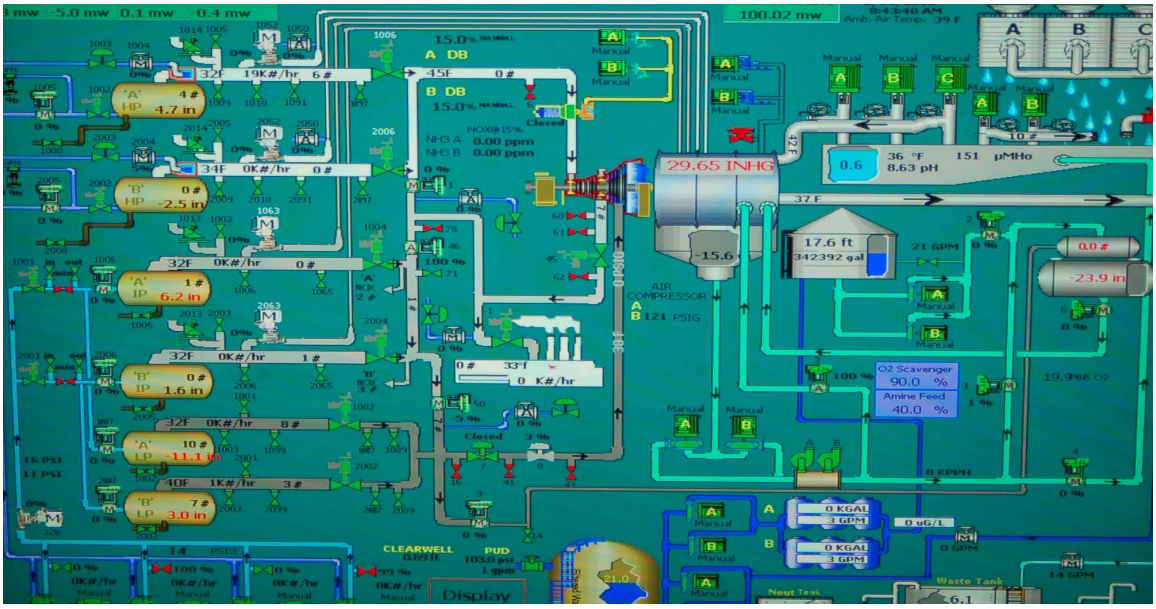
Lessons In Industrial Instrumentation



© 2008-2024 by Tony R. Kuphaldt – under the terms and conditions of the Creative Commons Attribution 4.0 International Public License

Version 2.33 (development) – Last update 9 October 2024

Lessons In Industrial Instrumentation © 2008-2024 by Tony R. Kuphaldt

This book is a copyrighted work, but licensed under the Creative Commons Attribution 4.0 International Public License. To view a copy of this license, turn to Appendix F, or visit `http://creativecommons.org/licenses/by/4.0/` or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA. The terms and conditions of this license allow for free copying, distribution, and/or modification of all licensed works by the general public.

Revision history[1]

- Version 0.1 – July 2008 to September 2008 (initial development)
- Version 0.2 – released September 29, 2008 for Fall quarter student use (SafeCreative registration code `0810111072182`)
- Version 0.4 – released January 12, 2009 for Winter quarter student use (SafeCreative registration code `0901122394919`)
- Version 0.6 – released April 21, 2009 for public use (SafeCreative registration code `0904213106101`)
- Version 0.8 – released September 8, 2009 for public use (SafeCreative registration code `0909094400980`)
- Version 1.0 – released September 28, 2009 for public use (SafeCreative registration code `0909284601913`)
- Version 1.2 – released January 12, 2010 for public use (SafeCreative registration code `1001125303242`)
- Version 1.4 – released April 11, 2010 for public use (SafeCreative registration code `1004125976416`)
- Version 1.6 – released May 4, 2010 for public use (SafeCreative registration code `1005046194859`)
- Version 1.8 – released June 1, 2010 for public use (SafeCreative registration code `1006016477958`)
- Version 1.10 – released June 28, 2010 for public use (SafeCreative registration code `1006286691573`)
- Version 1.12 – released September 20, 2010 for public use (SafeCreative registration code `1009217397803`)

[1] Version numbers ending in odd digits are developmental (e.g. 0.7, 1.23, 4.5), with only the latest revision made accessible to the public. Version numbers ending in even digits (e.g. 0.6, 1.0, 2.14) are considered "public-release" and will be archived. Version numbers beginning with zero (e.g. 0.1, 0.2, etc.) represent early editions that were substantially incomplete.

Chapter 27

Control valves

One of the most common final control elements in industrial control systems is the *control valve*. A "control valve" works to restrict the flow of fluid through a pipe at the command of a remotely sourced signal, such as the signal from a loop controller or logic device (such as a PLC), or even a manual ("hand") interface controlled by a human operator. Some control valve designs are intended for discrete (on/off) control of fluid flow, while others are designed to *throttle* fluid flow somewhere between fully open and fully closed (shut), inclusive. The electrical equivalent of an on/off valve is a switch, while the electrical equivalent of a throttling valve is a variable resistor.

Control valves are comprised of two major parts: the *valve body*, containing all the mechanical components necessary to influence fluid flow; and the *valve actuator*, providing the mechanical power necessary to move the valve body components. Often times, the major difference between an on/off control valve and a throttling control valve is the type of actuator applied to the valve[1]: on/off actuators need only position a valve mechanism two one of two extreme positions (fully open or fully closed). Throttling actuators must be able to accurately position a valve mechanism anywhere between those extremes.

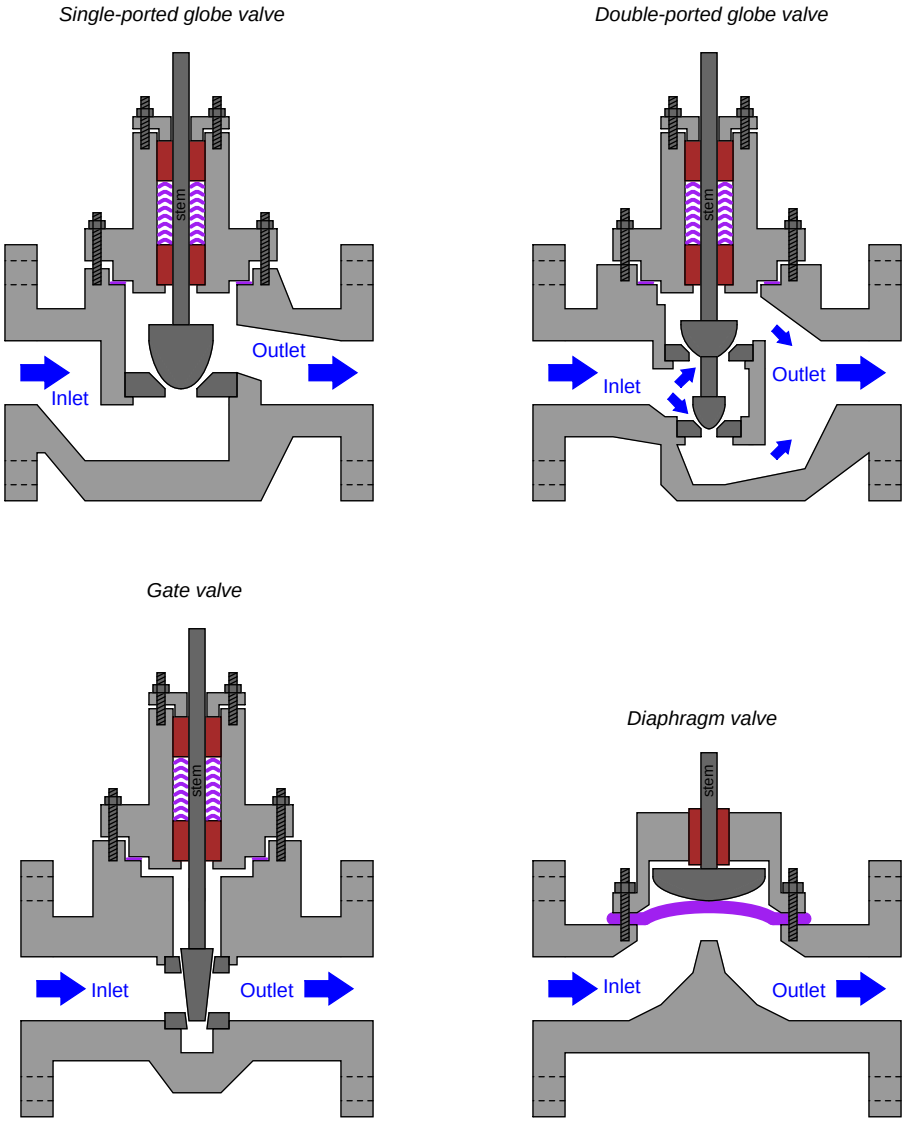
Within a control valve body, the specific components performing the work of throttling (or completely shutting off) of fluid flow are collectively referred to as the valve *trim*. For each major type of control valve, there are usually many variations of trim design. The choice of valve type, and of specific trim for any type of valve, is a decision dictated by the type of fluid being controlled, the nature of the control action (on/off versus throttling), the process conditions (expected flow rate, temperature, pressures, etc.), and economics.

An appendix of this book (Appendix C beginning on page 3183) photographically documents the complete disassembly of a typical control valve. The valve happens to be a Fisher E-body globe valve with a pneumatic diaphragm actuator.

[1] To be honest, there are some valve body designs that work far better in on/off service (e.g. ball valves and plug valves) while other designs do a better job at throttling (e.g. double-ported globe valves). Many valve designs, however, may be pressed into either type of service merely by attaching the appropriate actuator.

27.1 Sliding-stem valves

A *sliding-stem* valve body is one where the moving parts slide with a linear motion. Some examples of sliding-stem valve body designs are shown here:



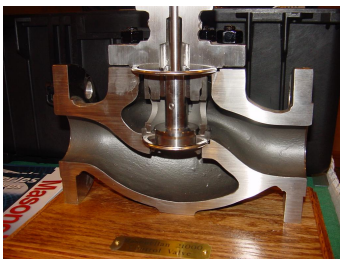
Most sliding-stem control valves are *direct acting*, which means the valve opens up wider as the stem is drawn out of the body. Conversely, a direct-acting valve shuts off (closes) when the stem is pushed into the body. Of course, a *reverse-acting* valve body would behave just the opposite: opening up as the stem is pushed in and closing off as the stem is drawn out.

27.1.1 Globe valves

Globe valves restrict the flow of fluid by altering the distance between a movable plug and a stationary seat (in some cases, a pair of plugs and matching seats). Fluid flows through a hole in the center of the seat, and is more or less restricted by the plug's proximity to that hole. The globe valve design is one of the most popular sliding-stem valve designs used in throttling service. A photograph of a small (2 inch) globe valve body appears here:



A set of three photographs showing a cut-away Masoneilan model 21000 globe valve body illustrates just how the moving plug and stationary seat work together to throttle flow in a direct-acting globe valve. The left-hand photo shows the valve body in the fully closed position, while the middle photo shows the valve half-open, and the right-hand photo shows the valve fully open:

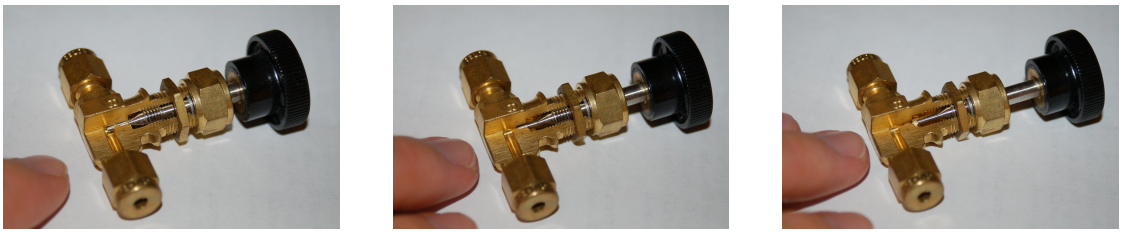




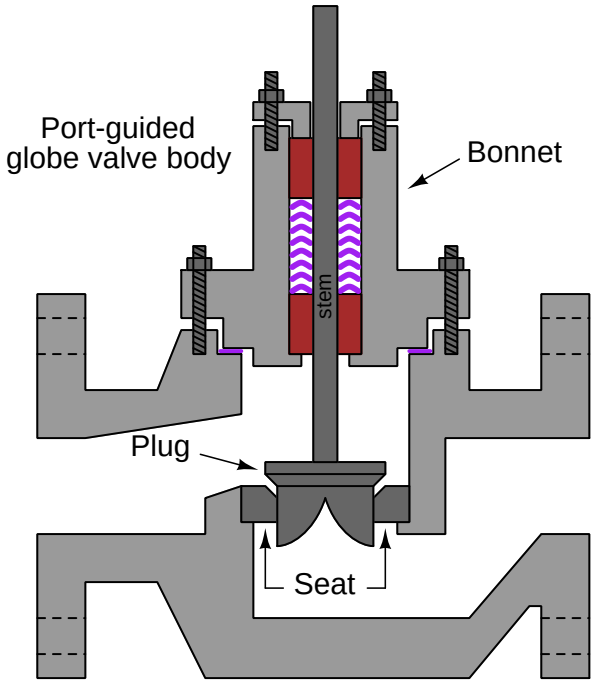


As you can see from these photographs, the valve plug is guided by the stem to maintain alignment with the centerline of the seat. For this reason, this particular style of globe valve is called a *stem-guided* globe valve.

A variation on the stem-guided globe valve design is the *needle valve*, where the plug is extremely small in diameter and usually fits well into the seat hole rather than merely sitting on top of it. Needle valves are very common as manually-actuated valves used to control low flow rates of air or oil. A set of three photographs shows a needle valve in the fully-closed, mid-open, and fully-open positions (left-to-right):



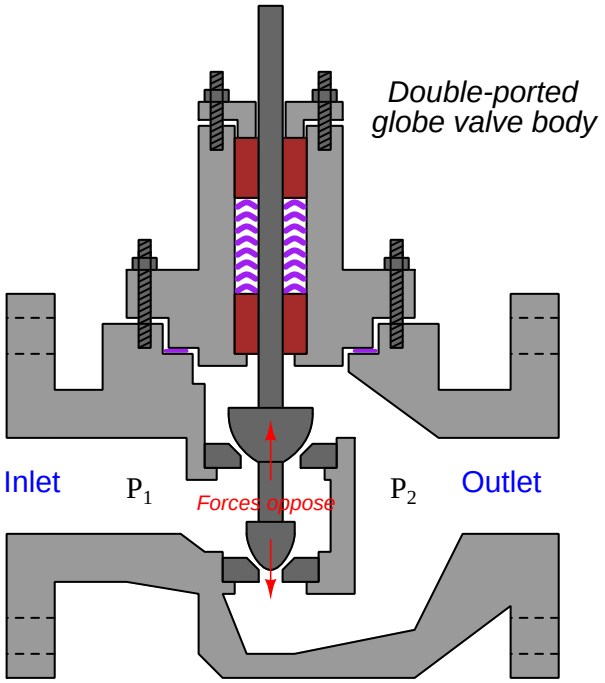
Yet another variation on the globe valve design is the *port-guided* valve, where the plug has an unusual shape, projecting into the seat. Thus, the seat ring acts as a guide for the plug to keep the centerlines of the plug and seat always aligned, minimizing guiding stresses that would otherwise be placed on the stem. This means that the stem may be made smaller in diameter than if the valve trim were stem-guided, minimizing sliding friction and improving control behavior.



A photograph showing a small port-guided globe valve plug appears in the following photograph:

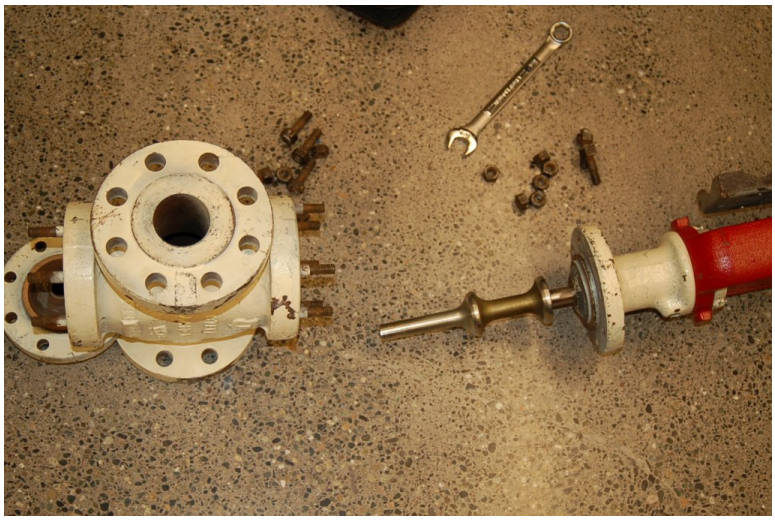


Some globe valves use a pair of plugs (on the same stem) and a matching pair of seats to throttle fluid flow. These are called *double-ported* globe valves. The purpose of a double-ported globe valve is to minimize the force applied to the stem by process fluid pressure across the plugs:



Differential pressure of the process fluid ($P_1 - P_2$) across a valve plug will generate a force parallel to the stem as described by the formula $F = PA$, with A being the plug's effective area presented for the pressure to act upon. In a single-ported globe valve, there will only be one force generated by the process pressure. In a double-ported globe valve, there will be *two opposed* force vectors, one generated at the upper plug and another generated at the lower plug. If the plug areas are approximately equal, then the forces will likewise be approximately equal and therefore nearly cancel. This makes for a control valve that is easier to actuate (i.e. the stem position is less affected by pressure drop across the valve).

The following photograph shows a disassembled Fisher "A-body" double-ported globe valve, with the double plug plainly visible on the right:



This particular double-ported globe valve happens to be stem-guided, with bushings guiding the upper stem and also a lower stem (on the bottom side of the valve body). Double-ported, port-guided control valves also exist, with two sets of port-guided plugs and seats throttling fluid flow.

While double-ported globe valves certainly enjoy the advantage of easier actuation compared to their single-ported cousins, they also suffer from a distinct disadvantage: the near impossibility of tight shut-off. With *two* plugs needing to come to simultaneous rest on *two* seats to achieve a fluid-tight seal, there is precious little room for error or dimensional instability. Even if a double-ported valve is prepared in a shop for the best shut-off possible[2], it may not completely shut off when installed due to dimensional changes caused by process fluid heating or cooling the valve stem and body. This is especially problematic when the stem is made of a different material than the body. Globe valve stems are commonly manufactured from stainless steel bar stock, while globe valve bodies are commonly made of cast steel. Cold-formed stainless steel has a different coefficient of thermal expansion than hot-cast steel, which means the plugs will no longer simultaneously seat once the valve warms or cools much from the temperature it was at when it seated tightly.

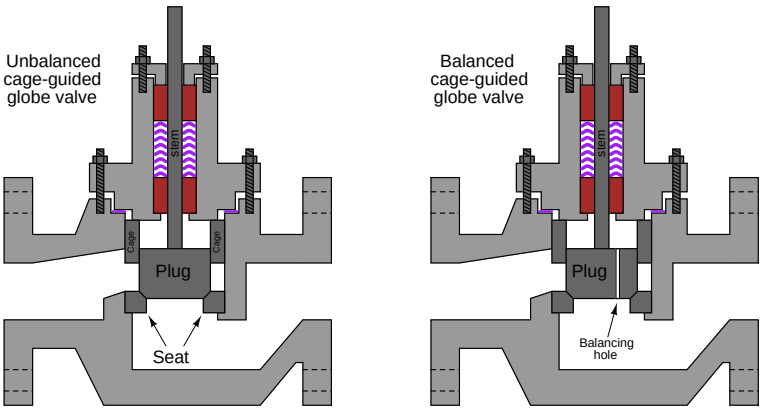
[2]The standard preparatory technique is called *lapping*. To "lap" a valve plug and seat assembly, an abrasive paste known as *lapping compound* is applied to the valve plug(s) and seat(s) at the areas of mutual contact when the valve is disassembled. The valve mechanism is reassembled, and the stem is then rotated in a cyclic motion such that the plug(s) grind into the seat(s), creating a matched fit. The precision of this fit may be checked by disassembling the valve, cleaning off all remaining lapping compound, applying a metal-staining compound such as *Prussian blue*, then reassembling. The stem is rotated once more such that the plug(s) will rub against the seat(s), wearing through the applied stain. Upon disassembly, the worn stain may be inspected to reveal the extend of metal-to-metal contact between the plug(s) and the seat(s). If the contact area is deemed insufficient, the lapping process may be repeated.

A more modern version of the globe valve design uses a piston-shaped plug inside a surrounding *cage* with ports cast or machined into it. These *cage-guided* globe valves throttle flow by uncovering more or less of the port area in the surrounding cage as the plug moves up and down. The cage also serves to guide the plug so the stem need not be subjected to lateral forces as in a stem-guided valve design. A photograph of a cut-away control valve shows the appearance of the cage (in this case, with the plug in the fully closed position). Note the "T"-shaped ports in the cage, through which fluid flows as the plug moves up and out of the way:

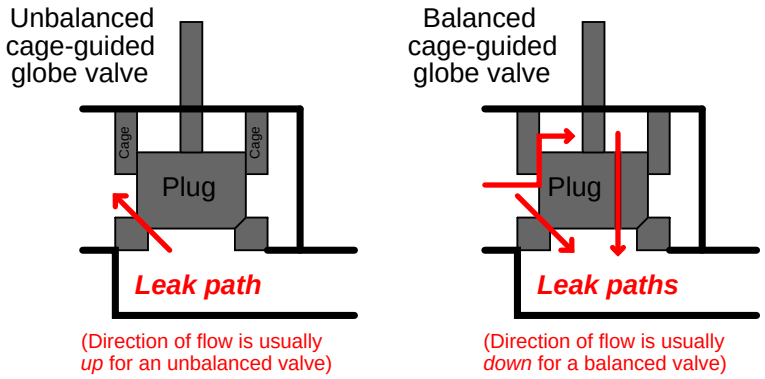


An advantage of the cage-guided design is that the valve's flowing characteristics may be easily altered just by replacing the cage with another having different size or shape of holes. By contrast, stem-guided and port-guided globe valves are characterized by the shape of the plug, which requires further disassembly to replace than the cage in a cage-guided globe valve. With most cage-guided valves all that is needed to replace the cage is to separate the bonnet from the rest of the valve body, at which point the cage may be lifted out of the body and swapped with another cage. In order to change a globe valve's plug, you must first separate the bonnet from the rest of the body and then de-couple the plug and plug stem from the actuator stem, being careful not to disturb the packing inside of the bonnet as you do so. After replacing a plug, the "bench-set" of the valve must be re-adjusted to ensure proper seating pressure and stroke calibration.

Cage-guided globe valves are available with both *balanced* and *unbalanced* plugs. A balanced plug has one or more ports drilled from top to bottom, allowing fluid pressure to equalize on both sides of the plug. This helps minimize the forces acting on the plug which must be overcome by the actuator:

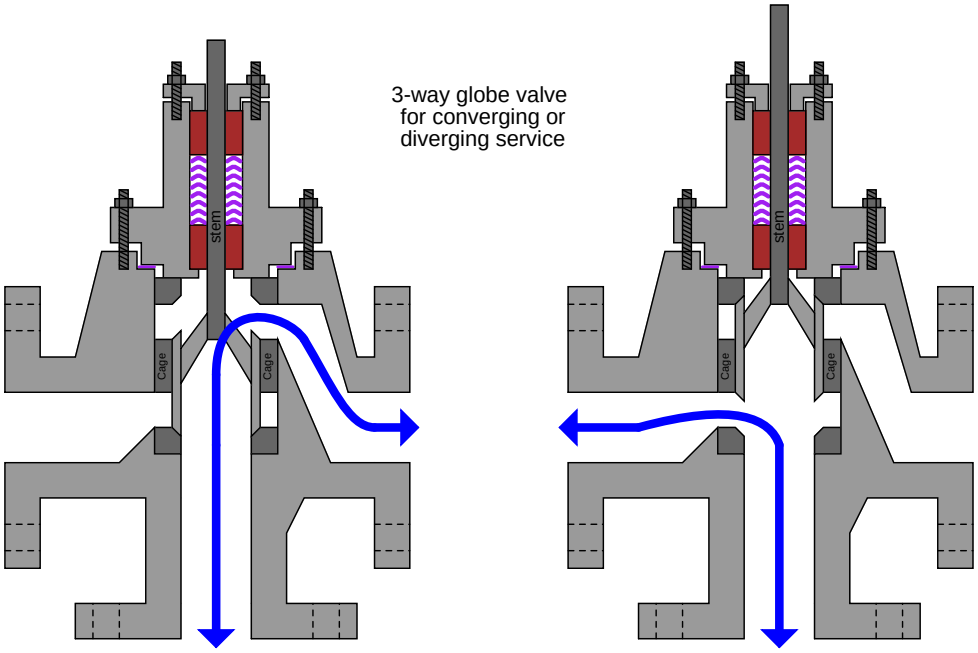


Unbalanced plugs generate a force equal to the product of the differential pressure across the plug and the plug's area ($F = PA$), which may be quite substantial in some applications. Balanced plugs do not generate this same force because they equalize the pressure on both sides of the plug, however, they exhibit the disadvantage of one more leak path when the valve is in the fully closed position (through the balancing ports, past the piston ring, and out the cage ports):



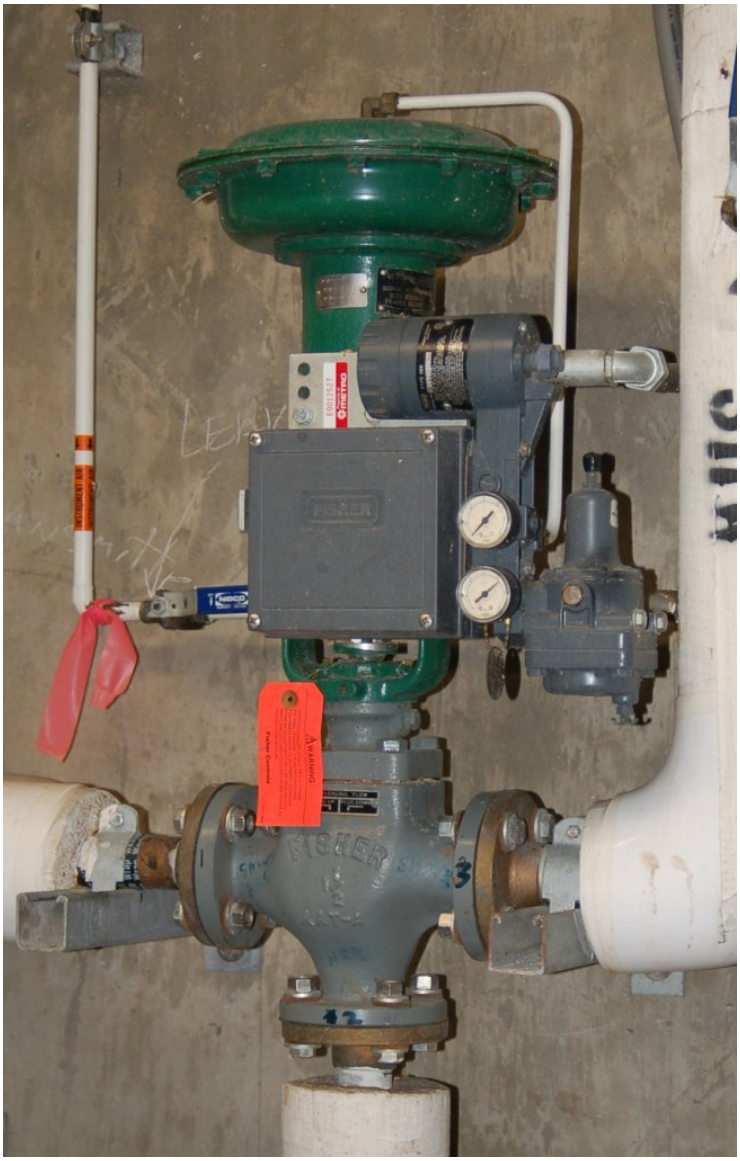
Thus, balanced and unbalanced cage-guided globe valves exhibit similar characteristics to double-ported and single-ported stem- or port-guided globe valves, and for similar reasons. Balanced cage-guided valves are easy to position, just like double-ported stem-guided and port-guided globe valves. However, balanced cage-guided valves tend to leak more when in the shut position due to a greater number of leak paths, much the same as with double-ported stem-guided and port-guided globe valves.

Another style of globe valve body is the *three-way* body, sometimes called a *mixing* or a *diverting* valve. This valve design has three ports on it, with the plug (in this particular case, a cage-guided plug) controlling the degree to which two of the ports connect with the third port:



This dual illustration shows a three-way valve in its two extreme stem positions. If the stem is positioned between these two extremes, all three ports will be "connected" to varying degrees. Three-way valves are useful in services where a flow stream must be diverted (split) between two different directions, or where two flow streams must converge (mix) within the valve to form a single flow stream.

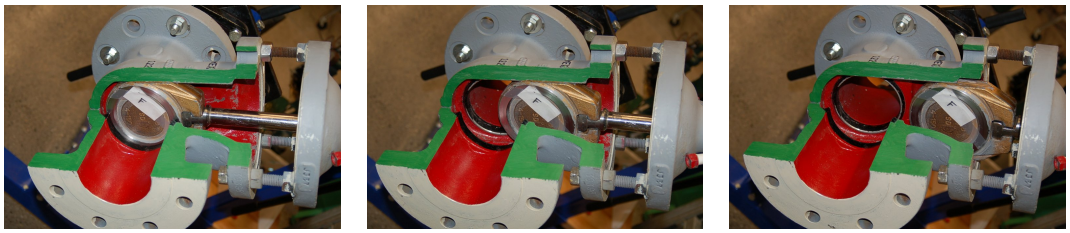
A photograph of a three-way globe valve mixing hot and cold water to control temperature is shown here:



27.1.2 Gate valves

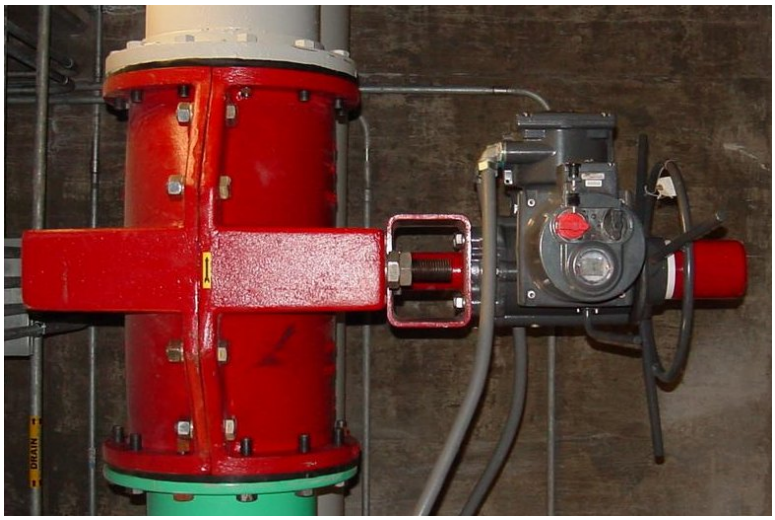
Gate valves work by inserting a dam ("gate") into the path of the flow to restrict it, in a manner similar to the action of a sliding door. Gate valves are more often used for on/off control than for throttling.

The following set of photographs shows a hand-operated gate valve (cut away and painted for use as an instructional tool) in three different positions, from full closed to full open (left to right):

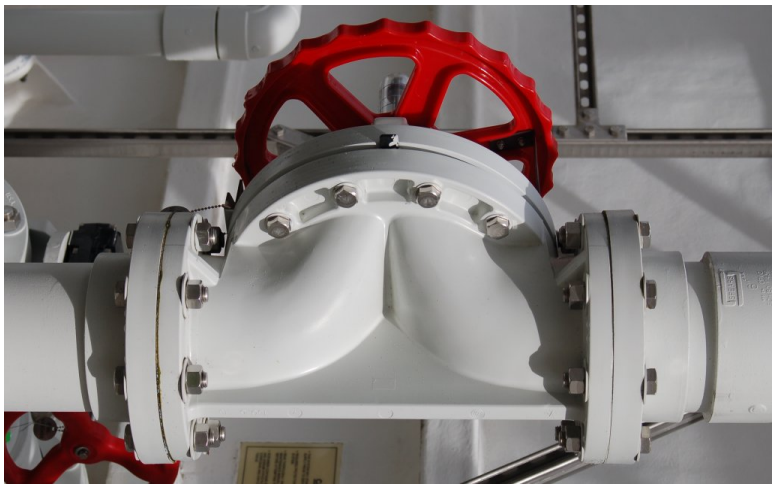


27.1.3 Diaphragm valves

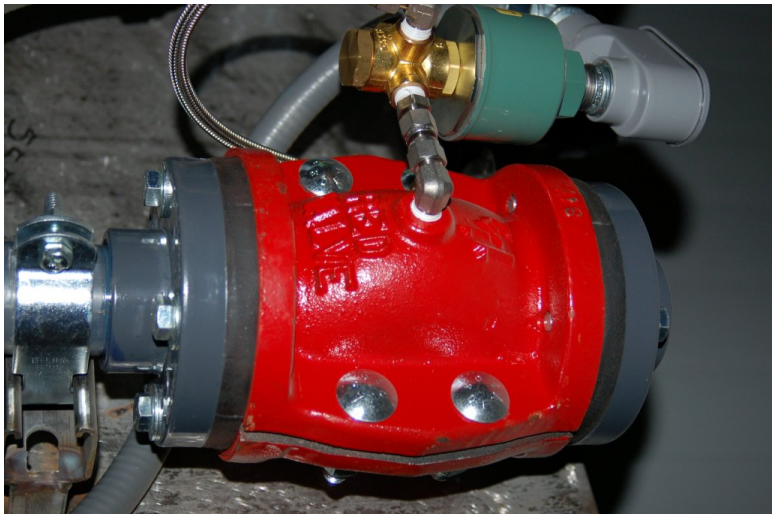
Diaphragm valves use a flexible sheet pressed close to the edge of a solid dam to narrow the flow path for fluid. Their operation is not unlike controlling the flow of water through a flexible hose by pinching the hose. These valves are well suited for flows containing solid particulate matter such as slurries, although precise throttling may be difficult to achieve due to the elasticity of the diaphragm. The next photograph shows a diaphragm valve actuated by an electric motor, used to control the flow of treated sewage:



The following photograph shows a hand-actuated diaphragm valve, the external shape of the valve body revealing the "dam" structure against which the flexible diaphragm is pressed to create a leak-tight seal when shut:



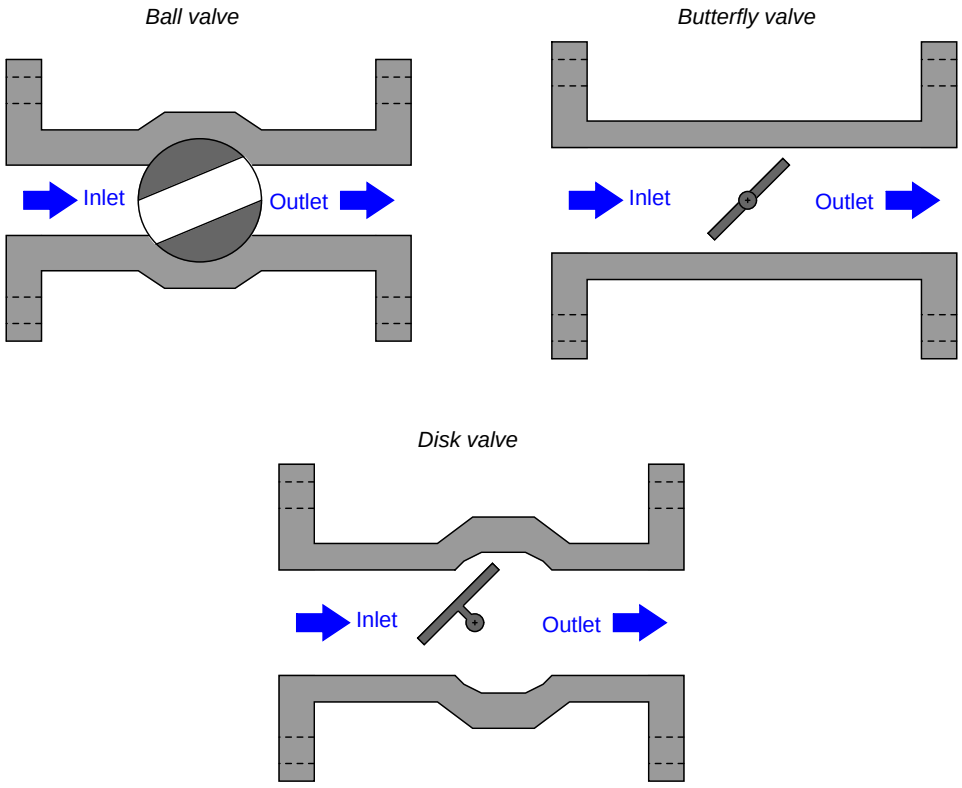
Some diaphragm valves are pneumatically actuated, using the force of compressed air on one side of the diaphragm to press it against the dam (on the other side) to shut off flow. This next example is of a small air-actuated diaphragm valve, controlling the flow of water through a 1-inch pipe:



The actuating air for this particular diaphragm valve comes through an electric solenoid valve. The solenoid valve in this photograph has a brass body and a green-painted solenoid coil.

27.2 Rotary-stem valves

A different strategy for controlling the flow of fluid is to insert a rotary element into the flow path. Instead of sliding a stem into and out of the valve body to actuate a throttling mechanism, rotary valves rely on the rotation of a shaft to actuate the trim. An important advantage of rotary control valves over sliding-stem designs such as the globe valve and diaphragm valve is a virtually obstructionless path for fluid when the valve is wide-open[3].



[3] Of course, gate valves also offer obstructionless flow when wide-open, but their poor throttling characteristics give most rotary valve designs the overall advantage.

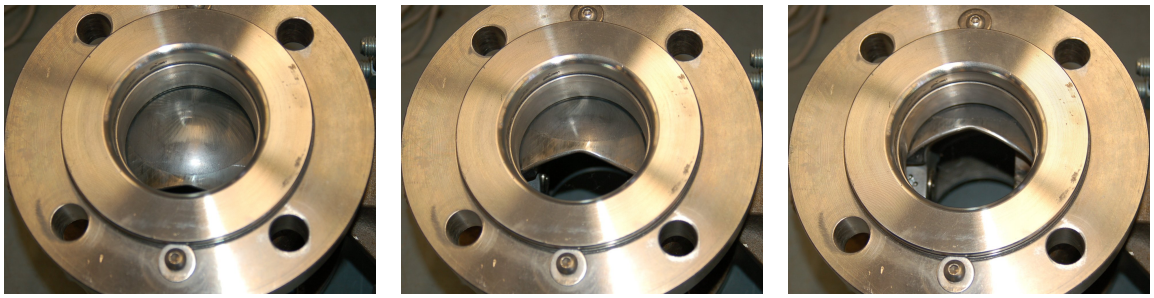
27.2.1 Ball valves

In the ball valve design, a spherical ball with a passageway cut through the center rotates to allow fluid more or less access to the passageway. When the passageway is parallel to the direction of fluid motion, the valve is wide open; when the passageway is aligned perpendicular to the direction of fluid motion, the valve is fully shut (closed).

The following set of photographs shows a hand-operated ball valve in three different positions, from nearly full closed to nearly full open (left to right):



Simple ball valves with full-sized bores in the rotating ball are generally better suited for on/off service than for throttling (partially-open) service. A better design of ball valve for throttling service is the *characterized* or *segmented* ball valve, shown in various stages of opening in the following set of photographs:



The V-shaped notch cut into the opening lip of the ball provides a narrower area for fluid flow at low opening angles, providing more precise flow control than a plain-bore ball valve.

27.2.2 Butterfly valves

Butterfly valves are quite simple to understand: the "butterfly" element is a disk that rotates perpendicular to the path of fluid flow. When parallel to the axis of flow, the disk presents minimal obstruction; when perpendicular to the axis, the disk completely blocks any flow. Fluid-tight shut-off is difficult to obtain in the classic butterfly design unless the seating area is lined with a soft (elastic) material.

27.2.3 Disk valves

Disk valves (often referred to as *eccentric disk valves*, or as *high-performance butterfly valves*) are a variation on the butterfly design intended to improve seat shut-off. The disk's center is offset from the shaft centerline, causing it to approach the seat with a "cam" action that results in high seating pressure. Thus, tight shut-off of flow is possible even when using metal seats and disks.

The following photograph shows the body of a Fisher E-plug control valve, with the disk in a partially-open position:



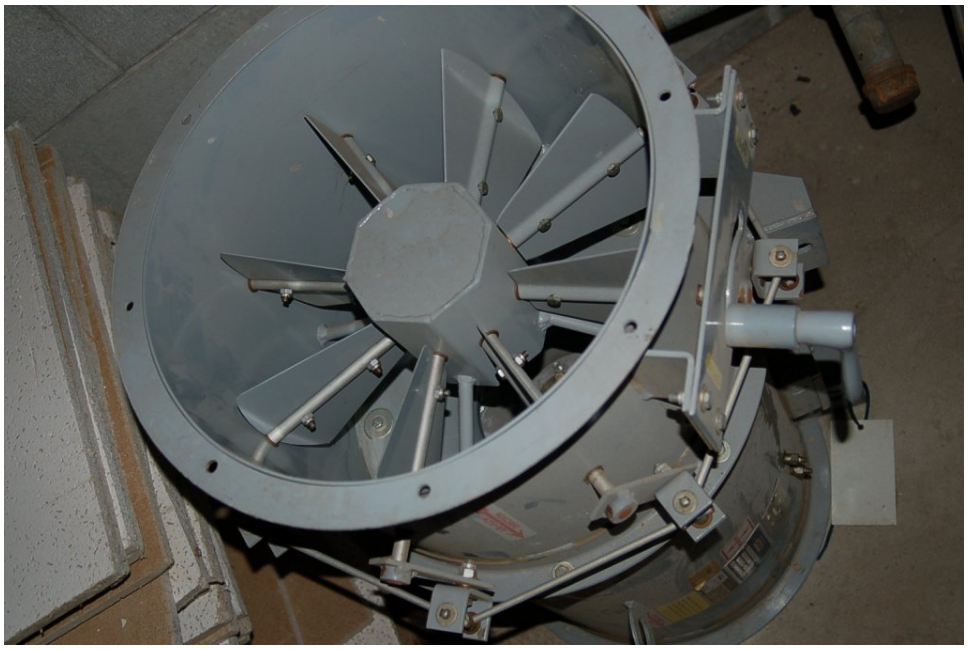
27.3 Dampers and louvres

A *damper* (otherwise known as a *louvre*) is a multi-element flow control device generally used to throttle large flows of air at low pressure. Dampers find common application in furnace and boiler draft control, and in HVAC (Heating, Ventilation, and Air Conditioning) systems.

Common damper designs include parallel and radial. Parallel-vane dampers resemble a Venetian blind, with multiple rectangular vanes synchronously rotated to throttle flow through a rectangular opening. A photograph of a parallel-vane damper is shown here, part of an induced-draft (suction) air fan system on a separator at a cement plant. The vanes are not visible in this photograph because they reside inside the metal air duct, but the actuator mechanism and linkages connecting seven vane shafts together are:



Radial-vane dampers use multiple vanes arranged like petals of a flower to throttle flow through a circular opening. A photograph of a radial-vane damper is shown here (note the levers and linkages on the periphery of the tube, synchronizing the motions of the eight vanes so they rotate at the same angle):



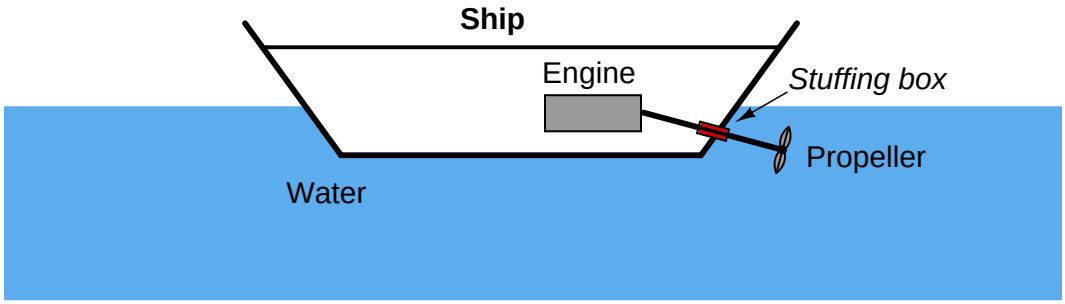
Dampers find use in many non-industrial applications as well. Take for instance these greenhouse vents, actuated by pneumatic (air-powered) piston actuators:



27.4 Valve packing

Regardless of valve type, all stem-actuated control valves require some form of seal allowing motion of the stem from some external device (an *actuator*) while sealing process fluid so no leaks occur between the moving stem and the body of the valve. The general term for this sealing mechanism is *packing*.

This mechanical feature is not unlike the *stuffing box* used to seal seawater from entering a boat or ship at the point where the propeller shaft penetrates the hull:



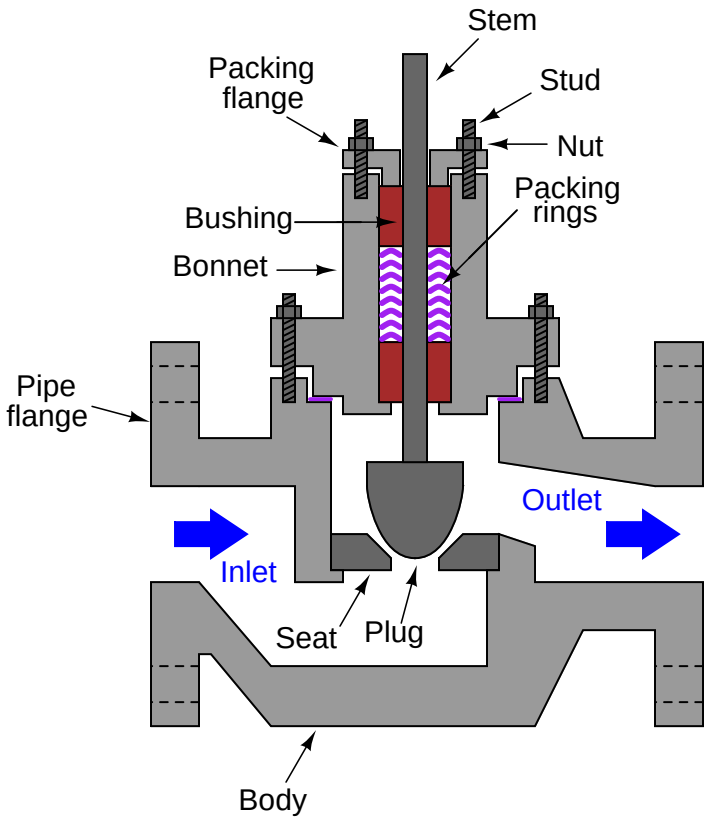
The fundamental problem is the same for the ship as it is for the control valve: how to allow a moving shaft to pass through what needs to be an impenetrable barrier to some fluid (in the case of the ship's hull, the fluid is seawater). The solution is to wrap the shaft in a flexible material that maintains a close fit to the shaft without binding its motion. A traditional packing material for ship propeller shafts is *flax rope*. Some form of lubrication is usually provided so this packing material does not impose excessive friction on the shaft's motion[4].

Modern marine stuffing boxes use advanced materials such as Teflon (PTFE) or graphite instead of flax, which wear longer and leak less water. In the world of control valves, the traditional packing material used to be asbestos (shaped into rings or ropes, much like flax used to be shaped for use in stuffing boxes), but is now commonly Teflon or graphite as well.

In the case of a ship's stuffing box, a little bit of water leakage is not a problem since all ships are equipped with bilge pumps to pump out collected water over time. However, leakage is simply unacceptable in many industrial control valve applications where we must minimize *fugitive emissions*. A "fugitive emission" is any unwanted escape of process substance into the surrounding environment, usually from leaks around pump and valve shafts. Special "environmental" packing sets are available for control valve applications where this is a concern.

[4] Some packing materials, most notably Teflon and graphite, tend to be *self-lubricating*.

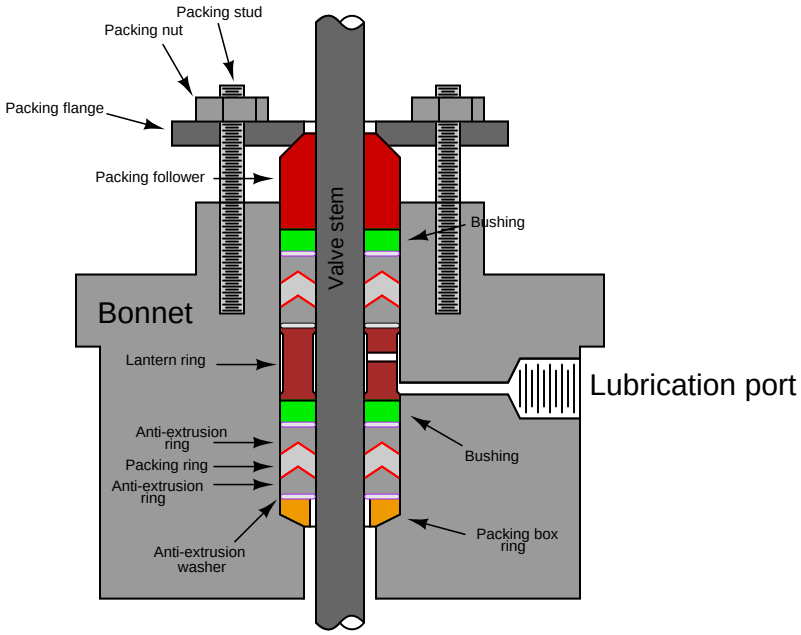
Packing in a sliding-stem valve fits in a section of the valve body called the *bonnet*, shown in this simplified diagram of a single-ported, stem-guided globe valve:



Here, the packing material takes the form of several concentric rings, stacked on the valve stem like washers on a bolt. These packing rings are forced down from above by the *packing flange* to apply a compressive force around the circumference of the valve stem. This compressive force is necessary to generate mechanical stress in the packing material to make it seal tightly against the stem of the valve and the interior wall of the bonnet.

Two nuts threaded onto studs maintain proper force on the packing rings. Care must be taken not to over-tighten these nuts and over-compress the packing material, or else the packing will create excessive friction on the valve stem. Not only will this friction impede precise valve stem motion, but it will also create undue wear on the stem and packing, increasing the likelihood of future packing leakage. Insufficient packing flange force will lead to poor sealing, with process fluid potentially leaking past the packing and out of the valve.

A closer look at the bonnet shows a multitude of components working together to form a low-friction, pressure-tight seal for the moving valve stem:

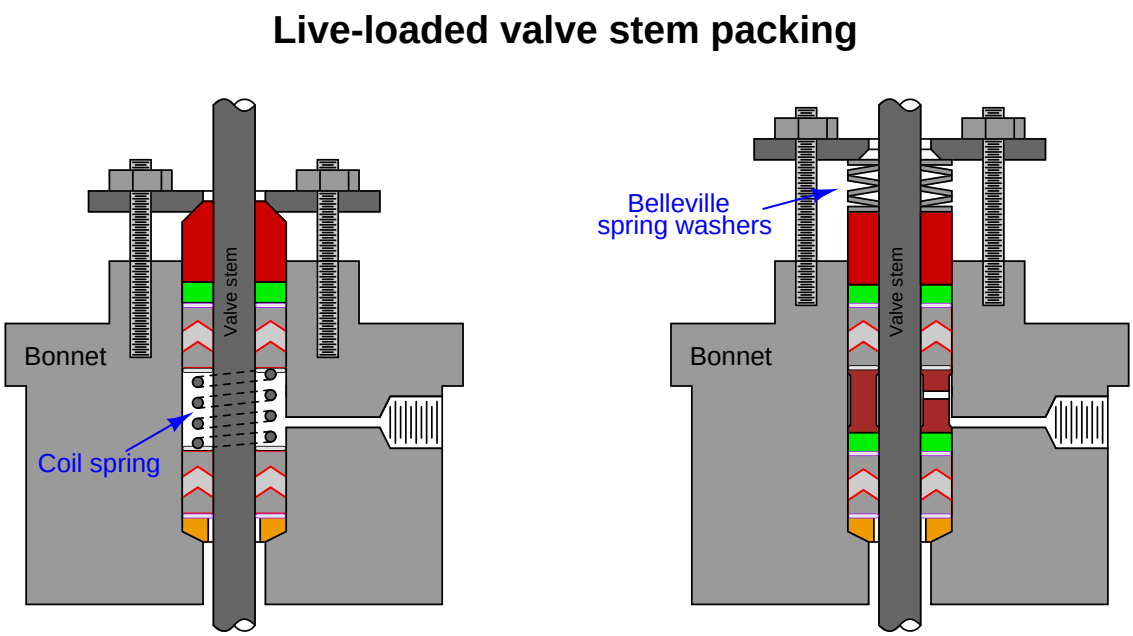


In this diagram, we see two sets of packing rings separated by a metal piece called a *lantern ring*. The lantern ring acts as a spacer allowing lubricant introduced through the lubrication port to enter into both packing sets from the middle of the bonnet.

The packing shown here is "loaded" by the compressive force exerted by the packing follower. The only elasticity in this particular system resides in the packing material itself. This is called *stationary loading*, otherwise known as *jam* packing. Over time, as the packing material wears and fatigues, the packing follower must be re-compressed by carefully tightening the packing nuts.

One must be very careful when torquing the packing nuts on a stationary-loaded packing set. Insufficient torque (which translates into insufficient stress applied to the packing) will result in process fluid leakage. Excessive torque (causing excessive stress on the packing) will result in high valve stem friction and premature packing failure. The latter scenario is what one usually finds in industrial settings, where well-intentioned but uninformed personnel over-tighten valve packing in an effort to prevent leaks. The proper remedy for a packing assembly that leaks despite having been properly torqued is replacement, not further tightening.

An alternative to "stationary" loading is to insert a metal spring into the packing assembly, so that the elasticity of the spring helps to maintain an appropriate amount of packing stress as the packing material wears and ages. This is called *live loading*, examples of which are shown here:



In one of these examples we see a coil spring inside the bonnet used to live-load the packing. In the other example we see a set of spring-steel washers known as a *Belleville* spring. Belleville springs have a concave profile, giving them resistance to compression along the shaft axis. These spring washers are always stacked in opposed pairs (concave against concave, convex against convex) so the washers have room to compress.

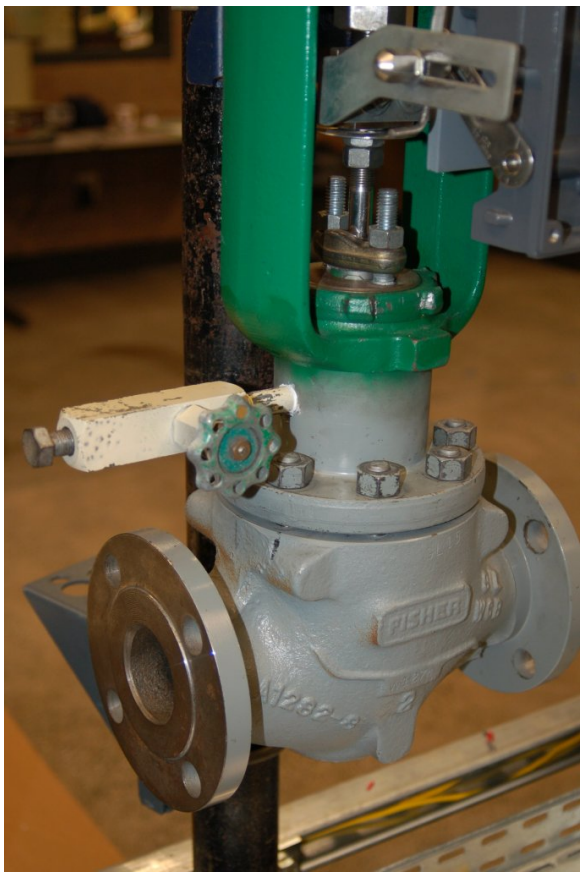
Photographs taken of an actual valve packing assembly removed from the bonnet (left), and reassembled on the valve stem (right) reveal the structure of the packing and associated components.





There is no lantern ring in this packing assembly, but there is a coil spring. This makes it a *live-loaded* packing as opposed to a *jam* packing.

In packing applications requiring external lubrication, a *stem packing lubricator* may be connected to the lubrication port on the bonnet. This device uses a long, threaded bolt as a piston to push a quantity grease into the packing assembly:



To operate a lubricator, the hand valve on the lubricator is first secured in the closed (shut) position, then the bolt is fully unscrewed until it falls out of the lubricator body. An appropriate lubricating grease is squeezed into the bolt hole in the lubricator body, and the bolt threaded back into place until hand-tight. Using a wrench or socket to tighten the bolt a bit more (generating pressure in the grease) and opening the hand valve allows grease to enter the packing chamber. The bolt is then screwed in fully, pushing the entire quantity of grease into the packing. As a final step, the hand valve is fully shut so there is no way for process liquid to leak out past the bolt threads.

The two most common packing materials in use today are Teflon (PTFE) and graphite. Teflon is the better of the two with regard to fluid sealing, stem friction, and stem wear[5]. Teflon is also quite resistant to attack from a wide variety of chemical substances. Unfortunately, it has a limited temperature range and cannot withstand intense nuclear radiation (making it unsuitable for use near reactors in nuclear power plants). Graphite is another self-lubricating packing material, and it has a far greater temperature range than Teflon[6] as well as the ability to withstand harsh nuclear radiation, but creates much more stem friction than Teflon. Graphite packing also has the unfortunate property of permitting *galvanic corrosion* between the stem and bonnet metals due to its electrical conductivity. Sacrificial zinc washers are sometimes added to graphic packing assemblies to help mitigate this corrosion, but this only postpones rather than prevents corrosive damage to the stem.

The following photographs show samples of woven graphite (left) and Teflon (right) "rope" packing, longer pieces of which would normally be found bent around valve stems to form seals. The graphite packing has a shiny finish and flakes easily, while the Teflon packing is plain white in color and maintains its integrity. Both feel slippery to the touch:





Hybrid packing materials, such as carbon-reinforced Teflon, have been developed in an attempt to combine the best characteristics of both materials.

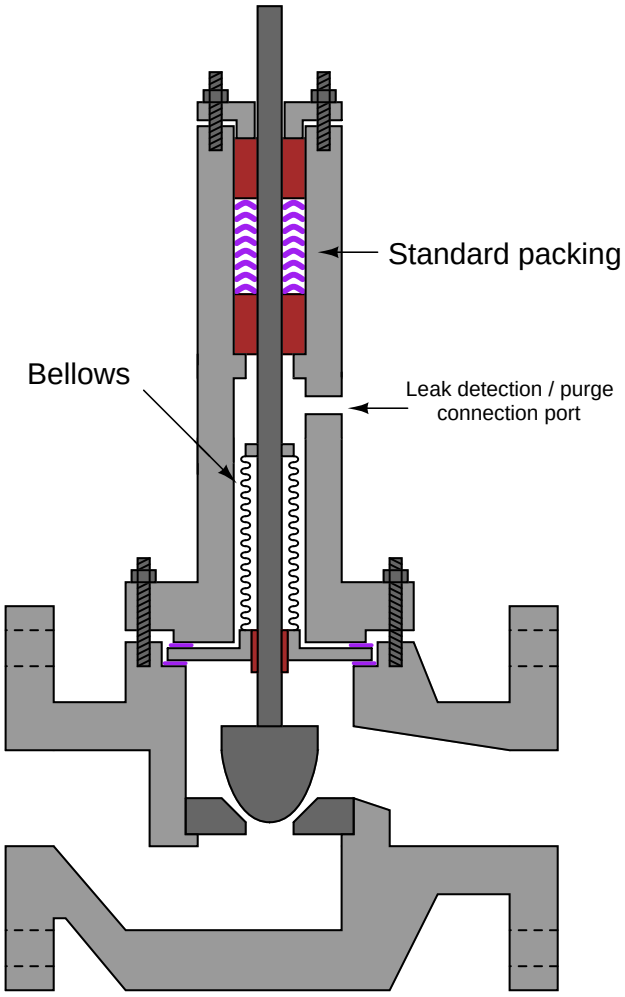
A legacy valve packing material is *asbestos*, woven into packing rings much the same way as graphite fibers are woven into modern packing rings. Asbestos is a mineral, which made it suitable for high-temperature process applications. Its electrical non-conductivity eliminated the galvanic corrosion problem inherent to graphite. Unfortunately, its classification as a hazardous substance[7] precludes its use as a packing material for contemporary applications.

[5] Based on friction values shown on page 131 of Fisher's *Control Valve Handbook* (Third Edition), Teflon packing friction is typically 5 to 10 times less than graphite packing for the same stem size!

[6] Graphite packing is usable in services ranging from cryogenic temperatures to 1200 degrees Fahrenheit, as opposed to Teflon which is typically rated between -40 ^{o}F and 450 ^{o}F.

[7] Asbestos fibers have the ability to permanently lodge in the air sacs of human lungs, leading to long-term health problems if those fibers are inhaled.

A completely different approach to packing is a device called a *bellows seal*: an accordion-like metal tube fastened to the valve stem and to the bonnet, forming a leak-proof seal with negligible friction. The accordion ribs give the bellows seal an ability to stretch and compress with a sliding stem's linear motion. Since the bellows is an uninterrupted metal tube, there is no place at all for leaks to develop:

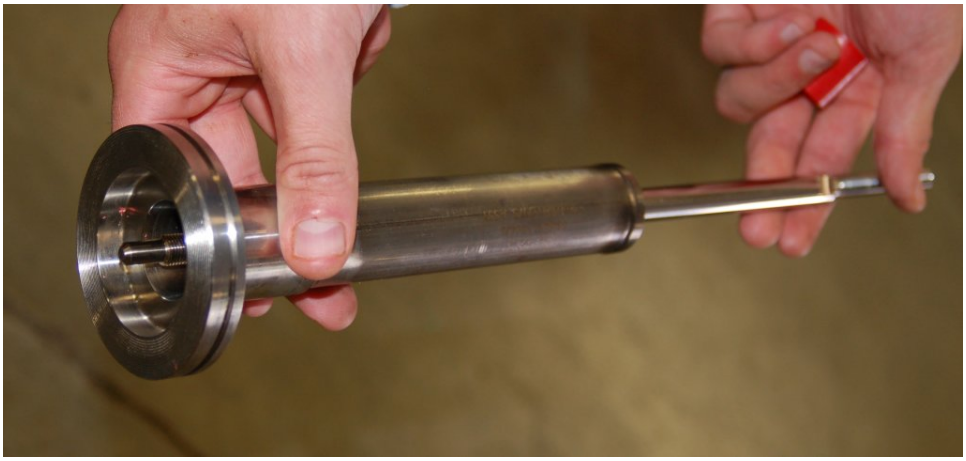


The port on the extended bonnet serves as a point of connection for process fluid leak detection sensors, to sound an alarm and/or take action in the event of a ruptured bellows. The sensor may be as simple as a pressure switch, calibrated to "trip" at some modest pressure value below that of the normal process operating pressure. When the bellows seal breaks[8], the sensor will detect the leak and the standard packing assembly will maintain a reasonable seal until repairs are made on

[8] Bellows have a limited service life, which means the possibility of a rupture is likely. This is why a conventional packing assembly is always included in a bellows-equipped bonnet.

the valve.

An actual bellows seal unit appears in this photograph:



The accordion-shaped bellows is contained and protected inside the thick metal tube visible in this photograph. One end of the bellows is welded to the valve stem, and the other end is welded to the protective tube. With the wide flange of the tube firmly clamped in the bonnet of the valve, a leak-free seal exists.

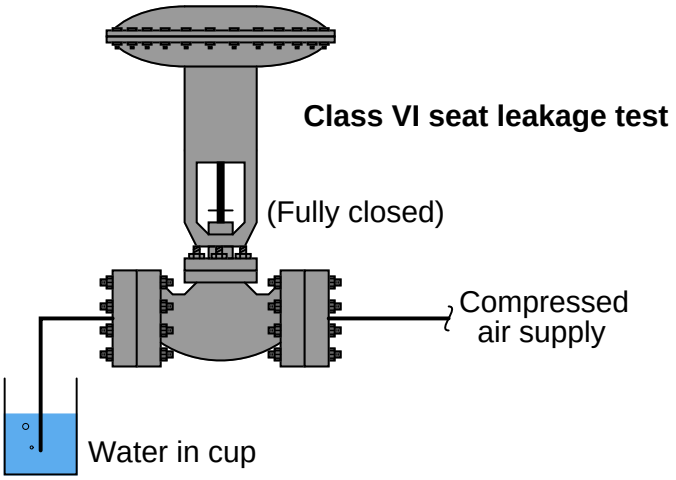
27.5 Valve seat leakage

In some process applications, it is important that the control valve be able to completely stop fluid flow when placed in the "closed" position. Although this may seem to be a fundamental requirement of any valve, it is not necessarily so. Many control valves spend most of their operating lives in a partially-open state, rarely opening or closing fully. Additionally, some control valve designs are notorious for the inability to completely shut off (e.g. double-ported globe valves). Given the common installation of manual "block" valves upstream and downstream of a control valve, there is usually a way to secure zero flow through a pipe even if a control valve is incapable of tight shut-off. For some applications, however, tight control valve shut-off is mandatory.

For this reason we have several classifications for control valves, rating them in their ability to fully shut off. Seat leakage tolerances are given roman numeral designations, as shown in this table[9]:

Class	Maximum allowable leakage rate	Test pressure drop
I	(no specification given)	(no specification given)
II	0.5% of rated flow capacity, air or water	45-60 PSI or max. operating
III	0.1% of rated flow capacity, air or water	45-60 PSI or max. operating
IV	0.01% of rated flow capacity, air or water	45-60 PSI or max. operating
V	0.0005 ml/min water per inch orifice size per PSI	Max. operating
VI	Bubble test, air or nitrogen	50 PSI or max. operating

The "bubble test" used for Class VI seat leakage is based on the leakage rate of air or nitrogen gas past the closed valve seat as measured by counting the rate of gas bubbles escaping a bubble tube submerged under water. For a 6 inch valve, this maximum bubble rate is 27 bubbles per minute (or about 1 bubble every two seconds):



It is from this leakage test procedure that the term *bubble-tight shut-off* originates. Class VI shut-off is often achievable only through the use of "soft" seat materials such as Teflon rather than

[9]Data in this table taken from Fisher's *Control Valve Handbook*.

hard metal-to-metal contact between the valve plug and seat. Of course, this method of achieving bubble-tight shut-off comes at the price of limited operating temperature range and the inability to withstand nuclear radiation exposure.

Special test fixtures are typically used in control valve rebuild shops to test the leakage rates of a rebuilt valve. One such test bench appears in this photograph:



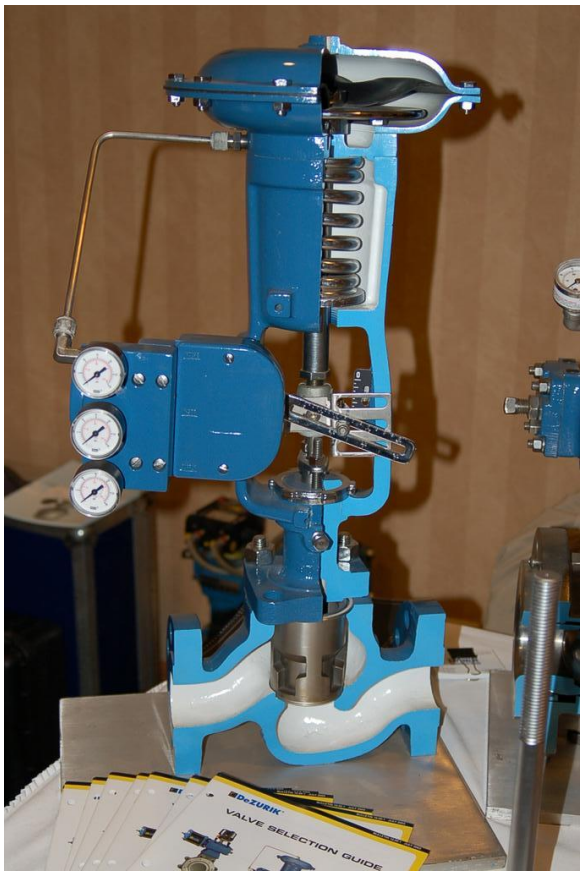
In the foreground of this photograph we see a special "vise" used to make quick, pressure-tight connections to the flange of any control valve placed within it. A movable flange sandwiches the control valve against a stationary flange, both flanges faced with high-density plastic for a pressure-tight fit against the valve body flange faces. A stainless-steel panel in the background provides a set of air pressure regulators, hand valves, rotameters, pressure gauges, and even a "bubble" flow indicator to measure leakage flow rate through the control valve under varying pressure conditions.

27.6 Control valve actuators

The purpose of a control valve *actuator* is to provide the motive force to operate a valve mechanism. Both sliding-stem and rotary control valves enjoy the same selection of actuators: *pneumatic*, *hydraulic*, *electric motor*, and *hand* (manual).

27.6.1 Pneumatic actuators

Pneumatic actuators use air pressure pushing against either a flexible diaphragm or a piston to move a valve mechanism. The following photograph shows a cut-away control valve, with a pneumatic diaphragm actuator mounted above the valve body. You can see the large coil spring providing default positioning of the valve (air pressure acting against the diaphragm moves the valve against the spring) and the rubber diaphragm at the very top. Air pressure applied to the bottom side of the diaphragm lifts the sliding stem of the valve in the upward direction, against the spring's force which tries to push the stem down:



The amount of force (F) in units of pounds generated by any fluid pressing against any surface is equal to the fluid's pressure (P) in units of PSI multiplied by the surface area (A) in units of square inches ($F = PA$). In the case of a circular diaphragm, with area equal to πr^2, the complete formula for force is $F = P\pi r^2$. For example, a control valve diaphragm 14 inches in diameter (radius = 7 inches) with an applied air pressure of 15 PSI generates a linear force of 2309 pounds.

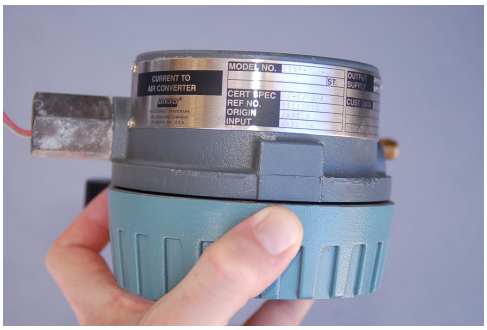
Air pressure required to motivate a pneumatic actuator may come directly from the output of a pneumatic process controller, or from a *signal transducer* (or *converter*) translating an electrical signal into an air pressure signal. Such transducers are commonly known as *I/P* or "I to P"

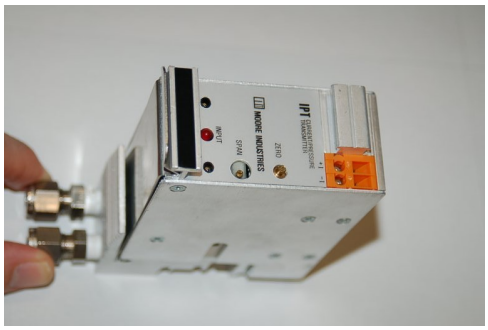
converters, since they typically translate an electric current signal (I) of 4 to 20 mA DC into an air pressure signal (P) of 3 to 15 PSI.

The following photographs show I/P transducers of different make and model. A Fisher model 846 appears in the upper-left photograph, while an older Fisher model 546 appears in the upper-right (with cover removed). A Foxboro model E69F I/P appears in the lower-left photograph, while a Moore Industries model IPT appears in the lower-right:



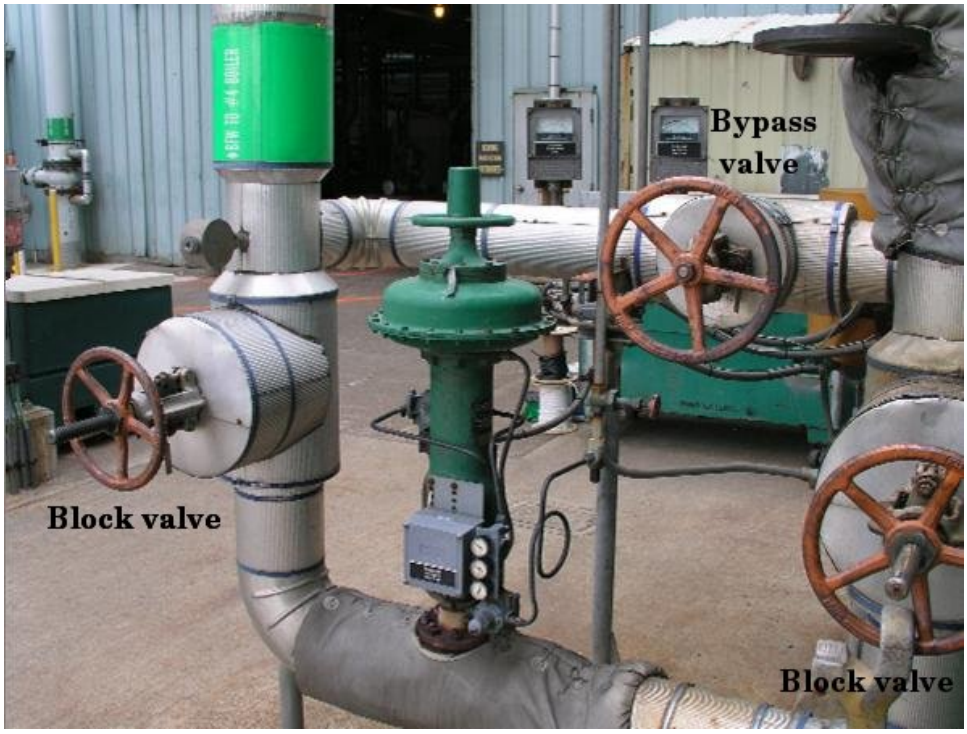






Despite their differing designs and appearances, they all function the same: accepting an analog DC current signal input and a clean supply air pressure of about 20 PSI, outputting a variable air pressure signal proportional to the electric current input. An interesting feature to compare between these four I/P transducers is their relative ruggedness. Every transducer shown except the Moore Industries model (lower-right) is built to withstand direct exposure to a process atmosphere, hence the heavy cast-metal housings and electrical conduit fittings. The Moore Industries unit is intended for a sheltered location, and may be plugged in to a "manifold" with several other I/P transducers to form a compact bank of transducers capable of driving air pressure signals to several valve actuators.

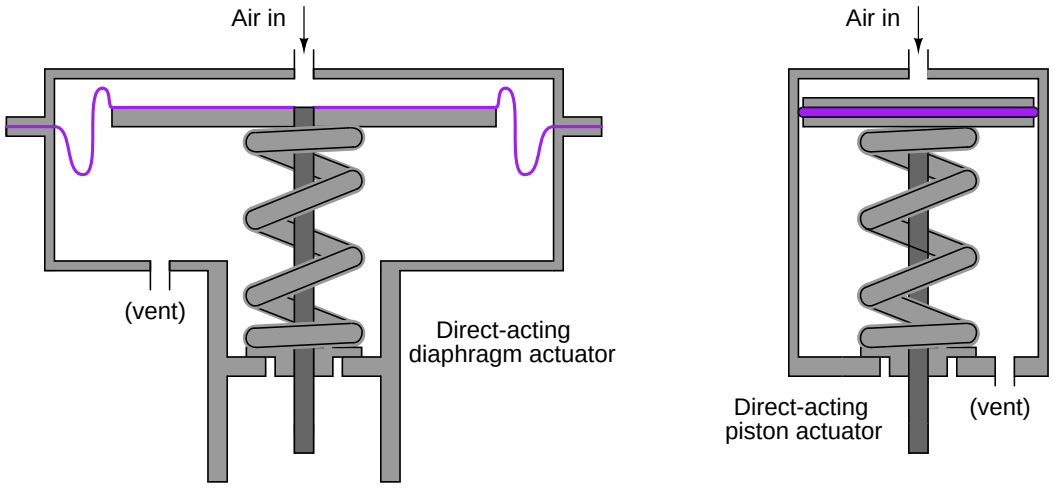
Some pneumatic valve actuators are equipped with *handwheels* which are used to manually position the valve in the event of air pressure failure. The next photograph shows a sliding-stem control valve with pneumatic diaphragm actuator and a "handwheel" on the top:



Note the three manual valves located around the control valve: two to *block* flow through the control valve and one to *bypass* flow around the control valve in the event of control valve failure or maintenance. These manual valves happen to be of the *gate* design, with *rising-stem* actuators to clearly show their status (stem protruding = open valve ; stem hidden = closed valve). Such block-and-bypass manual valve arrangements are quite common in the process industries where control valves fulfill critical roles and some form of manual control is needed as an emergency alternative.

Note also the air pressure tubing between the valve actuator and the air supply pipe, bent into a loop. This is called a *vibration loop*, and it exists to minimize strain on the metal tubing from vibration that may occur.

Pneumatic actuators may take the form of pistons rather than diaphragms. Illustrations of each type are shown here for comparison:

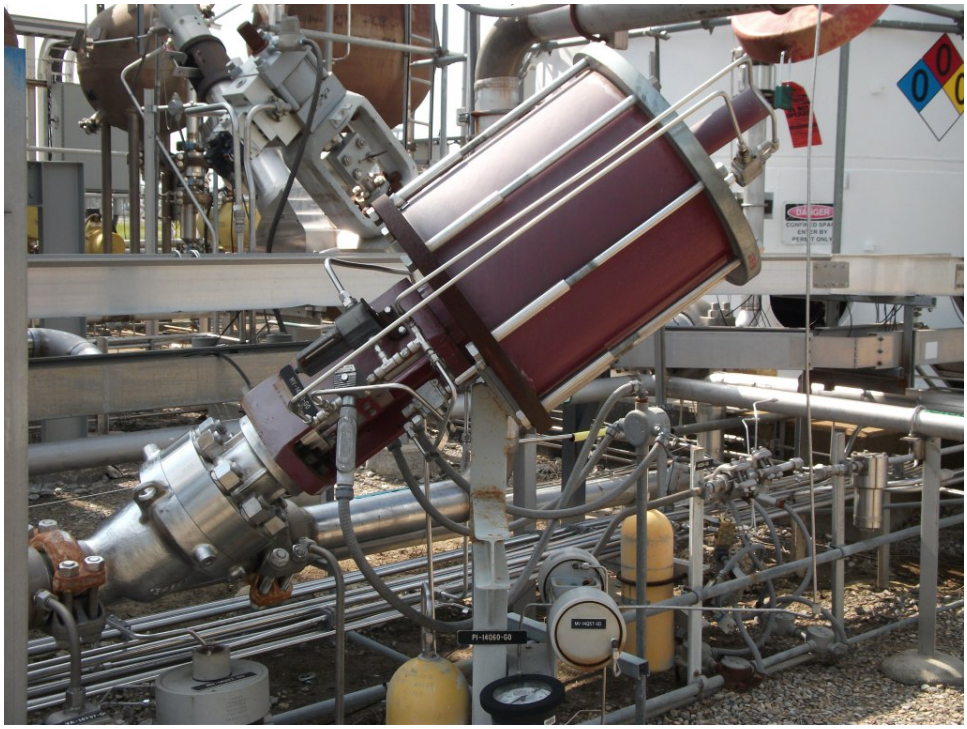


Piston actuators generally have longer stroke lengths than diaphragm actuators, and are able to operate on much greater air pressures[10]. Since actuator force is a function of fluid pressure and actuator area ($F = PA$), this means piston actuators are able to generate more force than diaphragm actuators of the same diameter. A 14 inch diaphragm operating at a maximum pressure of 35 PSI generates 5388 pounds of force, but the same size piston operating at a maximum pressure of 150 PSI generates 23091 pounds of force. The combination of greater force and greater displacement yields more work potential for piston actuators than diaphragm actuators of equivalent size, since mechanical work is the product of force and displacement ($W = Fx$).

Diaphragm actuators enjoy the definite advantage of less friction than piston actuators. Less friction means greater precision in positioning the valve stem, which gives diaphragm actuators an advantage over piston actuators where precise valve positioning is important, all other factors being equal.

[10]The greater pressure rating of a piston actuator comes from the fact that the only "soft" component (the sealing ring) has far less surface area exposed to the high pressure than a rolling diaphragm. This results in significantly less stress on the elastic ring than there would be on an elastic diaphragm exposed to the same pressure. There really is no limit to the stroke length of a piston actuator as there is with the stroke length of a diaphragm actuator. It is possible to build a piston actuator miles long, but such a feat would be impossible for a diaphragm actuator, where the diaphragm must stretch (or roll) the entire stroke length.

The following photograph of an ultra-high pressure oxygen valve shows a large pneumatic piston actuating a relatively tiny valve body:



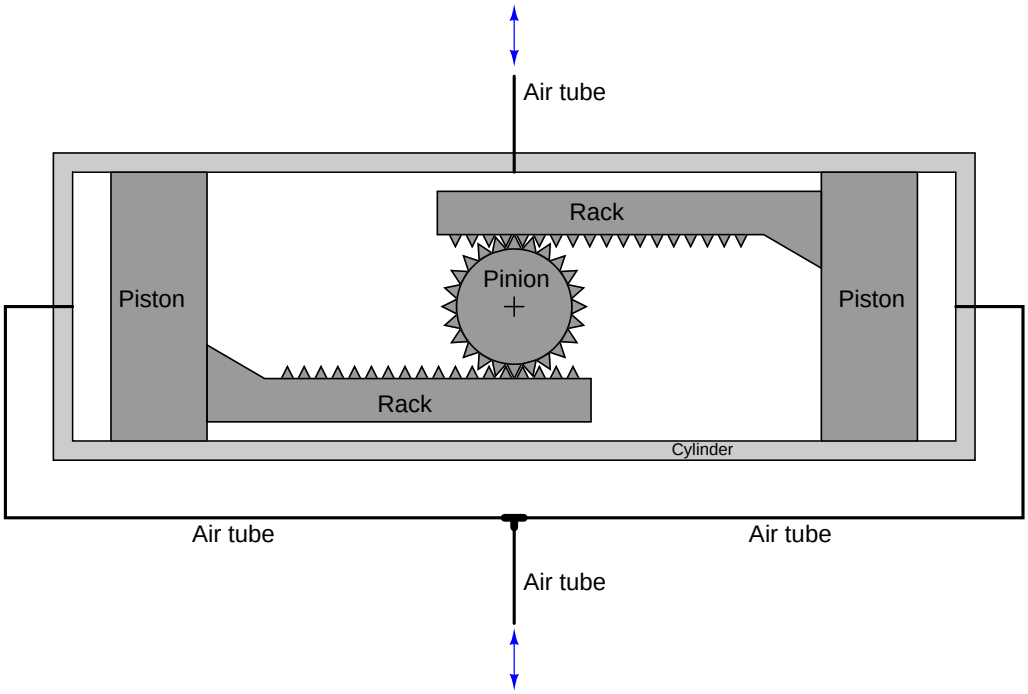
Since the only rationale for selecting such a large piston actuator is to generate large actuating force, we may conclude that this relatively small valve body requires an unusually high force to actuate. This is indeed the case, as the process fluid pressure drop across the valve trim in this application happens to be *several thousand PSI*. This great of a pressure differential, dropped across even a small valve plug, generates substantial force. The actuator must generate even more force than this in order to successfully move the valve, and must do so while limited to the typical instrument air pressure value of 100 PSI. Thus, the only way for the actuator to generate a superior force to the valve plug while working with much less fluid pressure is for the actuator piston to have a much greater area than the plug.

A double-piston pneumatic actuator appears in the next photograph, providing the mechanical force needed to turn an on/off butterfly valve:



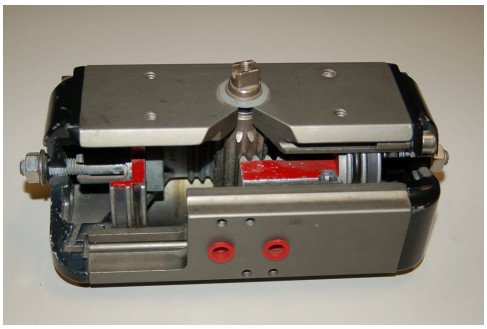
In this particular actuator design, a pair of pneumatically-actuated pistons move a rack-and-pinion mechanism to convert linear piston motion into rotary shaft motion to move the butterfly trim. Note the rotary indicator (yellow in color) at the end of the rotary valve stem, showing what position the butterfly valve is in. Note also the travel switch box (black in color) housing multiple limit switches providing remote indication of valve position to the control room.

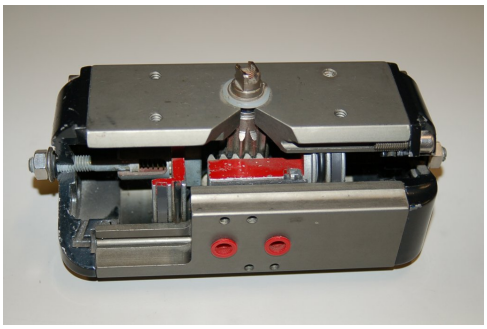
A rack-and-pinion mechanism looks like this, as viewed looking into the axis of the rotary shaft:



Compressed air applied to the bottom tube (with the top tube vented) pushes both pistons toward the center, spinning the pinion gear counter-clockwise. Applying compressed air to the top tube (with the bottom tube vented) pushes both pistons outward, rotating the pinion gear clockwise.

An example of this actuator design, cut away to reveal its inner workings, appears here:





Another pneumatic piston actuator design uses a simple crank lever instead of a rack-and-pinion gear set to convert linear piston motion into rotary motion. This next photograph shows such a piston actuator connected to a ball valve:



Perhaps the greatest disadvantage of piston actuators as applied to control valves is friction between the piston's pressure-sealing ring and the cylinder wall. This is not a problem for on/off control valves, but it may be a significant problem for throttling valves where precise positioning is desired. Diaphragm actuators do not exhibit the same degree of friction as piston actuators because the elastic diaphragm rolls and flexes rather than rubs against a stationary surface as is the case with piston sealing rings.

27.6.2 Hydraulic actuators

Hydraulic actuators use liquid pressure rather than gas pressure to move the valve mechanism. Nearly all hydraulic actuator designs use a piston rather than a diaphragm to convert fluid pressure into mechanical force. The high pressure rating of piston actuators lends itself well to typical hydraulic system pressures, and the lubricating nature of hydraulic oil helps to overcome the characteristic friction of piston-type actuators. Given the high pressure ratings of most hydraulic pistons, it is possible to generate tremendous actuating forces with a hydraulic actuator, even if the piston area is modest. For example, an hydraulic pressure of 2000 PSI applied to one side of a 3 inch diameter piston will generate a linear thrust exceeding 14000 pounds (7 tons)!

In addition to the ability of hydraulic actuators to easily generate extremely large forces, they also exhibit very *stable* positioning owing to the non-compressibility of hydraulic oil. Unlike pneumatic actuators, where the actuating fluid (air) is "elastic," the oil inside a hydraulic actuator cylinder does not yield appreciably under stress. If the passage of oil to and from a hydraulic cylinder is blocked by small valves, the actuator will become firmly "locked" into place. This is an important feature for certain valve-positioning applications where the actuator must firmly hold the valve position in one position.

Some hydraulic actuators contain their own electrically-controlled pumps to provide the fluid power, so the valve is actually controlled by an electric signal. Other hydraulic actuators rely on a separate fluid power system (pump, reservoir, cooler, hand or solenoid valves, etc.) to provide hydraulic pressure on which to operate. Hydraulic pressure supply systems, however, tend to be more limited in physical span than pneumatic distribution systems due to the need for thick-walled tubing (to contain the high oil pressure), the need to purge the system of all gas bubbles, and the problem of maintaining a leak-free distribution network. It is usually not practical to build a hydraulic oil supply and distribution system large enough to cover the entirety of an industrial facility. Another disadvantage of hydraulic systems compared to pneumatic is lack of intrinsic power storage. Compressed air systems, by virtue of air's compressibility (elasticity), naturally store energy in any pressurized volumes, and so provide a certain degree of "reserve" power in the event that the main compressor shut down. Hydraulic systems do not naturally exhibit this desirable trait.

A hydraulic piston actuator attached to a large shut-off valve (used for on/off control rather than throttling) appears in the next photograph. Two hydraulic cylinders may be seen above the round valve body, mounted horizontally. Like the pneumatic piston valve shown earlier, this valve actuator uses a rack-and-pinion mechanism to convert the hydraulic pistons' linear motion into rotary motion to turn the valve trim:



A feature not evident in this photograph is a hydraulic hand pump that may be used to manually actuate the valve in the event of hydraulic system failure.

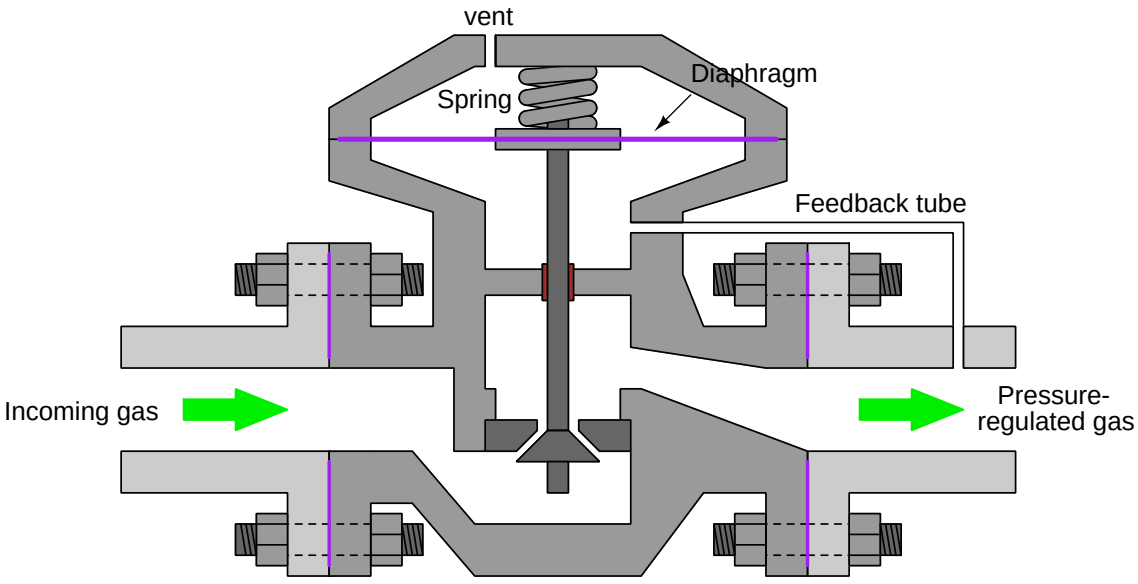
27.6.3 Self-operated valves

Although not a type of actuator itself, a form of actuation worthy of mention is where the process fluid pressure itself actuates a valve mechanism. This self-operating principle may be used in throttling applications or on/off applications, in gas or liquid services alike. The process fluid may be directly tubed to the actuating element (diaphragm or piston), or passed through a small mechanism called a *pilot* to modulate that pressure before reaching the valve actuator. This latter design allows the main valve's motion to be controlled by an adjustable device (the pilot).

A very common application for pilot-operated control valves is gas pressure regulation, especially for fuel gas such as propane or natural gas used to fuel large industrial burners. This next photograph shows a Fisher gas pressure regulator used for regulating the pressure of natural gas fueling an industrial burner:



The following diagram shows how a self-operated, spring-loaded gas pressure regulating valve functions:

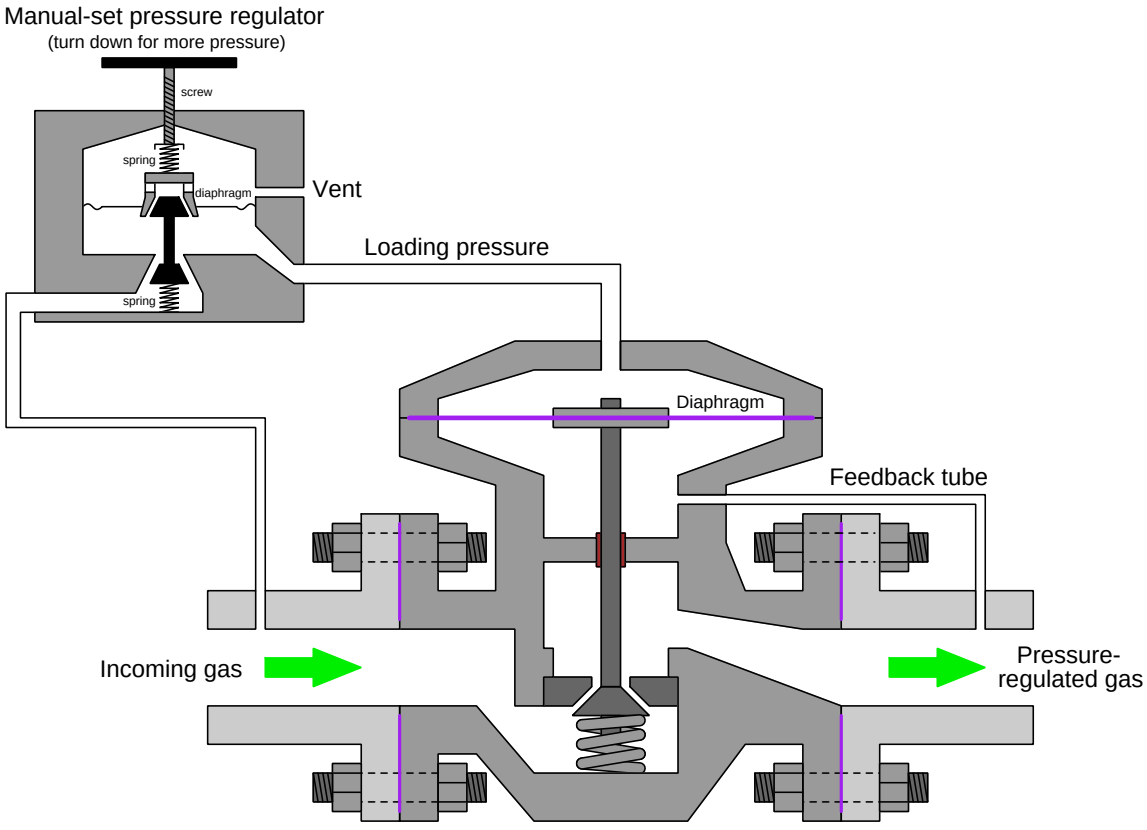


A spring tries to force the plug off the seat, while "feedback" gas pressure from the downstream side of the valve acts against a flexible diaphragm to move the plug toward the seat. The less

downstream pressure, the more the trim opens up; the more downstream pressure, the more the trim shuts off. This spring establishes the pressure-regulating "setpoint" value for the regulator. If a different setpoint is desired, the spring must be replaced with one having a different stiffness.

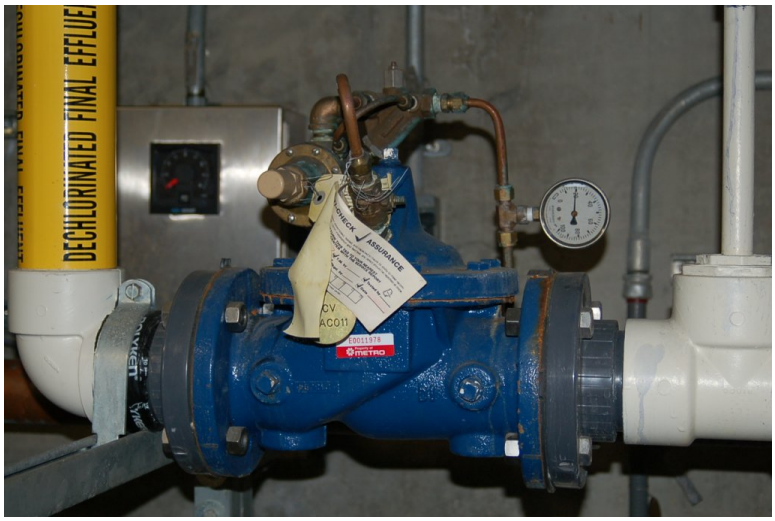
It is worth noting that the feedback tube may be extended to any point downstream of the regulator, in order to ensure proper gas pressure at the point of use rather than merely at the regulator's downstream flange. This "remote sensing" option is useful if substantial pressure losses exist between the regulator and the point of use, for example if the regulated gas must pass through a restrictive heat exchanger and/or a long run of pipe before reaching the point of use. By running the feedback tube between the point of use and the regulator, the regulator is able to sense any degradation in pressure caused by such piping losses and automatically compensate.

A useful modification is the *pilot-loaded* or *externally-loaded* pressure regulator, using a source of external gas pressure to establish the pressure regulation setpoint rather than a spring. Here, a simple manual-adjustment pressure regulator serves as the "pilot" device to send a loading pressure to the top of the main regulator's actuating diaphragm:



Since this loading pressure is easily adjusted by turning the knob on the manual-set pressure regulator, the main regulator now becomes adjustable as well. The pilot mechanism controls the main gas throttling mechanism, hence the name *pilot*.

This next pilot-operated valve is used in a liquid (wastewater) service rather than gas. It does not throttle like a gas pressure regulator, but instead acts in an on/off fashion, controlled by a small electric solenoid valve. The solenoid valve sends water pressure to the actuating diaphragm of the large valve, enabling a small electrical signal to exert control over a large mechanism:



A consumer-grade application of this concept is lawn irrigation control, where the solenoid valves used to switch water flow on and off to sprinkler heads use pilot mechanisms rather than operate the valve mechanism directly with magnetic force. A small solenoid valve opens and closes to send water pressure to an actuating diaphragm, which then operates the larger valve mechanism to start and stop the flow of water to the sprinkler. The use of a pilot allows a relatively small amount of electrical power to control the valve, compared to the amount of electrical power that would be necessary if the solenoid coil were built large enough to actuate the main water valve directly.

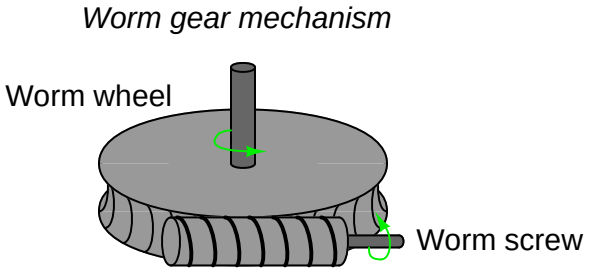
A special case of self-operated valve is the *Pressure Relief Valve* (PRV) or *Pressure Safety Valve* (PSV). These valves are normally shut, opening only when sufficient fluid pressure develops across them to relieve that process fluid pressure and thereby protect the pipes and vessels upstream. Like the other self-operated valves, these safety valves may directly actuate using process fluid pressure or they may be triggered by a pilot mechanism sending process fluid pressure to the actuator only above certain pressures. Pilot-operated valves have the advantage of being widely adjustable, whereas non-piloted valves usually have limited adjustment ranges.

For more information on overpressure protection devices (including PRVs and PSVs) refer to section 32.5 beginning on page 2660.

27.6.4 Electric actuators

Electric motors have long been used to actuate large valves, especially valves operated as on/off ("shutoff") devices. Advances in motor design and motor control circuitry have brought motor-operated valve (MOV) technology to the point where it now competes with legacy actuator technologies such as pneumatic in actuating *throttling* valves as well.

Most electric valve actuators use a *worm gear* set to reduce the high rotational speed of the electric motor to a slow rotation suitable for moving a large valve mechanism. An illustration of a worm gear set appears here:



The *worm screw* looks much like a threaded fastener, with its "threads" properly pitched to engage with the teeth of the *worm wheel* gear. As the worm screw turns, it slowly pushes or pulls the circumference of the worm wheel, resulting in a large gear ratio (i.e. many turns of the screw are required to produce a single turn of the wheel). This slow-turning wheel may then be used to move a sliding-stem valve by means of a threaded shaft (another screw) or used to directly turn a rotary valve (e.g. butterfly, ball, plug).

An electric actuator appears in the next photograph, providing on/off rotary actuation to a ball valve. This particular electric actuator comes with a hand crank for manual operation, in the event that the electric motor (or the power provided to it) fails:



A small lever to the left of the hand crank actuates a *clutch* mechanism to engage or disengage the valve mechanism from the electric motor and the hand wheel. This clutch "selects" either the motor or the hand wheel as the prime mover for the valve, to avoid having the hand wheel spin as the motor turns. Unless this lever is first moved to the "manual" position, turning the hand wheel accomplishes nothing.

The next photograph shows an electric valve actuator coupled to a large butterfly valve. Although nothing visible in this photograph betrays the nature of this actuator's signaling, it happens to be *digital* rather than analog, receiving position commands through a *Profibus* digital network rather than an analog 4-20 mA current signal:



The shape of the actuator's metal casing reveals the final gear drive of this actuator as a *worm gear* set. Note the round shape of the casing where the open/close indicator is located (at the centerline of the butterfly valve stem): this contains the worm wheel. Note immediately to the left of that round casing is a vertically-oriented cylinder shape: this contains the worm screw, which engages with the teeth of the worm wheel. Above the worm screw is a parallel-shaft *spur gear* set which acts to further reduce the speed of the actuator's electric motor: the motor shaft terminates in a small gear, which meshes with a larger gear that turns the worm screw shaft. Above the parallel-shaft gear set is the main casing of the actuator, which actually contains its own internal worm gear set. This multiple-stage gear reduction means that the motor spins very fast in comparison to the butterfly element inside the valve, and that the butterfly is capable of exerting a fantastic amount of torque in comparison to the torque rating of the electric motor.

Electric motors require no external fluid power system to function, unlike pneumatic or hydraulic actuators. All they require is a source of electrical power (often 480 volts AC, three-phase). Some electric valve actuators even have the capability of operating from the power of an electric battery pack, for reliable operation in the event of a power system outage.

Virtually all electric valve actuators require some form of *feedback* to indicate the valve's position.

At minimum, this consists of *limit switches* to indicate when the valve is fully shut and fully open. For throttling services, an electric actuator requires an actual valve position sensor so that it may precisely adjust the valve to any desired state. This sensor may take the form of a potentiometer, or a variable differential transformer (LVDT or RVDT), or pulse encoder.

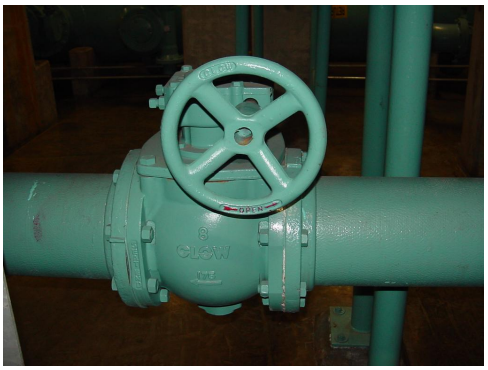
This Rotork brand MOV has a digital display at one end showing its closed status both in text ("Closed Limit") and symbolically (by the vertical line, which is supposed to represent a closed butterfly element):

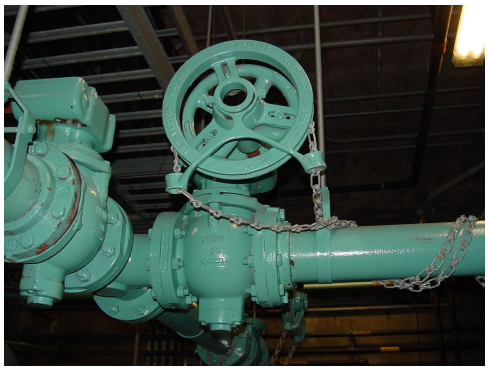


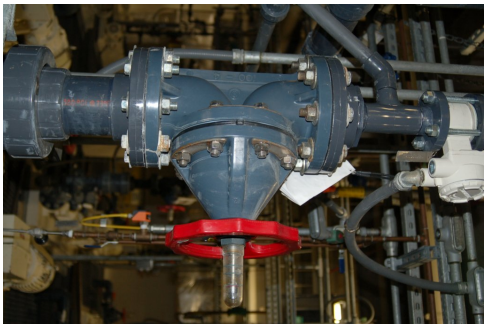
In addition to visual indicators of status, electric valve actuators commonly provide auxiliary electrical contacts signaling fully-open and fully-closed positions, which may be used to energize remote indicator lights or discrete input channels of control systems (e.g. PLC). Throttling-service MOVs also provide analog (and/or digital) signaling of valve stem position for remote indication or feedback to an electronic control system.

27.6.5 Hand (manual) actuators

Valves may also be actuated by hand power alone. The following valves are all "manual" valves, requiring the intervention of a human operator to actuate:









Note the threaded stem of the lower-left valve. This stem rises and falls with the handle's turning, providing visual indication of the valve's status. Such an actuator is called a *rising-stem* design. Also note the chain on the upper-right actuator wheel, allowing operation of the high-mounted valve from a ground-level position.

A hybrid of hand and pneumatic valve actuation is seen on this Valtek brand control valve, where the control valve assembly is actuated by a pneumatic piston actuator, but is also equipped with a manually-operated "handwheel":



A handwheel mechanism may be used to override the pneumatic actuator simply by overpowering it in either direction (i.e. providing a greater force on the valve stem than the piston actuator provides) or it may be left in a neutral position to allow the pneumatic actuator full control over valve stem position. Handwheels may be used to override the control valve's pneumatic actuator to either the full-open or full-closed positions, or it may simply be used to assert a high- or low-limit "stop" to the valve stem to prohibit stem motion beyond a certain position.

Note the "lockout" tab flipped to the horizontal position near the handwheel, located between two of the handwheel's spokes. This simple mechanism permits the handwheel to be locked out to prevent accidental turning.

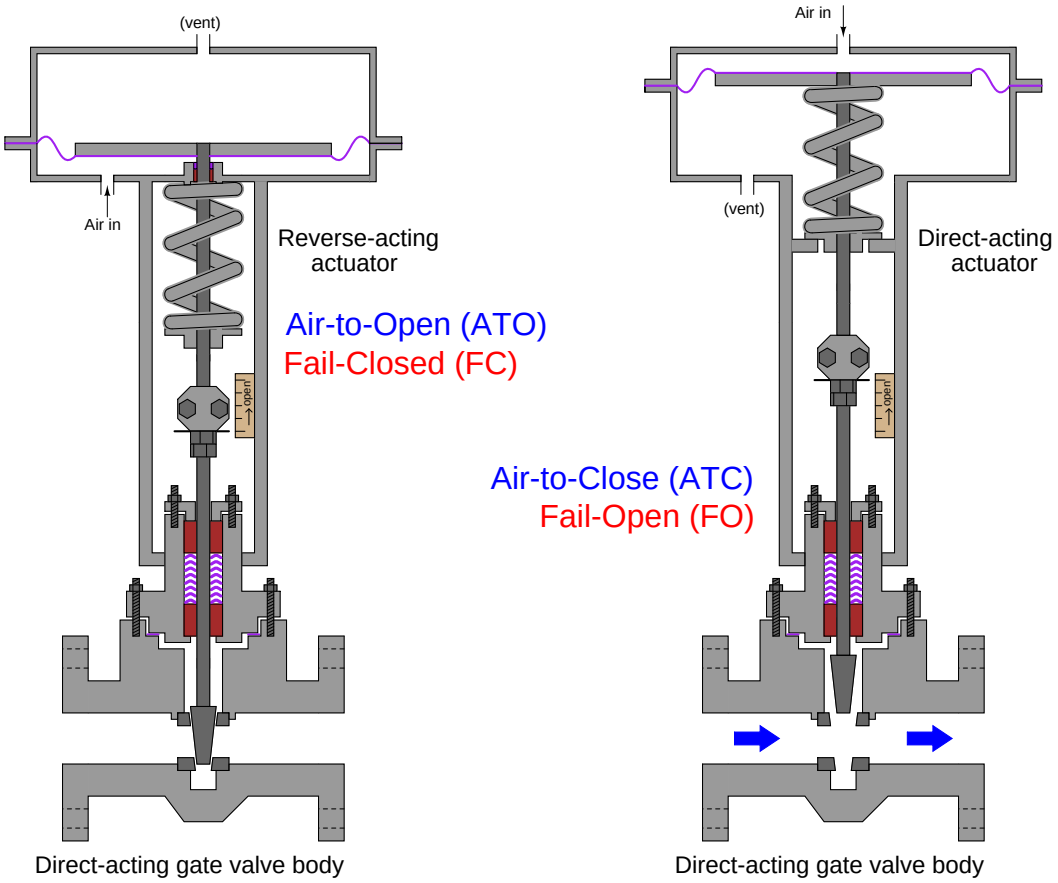
27.7 Valve failure mode

An important design parameter of a control valve is the position it will "fail" to if it loses motive power. For electrically actuated valves, this is typically the last position the valve was in before loss of electric power. For pneumatic and hydraulic actuated valves, the option exists of having a large spring provide a known "fail-safe" position (either open or closed) in the event of fluid pressure (pneumatic air pressure or hydraulic oil pressure) loss.

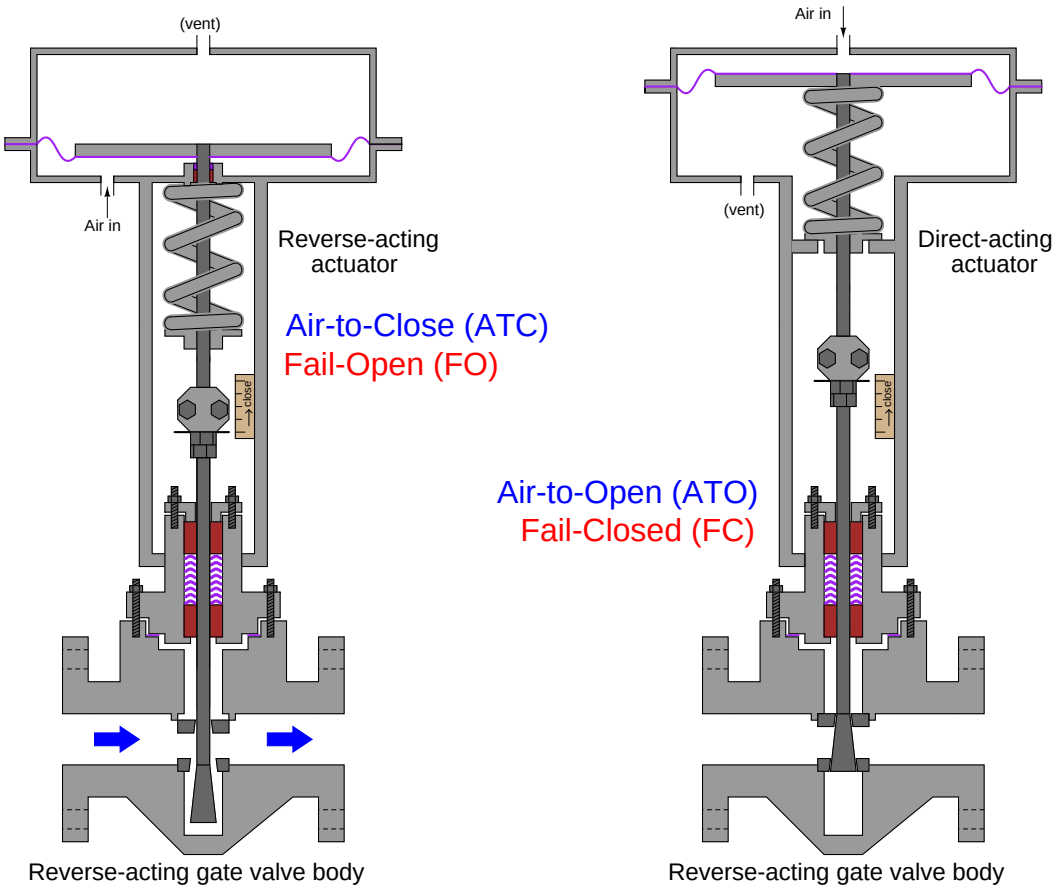
27.7.1 Direct/reverse actions

The fail-safe mode of a pneumatic/spring valve is a function of both the actuator's action and the valve body's action. For sliding-stem valves, a *direct-acting* actuator pushes down on the stem with increasing pressure while a *reverse-acting* actuator pulls up on the stem with increasing pressure. Sliding-stem valve bodies are classified as *direct-acting* if they open up when the stem is lifted, and classified as *reverse-acting* if they shut off (close) when the stem is lifted. Thus, a sliding-stem, pneumatically actuated control valve may be made *air-to-open* or *air-to-close* simply by matching the appropriate actuator and body types.

The most common combinations mix a direct-acting valve body with either a reverse- or direct-acting valve actuator, as shown in this illustration:



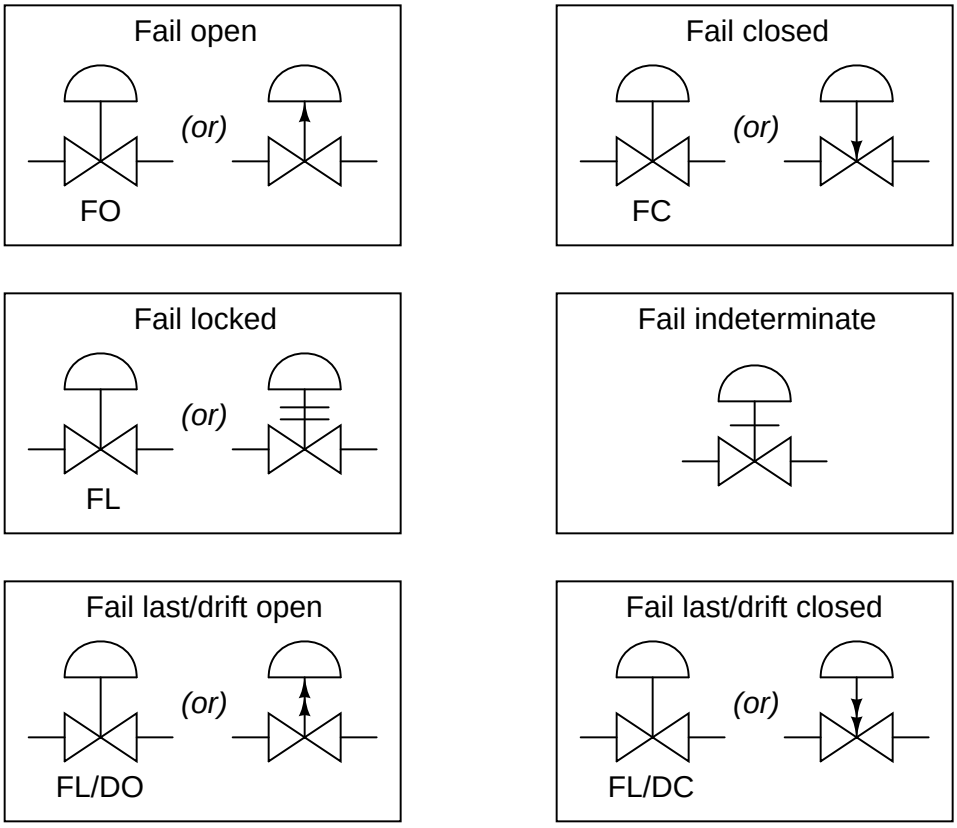
Reverse-acting valve bodies may also be used, with opposite results:



The reverse-acting gate valve body shown in the left-hand illustration is open, with fluid flowing *around* the stem while the wide plug sits well below the seat area. Reverse-acting valve bodies tend to be more complex in construction than direct-acting valve bodies, and so they are less common in control valve applications. An interesting exception to this trend – although not technically a *control* valve but rather a self-actuated device – is the Fisher model 1098EGR pilot-operated pressure regulator which uses a reverse-acting valve body to throttle the flow of gas through it.

27.7.2 Available failure modes

Valve fail mode may be shown in instrument diagrams by either an arrow pointing in the direction of failure (assuming a direct-acting valve body where stem motion toward the body closes and stem motion away from the body opens the valve trim) and/or the abbreviations "FC" (fail closed) and "FO" (fail open). Other failure modes are possible, as indicated by this set of valve symbols:



In order for a pneumatic or hydraulic valve to fail in the *locked* state, an external device must trap fluid pressure in the actuator's diaphragm or piston chamber in the event of supply pressure loss.

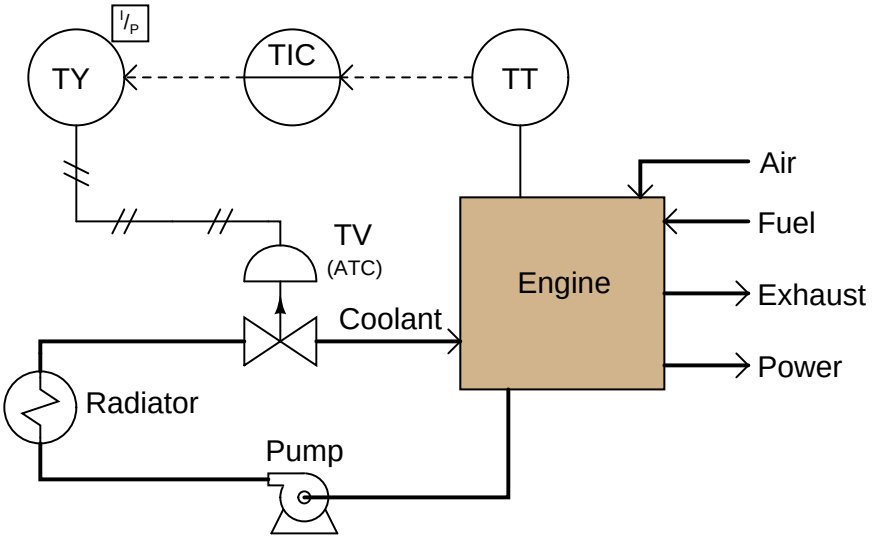
Valves that fail in place and drift in a particular direction are usually actuated by double-acting pneumatic piston actuators. These actuators do not use a spring to provide a definite fail mode, but rather use air pressure both to open and to close the valve. In the event of an air pressure loss, the actuator will neither be able to open nor close the valve, and so it will tend to remain in position. If the valve is of the globe design with unbalanced trim, forces exerted on the valve plug will move it in one direction (causing drift).

27.7.3 Selecting the proper failure mode

It is important to note how the failure mode of a valve is often linked to its control action (air-to-open, air-to-close)[11]. That is, an air-to-open pneumatic control valve will fail closed on loss of air pressure, and vice-versa. This is an important fact because good safety engineering demands that the risk factors of the process determine proper valve failure mode rather than control system convention or habit. People may find it easier to understand the operation of an air-to-open control valve than an air-to-close valve (more signal = more process fluid flow), but this should not be a guiding principle in valve selection. Air-to-open control valves naturally fail closed which means they are appropriate for a particular process control application *only* if that process is safer with a failed-closed valve than with a failed-open valve. If the process is safer with a fail-open valve, then the pneumatically-actuated control valve specified for that application needs to be air-to-close.

In fact, this basic principle forms the basis – or at least it *should* form the basis – of decisions made for all instrument actions in critical control loops: first determine the safest mode of valve failure, then select and/or configure instrument actions in such a way that the most probable modes of signal path failure will result in the control valve consistently moving to that (safest) position.

For example, consider this automated cooling system for a large power-generating engine:



Clearly, it is more hazardous to the engine for the valve to fail closed than it would be for the valve to fail open. If the valve fails closed, the engine will surely overheat from lack of cooling. If it fails open, the engine will merely run cooler than designed, the only negative consequence being decreased efficiency. With this in mind, the only sensible choice for a control valve is one that fails open (air-to-close).

However, our choices in instrument action do not end with the control valve. How should the temperature transmitter, the controller, and the I/P transducer be configured to act? In each case,

[11] Exceptions exist for valves designed to fail in place, where a valve may be engineered to "lock" in position through the action of an external device whether the valve itself is air-to-open or air-to-close.

the answer should be to act in such a way that the valve will default to its fail-safe position (wide open) in the event of the most likely input signal fault.

Stepping "backward" through the control system from the valve to the temperature sensor, the next instrument we encounter is the I/P transducer. Its job, of course, is to convert a 4-20 mA current signal into a corresponding pneumatic pressure that the valve actuator can use. Since we know that the valve's failure mode is based on a loss of actuating air pressure, we want the I/P to be configured in such a way that it outputs minimum pressure in the event of an electrical fault in its 4-20 mA input signal wiring. Whether the wiring fails shorted or fails open, the result will be 0 milliamps at the I/P input terminals. Thus, the configuration of the I/P transducer should be *direct*, such that a 4 to 20 mA input signal produces a 3 to 15 PSI output pressure, respectively (i.e. minimum input current yields minimum output pressure).

The next instrument in the loop is the controller. Here, we want the most likely input signal failure to result in a minimum output signal, so the valve will (once again) default to its "fail safe" position. Consequently, we should configure the controller for *direct* action just like we did with the I/P transducer (i.e. a decreasing PV signal from a broken wire or loose connection in the input circuit results in a decreasing output signal).

Finally, we come to the last instrument in the control loop: the temperature transmitter (TT). As with most instruments, we have the option of configuring it for direct or reverse action. Should we choose direct (i.e. hotter engine = more mA output) or reverse (hotter engine = less mA output)? Here, our choice needs to be made in such a way that the overall effect of the control system is *negative feedback*. In other words, we need to configure the transmitter such that a hotter engine results in increased coolant flow (a wider-open control valve). Since we know the rest of the system has been designed so a minimum signal anywhere tends to drive the valve to its fail-safe mode (wide open), we must choose a *reverse-acting* transmitter, so a hotter engine results in a decreased milliamp signal from the transmitter. If the transmitter has a sensor "burnout" mode switch, we should flip this switch into the low-scale burnout position, so a burned-out sensor will result in 4 mA output (low end of the 4-20 mA scale), thus driving the valve into its safest (wide-open) position.

Such a configuration – with its air-to-close control valve and a reverse-acting transmitter – may seem strange and counter-intuitive, but it is the safest design for this engine cooling system. We arrived at this "odd" configuration of instruments by first choosing the safest control valve failure mode, then choosing instrument actions in such a way that the most likely signal-path failures anywhere in the system would result in the same, consistent valve response. Of course it should go without saying that accurate documentation in the form of a loop diagram with instrument actions clearly shown is an absolutely essential piece of the whole system. If the safety of a control system depends on using any "non-standard" instrument configurations, those configurations had better be documented so those maintaining the system in the future will know what to expect!

Another important detail in this system is to configure the controller such that the operator display for the output signal still registers in an intuitive way: 0% should still represent a shut control valve, while 100% should still represent a wide-open valve. With the valve being air-to-close (signal-to-close from the controller's perspective), this means the controller should be configured for *reverse indication*[12] on the output display, so that an output of 4 mA (wide-open valve) reads 100% open, and an output of 20 mA (fully shut valve) reads 0%. As confusing as this might be for

[12]Note that reverse *indication* is not the same thing as reverse *action* for a loop controller. Reverse indication simply means the output display shows 100% valve position at 4 mA output, and 0% valve position at 20 mA output. Reverse action means the output decreases when the input (process variable) increases.

the technician who must service the controller, it is more important that the operator using this controller every working day sees something that makes intuitive sense. "Minor" details such as this become critically important if an emergency ever occurs, and the operator must make split-second decisions based on the indications they see!

27.8 Actuator bench-set

Valve actuators provide force to move control valve trim. For precise positioning of a control valve, there must be a calibrated relationship between applied force and valve position. Most pneumatic actuators exploit *Hooke's Law* to translate applied air pressure to valve stem position.

$$F = kx$$

Where,

F = Force applied to spring in newtons (metric) or pounds (British)

k = Constant of elasticity, or "spring constant" in newtons per meter (metric) or pounds per foot (British)

x = Displacement of spring in meters (metric) or feet (British)

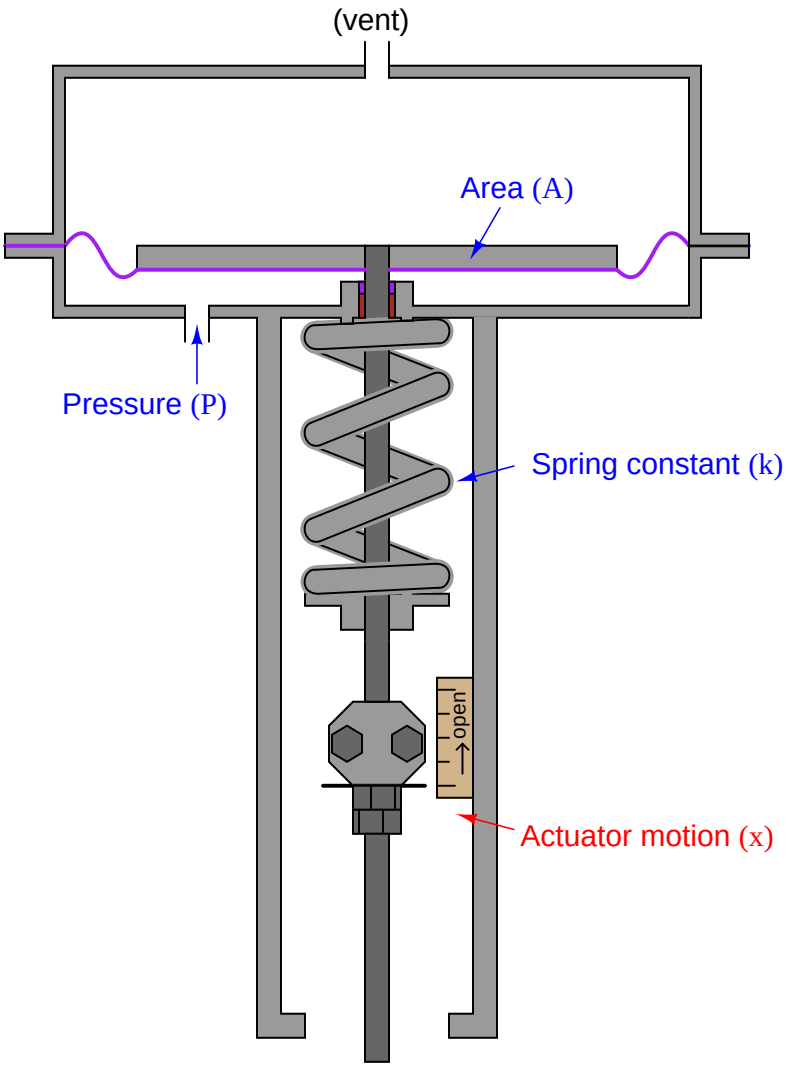
Hooke's Law is a linear function, which means that spring motion will be linearly related to applied force from the actuator element (piston or diaphragm). Since the working area of a piston or diaphragm is constant, the relationship between actuating fluid pressure and force will be a simple proportion ($F = PA$). By algebraic substitution, we may alter Hooke's Law to include pressure and area:

$$F = kx$$

$$PA = kx$$

Solving for spring compression as a function of pressure, area, and spring constant:

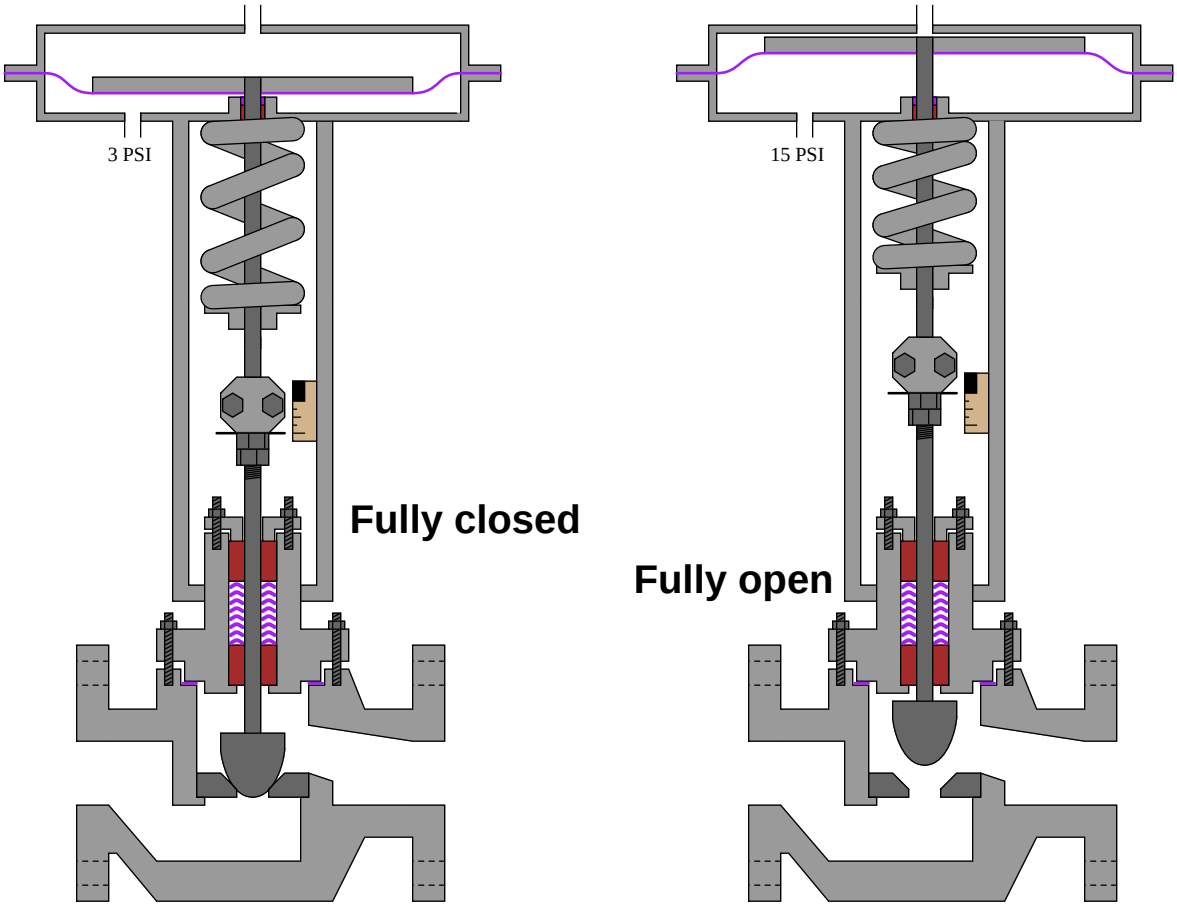
$$x = \frac{PA}{k}$$



When a control valve is assembled from an actuator and a valve body, the two mechanisms must be coupled together in such a way that the valve moves between its fully closed and fully open positions with an expected range of air pressures. A common standard for pneumatic control valve actuators is 3 to 15 PSI[13].

[13] 3 PSI could mean fully closed and 15 PSI fully open, or vice-versa, depending on what form of actuator is coupled to what form of valve body. A direct-acting actuator coupled to a direct-acting valve body will be open at low pressure and closed at high pressure (increasing pressure pushing the valve stem toward the body, closing off the valve trim),

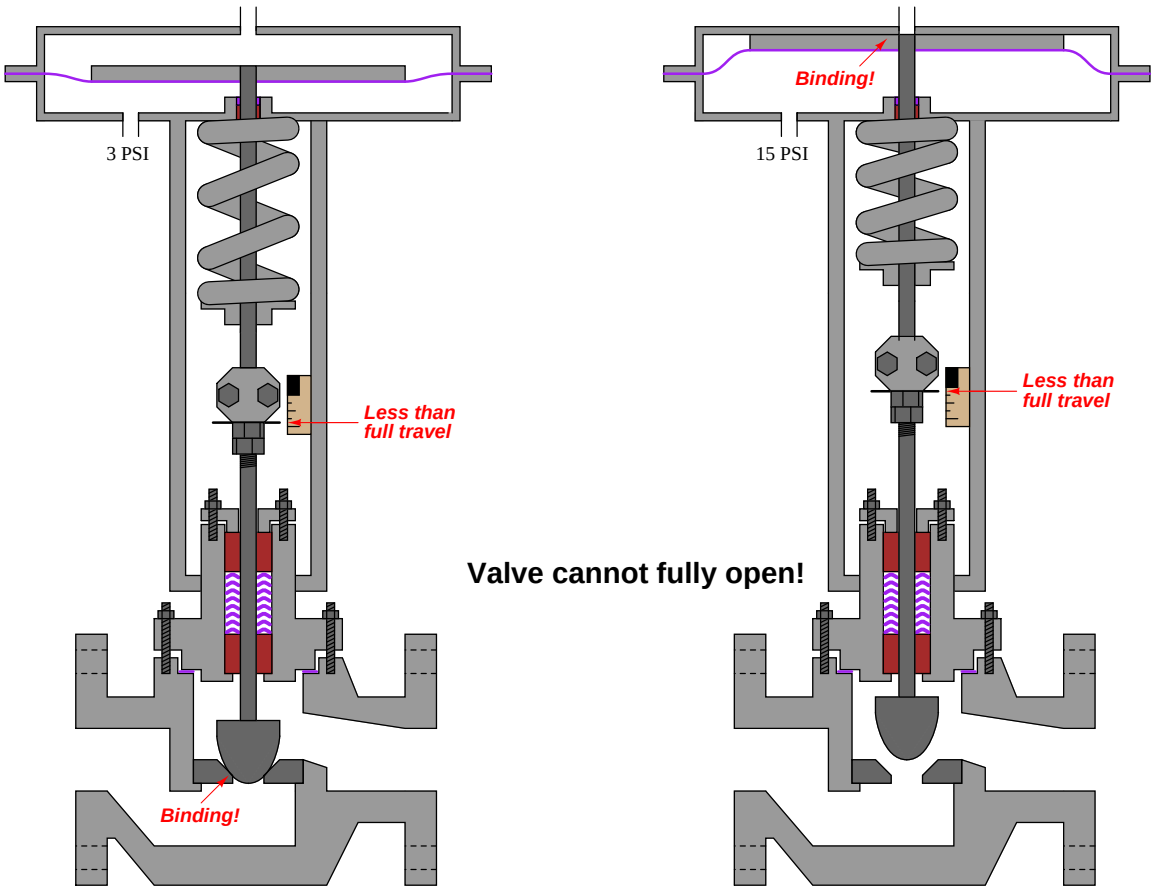
There are really only two mechanical adjustments that need to be made when coupling a pneumatic diaphragm actuator to a sliding-stem valve: the *stem connector* and the *spring adjuster*. The stem connector mechanically joins the sliding stems of both actuator and valve body so they move together as one stem. This connector must be adjusted so neither the actuator nor the valve trim prevents full travel of the valve trim:



Note how the plug rests fully on the seat when the valve is closed, and how the travel indicator indicates fully open at the point where the actuator diaphragm nears its fully upward travel limit. This is how things should be when the stem connector is properly adjusted.

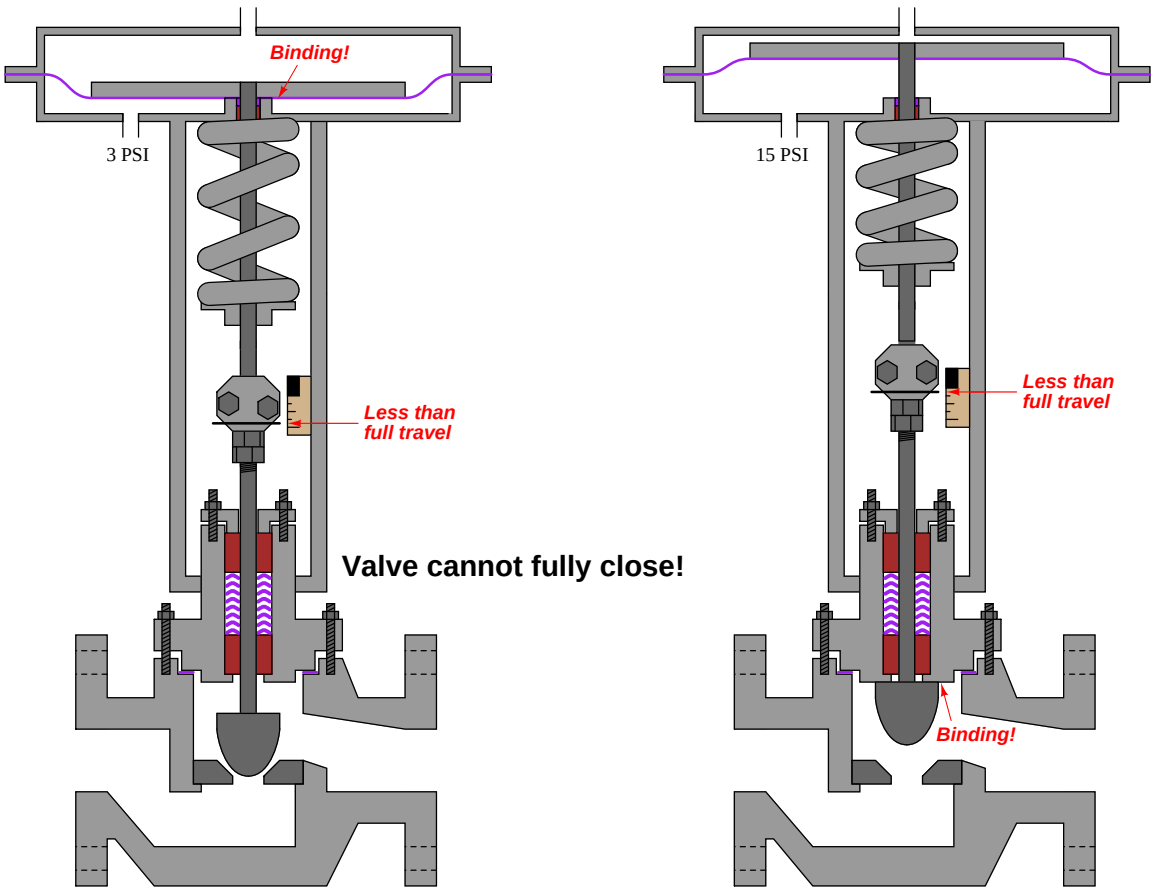
resulting in *air-to-close* action. Reversing either actuator or valve type (e.g. reverse actuator with direct valve or direct actuator with reverse valve) will result in *air-to-open* action.

If the stem connector is set with the actuator and valve stems spaced too far apart (i.e. the total stem length is too long), the actuator diaphragm will bind travel at the upper end and the valve plug will bind travel at the lower end. The result is a valve that cannot ever fully open:



A control valve improperly adjusted in this manner will never achieve full-flow capacity, which may have an adverse impact on control system performance.

If the stem connector is set with the actuator and valve stems too closely coupled (i.e. the total stem length is too short), the actuator diaphragm will bind travel at the lower end and the valve plug will bind travel at the upper end. The result is a valve that cannot ever fully close:



This is a very dangerous condition: a control valve that lacks the ability to fully shut off. The process in which this valve is installed may be placed in jeopardy if the valve lacks the ability to stop the flow of fluid through it!

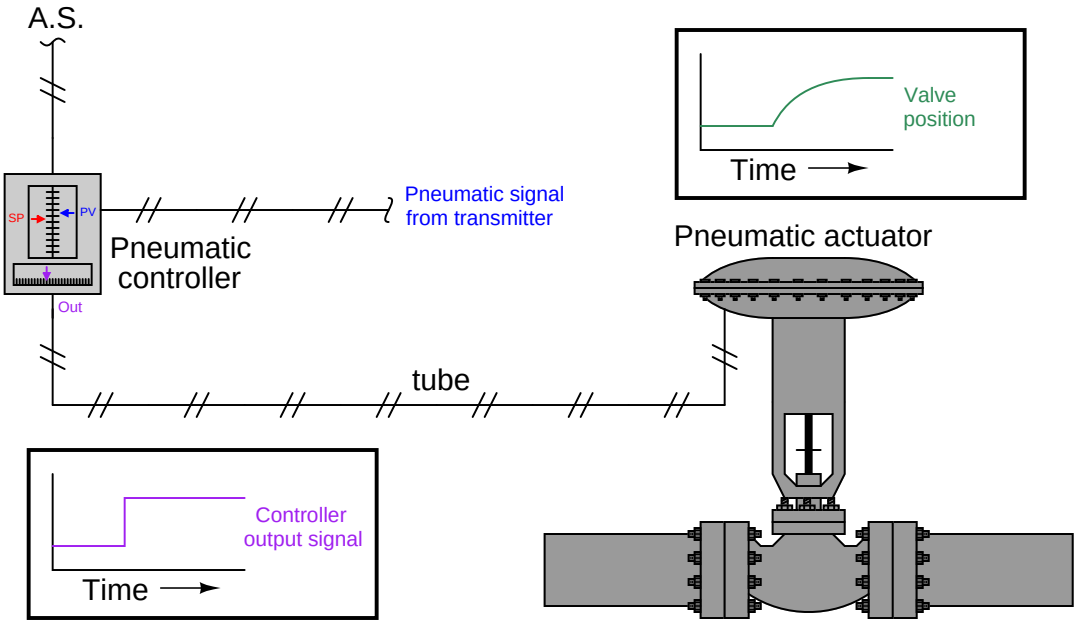
Once the stem length has been properly set by adjusting the stem connector, the spring adjuster must be set for the proper *bench set* pressure. This is the pneumatic signal pressure required to lift the plug off the seat. For an air-to-open control valve with a 3 to 15 PSI signal range, the "bench set" pressure would be 3 PSI.

Bench set is a very important parameter for a control valve because it establishes the seating force (*seat load*) of the plug against the seat when the valve is fully closed. Proper seating pressure is critical for tight shut-off, which carries safety implications in some process services. Consult the manufacturer's instructions when adjusting the bench set pressure for any sliding-stem control valve.

These instructions will typically guide you through both the stem connector and the spring adjuster procedures, to ensure both parameters are correctly set.

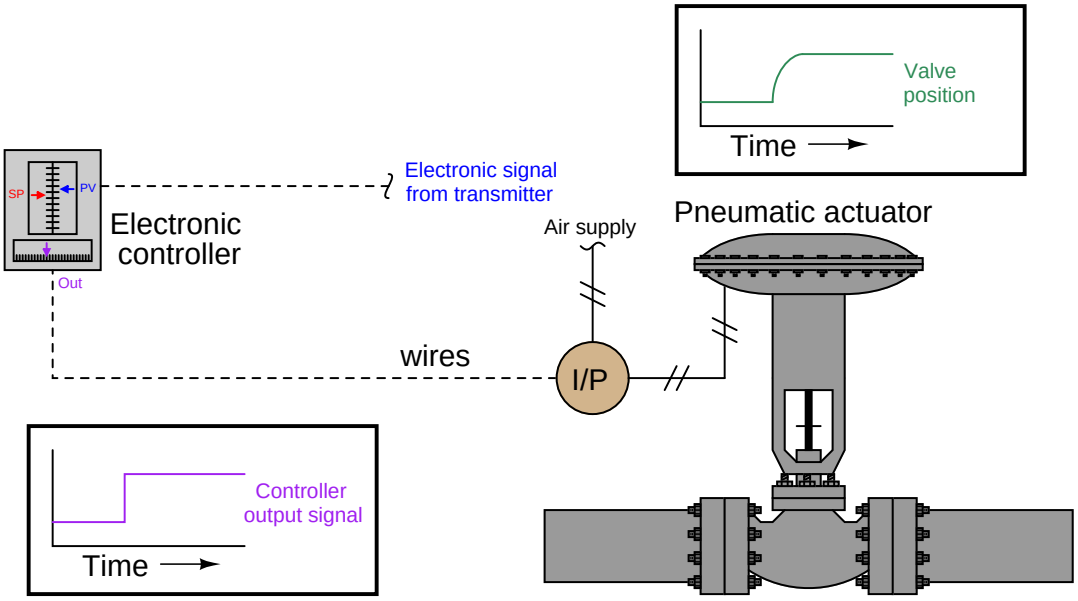
27.9 Pneumatic actuator response

A limitation inherent to pneumatic valve actuators is the amount of air flow required to or from the actuator to cause rapid valve motion. This is an especially acute problem in all-pneumatic control systems, where the distance separating a control valve from the controller may be substantial:



The combined effect of air-flow friction in the tube, flow limitations inherent to the controller mechanism, and volume inside the valve actuator conspire to create a sluggish valve response to sudden changes in controller output signal, not unlike the response of an RC (resistor-capacitor) time-delay circuit where a step-change in voltage input results in an inverse exponential output signal.

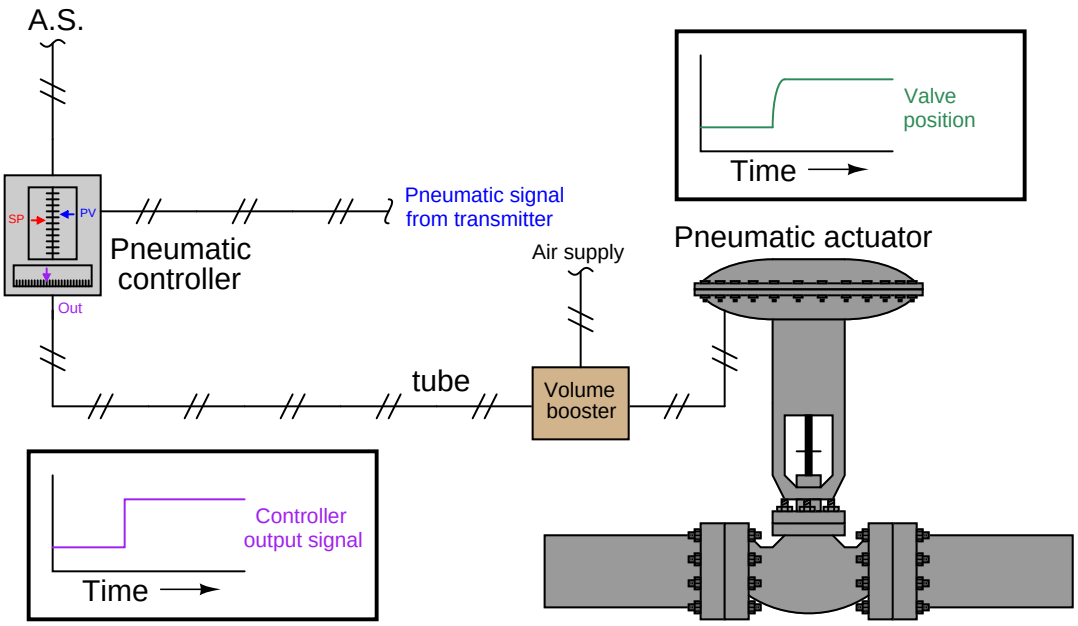
If the pneumatic valve actuator is driven by an I/P transducer instead of directly by a pneumatic controller, the problem is lessened by the ability to locate the I/P close to the actuator, thus greatly minimizing tube friction and thus minimizing the "time constant" (τ) of the control valve's response:



Still, if the pneumatic actuator is particularly large in volume, an I/P transducer may experience trouble supplying the necessary air flow rate to rapidly actuate the control valve. Certainly the problem of time delay is reduced, but not eliminated, by the close-coupled location of the I/P transducer to the actuator.

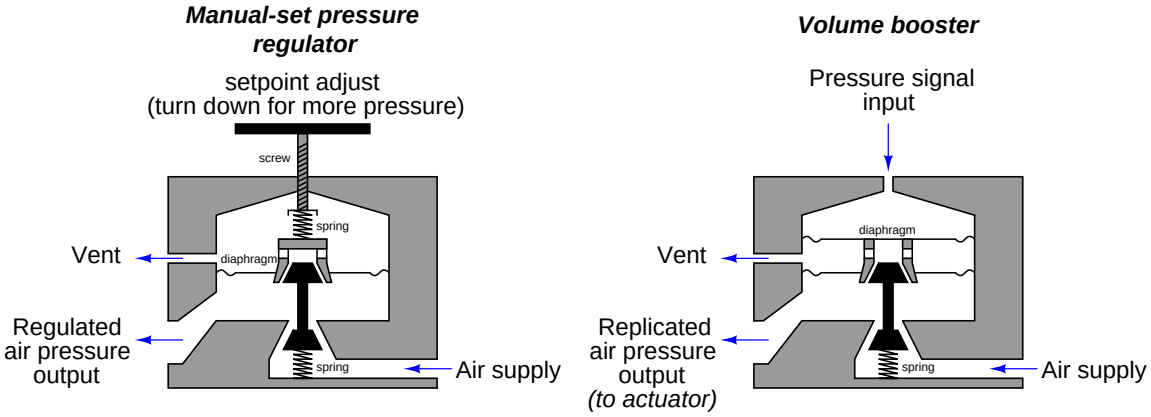
One way to improve valve response in either type of system (full-pneumatic or I/P-driven) is to use a device known as a *volume booster* to source and vent compressed air for the valve actuator. A "volume booster" is a pneumatic device designed to reproduce a pneumatic pressure signal (1:1 ratio), but with far greater output flow capacity. In electrical terms, a volume booster is analogous to a *voltage follower*: a circuit designed to boost current to a load, without boosting or diminishing voltage. A 3 to 15 PSI pneumatic pressure signal applied to the input of a volume booster will result in an identical output signal (3 to 15 PSI), but with greatly enhanced flow capacity.

A pneumatic control system equipped with a volume booster would look something like this:



Of course, enhanced air flow to and from the actuator does not completely eliminate time delays in valve response. So long as the flow rate into or out of an actuator is finite, some time will be required to change pressure inside the actuator and thus change valve position. However, if the actuator volume cannot be reduced for practical reasons of actuating force (larger diaphragm or piston area needed for more force, also resulting in more volume for any given stroke length), then the only variable capable of reducing time lag is increased air flow rate, and a volume booster directly addresses that deficiency.

Internally, a volume booster's construction is not unlike a manually-adjusted pressure regulator[14]:



In either mechanism, an internal diaphragm senses output pressure and acts against a restraining force (either a spring preloaded by a hand adjustment screw or an external pressure signal acting on another diaphragm) to position an air flow throttling/venting mechanism. If the output pressure is less than desired, the diaphragm moves down to open the air sourcing plug and supply additional air volume to the output. If the output pressure is greater than desired, the diaphragm moves up to shut off the sourcing plug and open up the venting port to relieve air pressure to atmosphere.

[14]The volume booster design shown here is loosely based on the Fisher model 2625 volume boosting relay.

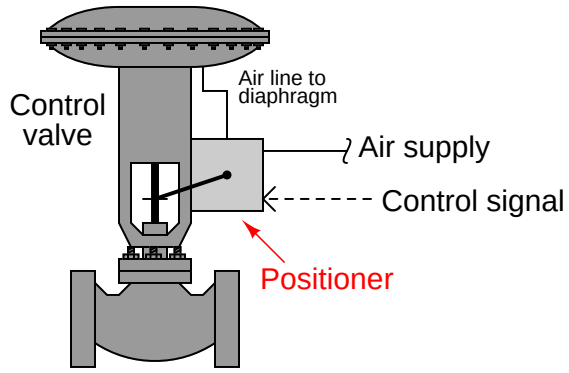
27.10 Valve positioners

The reason why a pneumatic control valve's stem position corresponds linearly to the amount of air pressure applied to the actuator is because mechanical springs tend to follow *Hooke's Law*, where the amount of spring motion (x) is directly proportional to applied force ($F = kx$). A pneumatic actuator applies force as a function of air pressure and piston/diaphragm area ($F = PA$), and the spring in turn compresses or stretches to generate an equal and opposite reaction force. The end-result is that actuator pressure linearly translates into valve stem motion ($x = \frac{PA}{k}$).

This linear and repeatable relationship between pneumatic signal pressure and valve stem position holds true if and only if the actuating diaphragm/piston and spring are the sole forces at work on the valve stem. If any *other* force acts upon this mechanism, the relationship between signal pressure and valve stem position will no longer be ideal.

Unfortunately, there exist many other forces acting on a valve stem besides the actuator force and the spring's reaction force. Friction from the stem packing is one such force, and reaction force at the valve plug caused by differential pressure across the plug's area is another[15]. These forces conspire to re-position the valve stem so stem travel does not precisely correlate to actuating fluid pressure.

A common solution to this dilemma is to add a *positioner* to the control valve assembly. A positioner is a motion-control device designed to actively compare stem position against the control signal, adjusting pressure to the actuator diaphragm or piston until the correct stem position is reached:



Positioners essentially act as control systems within themselves[16]: the valve's stem position is the process variable (PV), the command signal to the positioner is the setpoint (SP), and the positioner's signal to the valve actuator is the manipulated variable (MV) or output. Thus, when a process controller sends a command signal to a valve equipped with a positioner, the positioner receives that command signal and applies as much or as little air pressure to the actuator as needed

[15] One way to minimize dynamic forces on a globe valve plug is to use a *double-ported* plug design, or to use a *balanced* plug on a cage-guided globe valve. A disadvantage to both these valve plug designs, though, is greater difficulty achieving tight shut-off.

[16] The technical term for this type of control system is *cascade*, where one controller's output becomes the setpoint for a different controller. In the case of a valve positioner, the positioner receives a valve stem position setpoint from the main process controller. We could say that the main process controller in this case is the *primary* or *master* controller, while the valve positioner is the *secondary* or *slave* controller.

in order to achieve that desired stem position. Thus, the positioner will "fight" against any other forces acting on the valve stem to achieve crisp and accurate stem positioning according to the command signal. A properly functioning positioner ensures the control valve will be "well-behaved" and obedient to the command signal.

The following photograph shows a Fisher model 3582 pneumatic positioner mounted to a control valve. The positioner is the grey-colored box with three pressure gauges on its right-hand side:



On the left-hand side of this positioner may be seen part of the feedback mechanism: a metal bracket bolted to the valve stem connector, linking to an arm coming out of the positioner's side. Every control valve positioner must be equipped with some means to sense the position of the valve's stem, otherwise the positioner could not compare the valve stem's position against the command signal.

A more modern positioner appears in the next photograph, the Fisher DVC6000 (again, the grey-colored box with pressure gauges on its right-hand side):



Like the older model 3582 positioner, this DVC6000 model uses a feedback linkage on the left-hand side to sense the valve stem's position. The even newer model DVC6200 uses a magnetic *Hall Effect* sensor to sense the position of a magnet bolted to the valve stem. This non-mechanical position feedback design eliminates backlash, wear, interference, and other potential problems associated with mechanical links. Better feedback is essential to better valve positioning.

Control valve positioners are typically constructed in such a way to source and vent high air flow rates, such that the positioner also fulfills the functionality of a volume booster[17]. Thus, a positioner not only ensures more precise valve stem positioning, but also faster stem velocity (and shorter time delays) than if the valve actuator were directly "powered" by an I/P transducer.

[17] This is not to say valve positioners have no need for external volume boosters, just that the actuating air flow capacity of a typical positioner greatly exceeds the air flow capacity of a typical I/P transducer.

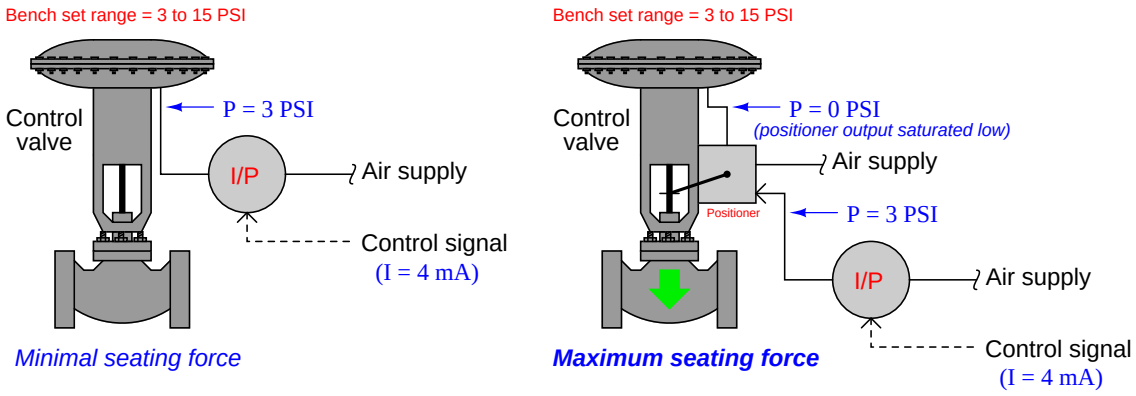
Another advantage of adding a positioner to a pneumatically actuated control valve is superior valve seating (tight shutoff). This benefit is not obvious at first inspection, and so some explanation is in order.

First, one must understand that mere contact between the plug and seat within a sliding-stem valve is not enough to ensure tight shut-off. Rather, the plug must be *forcefully* pressed down onto the seat in order to fully shut off all flow through the valve. Anyone who has ever tightened the handle on a leaking hose bib (garden spigot) intuitively understands this principle: a certain amount of contact force between the plug and the seat is necessary in order to slightly deform and thereby mold those two components to a perfect fluid-tight fit. The technical term for this mechanical requirement is *seat load*.

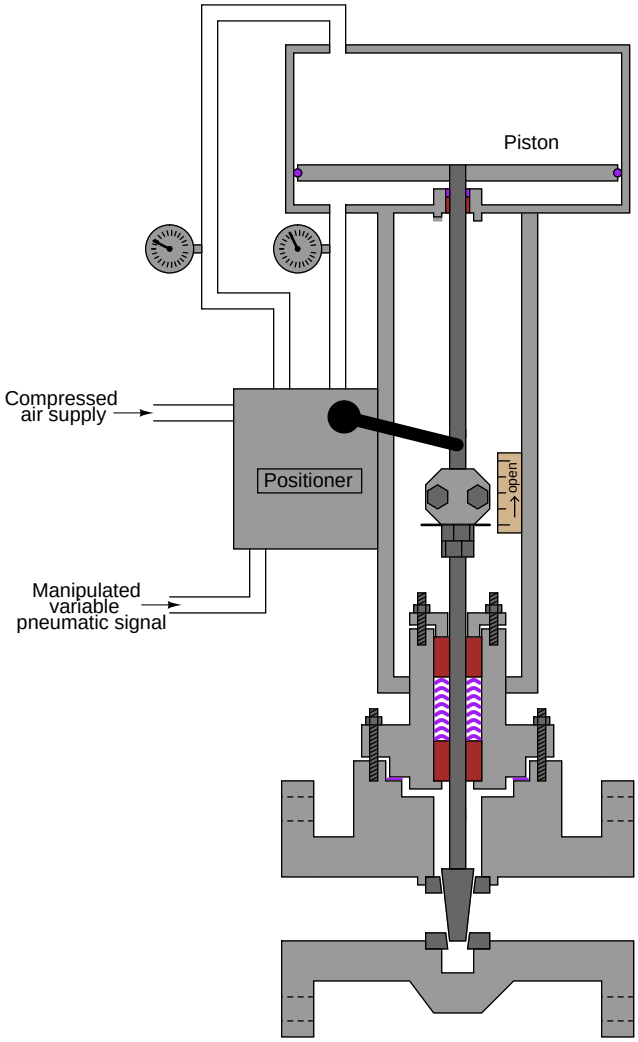
Imagine if you will a diaphragm-actuated, sliding-stem, air-to-open control valve with a bench set range of 3 to 15 PSI. At an applied actuator pressure of 3 PSI, the diaphragm generates just enough force to exactly overcome the actuator spring's pre-load force, but not enough force to actually move the plug off the seat. In other words, at 3 PSI diaphragm pressure, the plug is touching the seat but with little or no force to provide a tight shut-off seal. If this control valve is directly powered by an I/P transducer with a 3-15 PSI calibrated range, it means the valve will be barely shut at a 0% signal value (3 PSI) rather than tightly shut off. In order to fully force the valve plug against the valve seat to achieve a tight seal, *all* air pressure would have to be vented from the diaphragm to ensure no diaphragm force opposing the spring. This is impossible with an I/P having a calibrated range of 3-15 PSI.

Now imagine that exact same valve equipped with a positioner, taking the 3-15 PSI signal from the I/P and using it as a command (setpoint) for valve stem position, applying as much or as little pressure to the diaphragm as necessary to achieve the desired stem position. Proper positioner calibration is such that the valve stem does not begin to lift until the signal has risen slightly *above* 0%, which means at 0% (4 mA) the positioner will be trying to force the valve to a slightly *negative* stem position. In attempting to achieve this impossible demand, the positioner's output will saturate low, applying no pressure whatsoever to the actuating diaphragm, resulting in full spring force applied by the plug against the seat. A comparison of the two scenarios is shown here:

Air-to-open control valve at 0% signal, positioner versus non-positioner



While positioners are beneficial on spring-equipped valve actuators, they are absolutely essential for some other styles of actuators. Consider the following double-acting pneumatic piston actuator which has no spring:

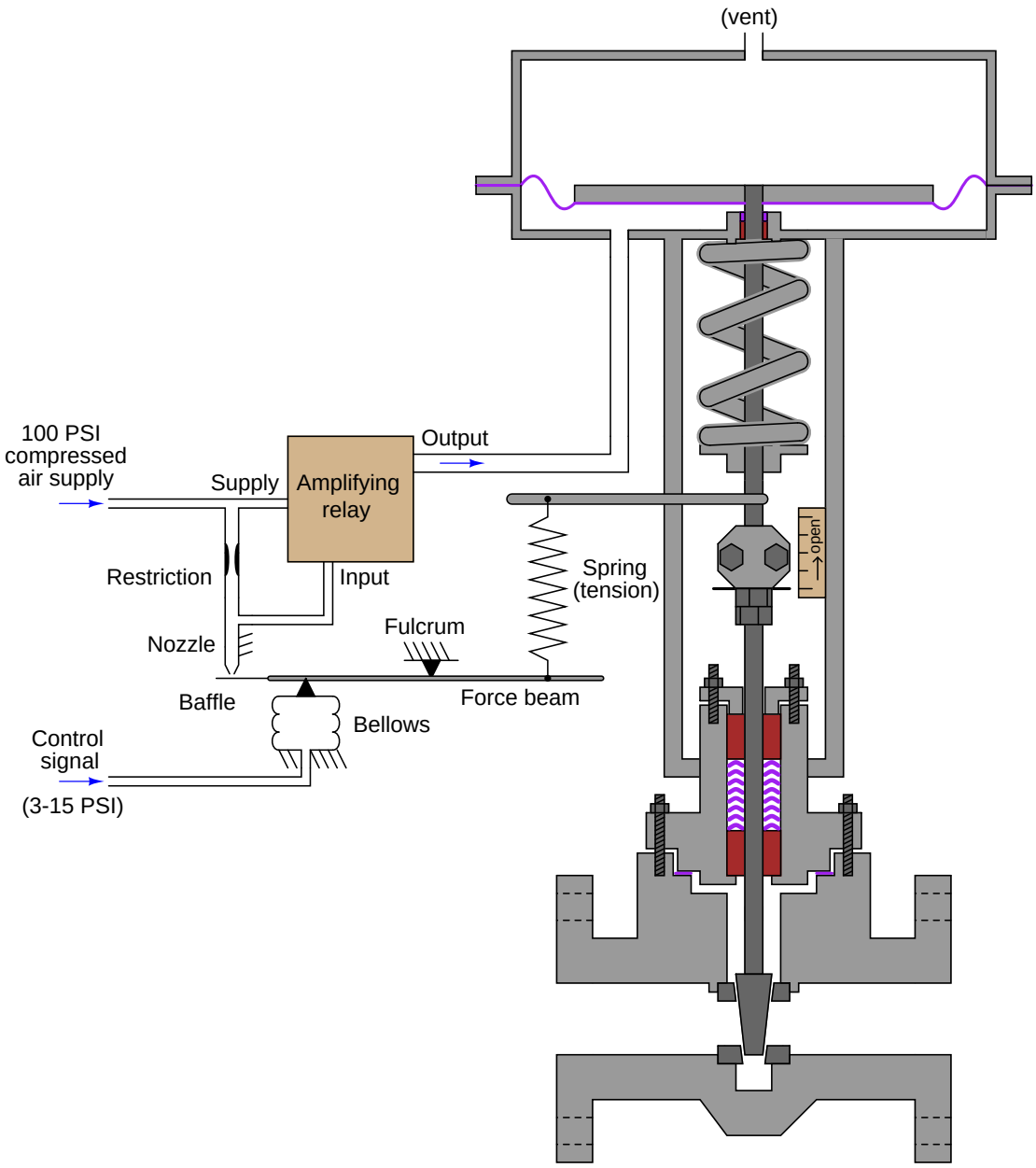


Without a spring providing a restraining force to return the valve to a "fail-safe" position, there exists no Hooke's Law relationship between applied air pressure and stem position. A positioner must alternately apply air pressure to *both* surfaces of the piston to raise and lower the valve stem.

Electric control valve actuators are another class of actuator design absolutely requiring some form of positioner system, because an electric motor is not "aware" of its own shaft position in order that it may precisely move a control valve. Thus, a positioner circuit using a potentiometer or LVDT/RVDT sensor to detect valve stem position and a set of transistor outputs to drive the motor is necessary to make an electric actuator responsive to an analog control signal.

27.10.1 Force-balance pneumatic positioners

A simple force-balance pneumatic valve positioner design appears in the following cutaway illustration:



The control signal for this valve is a 3 to 15 PSI pneumatic signal, coming from either an I/P

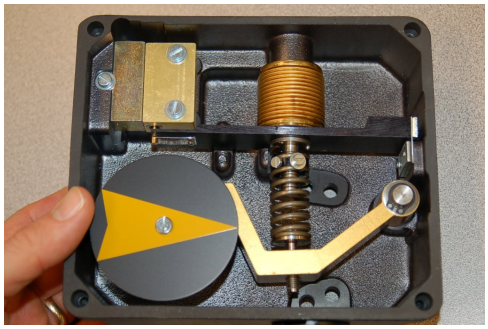
transducer or a pneumatic controller (neither one shown in the illustration). This control signal pressure applies an upward force on the force beam, such that the baffle tries to approach the nozzle. Increasing backpressure in the nozzle causes the pneumatic amplifying relay to output a greater air pressure to the valve actuator, which in turn lifts the valve stem up (opening up the valve). As the valve stem lifts up, the spring connecting the force beam to the valve stem becomes further stretched, applying additional force to the right-hand side of the force beam. When this additional force balances the bellows' force, the system stabilizes at a new equilibrium.

Like all force-balance systems, the force beam motion is constrained by the balancing forces, such that its motion is negligible for all practical purposes. In the end, equilibrium is achieved by one force balancing another, like two teams of people pulling oppositely on a length of rope[18]: so long as the two teams' forces remain equal in magnitude and opposite in direction, the rope will not deviate from its original position.

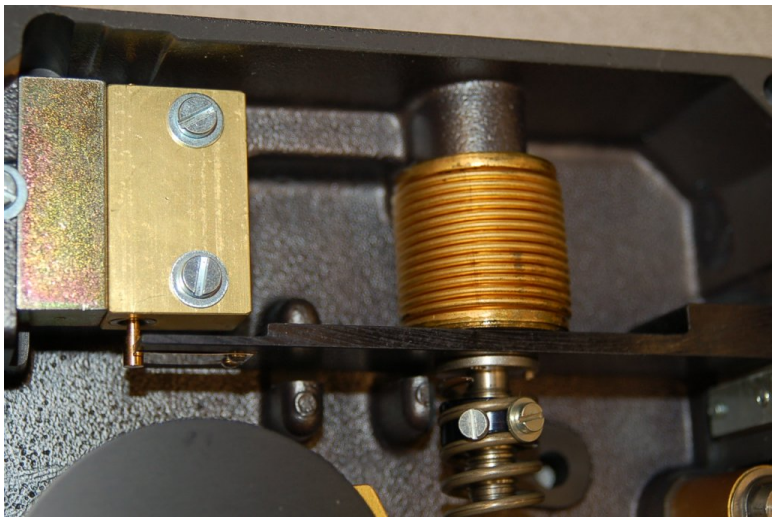
[18] In an earlier chapter of this book, force- and motion-balance pneumatic mechanisms were likened to "tug-of-war" contestants versus ballroom dancers, respectively. Force-balance mechanisms pit force against force to achieve mechanical balance, like two teams competing in a tug-of-war where opposing forces are perfectly balanced and no motion takes place. Motion-balance mechanisms match one motion with another motion to achieve mechanical balance, like two ballroom dancers moving across a dance floor while maintaining a constant distance between each other. All valve positioner mechanisms require motion on the part of the valve stem, and so it is natural to assume all valve positioner mechanisms will be motion-balance because unlike a tug-of-war something is definitely moving. However, if we examine the simple force-balance positioner mechanism closely we will see that *only* the valve stem moves in this mechanism, while nothing else does. To apply the tug-of-war analogy to this application, it is as if one team of contestants pulls on a stiff rope while the other team pulls on an elastic rope, the two ropes tied together in a knot at the center. In order to achieve a perfect balance of forces so the knot won't move to one side or the other, the team holding the elastic rope must *stretch their rope* further in order to balance an increased force from the team holding the stiff rope. The fact that one team is moving does not negate the fact that balance between the two teams is still a matter of force against force. To illustrate this point more vividly, we may ask the question: if the elastic rope is replaced by one that is even more elastic than before, will it advantage one team of contestants over the other? The answer to this question is *no*, as the two teams will still be equally matched if they were equally matched before. The only difference now is that the team holding the elastic rope will have to stretch the rope further than before to apply the same force as before. In a true motion-balance system, a greater motion imparted by one portion of the mechanism must be matched by a greater motion in the other portion of the mechanism.

The following photograph shows a PMV model 1500 force-balance positioner used to position a rotary valve actuator, with the cover on (left) and removed (right):

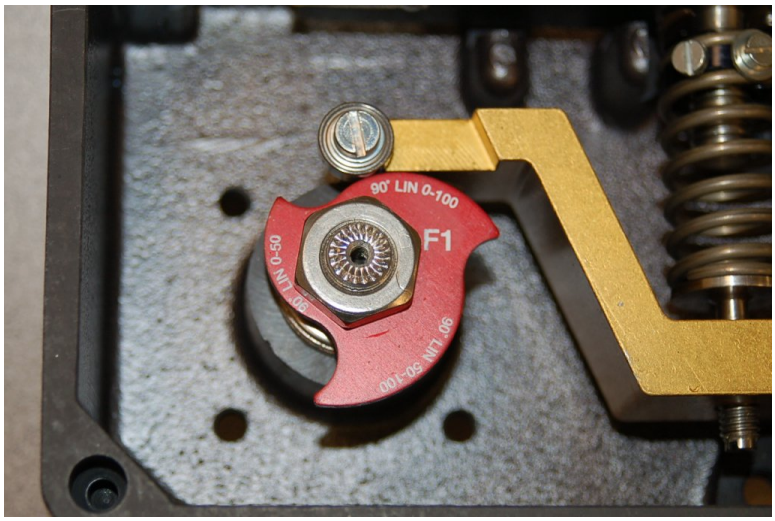




The 3-15 PSI pneumatic control signal enters into the bellows, pushing downward on the horizontal force beam (colored black). A pneumatic pilot valve assembly at the left-hand side of the force beam detects any motion, increasing air pressure to the valve actuating diaphragm if any downward motion is detected and releasing air pressure from the actuator if any upward motion is detected:



As compressed air is admitted to the valve actuator by this pilot valve assembly, the rotary valve will begin to rotate in the open direction. The shaft's rotary motion is converted into a linear motion inside the positioner by means of a *cam*: a disk with an irregular radius designed to produce linear displacement from angular displacement:



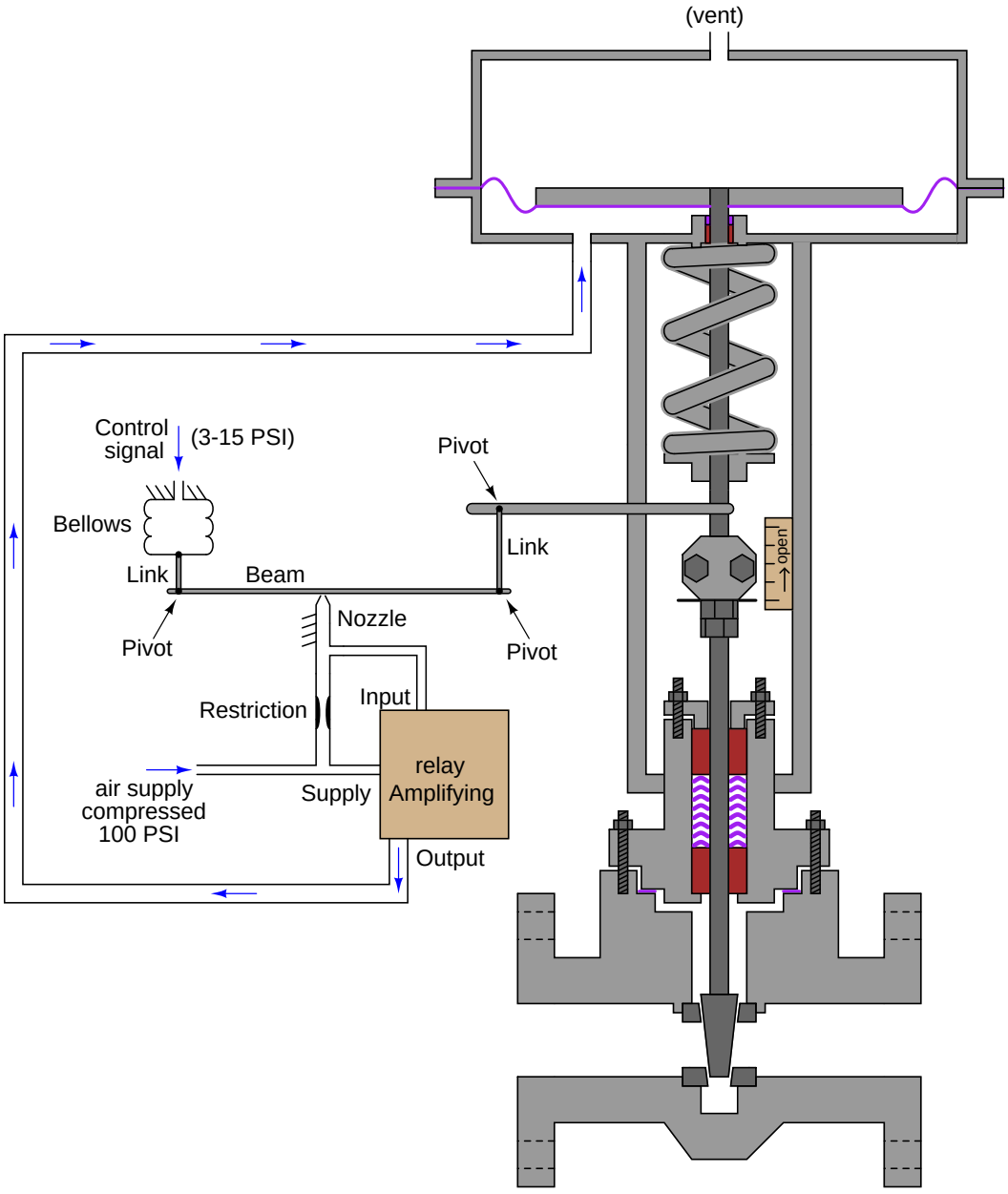
A roller-tipped *follower* at the end of a gold-colored beam rides along the cam's circumference. Cam motion is translated into linear force by the compression of a coil spring directly against the force of the pneumatic bellows on the force beam. When the cam moves far enough to compress the spring enough to balance the additional force generated by the bellows, the force beam return to its equilibrium position (very nearly where it began) and the valve will stop moving.

If you closely examine this last photograph, you will see the positioner's *zero* screw adjustment: the threaded rod extending below the gold-colored beam. This screw adjustment biases the amount of spring compression, making the positioner mechanism "think" the cam is in a different position. For example, turning this threaded rod clockwise (as viewed from the slotted end where a screwdriver would engage) further compresses the spring, pushing up with greater force on the dark-colored bar, achieving the same effect as if the cam had rotated counter-clockwise slightly. This makes the positioner take action to rotate the cam clockwise to compensate, closer toward the 0% valve stem position.

Even though the cam and follower in this positioner mechanism actually do move with valve stem motion, it is still considered a *force-balance* mechanism because the beam connected to the pilot valve does not move appreciably. The pilot valve always comes to rest at its equilibrium position through a balancing of *forces* on the beam.

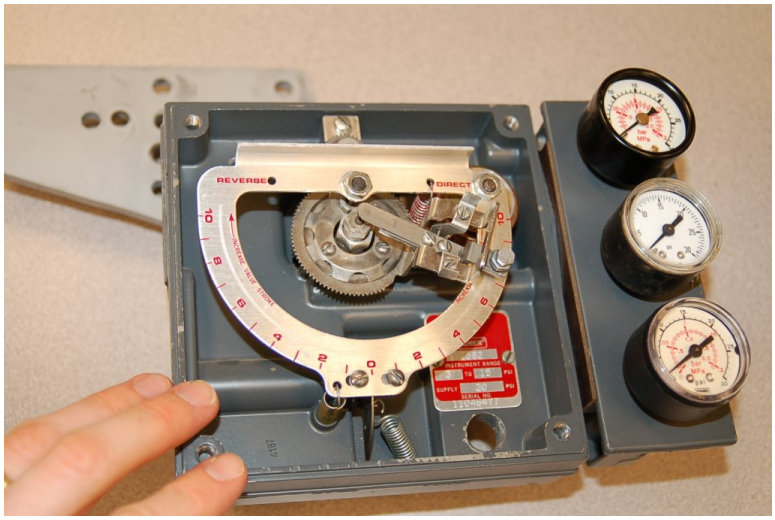
27.10.2 Motion-balance pneumatic positioners

Motion-balance pneumatic valve positioner designs also exist, whereby the motion of the valve stem counteracts motion (not force) from another element. The following cutaway illustration shows how a simple motion-balance positioner would work:

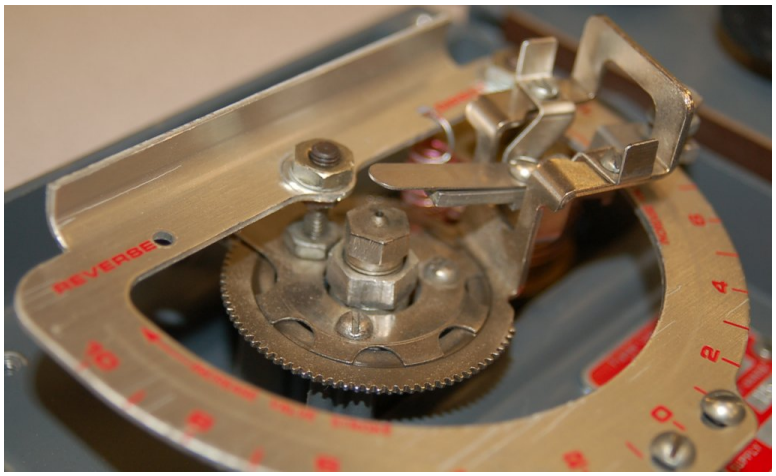


In this mechanism, an increasing signal pressure causes the beam to advance toward the nozzle, generating increased nozzle backpressure which then causes the pneumatic amplifying relay to send more air pressure to the valve actuator. As the valve stem lifts up, the upward motion imparted to the right-hand end of the beam counters the beam's previous advance toward the nozzle. When equilibrium is reached, the beam will be in an angled position with the bellows' motion balanced by valve stem motion.

The following photograph shows a close view of a Fisher model 3582 pneumatic motion-balance positioner's mechanism:



At the heart of this mechanism is a D-shaped metal ring translating bellows motion and valve stem motion into flapper (baffle) motion. As the bellows (located underneath the upper-right corner of the D-ring) expands with increasing pneumatic signal pressure, it rocks the beam along its vertical axis. With the positioner set for direct-acting operation, this rocking motion drives the flapper closer to the nozzle, increasing backpressure and sending more compressed air to the valve actuator:



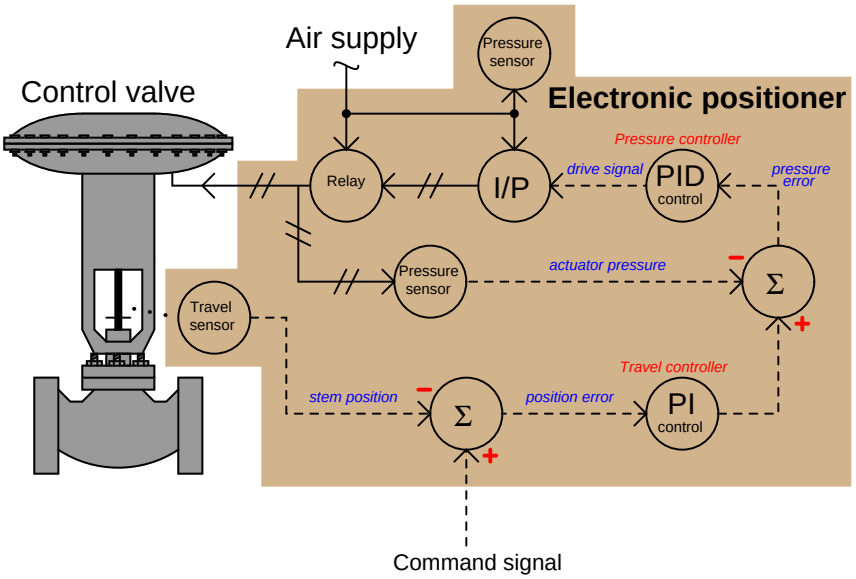
As the valve stem moves, a feedback lever rotates a cam underneath the bottom-most portion of the D-ring. A roller-tipped "follower" riding on that cam translates the valve stem's motion to another rocking motion on the beam, this time along the horizontal axis. Depending on how the cam has been fixed to the feedback shaft, this motion may rock the flapper farther away from the nozzle or closer toward the nozzle. This selection of cam orientation must match the action of the actuator: either direct (air to extend the stem) or reverse (air to retract the stem).

The D-ring mechanism is rather ingenious, as it allows convenient adjustment of span by angling the flapper (baffle) assembly at different points along the ring's circumference. If the flapper assembly is set close to horizontal, it will be maximally sensitive to bellows motion and minimally sensitive to valve stem motion, forcing the valve to move farther to balance small motions of the bellows (long stroke length). Conversely, if the flapper assembly is set close to vertical, it will be maximally sensitive to valve stem motion and minimally sensitive to bellows motion, resulting in little valve stroke (i.e. the bellows needs to expand greatly in order to balance a small amount of stem motion).

27.10.3 Electronic positioners

Recall that the purpose of a *valve positioner* is to ensure the mechanical valve's position matches the command signal at all times. Thus, a valve positioner is actually a closed-loop control system in its own right: applying as much or as little pressure to the actuator in order to achieve the commanded valve stem position at all times. Mechanical valve positioners use levers, cams, and other physical components to achieve this closed-loop control.

Electronic valve positioners, such as the Fisher model DVC6000, use an electronic sensor to detect valve stem position, a microprocessor to compare that sensed stem position against the control signal by mathematical subtraction (error = position − signal), then a pneumatic signal converter and relay(s) to send air pressure to the valve actuator. A simplified diagram of a generic electronic valve positioner is shown here:



As you can see from this diagram, there is a lot going on inside an electronic positioner. We have not just one, but *two* control algorithms working together to maintain proper valve position: one monitoring and controlling pressure applied to the actuator (compensating for changes in air supply pressure that might otherwise affect the valve's position) and the other monitoring and controlling stem position itself, sending a cascaded control signal to the pressure control components.

The command signal (sent from the process loop controller, PLC, or other control system) tells the positioner where the valve stem *should* be positioned. The first controller inside the positioner (PI) calculates how much air pressure at the actuator should be needed to achieve the requested stem position. The next controller (PID) drives the I/P (current-to-pressure) converter as much as necessary to achieve that pressure. If anything causes the valve stem to not be at the commanded position, the two controllers inside the positioner work together to force the valve to its proper position.

Not only do electronic valve positioners achieve superior position control when compared to mechanical valve positioners, but their array of sensors and digital communication ability provides a new level of diagnostic data both to maintenance personnel and the supervising control system (if programmed to monitor and act on this data). Examples of diagnostic data provided by electronic positioners include:

- Supply air pressure
- Actuator air pressure
- Ambient temperature
- Position and pressure errors
- Total valve stem travel (like an odometer in an automobile)

Additionally, the microprocessor embedded within an electronic valve positioner is capable of performing self-tests, self-calibrations, and other routine procedures traditionally performed by instrument technicians on mechanical valve positioners. Having access to such measurements as total valve stem travel even allows an electronic positioner to predictively calculate packing wear-out time, automatically flagging a maintenance alarm notifying operators and/or instrument technicians when the valve's stem packing will need to be replaced!

A useful capability of some "smart" valve positioners – since they monitor actuator air pressure in addition to stem position – is the ability to maintain a respectable degree of valve control in the event of a stem position sensor failure. If the microprocessor detects a failed (off-scale) position feedback signal, it may be programmed to continue operating the valve based on pressure alone: adjusting the applied air pressure to the valve actuator according to the pressure/position function it has recorded in the past. While not strictly functioning as a *positioner* any more since it cannot sense valve stem position, it may still fulfill its role as a volume booster (compared to the flow capacity of a typical I/P) and give reasonable control over the valve where any other (non-smart) valve positioner would actually make matters worse in the event of losing its stem position feedback. With any purely mechanical positioner, the control valve will typically "saturate" either fully open or fully closed if the stem position feedback linkage falls off. Not so with the best "smart" positioners!

Perhaps the most significant diagnostic data provided by an electronic positioner is the comparison of actuator pressure versus stem position, usually expressed in the form of a graph. Actuator pressure is a direct reflection of force applied by the actuator to the valve stem, since the relationship between force and pressure for either a piston or a diaphragm is simply $F = PA$, where area (A) is constant. Thus, a comparison of actuator air pressure versus stem position is really an expression of force-versus-position for the valve. This so-called *valve signature* is incredibly useful in identifying and correcting such problems as excessive packing friction, valve trim interference, and plug/seat fit problems.

A screenshot showing a "valve signature" (taken from an Emerson software product called *ValveLink*, as part of their *AMS* suite) appears here, showing the behavior of an air-to-open Fisher E-body globe valve:



Two plots of actuator pressure versus stem position are shown in this graph, one red and one blue. The red graph shows the valve's response in the *opening* direction where additional pressure is required to overcome packing friction as the valve moves open (up). The blue graph shows the

valve as it *closes*, less pressure applied to the diaphragm now to allow the spring's compression to overcome[19] packing friction as the valve moves closed (down) to its resting state. The sharp turns at each end of this graph show where the valve stem reaches its end positions and cannot move farther despite further changes in actuator pressure.

Each plot is roughly linear in accordance with Hooke's Law describing the behavior of the valve spring, where the force applied to a spring is directly proportional to the displacement (compression) of that spring: $F = kx$. Any departure from a single linear plot indicates some *other* force(s) besides spring compression and pneumatic force acting on the valve stem. This is why we see two plots vertically offset from each other: packing friction is another force acting on the valve stem in addition to spring compression and the force exerted by air pressure on the actuator diaphragm. The relatively small magnitude of this offset as well as its consistency indicates that packing friction in this valve is "healthy." The more packing friction this valve experiences, the more vertically-offset the two plots will be.

The sharp down-turn at the left-hand end of the graph where the valve plug contacts the seat is called the *seating profile*. Located at the end of the plot where the valve closes off, the *seating profile* holds much useful information about the physical condition of the plug and seat. As these trim parts wear in a control valve, the shape of the seating profile changes accordingly. Irregular seating profiles may diagnose seat erosion, galling, or a number of other maladies.

[19] Recall from basic physics that friction force always *opposes* the direction of motion. Thus, when the valve is opening, friction works against the actuator's air pressure (assuming an air-to-open valve), requiring additional air pressure to maintain motion. When the valve is closing, though, packing friction works in the same direction as the actuator's air pressure "helping" the valve stay more open than it should. This is why the positioner must maintain *less* actuator air pressure for any given position while moving closed than while the valve moves open. The difference in air pressure moving open versus moving closed at any given stem position is proportional to *twice* the dynamic packing friction. Stated mathematically, $F_{packing} = \frac{1}{2}(P_{opening} - P_{closing})A$.

By zooming in on the lower-left end of the valve signature graph, the seating profile may be examined in fine detail. A seating profile taken of a Fisher E-body globe valve in pristine condition appears here:



If the maintenance staff of a particular facility are diligent enough to record the valve signatures of its control valves after assembly or rebuild, the "original" signature of any particular control valve may be compared against the signature of the same control valve taken at any later date, allowing qualitative determinations of wear without having to disassemble the valves for inspection.

Interestingly, this relationship of actuator pressure (force) to stem position is also available in the electronic positioners used with some modern electrically-actuated valves. In the case of an electric actuator, force applied to the valve stem directly relates to *motor current*, which is easily measured and interpreted by the electronic positioner. Thus, even with a different actuator technology, the same kind of diagnostic data may be presented in graphical form for the purpose of more easily diagnosing valve problems. These diagnostics apply even to open/close motor-operated valves not

used for throttling service[20], and are especially useful on gate, plug, and ball-type shut-off valves where seat engagement is substantial for tight shut-off.

27.11 Split-ranging

There are many process control applications in industry where it is desirable to have multiple control valves respond to the output of a common controller. Control valves configured to follow the command of the same controller are said to be *split-ranged*, or *sequenced*.

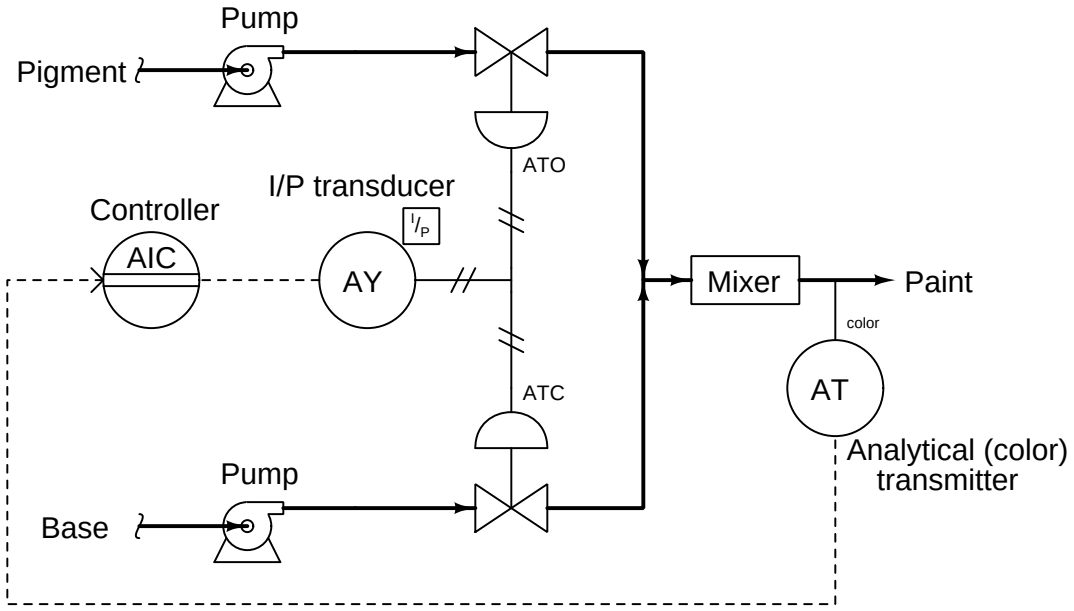
Split-ranged control valves may take different forms of sequencing. A few different modes of control valve sequencing are commonly seen in industry: *complementary*, *exclusive*, and *progressive*[21].

[20] Prior to the advent of motor-actuated valves, practically all shutoff valves in industrial facilities were manually operated. While this is an inconvenience for operations personnel, it did carry one advantage: the human operators tasked with closing these valves by hand could feel how each valve seated. The amount of effort and the onset of closing torque sensed while turning the valve handle shut gave operators tactile feedback on the condition of each valve seat. Motor-powered valve actuators eliminated the need for this routine manual labor, but also eliminated the routine collection of this valuable diagnostic information. Modern electric valve actuators now provide the best of both worlds: convenient and fast valve operation with accurate self-diagnostic assessment of valve seating.

[21] I have searched in vain for standardized names to categorize different forms of control valve sequencing. The names "complementary," "exclusive," and "progressive" are my own invention. If I have missed someone else's categorization of split-ranging in my research, I sincerely apologize.

27.11.1 Complementary valve sequencing

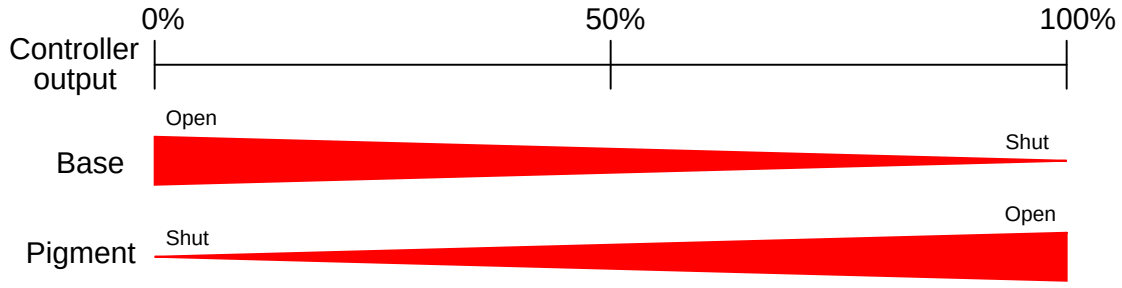
The first type of control valve sequencing is a mode where two valves serve to proportion a mixture of two fluid streams, such as this example where base and pigment liquids are mixed together to form colored paint:



Both base and pigment valves operate from the same 3 to 15 PSI pneumatic signal output by the I/P transducer (AY), but one of the valves is Air-To-Open while the other is Air-To-Close. The following table shows the relationship between valve opening for each control valve and the controller's output:

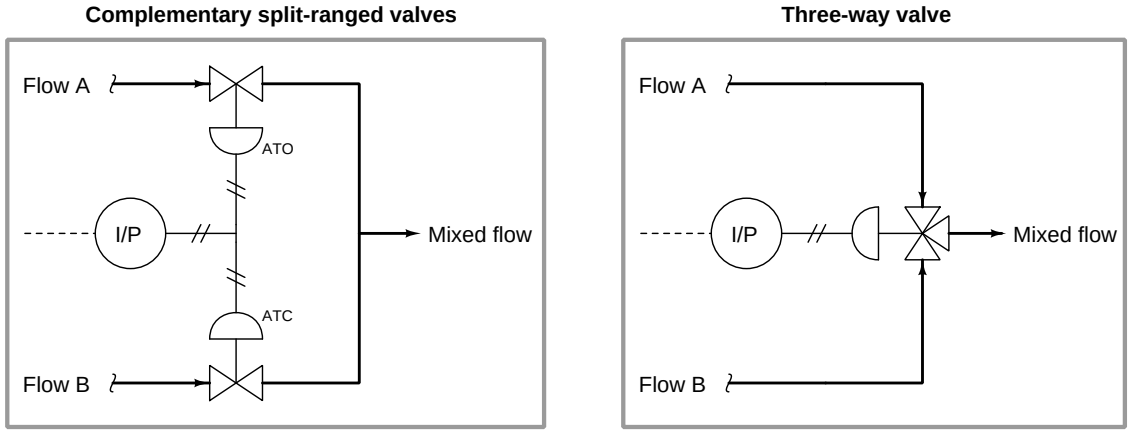
Controller output (%)	**I/P output** (PSI)	**Pigment valve** (stem position)	**Base valve** (stem position)
0 %	3 PSI	fully shut	fully open
25 %	6 PSI	25% open	75% open
50 %	9 PSI	half-open	half-open
75 %	12 PSI	75% open	25% open
100 %	15 PSI	fully open	fully shut

An alternative expression for this split-range valve behavior is a graph showing each valve opening as a colored stripe of varying width (wider representing further open). For this particular mode of split-ranging, the graph would look like this:



With this form of split-ranging, there is never a condition in the controller's output range where both valves are fully open or fully shut. Rather, each valve *complements* the other's position[22].

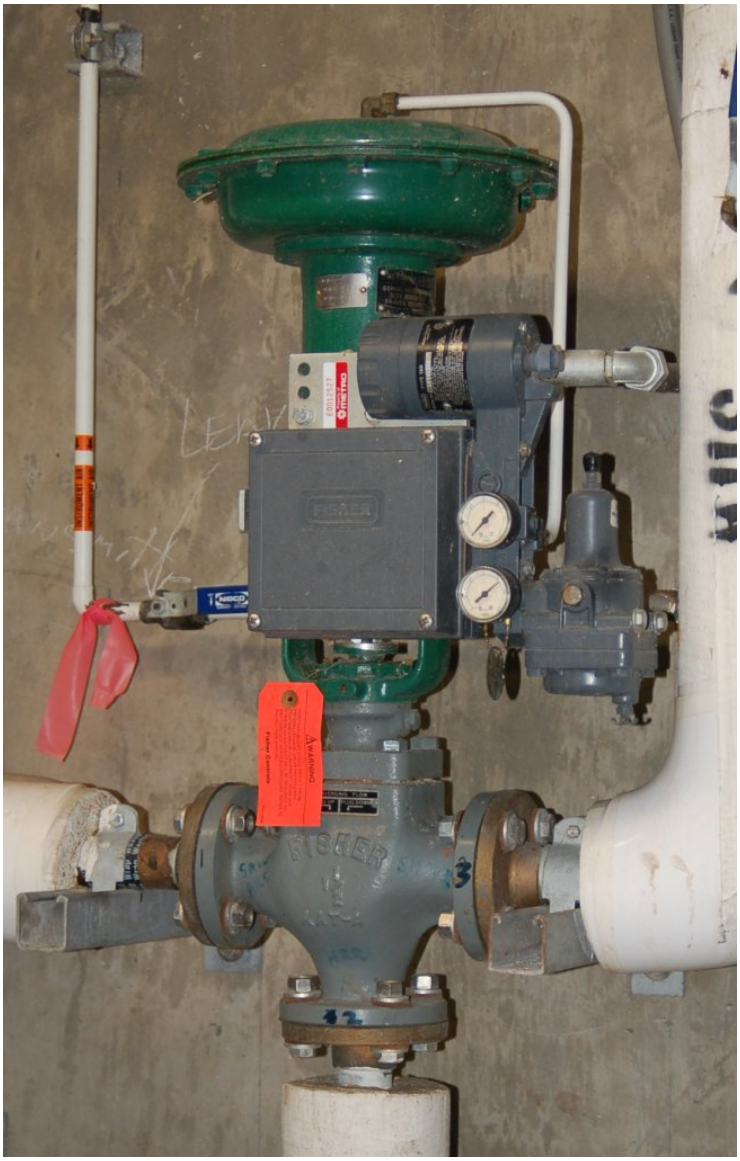
An alternative to complementary valve sequencing in a process where two fluid streams mix (or diverge) is to use one *three-way* valve[23] rather than a pair of two-way valves:



[22] In mathematics, a "complement" is a value whose sum with another quantity always results in a fixed total. Complementary angles, for instance, always add to 90° (a right angle).

[23] Also known as a *mixing valve* or a *diverting valve*, depending on how it is applied to process service.

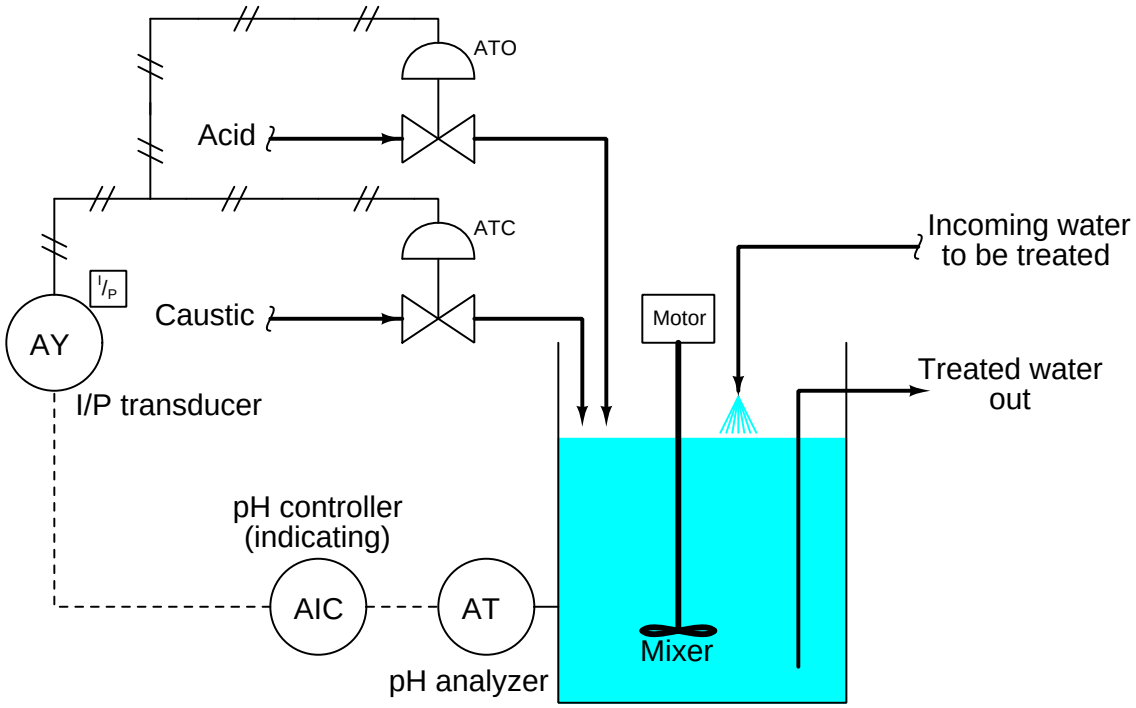
A photograph of a three-way globe valve mixing hot and cold water to control temperature is shown here:



27.11.2 Exclusive valve sequencing

Other applications for split-ranged control valves call for a form of valve sequencing where both valves are fully closed at a 50% controller output signal, with one valve opening fully as the controller output drives toward 100% and the other valve opening fully as the controller output goes to 0%. The nature of this valve sequencing is to have an "either-or" throttled path for process fluid. That is, *either* process fluid flows through one valve *or* through the other, but never through both at the same time.

A practical example of this form of split-ranging is reagent feed for a pH neutralization process, where the pH value of process liquid is brought closer to neutral by the addition of either acid or caustic reagent liquids:

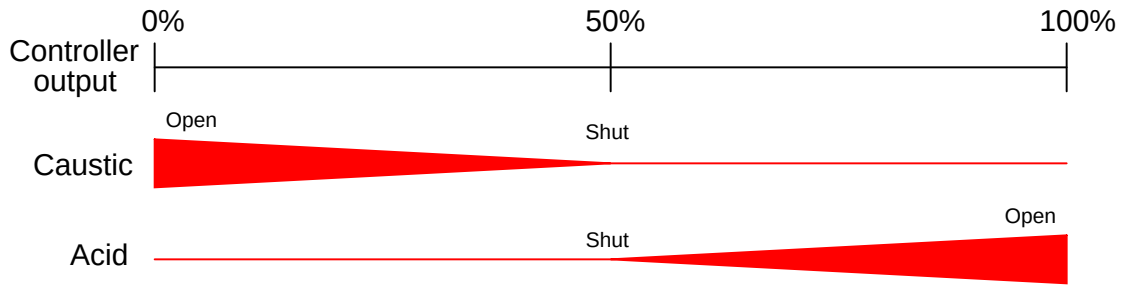


Here, a pH analyzer monitors the pH value of the liquid solution and a single pH controller commands two reagent valves to open when needed. If the process pH begins to increase, the controller output signal increases as well (direct action) to open up the acid valve. The addition of acid to the mixture will have the effect of lowering the mixture's pH value. Conversely, if the process pH begins to decrease, the controller output signal will decrease as well, closing the acid valve and then opening the caustic valve. The addition of caustic to the mixture will have the effect of raising the mixture's pH value.

Both reagent control valves operate from the same 3 to 15 PSI pneumatic signal output by the I/P transducer (AY), but the two valves' calibrated ranges are not the same. The Air-To-Open acid valve has an operating range of 9 to 15 PSI, while the Air-To-Close caustic valve has an operating range of 9 to 3 PSI. The following table shows the relationship between valve opening for each control valve and the controller's output:

Controller output (%)	**I/P output** (PSI)	**Acid valve** (stem position)	**Caustic valve** (stem position)
0 %	3 PSI	fully shut	fully open
25 %	6 PSI	fully shut	half-open
50 %	9 PSI	fully shut	fully shut
75 %	12 PSI	half-open	fully shut
100 %	15 PSI	fully open	fully shut

Again, we may express the two valves' exclusive relationship in the form of a graph, with colored stripes representing valve opening:



Exclusive-sequenced control valves are used in applications where it would be undesirable to have both valves open simultaneously. In the example given of a pH neutralization process, the goal here is for the controller to add either acid reagent or caustic reagent to "push" the pH value either direction as needed. However, simultaneously adding both acid and caustic to the process would be wasteful, as one reagent would simply neutralize the other with no benefit to the process liquid itself.

27.11.3 Progressive valve sequencing

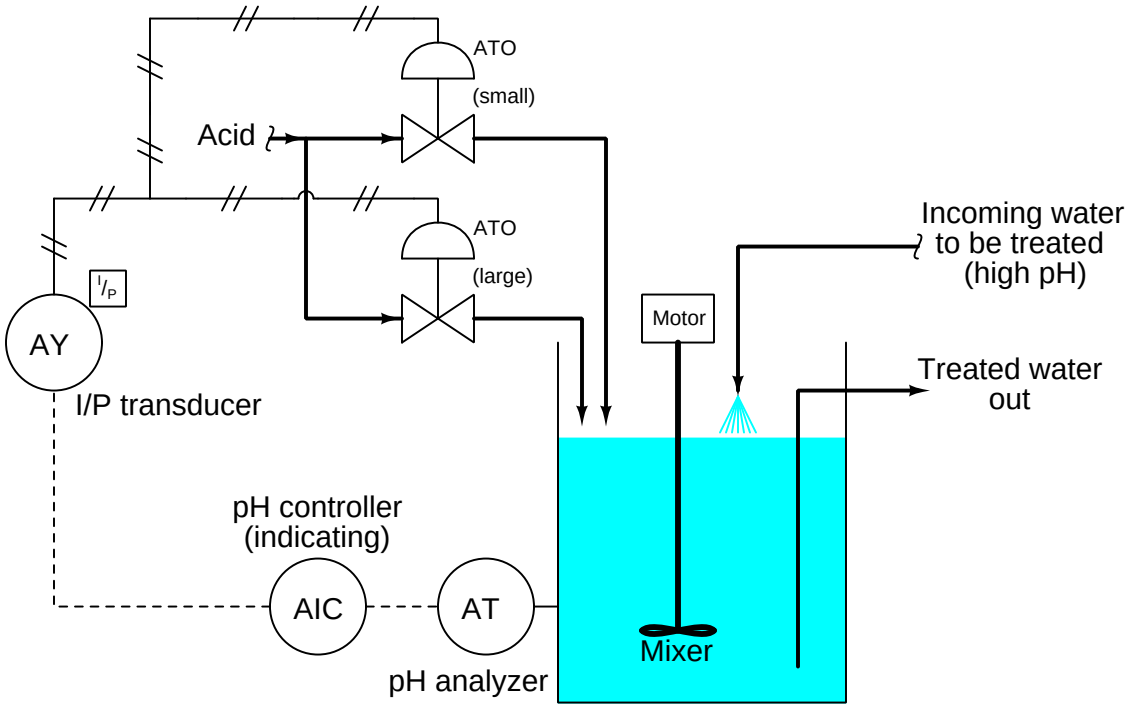
A third form of control valve sequencing is used to expand the operating range of flow control for some fluid beyond that which a single control valve could muster. Once again pH control provides a suitable example to illustrate an application of this form of sequencing.

pH is an especially challenging application of process control because the dynamic range of the process is enormous. Each unit of pH value change represents a *ten-fold* change in hydrogen ion concentration within the process liquid. This means the difference in ion concentration between a process liquid having a value of 10 pH and a process liquid having a value of 7 pH (a pH difference of 3) is a factor of *one thousand* (10^3)! Consequently, the flow rate of reagent necessary to neutralize a process liquid stream may vary widely. It is quite possible that a control valve sized to throttle minimum flow will simply be too small to meet the demands of high flow when needed. Yet, a control valve sized large enough to meet the maximum flow rate may be too large to precisely "turn down" when just a trickle of reagent is needed.

This same general control problem was encountered by automotive engineers in the days when *carburetors* were used to mix gasoline with air prior to combustion in an engine. A carburetor is a mechanical air flow control device using a "butterfly" valve element to throttle air flow into the engine, and a venturi element producing vacuum to aspirate fuel droplets into the air stream to create an air-fuel mixture. A carburetor with a butterfly valve and flow tube sized to idle well and respond to the needs of in-town driving would not flow enough air to provide good high-speed performance. Conversely, a large carburetor suitable for driving at racing speeds would offer poor control at low-speed and idling operation. Their solution to this problem was the *progressive carburetor*, having two butterfly valves to throttle the flow of air into the engine. One butterfly valve passed low amounts of air flow only, while a larger butterfly valve opened up only when the accelerator pedal was nearly at its maximum position. The combination of two differently-sized butterfly valves – progressively opened – gave drivers the best of both worlds. Now, an automobile engine could perform well both at low power levels and at high power levels.

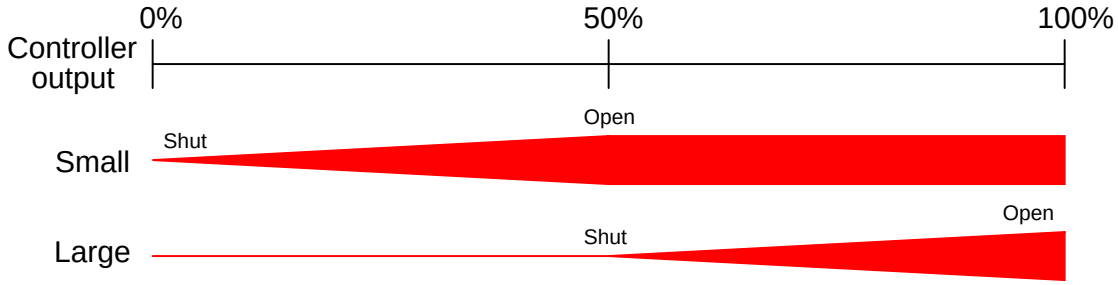
On a fundamental level, the problem faced in pH control as well as by early automotive engineers is the same thing: insufficient *rangeability*. Some processes demand a greater range of control than any single valve can deliver, and it is within these processes that a pair of progressively-sequenced control valves is a valid solution.

Applying this solution to a pH control process where the incoming liquid always has a high pH value, and must be neutralized with acid:



Proper sequencing of the small and large acid control valves is shown in the table and the graph:

Controller output (%)	**I/P output** (PSI)	**Small acid valve** (stem position)	**Large acid valve** (stem position)
0 %	3 PSI	fully shut	fully shut
25 %	6 PSI	half-open	fully shut
50 %	9 PSI	fully open	fully shut
75 %	12 PSI	fully open	half-open
100 %	15 PSI	fully open	fully open



With the two acid control valves sequenced progressively, the control system will have significantly more rangeability necessary to regulate pH under widely varying process conditions.

A photograph showing a pair of progressively sequenced control valves used to control boiler feedwater to the high-pressure ("HP") drum on a large power boiler is shown here:



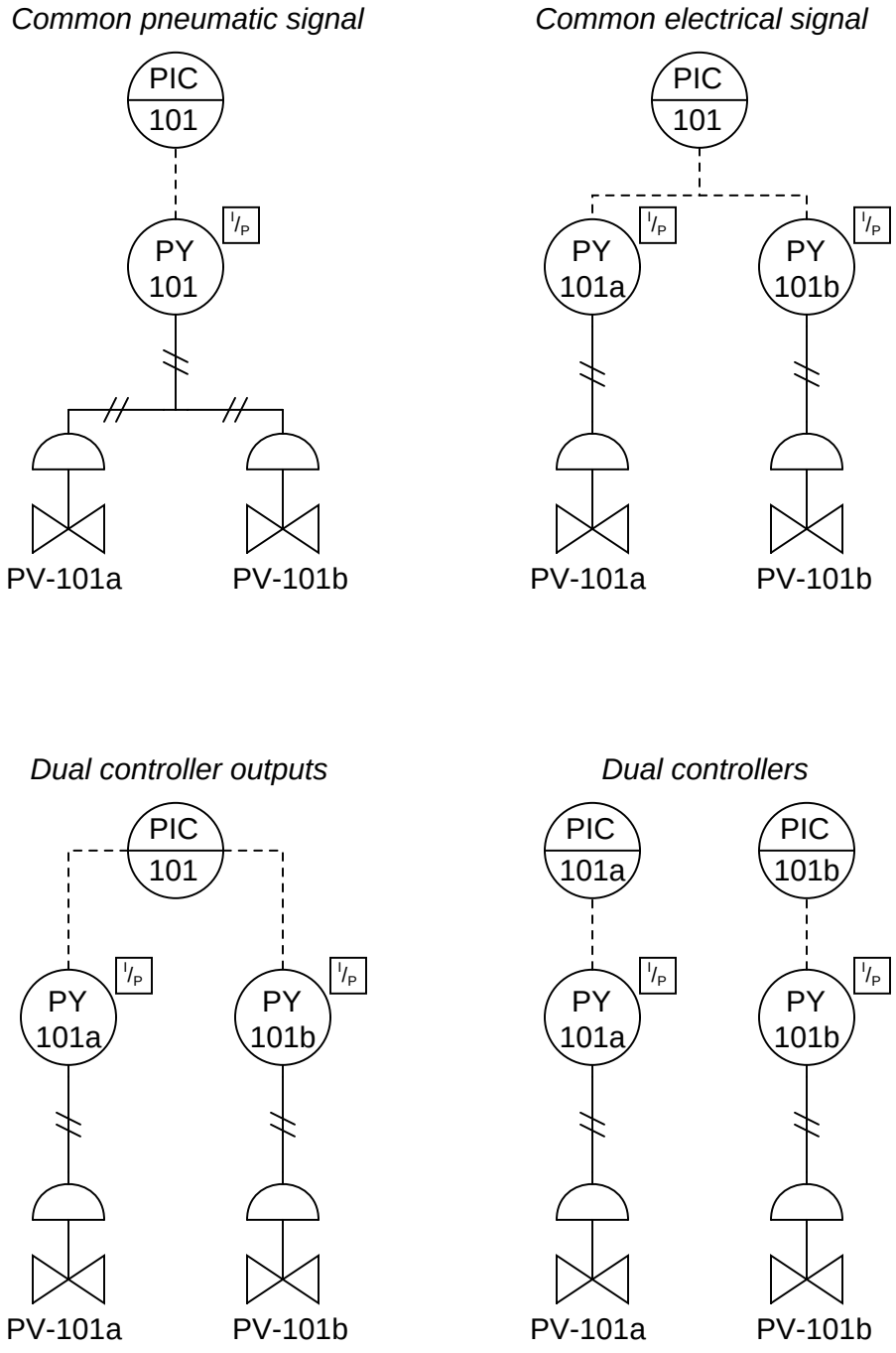
One small control valve (horizontal, with a green actuator) is the first to open from 4 to 12 mA. A larger control valve (vertical, with a red actuator) is the next to open from 12 mA to 20 mA. Under normal (full-power) operating conditions, the green valve is wide-open while the larger red valve throttles feedwater to the steam drum. When the boiler is operating in standby (low-power) mode, the red valve is shut while the smaller green valve throttles feedwater to the steam drum. The combination of these two control valves provides better steam drum level control over the boiler's full range of operation than any single control valve could do alone.

27.11.4 Valve sequencing implementations

In all previous control valve sequencing examples shown, both control valves received the same pneumatic signal from a common I/P (current-to-pressure) converter. This means each valve received the exact same pressure signal from the transducer for any given controller output value. Sequencing of the two valves (i.e. making each one respond differently to the same air pressure signal), therefore, was a matter of setting each valve to a different bench-set pressure range.

It should be understood, however, that setting up two control valves with different bench-set ranges is not the only way to split-range a pair of valves. Other ways exist as well, each with its own advantages and disadvantages.

The following illustrations depict several alternative methods for control valve sequencing (split-ranging):



The common pneumatic signal approach (one controller, one I/P transducer) is simple but suffers from the disadvantage of slow response, since one I/P transducer must drive two pneumatic actuators. Response time may be improved by adding a pneumatic *volume booster* between the I/P and the valve actuators, or by adding a positioner to at least one of the valves. Either of these solutions works by the same principle: reducing the air volume demand on the one common I/P transducer.

Wiring two I/P transducers in series so they share a common signal current is another way to split-range two control valves. This approach does not suffer from slow response, since each valve has its own dedicated I/P transducer to supply it with actuating air. We now have a choice where we implement the split ranges: we can do it in the I/P transducers (i.e. each I/P transducer having a different calibration) *or* in the control valves (i.e. each control valve having a different bench-set pressure range). Since it is generally easier to re-range an I/P than it is to rebuild a control valve with a different spring (to give it a different actuating pressure range), this approach has the advantage of convenient configuration.

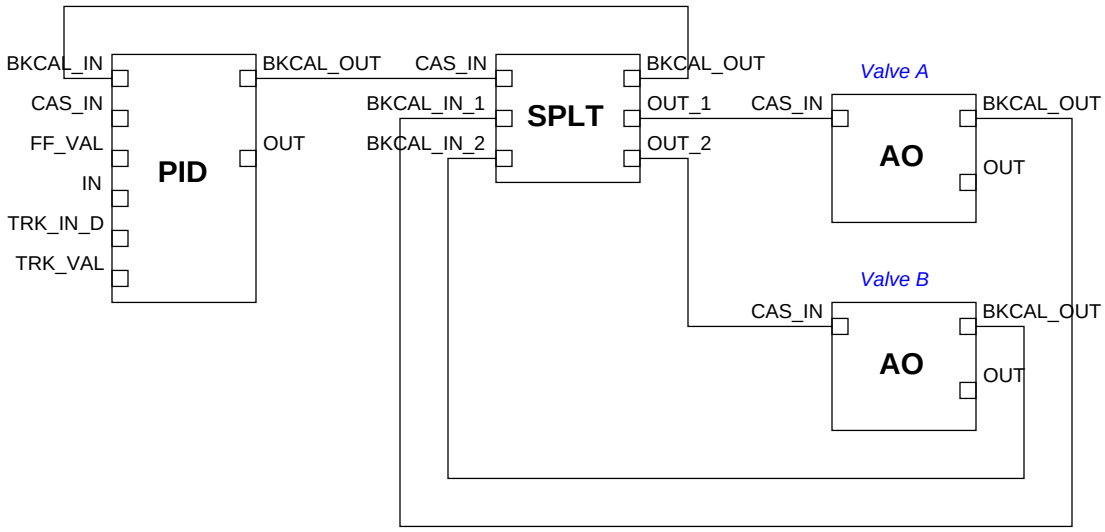
A disadvantage of the series-wired I/P strategy is the extra burden placed on the controller's output signal circuitry: one must be careful to ensure the two series-connected I/P converters do not drop too much voltage at full current, or else the controller may have difficulty driving both devices in series. Another (potential) disadvantage of series-connected valve devices in one current loop is the inability to install "smart" instruments communicating with the HART protocol, since multiple devices on the same loop will experience address conflicts[24]. HART devices can only work in hybrid analog/digital mode when there is one device per 4-20 mA circuit.

A popular way to implement split-ranging is to use multiple 4-20 mA outputs on the same controller. This is very easy to do if the controller is part of a large system (e.g. a DCS or a PLC) with multiple analog output channels. If multiple outputs are configured on one controller, each valve will have its own dedicated wire pair for control. This tends to result in simpler wiring than series-wired I/P transducers or positioners, since each valve loop is a standard 4-20 mA circuit just like any other (non-split-ranged) control valve loop circuit. It may also be the most practical way to implement split-ranging when "smart" valve positioners are used, since the dedicated loop circuits allow for normal operation of the HART protocol with no address conflicts.

An advantage of dual controller outputs is the ability to perform the split-range sequencing within the controller itself, which is often easier than re-ranging an I/P or calibrating a valve positioner. This way, the 4-20 mA signals going to each valve will be unique for any given controller output value. If sequenced as such, the I/P transducer calibration and valve bench-set values may be standard rather than customized. Of course, just because the controller is capable of performing the necessary sequencing doesn't mean the sequencing must be done within the controller. It is possible to program the controller's dual analog outputs to send the exact same current signal to each valve, configuring each valve (or each positioner, or each I/P transducer) to respond differently to the identical current signals.

[24] Although the HART standard does support "multidrop" mode where multiple devices exist on the same current loop, this mode is digital-only with no analog signal support. Not only do many host systems not support HART multidrop mode, but the relatively slow data communication rate of HART makes this choice unwise for most process control applications. If analog control of multiple HART valve positioner devices from the same 4-20 mA signal is desired, the address conflict problem may be resolved through the use of one or more *isolator* devices, allowing all devices to share the same analog current signal but isolating each other from HART signals.

A digital adaptation of the dual-output controller sequencing method is seen in FOUNDATION Fieldbus systems[25], where a special software function block called "SPLT" exists to provide split-ranged sequencing to two valves. The "SPLT" function block takes in a single control signal and outputs two signals, one output signal for each valve in a split-ranged pair. The function block diagram for such a system appears here:



In this Fieldbus system, a single PID control block outputs a signal to the SPLT block, which is programmed to drive two unique positioning signals to the two valves' AO (analog output) blocks. It should be noted that while each AO block is unique to its own control valve, the SPLT and even PID blocks may be located in any capable device within the Fieldbus network. With FOUNDATION Fieldbus, control system functions are not necessarily relegated to separate devices. It is possible, for example, to have a control valve equipped with a Fieldbus positioner actually perform its own PID control calculations and split-ranged sequencing by locating those function blocks in that one physical device!

Dual controllers are an option only for specialized applications requiring different degrees of responsiveness for each valve, usually for exclusive or progressive split-ranging applications only. Care must be taken to ensure the controllers' output signals do not wander outside of their intended ranges, or that the controllers do not begin to "fight" each other in trying to control the same process variable[26].

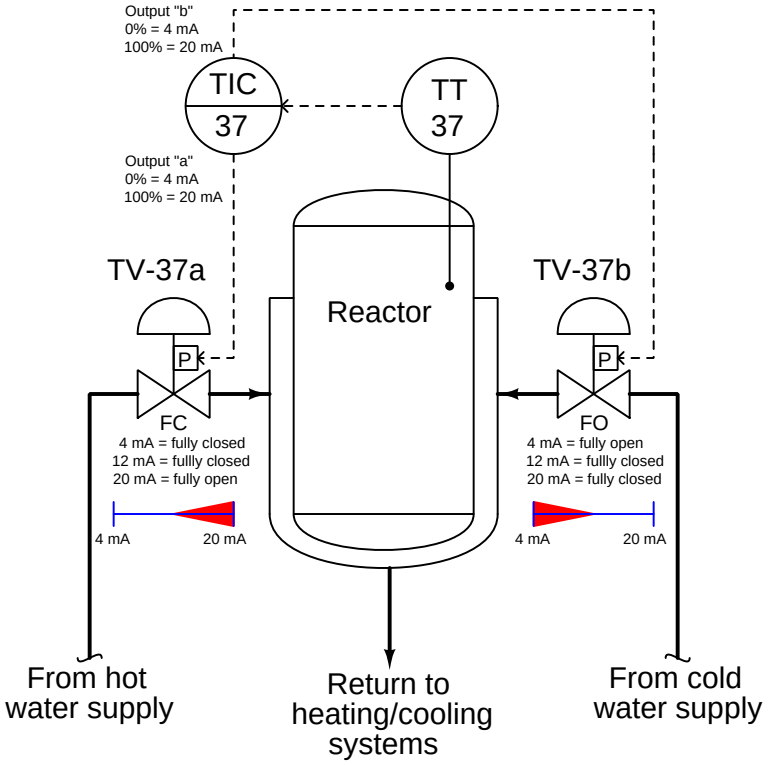
[25] To review, Fieldbus is an all-digital industrial control protocol, where instruments connect to a control system and to each other by means of a single network cable. Signals are routed not by specific wire connections, but rather by software entities called *function blocks* whereby the engineer or technician programs the instruments and control system what to do with those signals. The function blocks shown in this example would typically be accessed through the graphic display of a DCS in a real Fieldbus system, lines drawn between the blocks instructing the system where each of the instrument signals need to go.

[26] Both controllers should be equipped with provisions for reset windup control (or have no integral action at all), such that the output signal values are predictable enough that they behave as a synchronized pair rather than as two separate controllers.

An important consideration – and one that is easily overlooked – in split-range valve systems is *fail-safe mode*. As discussed in a previous section of this chapter (on section 27.7.3), the basis of fail-safe control system design is that the control valve(s) must be chosen to fail in the mode that is safest for the process in the event of actuating power loss or control signal loss. The actions of all other instruments in the loop should then be selected to complement the valves' natural operating mode.

In control systems where valves are split-ranged in either complementary or exclusive fashion, one control valve will be fully closed and the other will be fully open at each extreme end of the signal range (e.g. at 4 mA and at 20 mA). If the sequencing for a set of complementary or exclusive split-ranged control valves happens after the controller (e.g. different actuator actions) the valves must fail in opposite modes upon loss of controller signal. However, if it is deemed safer for the process to have the two valves fail in the same state – for example, to both fail closed in the event of air pressure or signal loss – we may use dual sequenced controller outputs, achieving either complementary or exclusive control action by driving the two valves with two different output signals. In other words, split-ranging two control valves so they normally behave in opposite fashion does *not* necessarily mean the two valves must fail in opposite states. The secret to achieving proper failure mode *and* proper split-range sequencing is to carefully locate where the sequencing takes place in the control system.

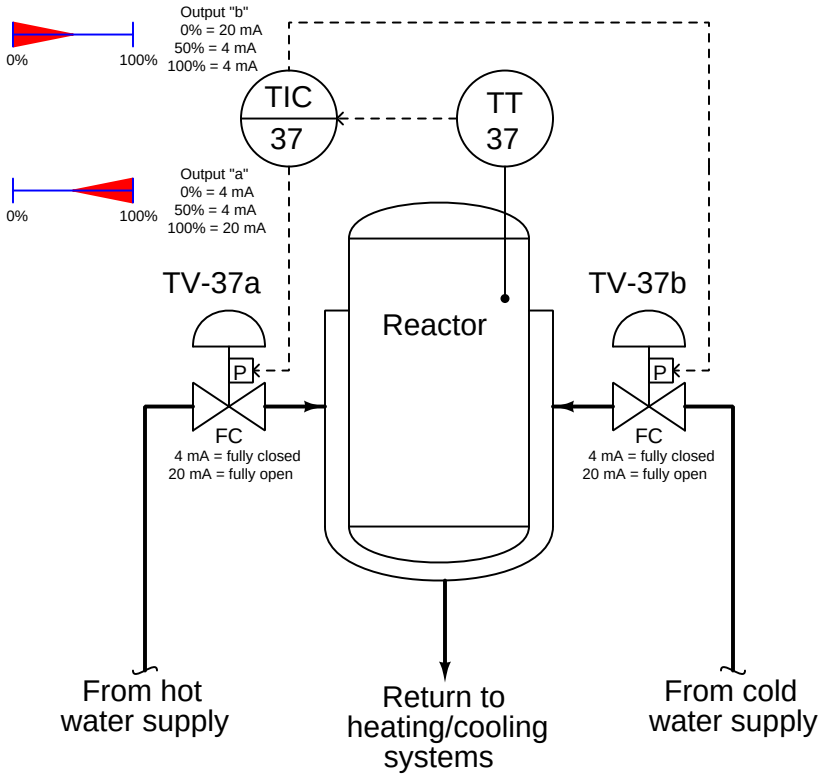
As an example of a split-ranged system with opposite valve failure modes, consider the following temperature control system supplying either hot water or chilled water to a "jacket" surrounding a chemical reactor vessel. The purpose of this system is to either add or remove heat from the reactor as needed to control the temperature of its contents. Chemical piping in and out of the reactor vessel has been omitted from this P&ID for simplicity, so we can focus just on the reactor's temperature control system:



Here, the controller has been configured for dual-output operation, where the output value drives two identical 4-20 mA signals to the control valve positioners, which directly input the current signals from the controller without the need for I/P transducers in between. The hot water valve (TV-37a) is fail-closed (FC) while the cold water valve (TV-37b) is fail-open (FO). Half-range positioner calibrations provide the exclusive sequencing necessary to ensure the two valves are never open simultaneously – TV-37b operates on the lower half of the 4-20 mA signal range (4-12 mA), while TV-37a operates on the upper half (12-20 mA).

Consider the effects from the controller (TIC-37) losing power. Both 4-20 mA signals will go dead, driving both valves to their fail-safe modes: hot water valve TV-37a will fully close, while cold water valve TV-37b will fully open. Now consider the effects of air pressure loss to both valves. With no air pressure to operate, the actuators will likewise spring-return to their fail-safe modes: once again hot water valve TV-37a will fully close, while cold water valve TV-37b will fully open. In both failure events, the two control valves assume consistent states, ensuring maximum cooling to the reactor in the event of an output signal or instrument air failure.

However, suppose we desired both of these valves fail in the closed position in the event of an output signal or instrument air failure, rather than have the cooling valve fail open while the heating valve fails closed. Clearly this would require both TV-37a and TV-37b to be fail-closed (FC), which would mean we must find some other way to sequence their operation to achieve split ranging. Examine this reconfiguration of the reactor temperature control system, using identical control valves (signal-to-open, fail-closed) for both hot and cold water supply, and a controller with exclusively-sequenced 4-20 mA output signals:



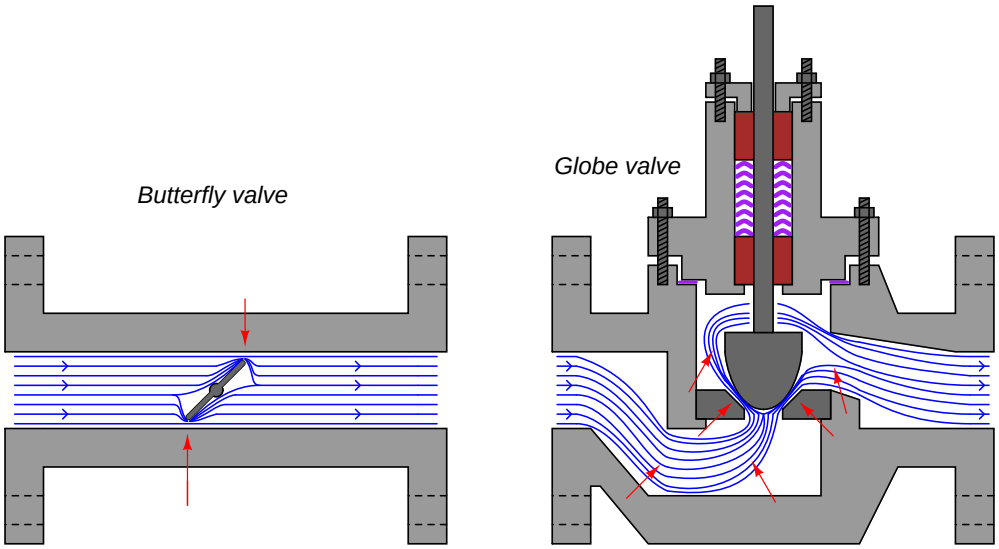
Consider the effects from the controller (TIC-37) losing power. Both 4-20 mA signals will go dead, driving both valves to their fail-safe modes: fully closed. Now consider the effects of air pressure loss to both valves. With no air pressure to operate, the actuators will spring-return to their fail-safe modes: once again both control valves fully close. In both failure events, the two control valves consistently close. The failure modes of both valves are still consistent regardless of the nature of the fault, but note how this scheme allows both valves to fail in the same mode if that is what we deem safest for the process.

As with all fail-safe system designs, we begin by choosing the proper fail-safe mode for each control valve *as determined by the safety requirements of the process*, not by what we would consider the simplest or easiest-to-understand instrument configurations. Only after we have chosen each valve's failure mode do we choose the other instruments' configurations. This includes split-range sequencing: where and how we sequence the valves is a decision to be made only after the valves' fail-safe states are chosen based on process safety.

27.12 Control valve sizing

When control valves operate between fully open and fully shut, they serve much the same purpose in process systems as resistors do in electric circuits: to dissipate energy. Like resistors, the form that this dissipated energy takes is mostly heat, although some of the dissipated energy manifests in the form of vibration and noise[27].

In most control valves, the dominant mechanism of energy dissipation comes as a result of turbulence introduced to the fluid as it travels through constrictive portions of the valve trim. The following illustration shows these constrictive points within two different control valve types (shown by arrows):



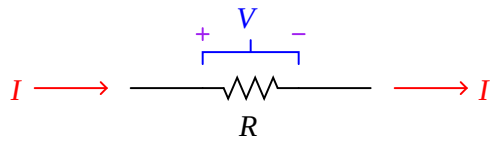
Note: red arrows show points of maximum fluid energy loss

The act of choosing an appropriate control valve for the expected energy dissipation is called *valve sizing*.

[27] Valve noise may be *severe* in some cases, especially in certain gas flow applications. An important performance metric for control valves is noise production expressed in decibels (dB).

27.12.1 Physics of energy dissipation in a turbulent fluid stream

Control valves are rated in their ability to throttle fluid flow much in the same way resistors are rated in their ability to throttle the flow of electrons in a circuit. For resistors, the unit of measurement for electron flow restriction is the *ohm*: 1 ohm of resistance results in a voltage drop of 1 volt across that resistance given a current through the resistance equal to 1 ampere:



Arrows point in direction of conventional flow

The mathematical relationship between current, voltage, and resistance for any resistor is *Ohm's Law*:

$$R = \frac{V}{I}$$

Where,
R = Electrical resistance in ohms
V = Electrical voltage drop in volts
I = Electrical current in amperes

Ohm's Law is a simple, linear relationship, expressing the "friction" encountered by electric charge carriers as they slowly drift through a solid object.

When a fluid moves turbulently through any restriction, energy is inevitably dissipated in that turbulence. The amount of energy dissipated is proportional to the kinetic energy of the turbulent motion, which is proportional to the square of velocity according to the classic kinetic energy equation for moving objects:

$$E_k = \frac{1}{2}mv^2$$

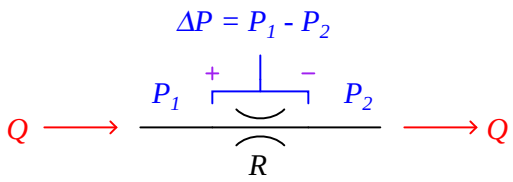
If we were to re-write this equation to express the amount of kinetic energy represented by a volume of moving fluid with velocity v, it would look like this:

$$\text{Kinetic energy per unit volume} = \frac{1}{2}\rho v^2$$

We know that the amount of energy dissipated by turbulence in such a fluid stream will be some proportion (k) of the total kinetic energy, so:

$$\text{Energy dissipated per unit volume} = \frac{1}{2}k\rho v^2$$

Any energy lost in turbulence eventually manifests as a loss in fluid pressure downstream of that turbulence. Thus, a control valve throttling a fluid flowstream will have a greater upstream pressure than downstream pressure (assuming all other factors such as pipe size and height above ground level being the same downstream as upstream):



This pressure drop ($P_1 - P_2$, or ΔP) is equivalent to the voltage drop seen across any current-carrying resistor, and may be substituted for dissipated energy per unit volume in the previous equation[28]. We may also substitute $\frac{Q}{A}$ for velocity v because we know volumetric flow rate (Q) is the product of fluid velocity and pipe cross-section area ($Q = Av$) for incompressible fluids such as liquids:

$$P_1 - P_2 = \frac{1}{2} k \rho \left(\frac{Q}{A} \right)^2$$

Next, we will solve for a quotient with pressure drop ($P_1 - P_2$) in the numerator and flow rate (Q) in the denominator so the equation bears a resemblance to Ohm's Law ($R = \frac{V}{I}$):

$$P_1 - P_2 = \frac{1}{2} k \rho \frac{Q^2}{A^2}$$

$$\frac{P_1 - P_2}{Q^2} = \frac{k \rho}{2 A^2}$$

$$\frac{\sqrt{P_1 - P_2}}{Q} = \sqrt{\frac{k \rho}{2 A^2}}$$

[28] In case you were wondering, it is appropriate to express energy loss per unit volume in the same units of measurement as pressure. For a more detailed discussion of dimensional analysis, see section 2.11.13 beginning on page 208 where Bernoulli's equation is examined and you will see how the units of $\frac{1}{2} \rho v^2$ and P are actually the same.

Either side of the last equation represents a sort of "Ohm's Law" for turbulent liquid restrictions: the left-hand side expressing fluid "resistance" in the state variables of pressure drop and volumetric flow, and the right-hand term expressing fluid "resistance" as a function of fluid density and restriction geometry. We can see how pressure drop ($P_1 - P_2$) and volumetric flow rate (Q) are not linearly related as voltage and current are for resistors, but that nevertheless we still have a quantity that acts like a "resistance" term:

$$R = \frac{\sqrt{P_1 - P_2}}{Q} \qquad\qquad R = \sqrt{\frac{k\rho}{2A^2}}$$

Where,

R = Fluid "resistance"
P_1 = Upstream fluid pressure
P_2 = Downstream fluid pressure
Q = Volumetric fluid flow rate
k = Turbulent energy dissipation factor
ρ = Mass density of fluid
A = Cross-sectional area of restriction

The fluid "resistance" of a restriction depends on several variables: the proportion of kinetic energy lost due to turbulence (k), the density of the fluid (ρ), and the cross-sectional area of the restriction (A). In a control valve throttling a liquid flow stream, only the first and last variables are subject to change with stem position, fluid density remaining relatively constant.

In a wide-open control valve, especially valves offering a nearly unrestricted path for moving fluid (e.g. ball valves, eccentric disk valves), the value of A will be at a maximum value essentially equal to the pipe's area, and k will be nearly zero[29]. In a fully shut control valve, A is zero, creating a condition of infinite "resistance" to fluid flow.

It is customary in control valve engineering to express the "restrictiveness" of any valve in terms of how much flow it will pass given a certain pressure drop and fluid specific gravity (G_f). This measure of valve performance is called *flow capacity* or *flow coefficient*, symbolized as C_v. A greater flow capacity value represents a less restrictive (less "resistive") valve, able to pass greater rates of flow for the same pressure drop. This is analogous to expressing an electrical resistor's rating in terms of conductance (G) rather than resistance (R): how many amperes of current it will pass with 1 volt of potential drop ($I = GV$ instead of $I = \frac{V}{R}$).

If we return to one of our earlier equations expressing pressure drop in terms of flow rate, restriction area, dissipation factor, and density, we will be able to manipulate it into a form expressing flow rate (Q) in terms of pressure drop and density, collecting k and A into a third term which will become flow capacity (C_v):

$$P_1 - P_2 = \frac{1}{2} k \rho \frac{Q^2}{A^2}$$

[29] In a case of minimal throttling, almost none of the fluid's kinetic energy is lost to turbulence, but rather passes right through the valve unrestricted.

First, we must substitute specific gravity (G_f) for mass density (ρ) using the following definition of specific gravity:

$$G_f = \frac{\rho}{\rho_{water}}$$

$$\rho_{water} G_f = \rho$$

Substituting and continuing with the algebraic manipulation:

$$P_1 - P_2 = \frac{1}{2} k \rho_{water} G_f \frac{Q^2}{A^2}$$

$$\frac{P_1 - P_2}{G_f} = \frac{1}{2} k \rho_{water} \frac{Q^2}{A^2}$$

$$\left(\frac{2A^2}{k\rho_{water}} \right) \left(\frac{P_1 - P_2}{G_f} \right) = Q^2$$

$$Q = \sqrt{\frac{2A^2}{k\rho_{water}}} \sqrt{\frac{P_1 - P_2}{G_f}}$$

The first square-rooted term in the equation, $\sqrt{\frac{2A^2}{k\rho_{water}}}$, is the valve capacity or C_v factor. Substituting C_v for this term results in the simplest form of valve sizing equation (for incompressible fluids):

$$Q = C_v \sqrt{\frac{P_1 - P_2}{G_f}}$$

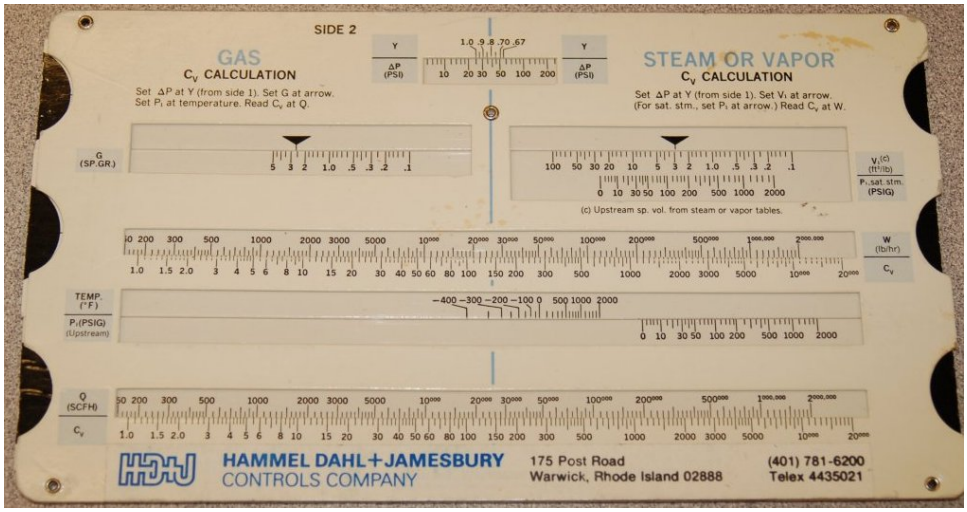
In the United States of America, C_v is defined as the number of gallons per minute of water that will flow through a valve with 1 PSI of pressure drop[30]. A similar valve capacity expression used with metric units rates valves in terms of how many cubic meters per hour of water will flow through a valve with a pressure drop of 1 bar. This latter flow capacity is symbolized as K_v.

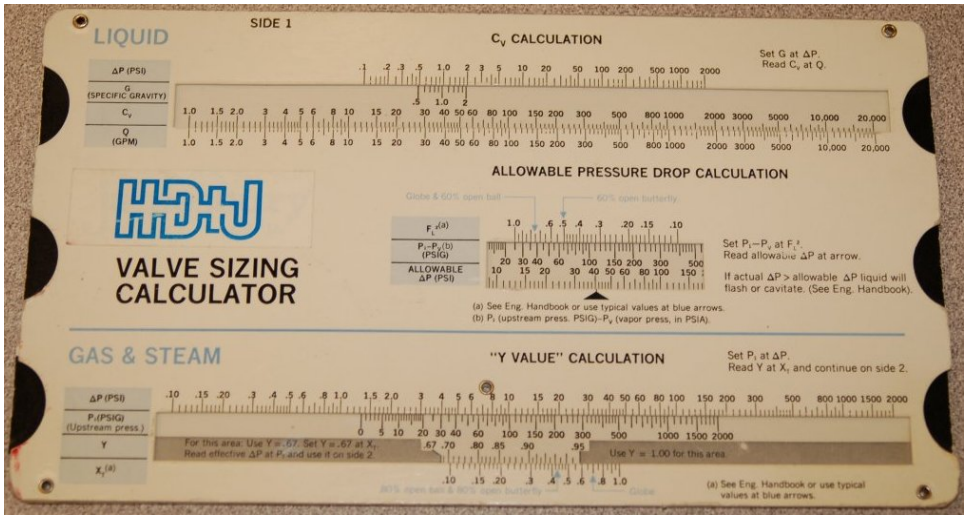
For the best results predicting required C_v values for control valves in any service, it is recommended that you use valve sizing software provided by control valve manufacturers. The formulae shown here do not account for all factors[31] influencing fluid flow rate and pressure drop, and therefore yield approximate values only. Modern valve sizing software is easy to use, especially when referenced to specific models of control valve sold by that manufacturer, and is able to account for a diverse multitude of factors affecting proper sizing.

[30] The specification of certain British units of measurement for flow and pressure drop means that there is more to C_v than just $\sqrt{\frac{2A^2}{k\rho_{water}}}$. C_v also incorporates a factor necessary to account for the arbitrary choice of units.

[31] Such factors include fluid compressibility, viscosity, specific heat, vapor pressure to name a few. Not only will modern valve sizing software more accurately predict valve sizes for particular applications than these simple formulae, but this software may also provide estimations of *noise levels* produced by the valve.

Control valve sizing is complex enough that some valve manufacturers used to give away "slide rule" calculator devices so customers could choose the C_v values they needed with relative ease. Photographs of a two-sided valve sizing slide rule are shown here for historical reference:





27.12.2 Importance of proper valve sizing

The flow coefficient of a control valve (C_v) is a numerical value expressing the number of gallons per minute flow of water the valve will pass with a constant pressure drop of 1 PSI from inlet to outlet. This rating is usually given for the valve in its wide-open state. For example, a control valve with a C_v rating of 45 should flow 45 gallons per minute of water through it with a 1 PSI pressure drop *when wide open*. The flow coefficient value for this same control valve will be less than 45 when the valve position is anything less than fully open. When the control valve is in the fully shut position, its C_v value will be zero. Thus, it should be understood that C_v is truly a variable – not a constant – for any control valve, even though control valves are often specified simply by their maximum flow capacity.

As previously shown, the basic liquid-flow equation relating volumetric flow rate to pressure drop and specific gravity is:

$$Q = C_v \sqrt{\frac{P_1 - P_2}{G_f}}$$

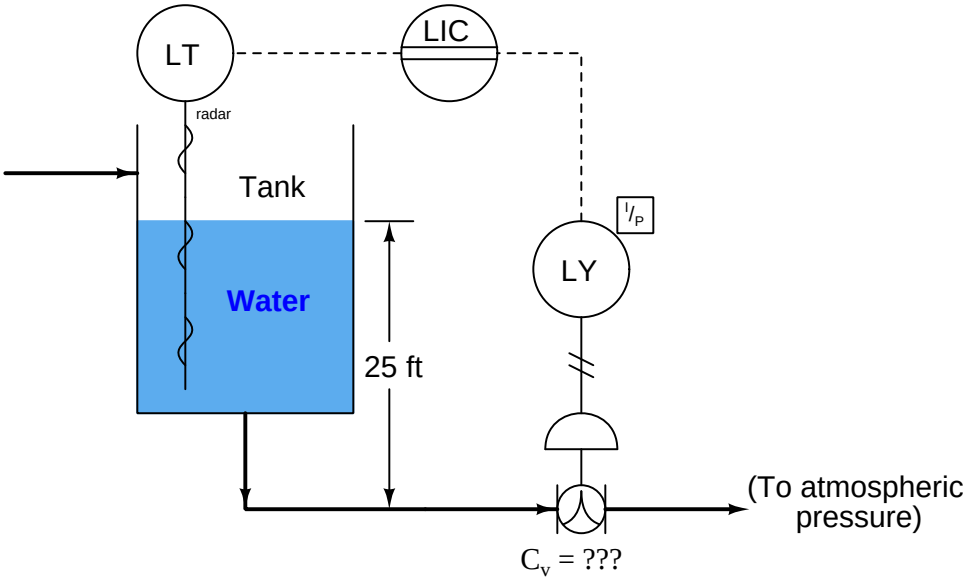
Where,

Q = Volumetric flow rate of liquid (gallons per minute, GPM)
C_v = Flow coefficient of valve
P_1 = Upstream pressure of liquid (PSI)
P_2 = Downstream pressure of liquid (PSI)
G_f = Specific gravity of liquid (ratio of liquid density to standard water density)

A common misconception is that C_v represents an absolute maximum flow rating for a valve. This is not true, any more than the resistance value of an electrical resistor reflects some maximum current value for that component. It is possible – at least in theory – to shove *any* rate of flow through *any* open valve given enough pressure to motivate that flow. C_v, like electrical resistance measured in ohms, merely states the proportion between flow rate and pressure drop. C_v is simply the maximum flow rate of water through a control valve *given a 1 PSI pressure difference from upstream to downstream.*

It should be obvious that any control valve must be sized large enough (i.e. possess sufficient maximum C_v capacity) to flow the greatest expected flow rate in any given process installation. A valve that is too small for an application will not be able to pass enough process fluid through it given the fluid pressure available.

Given this fact, it may seem safe to choose a valve sized much larger than what is needed, just to avoid the possibility of not having enough flow capacity. For instance, consider this control valve sizing problem, where a characterized ball valve controls the flow rate of water out of a surge tank to maintain a constant water level 25 feet higher than the height of the valve:



Suppose the process engineer designing this system specifies a flow of 470 GPM with the control valve wide-open. What should the full-open C_v rating be for this valve? To begin, we must know the expected pressure drop across the valve. The 25 foot water column height upstream provides us with the means to calculate P_1 by using the formula for calculating hydrostatic pressure (the pressure generated by a vertical column of liquid under the influence of gravity):

$$P = \gamma h$$

$$P_1 = (62.4 \text{ lb/ft}^3)(25 \text{ feet})$$

$$P_1 = 1560 \text{ PSF} = 10.8 \text{ PSI}$$

There is no need to calculate P_2, since the P&ID reveals the downstream side of the valve to be vented to atmosphere, and thus is guaranteed to be 0 PSI gauge pressure. This gives us a pressure drop of 10.8 PSI across the control valve, with an expected full-flow rate of 470 GPM. Manipulating our flow capacity equation to solve for C_v:

$$Q = C_v \sqrt{\frac{P_1 - P_2}{G_f}}$$

$$C_v = \frac{Q}{\sqrt{\frac{P_1 - P_2}{G_f}}}$$

$$C_v = \frac{470 \text{ GPM}}{\sqrt{\frac{10.8 \text{ PSI}}{1}}}$$

$$C_v = 143$$

This tells us we need a control valve with a C_v value of *at least* 143 to meet the specified flow rate of 470 GPM. A valve with insufficient C_v would not be able to flow the required 470 gallons per minute of water with only 10.8 PSI of pressure drop. As mentioned previously, these calculations yield approximate results only. An analysis using computer-based valve sizing software may require an even larger control valve for this application.

Does this mean we should over-size the valve, just to ensure we don't install one that is too small? For the sake of argument, would there be any problem with installing a control valve with a C_v value of 300? The general answer to these questions is that over-sized valves may create other problems. Not only is there the possibility of allowing too much flow under wide-open conditions (consider whatever process vessels and equipment lie downstream of the oversized valve), but also that the process will be difficult to control under low-flow conditions.

In order to understand how an over-sized control valve leads to unstable control, an exaggerated example is helpful to consider: imagine installing a fire hydrant valve on your kitchen sink faucet[32]. Certainly, a wide-open hydrant valve would allow sufficient water flow into your kitchen sink. However, due to the gross mismatch between a fire hydrant valve's flow capacity versus that of relatively skinny tubing providing water to the sink faucet, most of this valve's usable range of throttling will be limited to the first *percent* of stem travel. After the valve is opened just a few percent from fully shut, restrictions in the piping of your house's water system will have limited the flow rate to its maximum, thus rendering the rest of the valve's stem travel capacity utterly useless. It would be challenging indeed to try filling a drinking cup with water from this hydrant valve: just a little bit too much stem motion and the cup would be subjected to a full-flow stream of water!

An over-sized valve is therefore an overly-*sensitive* valve from the perspective of the control system driving it. With the upper end of the valve's travel being useless for control (having little or no effect), all the throttling action must take place within the first few percent of stem motion.

[32] This is a good example of a general problem-solving strategy in action: making some dramatic change to the scenario and then reasoning the consequences of that change to better understand general principles. For those readers who may be unfamiliar with American terminology, a *fire hydrant* is a large hand valve installed at intervals along public roadways, allowing connection of fire hoses to an underground water supply pipe in the event of an emergency fire. These valves are quite large, and would be comically oversized if installed inside a person's house, for any purpose.

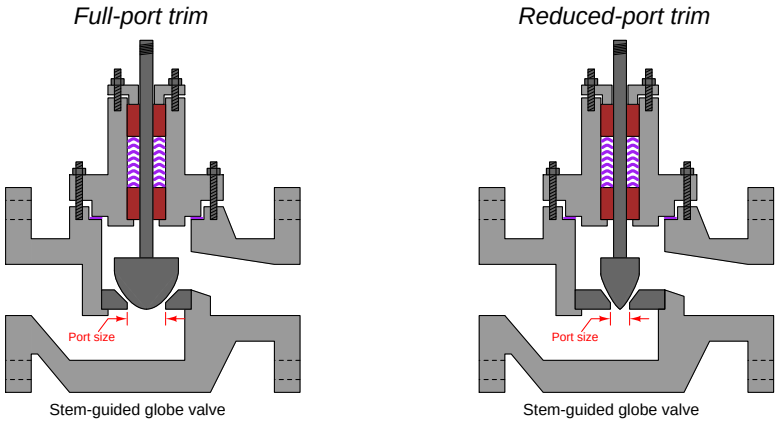
This makes precise control of flow rate more challenging than it should be. Typical valve problems such as stem friction, hysteresis, and calibration error therefore become amplified when the valve is over-sized, because any amount of imprecision in stem positioning becomes a greater *percentage* of the valve's useful travel range than if the valve were properly sized and able to use its full range of motion.

Control valve over-sizing is a common problem in industry, often created by future planning for expanded process flow. "If we buy a large valve now," so the reasoning goes, "we won't have to replace a smaller valve with a large valve when the time comes to increase our production rate." In the interim period when that larger valve must serve to control a meager flow rate, however, operational problems caused by poor control quality may end up costing the business more than the cost of an additional valve. The key here, as in so many other applications in business, is to carefully consider costs over the life-span of the device, not just the initial (capital) expense.

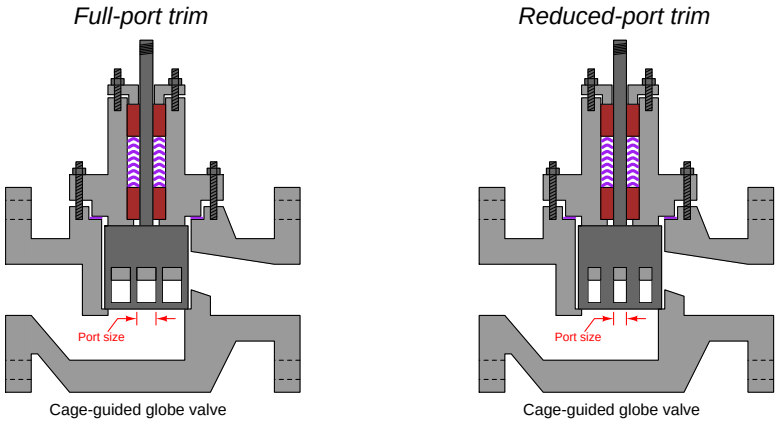
A practical solution to the problem of valve over-sizing – especially when larger flow rates will be required of the valve in the future – is to initially order the control valve with a body size suitable for the largest flow capacity it will need in the future, but equipped internally with *reduced-port* (or *restricted-capacity*) trim. This means trim having smaller holes ("ports") through which the fluid must flow. Such "reduced" trim is under-sized for the valve body, making the control valve's C_v rating significantly less than it would be with normal-sized trim installed. The benefit of installing reduced-port trim in a control valve is that the flow capacity of the valve may be upgraded simply by removing the reduced trim components and replacing them with *full-port* (full-sized) trim. Upgrading a control valve's trim to full-port size is significantly less expensive than replacing the entire control valve with a larger one[33].

[33] This is particularly true when one considers the piping changes usually necessary to accommodate a valve size change. Undersized valves installed in a pipe often require *reducer* fittings to "narrow" the full-bore size of the pipe down to the flange size of the control valve body. Upon replacement of the under-sized valve, these reducers must be removed to accommodate the larger valve body. The piping itself may need to be cut and re-welded to match the flange-to-flange dimensions of the new (larger) control valve. All of this requires time, labor, and material investment. If a large valve body with reduced-port trim were initially installed, however, most of this time, labor, and expense could be avoided when the time comes to replace the reduced-port trim with full-port trim.

Reduced-port trim for a stem-guided or port-guided globe valve takes the form of a new (smaller) plug and seat assembly. The seat is specially designed to match the plug for tight shutoff and good throttling behavior, while having the necessary external dimensions to fit the larger valve body casting:



Reduced-port trim for a cage-guided globe valve often consists of nothing more than a new cage, having smaller ports in the cage than standard. Thus, reduced-port trim for a cage-guided globe valve may[34] utilize the exact same plug and seat as the full-port trim for the same cage-guided globe valve:



[34] Reduced-port cage-guided trim may also take the form of a cage, plug, and seat of reduced diameter, with flanges attached in such a way that this smaller trim still fits inside the larger valve body. The example illustrated here, with a full-diameter cage having narrower ports on it, is just one way of achieving reduced flow capacity in a cage-guided design but certainly not the only way.

27.12.3 Gas valve sizing

Sizing a control valve for gas or vapor service is more complicated than for liquid service, due to the compressibility of gases and vapors. As a gas or vapor compresses with changes in pressure, its density changes correspondingly. In previous mathematical analyses of fluid flow restriction, one of our assumptions was that fluid density (ρ) remained constant. This assumption may hold true for some flowing gas conditions as well, provided minimal pressure changes within the path of flow. However, for most gas control valve applications where the very purpose of the valve is to introduce substantial pressure changes, the assumption of constant gas density is unrealistic.

Shown here is one of the simpler gas valve sizing equations you will encounter:

$$Q = 963\, C_v \sqrt{\frac{\Delta P(P_1 + P_2)}{G_g T}}$$

Where,

Q = Gas flow rate, in units of Standard Cubic Feet per Hour (SCFH)

C_v = Valve capacity coefficient

ΔP = Pressure dropped across valve, pounds per square inch differential (PSID)

P_1 = Upstream valve pressure, pounds per square inch absolute (PSIA)

P_2 = Downstream valve pressure, pounds per square inch absolute (PSIA)

G_g = Specific gravity of gas (ratio of gas density to standard air density)

T = Absolute temperature of gas in degrees Rankine (^{o}R), equal to degrees Fahrenheit plus 459.67

This equation holds true only for "subcritical" flow, where the moving gas stream velocity never approaches the speed of sound[35]. Other equations exist for calculating flow rates of gas through control valves in the presence of sonic flow regimes. Note the inclusion of absolute pressures in this equation, and not just differential pressure (ΔP, or $P_1 - P_2$). This is intended to correct for effects related to compression of the gas under pressure.

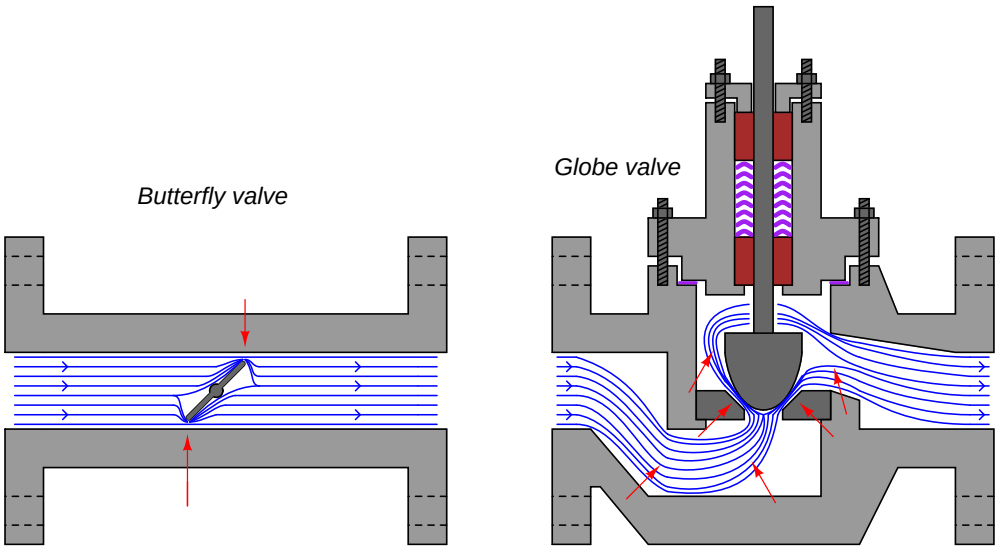
Valve sizing is complicated enough, both for liquid and gas service, that the use of valve sizing computer software is strongly recommended as opposed to hand-calculations. The number of important parameters, nonlinear factors, and alternative equations relevant to control valve sizing are numerous enough to bewilder most technicians (and more than a few engineers). Valve sizing software will also predict noise levels generated by the valve, and in many cases specify actual valve trim styles offered by the manufacturer for mitigating problems such as noise.

[35] The *ISA Handbook of Control Valves* cites this equation as being valid for conditions where the valve's downstream pressure (P_2) is equal to or greater than one-half the upstream pressure (P_1), with both pressures expressed in absolute units. In other words, $P_2 \geq 0.5P_1$ or $P_1 \leq 2P_2$. An upstream:downstream pressure ratio in excess of 2:1 usually means flow through a valve will become *choked*.

27.12.4 Relative flow capacity

The flow capacity of a valve (C_v) is a quantitative rating of its ability to pass a fluid flow for a set of given pressure and fluid density conditions, defined as the flow rate of water in gallons per minute through the valve given a constant pressure drop of 1 PSID. C_v may be predicted, or empirically measured, for any type of control valve given the proper information.

Not all control valve types exhibit the same C_v coefficients, however, for the same pipe size. A 4 inch butterfly valve, for example, has a much greater full-open C_v rating than a 4 inch globe valve, due to the much more direct path it offers to a moving fluid. A simple comparison of these two valve types clearly shows why this is true (note the "constriction" points labeled with arrows):



Note: red arrows show points of maximum fluid energy loss

A globe valve is simply more effective at generating fluid turbulence – and therefore dissipating fluid kinetic energy – than a butterfly valve of the same pipe size, because the globe valve design forces the fluid to change direction more often and in different ways.

One way to help quantify a particular valve design's ability to throttle fluid flow is to express this ability as a ratio of flow coefficient (C_v) versus cross-sectional pipe area. The basic principle here is that we should expect the C_v of any particular valve design to be proportional to pipe area (e.g. a ball valve with twice the pipe area should have twice the flow capacity, all other factors being equal), and therefore a ratio of these two quantities should be fairly constant for any valve design. Since we know the area of a pipe is proportional to the square of either radius or diameter ($A = \frac{\pi d^2}{4}$ or $A = \pi r^2$), we may simplify this ratio by omitting all constants such as π and simply relating C_v factor to the square of pipe diameter measured in *inches* (d^2). This ratio is called the *relative flow capacity*, or C_d:

$$C_d = \frac{C_v}{d^2}$$

Several approximate valve capacity factors (C_d) for different control valve types are shown here[36], assuming full-area (not reduced-port) trim, a wide-open position, and diameter measured in inches:

Valve design type	C_d
Single-port globe valve, ported plug	9.5
Single-port globe valve, contoured plug	11
Single-port globe valve, characterized cage	15
Double-port globe valve, ported plug	12.5
Double-port globe valve, contoured plug	13
Rotary ball valve, segmented	25
Rotary ball valve, standard port (diameter $\approx 0.8d$)	30
Rotary butterfly valve, 60°, no offset seat	17.5
Rotary butterfly valve, 90°, offset seat	29
Rotary butterfly valve, 90°, no offset seat	40

To calculate the approximate C_v for any valve, all we need to do is square that valve's pipe diameter (in inches) and multiply by the valve type's relative flow capacity (C_d):

$$C_v \approx d^2 C_d$$

Based on the figures in this table, for example, we may predict that a segmented ball valve with a pipe size of 3 inches will have a flow capacity (C_v) of approximately 225, or that a single-port cage-guided globe valve with an 8 inch pipe size will have a C_v of approximately 960.

As you can see from a comparison of C_d values, a no-offset butterfly valve has nearly *4 times* the flow capacity of a single-ported contoured-plug globe valve of the same pipe size ($C_d = 40$ versus $C_d = 11$). This makes butterfly valves advantageous in applications where large flow capacities must be achieved at minimal cost, such as in air handling (HVAC) systems for commercial buildings and combustion air controls for large industrial burners.

At first glance this may seem to make larger C_d valve types the superior choice for control valve applications, yielding the greatest C_v values for the smallest pipe sizes. However, there are other factors to consider such as ease of maintenance, noise and cavitation abatement, characterization, and valve seat leakage. For these reasons some of the lowest-C_d valve types (e.g. globe valves) remain popular choices for industrial control applications even though they require larger pipe sizes to achieve the same amount of flow compared to other valve types (e.g. ball valves).

27.13 Control valve characterization

Control valves are supposed to deliver reliable, repeatable control of process fluid flow rate over a wide range of operating conditions. As we will soon see, this is something of a challenge, as the rate of fluid flow through a control valve depends on more than just the position of its stem. This section discusses the problem of control valve behavior in real process applications, and explores the concept of *characterization* as a solution to the problem.

[36] Source for C_d factors: Chapter 4.17: Valve Sizing of Béla Lipták's *Instrument Engineers' Handbook, Process Control (Volume II), Third Edition*, page 590.

27.13.1 Inherent versus installed characteristics

When control valves are tested in a laboratory setting, they are connected to a piping system providing a nearly constant pressure difference between upstream and downstream ($P_1 - P_2 =$ constant). With a fluid of constant density and a constant pressure drop across the valve, flow rate becomes directly proportional to the valve's flow coefficient (C_v). This is clear from an examination of the basic valve capacity equation, if we replace the pressure and specific gravity terms with a single constant k:

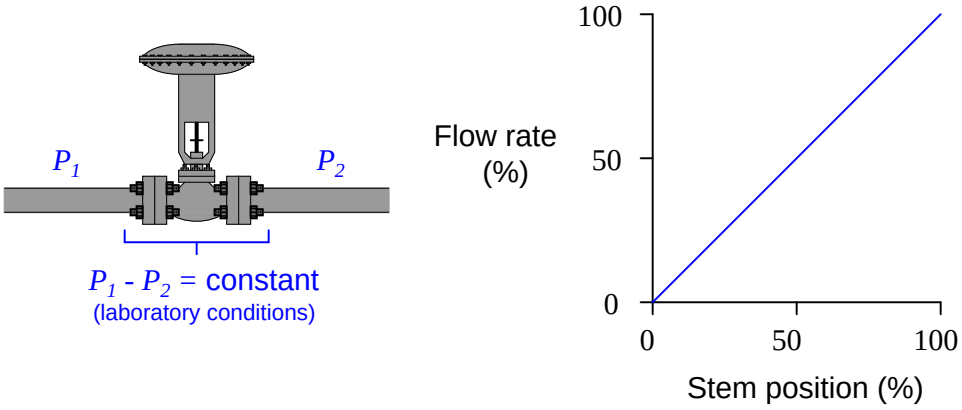
$$Q = C_v \sqrt{\frac{P_1 - P_2}{G_f}}$$

(If pressures and specific gravity are constant . . .)

$$Q = kC_v$$

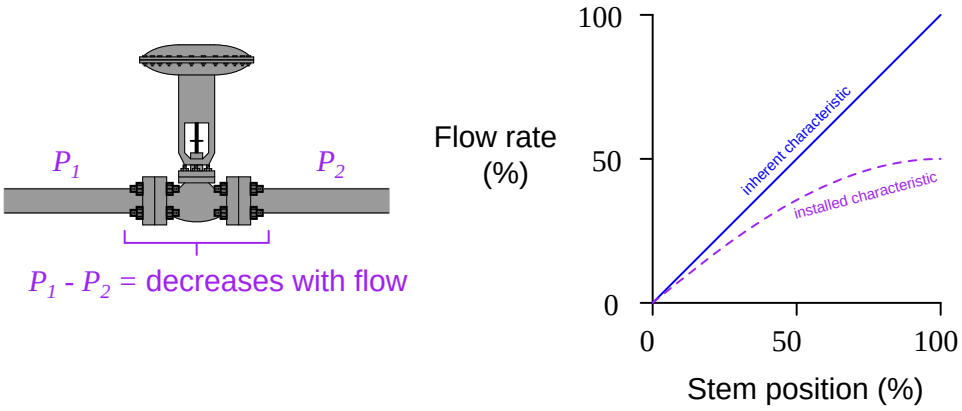
As discussed in an earlier section of this chapter (see section 27.12.1), the amount of "resistance" offered by a restriction of any kind to a turbulent fluid depends on the cross-sectional area of that restriction and also the proportion of fluid kinetic energy dissipated in turbulence. If a control valve is designed such that the combined effect of these two parameters vary linearly with stem motion, the C_v of the valve will likewise be proportional to stem position. That is to say, the C_v of a "linear" control valve will be approximately half its maximum rating with the stem position at 50%; approximately one-quarter its maximum rating with the stem position at 25%; and so on.

If such a valve is placed in a laboratory flow test piping system with constant differential pressure and constant fluid density, the relationship of flow rate to stem position will be linear. With constant pressure drop, the flow rate through any valve is directly proportional to that valve's C_v, and with a "linear" valve design the C_v (and therefore the flow rate as well) must be directly proportional to stem position:



However, most real-life valve installations do not provide the control valve with a constant pressure drop. Due to frictional pressure losses in piping and changes in supply/demand pressures that vary with flow rate, a typical control valve "sees" substantial changes in differential pressure as its controlled flow rate changes. Generally speaking, the pressure drop available to the control valve *decreases* as flow rate *increases*.

The result of this pressure drop versus flow relationship is that the actual flow rate of the same valve installed in a real process will *not* linearly track valve stem position. Instead, it will "droop" as the valve is further opened. This "drooping" graph is called the valve's *installed characteristic*, in contrast to the *inherent characteristic* exhibited in the laboratory with constant pressure drop:

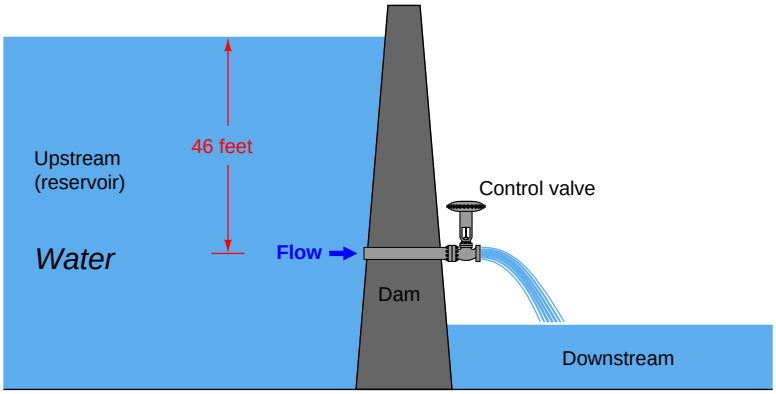


Each time the stem lifts up a bit more to open the valve trim further, flow increases, but not as much as at lower-opening positions. It is a situation of *diminishing returns*, where we still see increases in flow as the stem lifts up, but to a lesser and lesser degree.

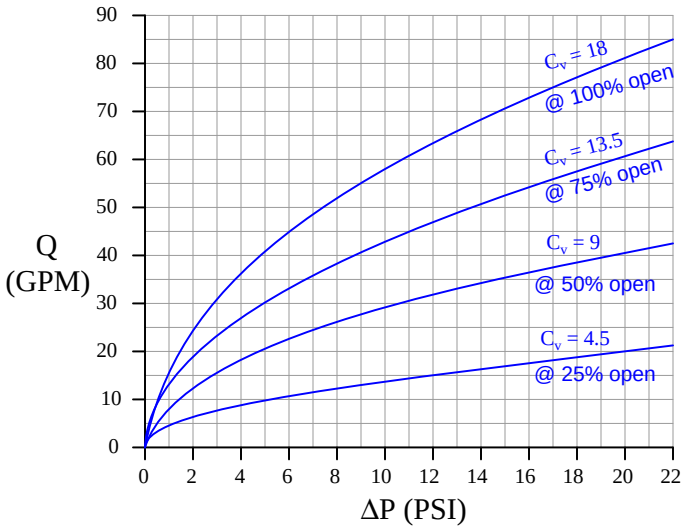
In my years of teaching, I have found this concept of "installed characteristic" to be especially challenging for many students. In the interest of clarifying the concept, the next two subsections will present a pair of contrasting valve performance scenarios.

27.13.2 Control valve performance with constant pressure

First, let us imagine a control valve installed at the base of a dam, releasing water from the reservoir. Given a constant height of water in the reservoir, the upstream (hydrostatic) pressure at the valve will likewise be constant. Let's assume this constant upstream pressure will be 20 PSI (corresponding to approximately 46 feet of water column above the valve inlet). With the valve discharging into the air, downstream pressure will essentially be zero. This set of upstream and downstream conditions guarantees a constant pressure drop of 20 PSI across our control valve at all times, for all flow conditions:



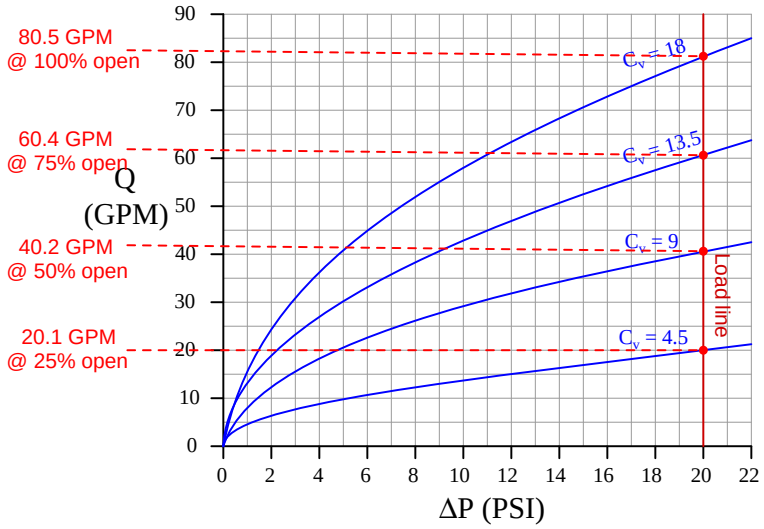
Furthermore, let us assume the control valve has a "linear" inherent characteristic and a maximum flow capacity (C_v rating) of 18. This means the valve's C_v will be 18 at 100% open, 13.5 at 75% open, 9 at 50% open, 4.5 at 25% open, and 0 at fully closed (0% open). We may plot the behavior of this control valve at these four stem positions by graphing the amount of flow through the valve for varying degrees of pressure drop across the valve. The result is a set of *characteristic curves*[37] for our hypothetical control valve:



Each curve on the graph traces the amount of flow through the valve at a constant stem position, for different amounts of applied pressure drop. For example, looking at the curve representing 50% open ($C_v = 9$), we can see the valve should flow about 42 GPM at 22 PSI, about 35 GPM at 15 PSI, about 20 GPM at 5 PSI, and so on. Of course, we can obtain these same flow figures simply by evaluating the formula $Q = C_v\sqrt{\Delta P}$ (which is in fact what I used to plot these curves), but the point here is to learn how to interpret the graph.

[37] For those readers with an electronics background, the concept of "characteristic curves" for a control valve is *exactly* the same as that of characteristic curves for transistors. Instead of plotting the amount of current a bipolar transistor will pass through its collector terminal (I_C) given varying amounts of collector-emitter voltage drop (V_{CE}), we are plotting the rate of water flow through the valve (Q) given varying amounts of supply pressure (ΔP).

We may use this set of characteristic curves to determine how this valve will respond in *any* installation by superimposing another curve on the graph called a *load line*[38], describing the pressure drop available to the valve at different flow rates. Since we know our hypothetical dam supplies a constant 20 PSI across the control valve for all flow conditions, the load line for the dam will be a vertical line at 20 PSI:



By noting the points of intersection[39] between the valve's characteristic curves and the load line, we may determine the flow rates from the dam at those stem positions:

Opening (%)	C_v	**Flow rate** (GPM)
0	0	0
25	4.5	20.1
50	9	40.2
75	13.5	60.4
100	18	80.5

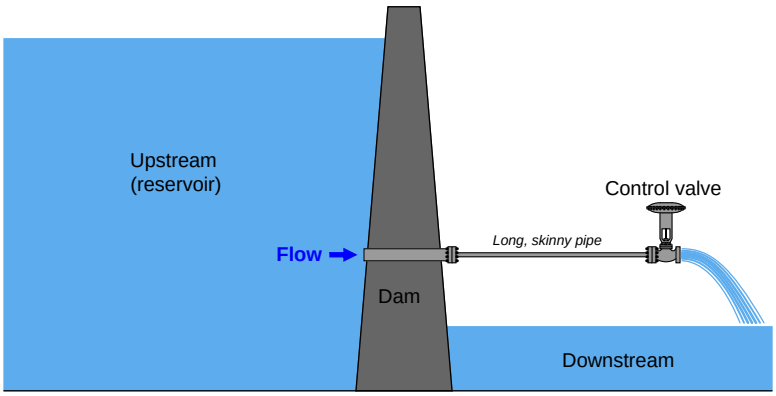
If we were to graph *this* table, plotting flow versus stem position, we would obtain a very linear graph. Note how 50% open gives us twice as much flow as 25% open, and 100% open nearly twice as much flow as 50% open. This tells us our control valve will respond linearly when operated under these conditions (i.e. operating with a constant pressure drop).

[38] Once again, the exact same concept applied in transistor circuit analysis finds application here in control valve behavior! The load line for a transistor circuit describes the amount of voltage available to the transistor under different current conditions, just like the load line here describes the amount of pressure available to the valve under different flow conditions.

[39] Load line plots are a graphical method of solving nonlinear, simultaneous equations. Since each curve represents a set of solutions to a particular equation, the intersection of two curves represents values uniquely satisfying both equations at the same time.

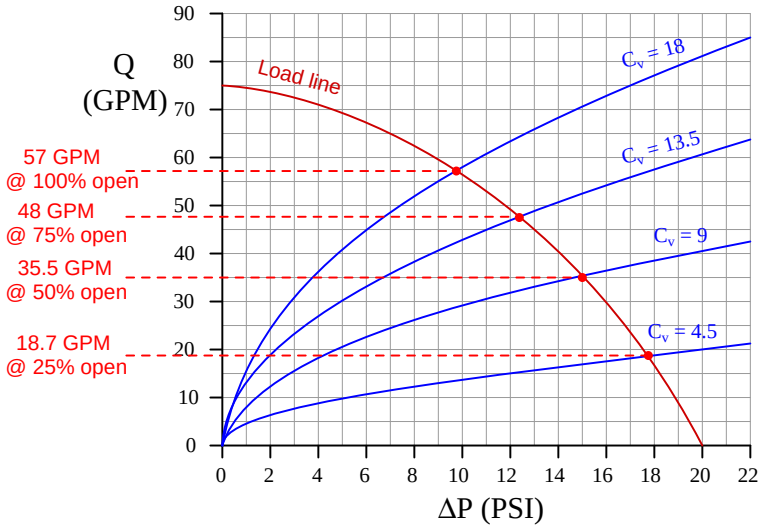
27.13.3 Control valve performance with varying pressure

Now let us consider a scenario where the pressure drop across the valve changes as the rate of flow through the valve changes. We may modify the previous example of a control valve releasing water from a dam to generate this effect. Suppose the valve is not closely coupled to the dam, but rather receives water through a narrow (restrictive) pipe:



In this installation, the narrow pipe generates a flow-dependent pressure drop due to friction between the turbulent water and the pipe walls, leaving less and less upstream pressure at the valve as flow increases. The control valve still drains to atmosphere, so its downstream pressure is still a constant 0 PSIG, but now its upstream pressure diminishes with increasing flow. How will this affect the valve's performance?

We may turn to the same set of characteristic curves to answer this question. All we need is a new load line describing the pressure available to the valve at different flow rates, then we may look for the points of intersection between this load line and the valve's characteristic curves. For the sake of our hypothetical example, I have sketched an arbitrary "load line" (actually a load *curve*) showing how the valve's pressure falls off as flow rises[40]:



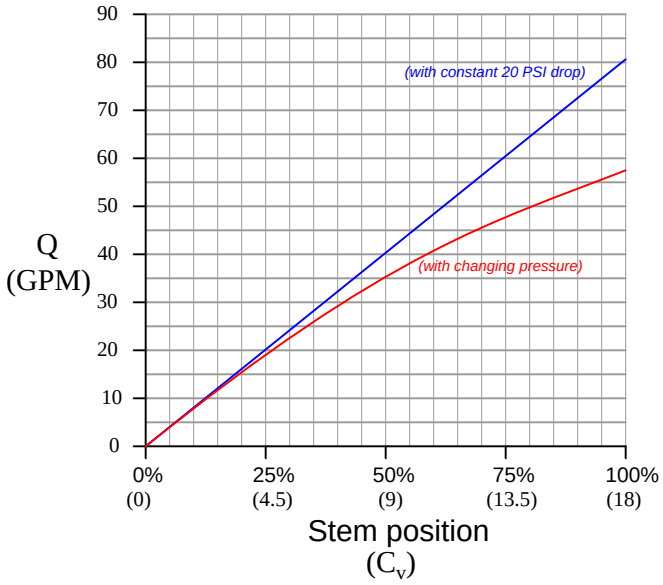
Now we see a definite nonlinearity in the control valve's behavior. No longer does a doubling of stem position (from 25% to 50%, or from 50% to 100%) result in a doubling of flow rate[41]:

Opening (%)	C_v	**Flow rate** (GPM)
0	0	0
25	4.5	18.7
50	9	35.5
75	13.5	48
100	18	57

[40] The precise determination of this curve is based on a model of the narrow pipe as a flow-restricting element, similar in behavior to an orifice, or to a control valve with a fixed stem position. Since pressure is dropped along the pipe's length as a function of turbulence (velocity), the load "line" curves for the exact reason the valve's own characteristic plots are curved: the relationship between fluid velocity and turbulent pressure loss is naturally non-linear.

[41] Not only is the response of the valve altered by this degradation of upstream pressure, but we can also see from the load line that a certain maximum flow rate has been asserted by the narrow pipe which did not previously exist: 75 GPM. Even if we unbolted the control valve from the pipe and let water gush freely into the atmosphere, the flow rate would saturate at only 75 GPM because that is the amount of flow where all 20 PSI of hydrostatic "head" is lost to friction in the pipe. Contrast this against the close-coupled scenario, where the load line was vertical on the graph, implying no theoretical limit to flow at all! With an absolutely constant upstream pressure, the only limit on flow rate was the maximum C_v of the valve (analogous to a perfect electrical voltage source with zero internal resistance, capable of sourcing any amount of current to a load).

If we plot the valve's performance in both scenarios (close-coupled to the dam, versus at the end of a restrictive pipe), we see the difference very clearly:



The "drooping" graph shows how the valve responds when it does not receive a constant pressure drop throughout the flow range. This is how the valve responds when *installed* in a non-ideal process, compared to the straight-line response it exhibits under *ideal* conditions of constant pressure. This what we mean by "installed" characteristic versus "ideal" or "inherent" characteristic.

Pressure losses due to fluid friction as it travels down pipe is just one cause of valve pressure changing with flow. Other causes exist as well, including pump curves[42] and frictional losses in other system components such as filters and heat exchangers. Whatever the cause, any piping system that fails to provide constant pressure across a control valve will "distort" the valve's inherent characteristic in the same "drooping" manner, and this must be compensated in some way if we desire linear response from the valve.

Not only does the diminishing pressure drop across the valve mean we cannot achieve the same full-open flow rate as in the laboratory (with a constant pressure drop), but it also means the control valve responds with different amounts of sensitivity at various points along its range. Note how the installed characteristic graph is relatively steep at the beginning where the valve is nearly closed, and how the graph grows "flatter" at the end where the valve is nearly full-open. The rate of response (rate-of-change of flow Q compared to stem position x, which may be expressed as the derivative $\frac{dQ}{dx}$) is much greater at low flow rates than it is at high flow rates, all due to diminished pressure

[42] The amount of fluid pressure output by any pump tends to vary with the fluid flow rate through the pump as well as the pump speed. This is especially true for centrifugal pumps, the most common pump design in process industries. Generally speaking, the discharge (output) pressure of a pump rises as flow rate decreases, and falls as flow rate increases. Variations in system fluid pressure caused by the pump constitutes one more variable for control valves to contend with.

drop at higher flow rates. This means the valve will respond more "sensitively" at the low end of its travel and more "sluggishly" at the high end of its travel.

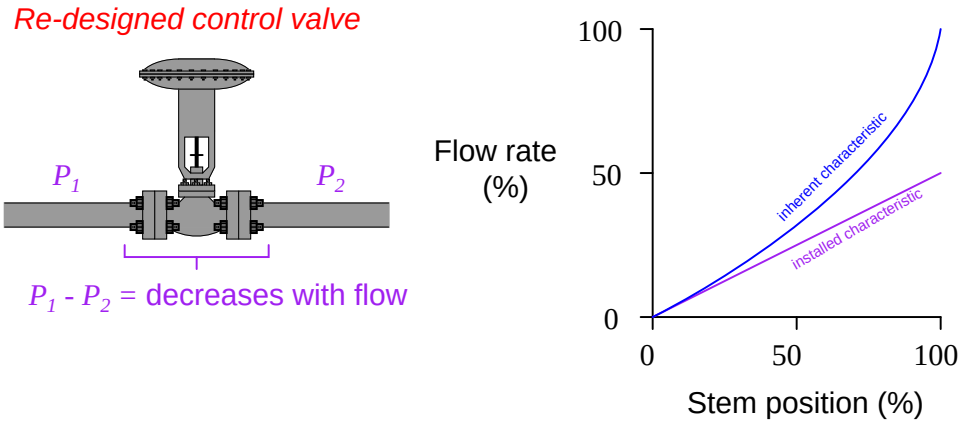
From the perspective of a feedback control system, this varying valve responsiveness means the system will be unstable at low flow rates and unresponsive at high flow rates. At low flow rates – where the valve is nearly closed – any small movement of the valve stem will have a relatively large effect on fluid flow. However, at high flow rates, a much greater stem motion will be required to achieve a comparable effect on fluid flow. Thus, the control system will tend to over-react at low flow rates and under-react at high flow rates, simply because the control valve fails to exert the same degree of control over process flow at different flow rates. Oscillations may occur at low flow rates, and excessive deviations from setpoint at high flow rates as a result of this "distorted" valve behavior.

27.13.4 Characterized valve trim

The root cause of the problem – a varying pressure drop caused by frictional losses in the piping and other factors – generally cannot be eliminated. This means there is no way to regain maximum flow capacity short of replacing the control valve with one having a greater C_v rating[43]. However, there is a clever way to flatten the valve's responsiveness to achieve a more linear characteristic, and that is to purposely design the valve such that its inherent characteristic complements the process "distortion" caused by changing pressure drop. In other words, we design the control valve trim so it opens up gradually during the initial stem travel (near the closed position), then opens up more aggressively during the final stages of stem travel (near the full-open position). With the valve made to open up in a nonlinear fashion inverse to the "droop" caused by the installed pressure changes, the two non-linearities should cancel each other and yield a more linear response.

[43] Even then, achieving the ideal maximum flow rate may be impossible. Our previous 100% flow rate for the valve was 80.5 GPM, but this goal has been rendered impossible by the narrow pipe, which according to the load line limits flow to an absolute maximum of 75 GPM (even with an infinitely large control valve).

This re-design will give the valve a nonlinear characteristic when tested in the laboratory with constant pressure drop, but the installed behavior should be more linear:



Now, control system response will be consistent at all points within the controlled flow range, which is a significant improvement over the original state of affairs.

Control valve trim is manufactured in a variety of different "characteristics" to provide the desired installed behavior. The two most common inherent characteristics are *linear* and *equal percentage*. "Linear" valve trim exhibits a fairly proportional relationship between valve stem travel and flow capacity (C_v), while "equal percentage" trim is decidedly nonlinear. A control valve with "linear" trim will exhibit consistent responsiveness only with a constant pressure drop, while "equal percentage" trim is designed to counter-act the "droop" caused by changing pressure drop when installed in a process system.

The C_v for linear and equal-percentage control valve trims are given by the following formulae[44]:

$$C_v = xC_{vm} \qquad \textbf{Linear trim}$$

$$C_v = C_{vm}R^{(x-1)} \qquad \textbf{Equal percentage trim}$$

Where,

C_v = Flow coefficient of control valve at stem position x

C_{vm} = Flow coefficient of control valve while wide-open ($x = 100\%$)

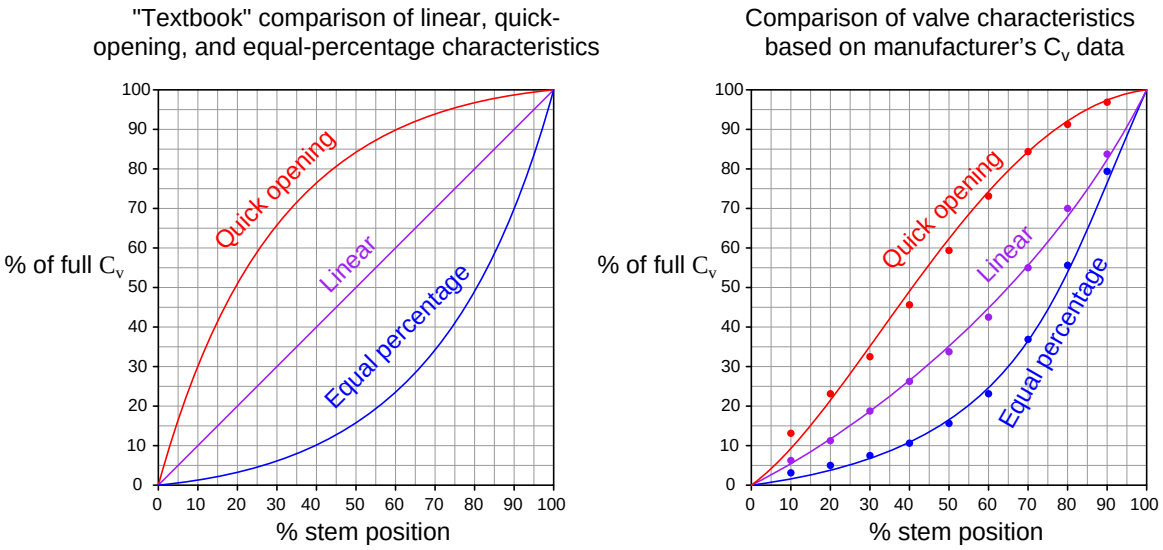
x = Stem position, as a *per unit* value (ranging from 0 to 1) inclusive

R = Rangeability coefficient of equal-percentage trim

[44] Note that the equal percentage formula given here can never achieve a C_v value of zero, regardless of stem position. This is untrue for real control valves, which of course achieve $C_v = 0$ when the stem is in the fully closed position. Therefore, the equal percentage formula shown here cannot be precisely trusted at small stem position values.

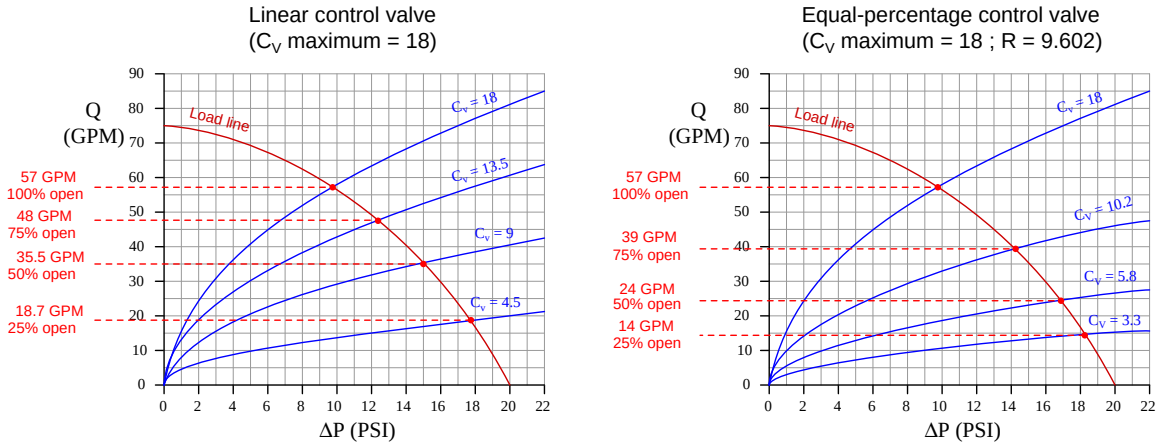
Another common inherent valve characteristic available from manufacturers is *quick-opening*, where the valve's C_v increases dramatically during the initial stages of opening, but then increases at a much slower rate for the rest of the travel. Quick-opening valves are often used in pressure-relief applications, where it is important to rapidly establish flow rate during the initial portions of valve stem travel.

The following pair of graphs show quick-opening, linear, and equal-percentage valve characteristics both as they are commonly presented in textbooks as well as based on real[45] control valve data from manufacturer's datasheets:



[45] Data for the three graphs were derived from actual C_v factors published in Fisher's ED, EAD, and EDR sliding-stem control valve product bulletin (51.1:ED). I did not copy the exact data, however; I "normalized" the data so all three valves would have the exact same full-open C_v rating of 50.

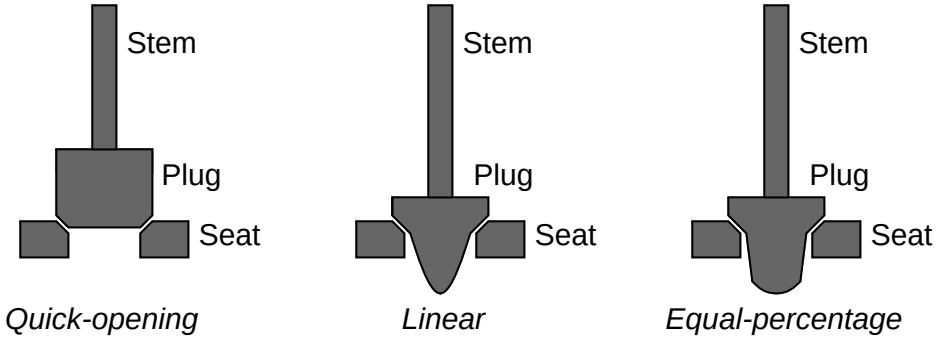
When we compare the performance of an equal-percentage control valve against the linear control valve from the previous scenario (where water flowed from a dam through a long, narrow pipe) using the "load line" plot to determine flow rates, we see that the equal-percentage valve yields a more linear installed response than the inherently linear valve. You can see how the blue curves on these graphs (representing each control valve's C_v at 25%, 50%, 75%, and 100% stem positions) are identical only at the wide-open position and differ at all other positions:



Stem position (%)	**Flow rate through linear valve** (GPM)	**Flow rate through equal-percent valve** (GPM)	**Ideal flow rate** (GPM)
0	0	0	0
25	18.7 *(+4.45% from ideal)*	14 *(−0.25% from ideal)*	14.25
50	35.5 *(+7% from ideal)*	24 *(−4.5% from ideal)*	28.5
75	48 *(+5.25% from ideal)*	39 *(−3.75% from ideal)*	42.75
100	57	57	57

Note that the equal-percentage valve with a maximum C_v of 18 does not yield any greater water flow rate at full-open than the linear valve with the same maximum C_v. No amount or type of valve characterization can make up for the pressure lost in restrictive piping. What equal-percentage characterization *does* accomplish is to make the relationship between flow rate and stem position closer to linear than it would be otherwise.

Different valve characterizations are be achieved by different valve trim shapes. For instance, the plug profiles of a single-ported, stem-guided globe valve may be modified to achieve the common quick-opening, linear, and equal-percentage characteristics:



Photographs of linear (left) and equal-percentage (right) globe valve plugs having the same port size are shown side-by-side for comparison:

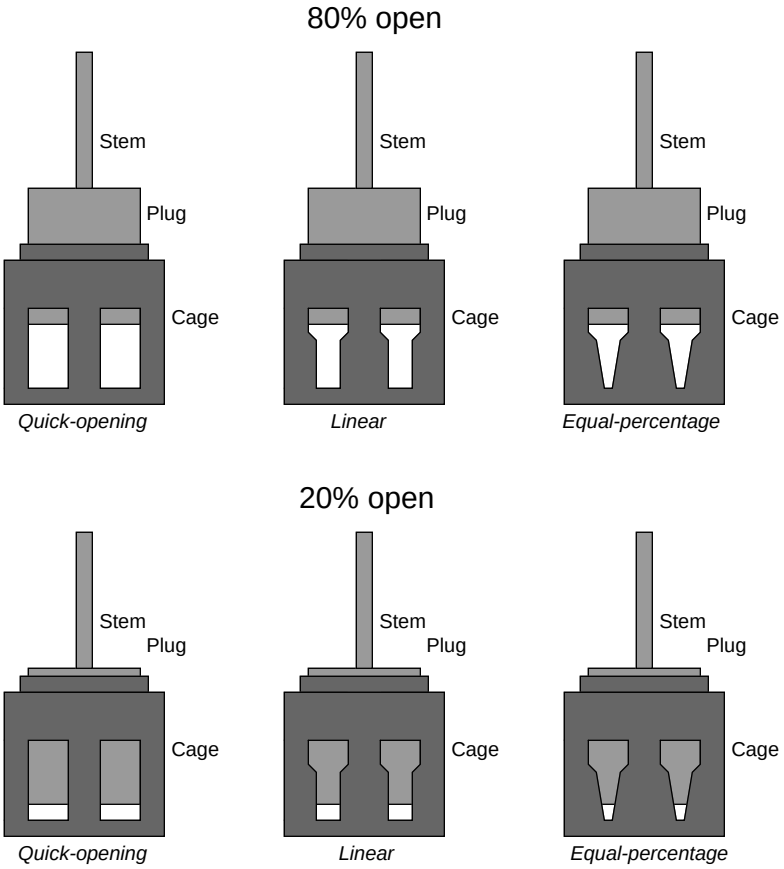




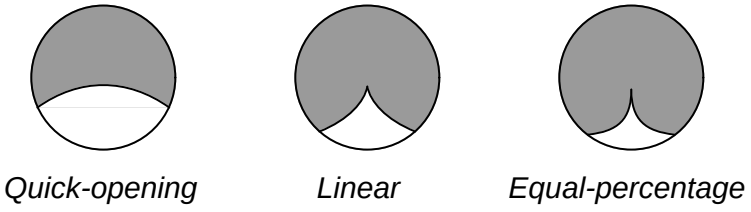
It should be clear[46] how the equal-percentage plug on the right-hand side retains more of its width along its length than the linear plug on the left-hand side. This means the equal-percentage plug is more restrictive than the linear plug for a greater portion of its withdrawal out of the seat. As each plug is drawn out of the seat's port by the actuator motion, the linear plug "opens up" more aggressively than the equal-percentage plug, even though both plugs are equally open when drawn fully out of the seat's port.

[46] Astute readers will also note how the stem diameter of the left-hand (linear) plug is significantly greater than the stem diameter of the right-hand (equal-percentage) plug. This has nothing to do with characterization, and is simply an irrelevant difference between the two plugs. The truth of the matter is these were the only two valve plugs I had on hand suitable for illustrating the difference between linear and equal-percentage trim. One just happened to have a thicker stem than the other.

Cage-guided globe valve trim characteristic is a function of port shape. As the plug rises up, the amount of port area uncovered determines the shape of the characteristic graph:



Ball valve trim characteristic is a function of notch shape. As the ball rotates, the amount of notch area opened to the fluid determines the shape of the characteristic graph. All valve trim in the following illustration is shown approximately half-open (50% stem rotation):



A different approach to valve characterization is to use a non-linear positioner function instead of a non-linear trim. That is, by "programming" a valve positioner to respond in a characterized fashion to command signals, it is possible to make an inherently linear valve behave as though it

were quick-opening, equal-percentage, or anywhere in between. All the positioner does is modify the valve stem position as per the desired characteristic function instead of proportionally follow the signal as it normally would.

This approach has the distinct advantage of convenience (especially if the valve is already equipped with a positioner) over changing the actual valve trim. However, if valve stem friction ever becomes a problem, its effects will be disproportionate along the valve travel range, as the positioner must position the valve more precisely in some areas of travel than others when pressed into service as a characterizer.

It should be noted that not all process control loops benefit from control valves with equal-percentage inherent characteristics. There are some process applications, for example, where the pressure drop across a control valve holds relatively constant over a wide range of valve flow rates[47], in which case an inherently linear control valve will indeed yield a linear installed characteristic. In other words, if the pressure drop across the control valve never changes much, what we have is an "ideal" scenario (i.e. no "droop" or "distortion") that doesn't need special valve characterization to behave linearly.

Some other process applications actually work quite well[48] with a quick-opening installed valve characteristic, in which case an inherently linear valve with varying pressure drop at different flow rates suffices. Ultimately, what matters is the valve stem position's effect on the process variable and whether that relationship is consistent (linear) over a wide range of operating conditions.

27.14 Control valve problems

Control valves are subject to a number of common problems. This section is dedicated to an exploration of the more common control valve problems, and potential remedies.

[47] Such applications are typically found when the purpose of the control valve is to regulate process fluid pressure. Consider, for example, a control valve regulating upstream gas pressure in a vessel by venting gas from that vessel to atmosphere. In such an application, the valve's upstream pressure (P_1) will be nearly constant due to the control loop's action, and the valve's downstream pressure (P_2) will be constant due to it being atmospheric pressure.

[48] An example of such a process is temperature control through a heat exchanger where the controlled fluid flow regime happens to transition from laminar to turbulent as the control valve opens further: at low stem positions (nearly shut) where the flow is laminar and heat transfer is impeded, large changes in flow rate may be necessary to effect modest changes in temperature; at high stem positions (nearly open) where the flow is turbulent and heat transfer is efficient, only small changes in flow rate are necessary to create modest changes in temperature. In such an application a quick-opening installed characteristic may actually yield more consistent behavior than a linear installed characteristic.

27.14.1 Mechanical friction

Control valves are mechanical devices with moving parts, and as such they are subject to *friction*, primarily between the valve stem and the stem packing. Some degree of friction is inevitable in valve packing[49] and in some types of trim where components must move past each other throughout the full range of valve stem travel (e.g. cage-guided globe valves, rotary ball valves) the trim itself adds additional friction beyond that which is imposed by the packing.

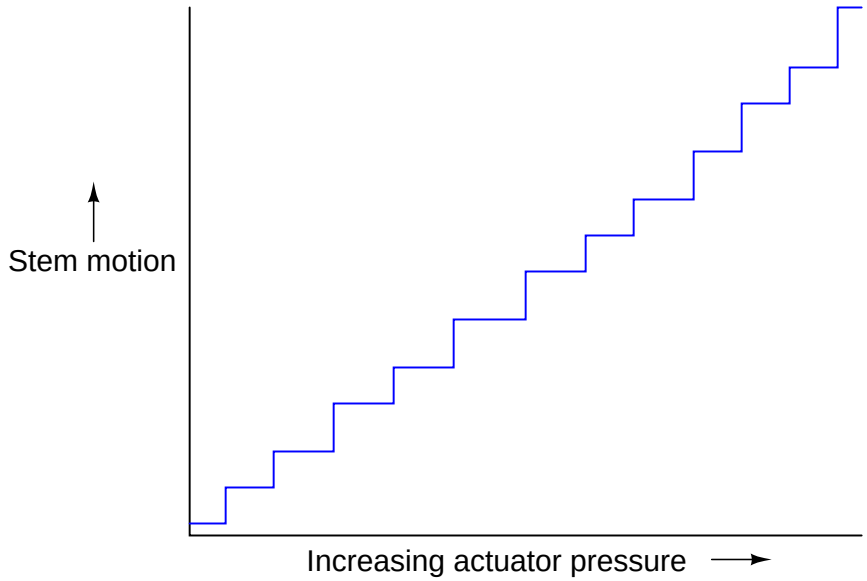
In physics, friction is classified as either *static* or *dynamic*. Static friction is defined as frictional force holding two stationary objects together. Dynamic friction is defined as frictional force impeding the motion of two objects sliding past each other. Static friction is always greater in magnitude than dynamic friction. Anyone who has ever pulled a sled through snow or ice knows that more force is required to "break" the sled loose from a stand-still (static friction) than is required to keep it moving (dynamic friction). The same holds true for friction in a control valve: the amount of force required to initially overcome static friction (i.e. initiate valve stem motion) usually exceeds the amount of force required to maintain the valve stem in motion.

The presence of friction in a control valve increases the force necessary from the actuator to cause valve movement. If the actuator is electric or hydraulic, the only real problem with increased force is the additional energy required from the actuator to move the valve (recall that mechanical *work* is the product of force and parallel displacement). If the actuator is pneumatic, however, a more serious problem arises from the combined effects of static and dynamic friction.

A simple "thought experiment" illustrates the problem. Imagine an air-to-open, sliding-stem control valve with the lower bench-set pressure applied to the pneumatic actuator. This should be the amount of pressure where the valve is just about to open from a fully-closed position. Now imagine slowly increasing the air pressure applied to the actuator. What should this valve do? If the spring tension is set properly, and there is negligible friction in the valve, the stem should smoothly rise from the fully-closed position as pressure increases beyond the bench-set pressure. However, what will this valve do if there is substantial friction present in the valve assembly? Instead of the stem smoothly lifting immediately as pressure exceeds the bench-set value, this valve will remain fully closed until enough *extra* pressure has accumulated in the actuator to generate a force large enough to overcome spring tension *plus* valve friction. Then, once the stem "breaks free" from static friction and begins to move, the stem will begin to accelerate because the actuator force now *exceeds* the sum of spring tension and friction, since dynamic friction is less than static friction. Compressed air trapped inside the actuator acts like a spring of its own, releasing stored energy. As the stem moves, however, the chamber volume in the diaphragm or piston actuator increases, causing pressure to drop, which causes the actuating force to decrease. When the force decreases sufficiently, the stem stops moving and static friction "grabs" it again. The stem will remain stationary until the applied pressure increases sufficiently again to overcome static friction, then the "slip-stick" cycle repeats.

[49] Bellows seals are theoretically frictionless, but in practice bellows seals are almost always combined with standard packing to prevent catastrophic blow-out in the event of the bellows rupturing, and so the theoretical advantage of low friction is never realized.

If we graph the mechanical response of a pneumatic actuator with substantial stem friction, we see something like this:



What should be a straight, smooth line is reduced to a series of "stair-steps" as the combined effect of static and dynamic friction, plus the dynamic effects of a pneumatic actuator, conspire to make precise stem positioning nearly impossible. This effect is commonly referred to as *stiction*.

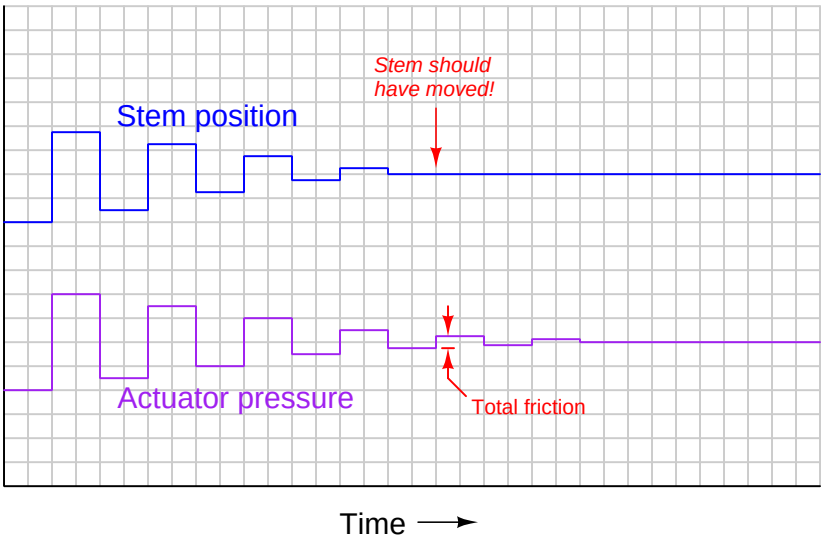
Even worse is the effect friction has on valve position when we *reverse* the direction of pressure change. Suppose that after we have reached some new valve position in the opening direction, we begin to ramp the pneumatic pressure downward. Due to static friction (again), the valve will *not* immediately respond by moving in the closed direction. Instead, it will hold still until enough pressure has been released to diminish actuator force to the point where there is enough unbalanced spring force to overcome static friction in the downward direction. Once this static friction is overcome, the stem will begin to accelerate downward because (lesser) dynamic friction will have replaced (greater) static friction. As the stem moves, however, air volume inside the actuating diaphragm or piston chamber will decrease, causing the contained air pressure to rise. Once this pressure rises enough that the stem stops moving downward, static friction will again "grab" the stem and hold it still until enough of a pressure change is applied to the actuator to overcome static friction.

What may not be immediately apparent in this second "thought experiment" is the amount of pressure change required to cause a reversal in stem motion compared to the amount of pressure change required to provoke continued stem motion in the same direction. In order to reverse the direction of stem motion, not only does the static friction have to be "relaxed" from the last movement, but additional static friction must be overcome in the opposite direction before the stem is able to move that way. We may clarify this concept by applying the problem-solving strategy of adding numerical quantities to the thought experiment. Suppose we have a control valve with a static packing friction of 50 pounds in either direction, and a diaphragm with a 12-inch diameter (113.1 square inches of area). According to the force/pressure/area formula ($F = PA$), an applied

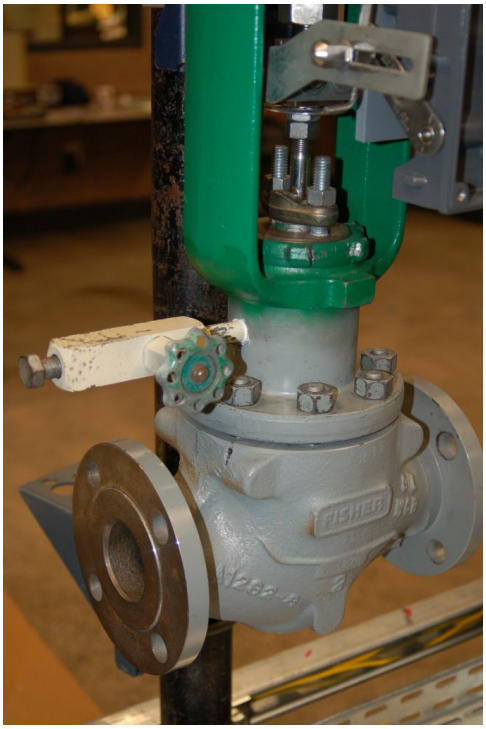
pressure of 0.442 PSI will be required to overcome this static friction in either direction. That is to say, when moving in the upward direction, we must apply 0.442 PSI *more* pressure than is ideally required to achieve any given valve position. If after achieving this stem position we desire to move the valve downward to some new stem position, we must not only decrease the pressure by 0.442 PSI to relax the tension on the packing, but we must also decrease it *another 0.442 PSI* to overcome packing friction in the downward direction before the valve moves at all from its last position. Thus, a pressure reversal of 0.884 PSI (i.e. *twice* the equivalent value of packing friction) is required to make the valve reverse its direction of motion. This constitutes "deadband" in the control valve's action, which degrades control behavior.

Thus, the effects of friction on a pneumatic control valve actuator may be quantified by subjecting the valve to small reversals in applied actuator pressure and measuring the resulting stem position. The largest increment of actuator pressure reversal resulting in zero stem motion represents the total amount of friction within the valve mechanism.

The following graph shows such a test, plotting actuator pressure over time as well as valve stem position over time. As the actuator pressure is stepped up and down in successively smaller intervals, the control valve's stem position is seen to respond with less and less motion until it fails to respond at all:



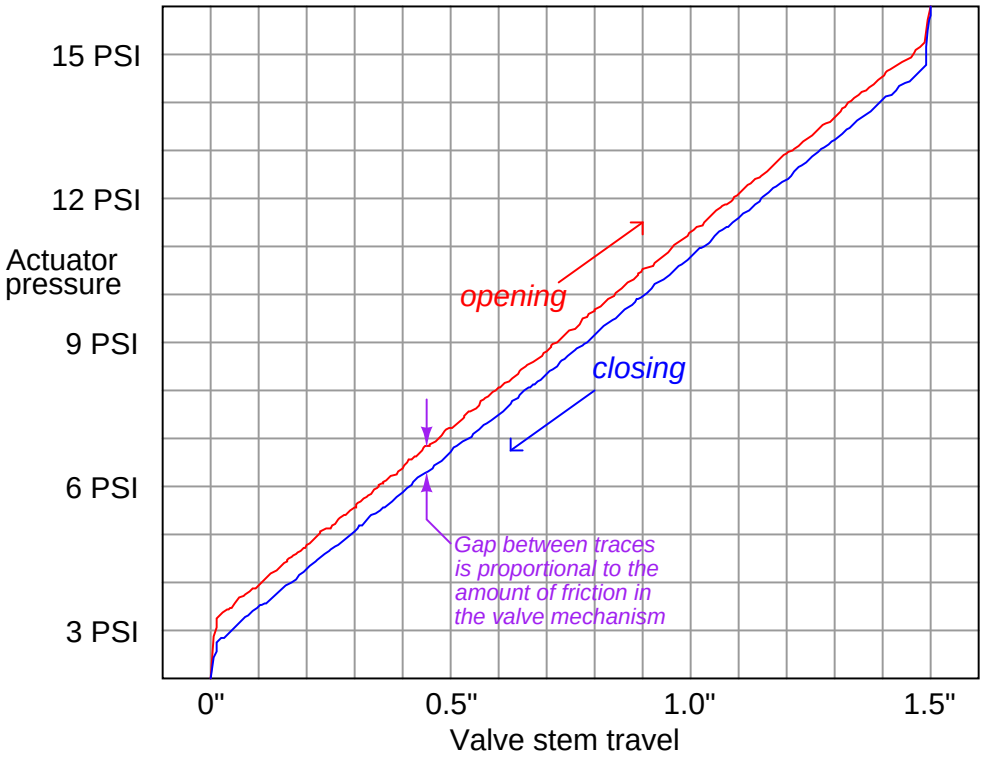
Short of rebuilding a "sticky" control valve to replace damaged or worn components, there is not much that may be done to improve valve stiction other than regular lubrication of the packing (if appropriate). Lubrication is applied to the packing by means of a special *lubricator* device threaded into the bonnet of the valve:



As one of the common sources of excessive packing friction is over-tightening of the packing nuts by maintenance personnel eager to prevent process fluid leaks, a great deal of trouble may be avoided simply by educating the maintenance staff on the "care and feeding" of control valve packing for long service life.

Many modern digital valve positioners have the ability to monitor the drive force applied by an actuator on a valve stem, and correlate that force against stem motion. Consequently, it is possible to perform highly informative diagnostic tests on a control valve's mechanical "health," at least with regard to friction[50]. For pneumatic and hydraulic actuators, actuator force is a simple and direct function of fluid pressure applied to the piston or diaphragm. For electric actuators, actuator force is an indirect function of electric motor current, or may be directly measured using load cells or springs and displacement sensors in the gear mechanism.

The following *valve signature* illustrates the kind of diagnostic "audit" that may be obtained from a digital control valve positioner based on actuator force (pneumatic air pressure) and stem motion:



This same diagnostic tool is useful for detecting trim seating problems in valve designs where there is sliding contact between the throttling element and the seat near the position of full closure (e.g. gate valves, ball valves, butterfly valves, plug valves, etc.). The force required to "seat" the valve into the fully-closed position will naturally be greater than the force required to move the throttling element during the rest of its travel, but this additional force should be smooth and consistent on the graph. A "jagged" force/travel graph near the fully-closed position indicates

[50] Other measures of a control valve's mechanical status, such as flow capacity, flow characterization, and seat shut-off, cannot be inferred from measurements of actuator force and stem position.

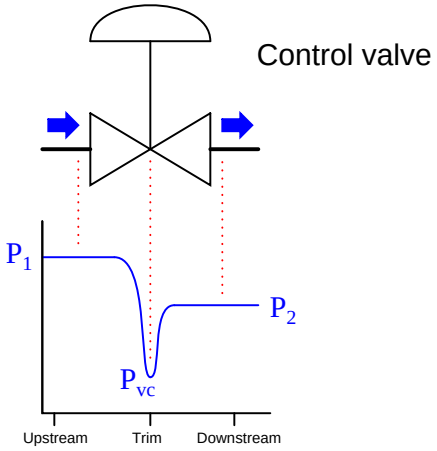
interference between the moving element and the stationary seat, providing information valuable for predicting the remaining service life of the valve before the next rebuild.

27.14.2 Flashing

When a fluid passes through the constrictive passageways of a control valve, its average velocity increases. This is predicted by the Law of Continuity, which states that the product of fluid density (ρ), cross-sectional area of flow (A), and average velocity ($\overline{v}$) must remain constant for any flowstream:

$$\rho_1 A_1 \overline{v_1} = \rho_2 A_2 \overline{v_2}$$

As fluid velocity increases through the constrictive passages of a control valve, the fluid molecules' kinetic energy increases. In accordance with the Law of Energy Conservation, potential energy in the form of fluid pressure must decrease correspondingly. Thus, fluid pressure decreases within the constriction of a control valve's trim as it throttles the flow, then increases (recovers) after leaving the constrictive passageways of the trim and entering the wider areas of the valve body:



If the fluid being throttled by the valve is a liquid (as opposed to a gas or vapor), and its absolute pressure ever falls below the vapor pressure[51] of that substance, the liquid will begin to boil. This phenomenon, when it happens inside a control valve, is called *flashing*. As the graph shows, the point of lowest pressure inside the valve (called the *vena contracta* pressure, or P_{vc}) is the location where flashing will first occur, if it occurs at all.

Flashing is almost universally undesirable in control valves. The effect of boiling liquid at the point of maximum constriction is that flow through the valve becomes "choked" by the rapid expansion of liquid to vapor as it boils, degrading the valve's flow capacity (i.e. decreasing the effective C_v). Flashing is also destructive to the valve trim, as boiling action propels tiny droplets of liquid at extremely high velocities past the plug and seat faces, eroding the metal over time.

[51] It should be noted that vapor pressure is a strong function of *temperature*. The warmer a liquid is, the more vapor pressure it will exhibit and thus the more prone it will be to flashing within a control valve.

A photograph showing a severely eroded valve plug (from a cage-guided globe valve) reveals just how destructive flashing can be:



A characteristic effect of flashing in a control valve is a "hissing" sound, reminiscent of what *sand* might sound like if it were flowing through the valve.

An important parameter predicting flashing in a control valve is the valve's *pressure recovery factor*, based on a comparison of the valve's total pressure drop from inlet to outlet versus the pressure drop from inlet to the point of minimum pressure within the valve.

$$F_L = \sqrt{\frac{P_1 - P_2}{P_1 - P_{vc}}}$$

Where,

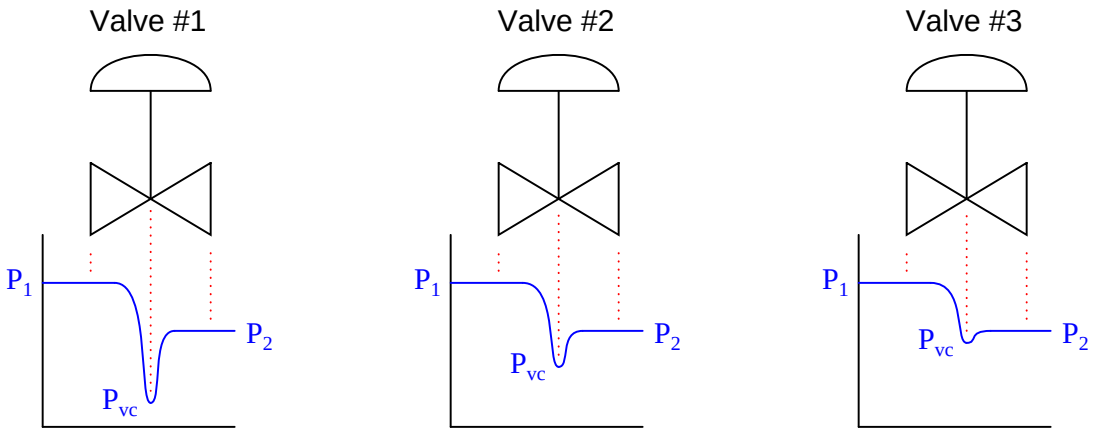
F_L = Pressure recovery factor (unitless)

P_1 = Absolute fluid pressure upstream of the valve

P_2 = Absolute fluid pressure downstream of the valve

P_{vc} = Absolute fluid pressure at the *vena contracta* (point of minimum fluid pressure within the valve)

The following set of illustrations shows three different control valves exhibiting the same permanent pressure drop ($P_1 - P_2$), but having different values of F_L:

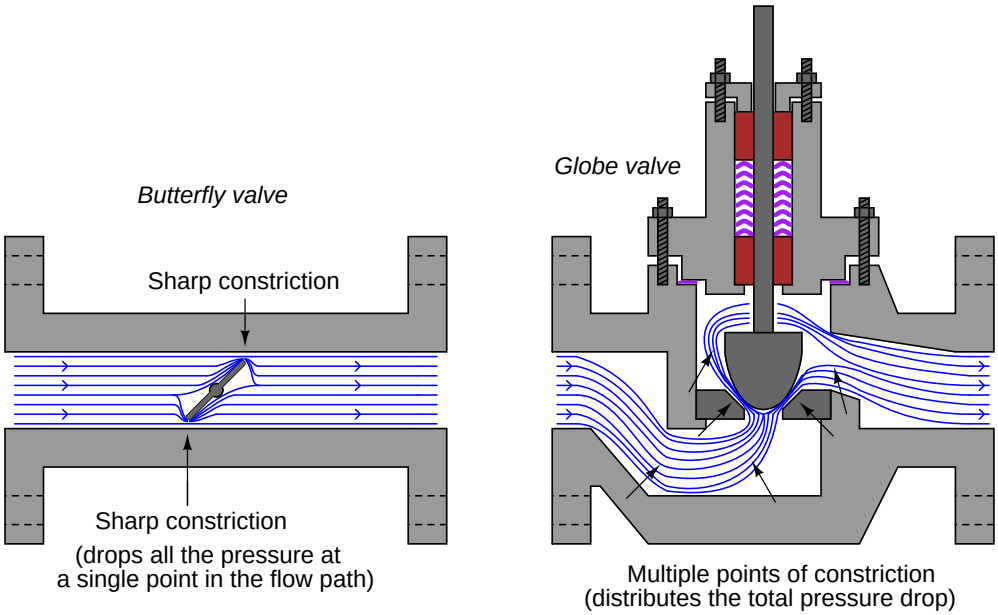


Valve #1 exhibits the greatest pressure recovery (i.e. the amount that fluid pressure *increases* from the minimum pressure at the vena contracta to the downstream pressure: $P_2 - P_{vc}$) and the lowest F_L value. It is also the valve most prone to flashing in liquid service, because the vena contracta pressure is so much lower (all other factors being equal) than in the other two valves. If any of these valves will experience flashing in liquid service, it would be valve #1.

Valve #3, by contrast, has very little pressure recovery, and a large F_L value (nearly equal to 1). From the perspective of avoiding flashing, it is the best of the three valves to use for liquid service.

The style of valve (ball, butterfly, globe, etc.) is very influential on pressure recovery factor. The more convoluted the path for fluid within a control valve, the more opportunities that fluid will have to dissipate energy in turbulent motion, resulting in the greatest permanent pressure drop for the least amount of restriction at any single point in the flow's path.

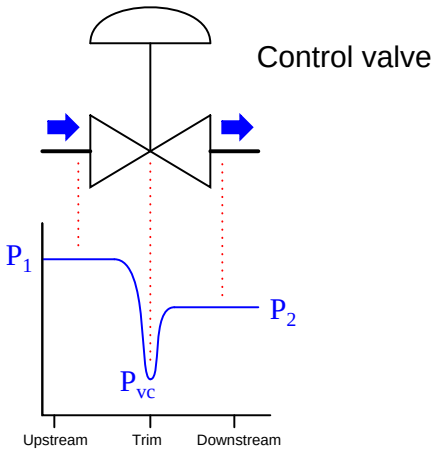
Compare these two styles of valve to see which will have lowest pressure recovery factor and therefore be most prone to flashing:



Clearly, the globe valve does a better job of evenly distributing pressure losses throughout the path of flow. By contrast, the butterfly valve can only drop pressure at the points of constriction between the disk and the valve body, because the rest of the valve body is a straight-through path for fluid offering little restriction at all. As a consequence, the butterfly valve experiences a much lower vena contracta pressure (i.e. greater pressure recovery, and a lower F_L value) than the globe valve for any given amount of permanent pressure loss, making the butterfly valve more prone to flashing than the globe valve with all other factors being equal.

27.14.3 Cavitation

Fluid passing through a control valve experiences changes in velocity as it enters the narrow constriction of the valve trim (increasing velocity) then enters the widening area of the valve body downstream of the trim (decreasing velocity). These changes in velocity result in the fluid molecules' kinetic energies changing as well, in accordance with the kinetic energy equation $E_k = \frac{1}{2}mv^2$. In order that energy be conserved in a moving fluid stream, any increase in kinetic energy due to increased velocity must be accompanied by a complementary decrease in potential energy, usually in the form of fluid pressure. This means the fluid's pressure will fall at the point of maximum constriction in the valve (the *vena contracta*, at the point where the trim throttles the flow) and rise again (or *recover*) downstream of the trim:



If fluid being throttled is a liquid, and the pressure at the vena contracta is less than the vapor pressure of that liquid at the flowing temperature, the liquid will spontaneously boil. This is the phenomenon of *flashing* previously described. If, however, the pressure recovers to a point greater than the vapor pressure of the liquid, the vapor will re-condense back into liquid again. This is called *cavitation*.

As destructive as flashing is to a control valve, cavitation is worse. When vapor bubbles re-condense into liquid they often do so asymmetrically, one side of the bubble collapsing before the rest of the bubble. This has the effect of translating the kinetic energy of the bubble's collapse into a high-speed "jet" of liquid in the direction of the asymmetrical collapse. These liquid "microjets" have been experimentally measured at speeds up to 100 meters per second (over 320 feet per second). What is more, the pressure applied to the surface of control valve components in the path of these microjets is intense. Each microjet strikes the valve component surface over a very small surface area, resulting in a very high pressure ($P = \frac{F}{A}$) applied to that small area. Pressure estimates as high as 1500 newtons per square millimeter (1.5 *giga*-pascals, or about 220000 PSI!) have been calculated for cavitating control valve applications involving water.

No substance known is able to continuously withstand this form of abuse, meaning that cavitation *will destroy* any control valve given enough time. The effect of each microjet impinging on a metal surface is to carve out a small pocket in that metal surface. Over time, the metal will begin to take on a "pock-marked" look over the area where cavitation occurs. This stands in stark contrast to the visual appearance of flashing damage, which is smooth and polished.

Photographs of a fluted valve plug and its matching seat are shown here as evidence of flashing and cavitation damage, respectively:





The plug of this valve has been severely worn by flashing and cavitation. The flashing damage is responsible for the relatively smooth wear areas seen on the plug. Cavitation damage is most prominent inside the seat, where almost all the damage is in the form of pitting. The mouth of the seat exhibits smooth wear caused by flashing, but deeper inside you can see the pock-marked surface characteristic of cavitation, where liquid microjets literally blasted away pieces of metal. This trim set belongs to a Fisher Micro-Flat Cavitation valve, designed with process liquid flow passing down instead of up (i.e. first past the wide body of the plug and then down through the seat, rather than first up through the seat and then past the wide body of the plug). This trim design does not prevent cavitation (as clearly evidenced by the photos), but it does "move" the area of cavitation damage down below the seat's sealing surface into a long tube extending below the seat. Although the ravages of flashing clearly took their toll on this valve's trim, the valve would have been rendered inoperable much sooner had cavitation been at work along the plug's length and at the sealing area where the plug contacts the seat.

The sound made by substantial liquid cavitation also contrasts starkly against the sound made by flashing. Whereas flashing sounds as though sand were flowing through the valve, cavitation produces a much louder "crackling" sound comprised of distinct impact pulses, reminiscent of what *gravel* or *rocks* might sound like if they were somehow forced to flow through the valve.

Sustained cavitation also has the detrimental effect of accelerating corrosion in certain process services. Bare metal surfaces are highly reactive with many chemical fluids, but become more resistant to further attack when a thin layer of reacted metal on the surface (the so-called *passivation layer*) acts as a sort of chemical barrier. Rust on steel, or the powdery-white oxide of aluminum are good examples: the initially bare metal surfaces react with their surrounding environment to form a protective outer layer, impeding further degradation of the metal beneath that layer. Cavitation works to blast away any protective layer that might otherwise accumulate, allowing corrosion to work at full speed until the entire thickness of the metal is corroded through. The

complementary destructive actions of cavitation and corrosion together is sometimes referred to as *cavitation corrosion*.

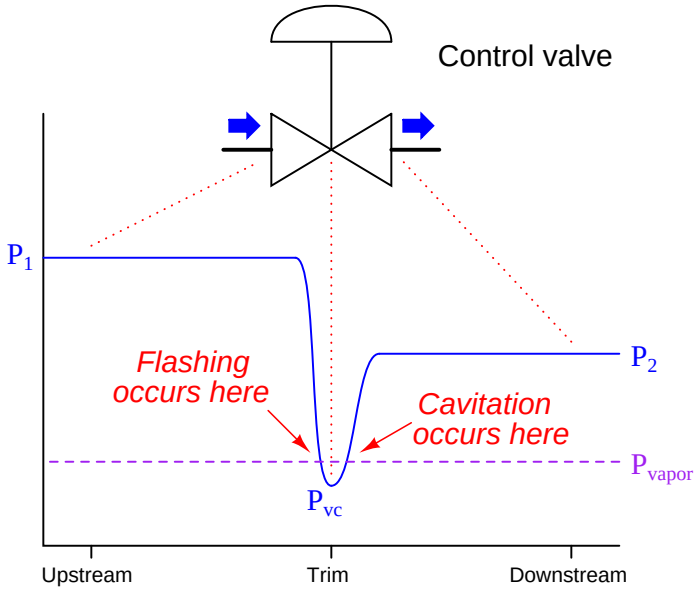
Several methods exist for abating cavitation in control valves:

1. Prevent flashing in the first place
2. Cushion with introduced gas
3. Sustain the flashing action (i.e. delay cavitation)

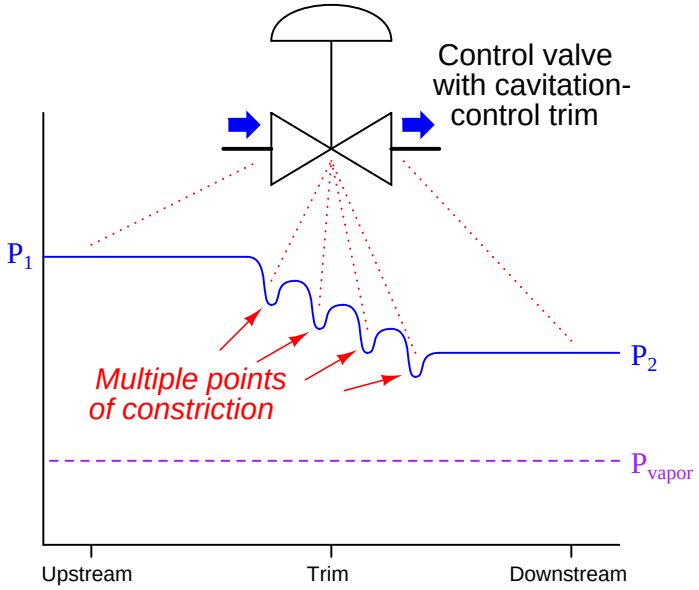
Cavitation abatement method #1 is quite simple to understand: if we prevent flashing from ever happening in a control valve, cavitation cannot follow. The key to doing this is making sure the vena contracta pressure never falls below the vapor pressure for the liquid. Several techniques exist for doing this:

- Select a control valve type having less pressure recovery (i.e. greater F_L value)
- Increase both upstream and downstream pressures by relocating the valve to a higher-pressure location in the process.
- Use multiple control valves in series to reduce the lowest pressure at either one
- Decrease the liquid's temperature (this decreases vapor pressure)
- Use cavitation-control valve trim

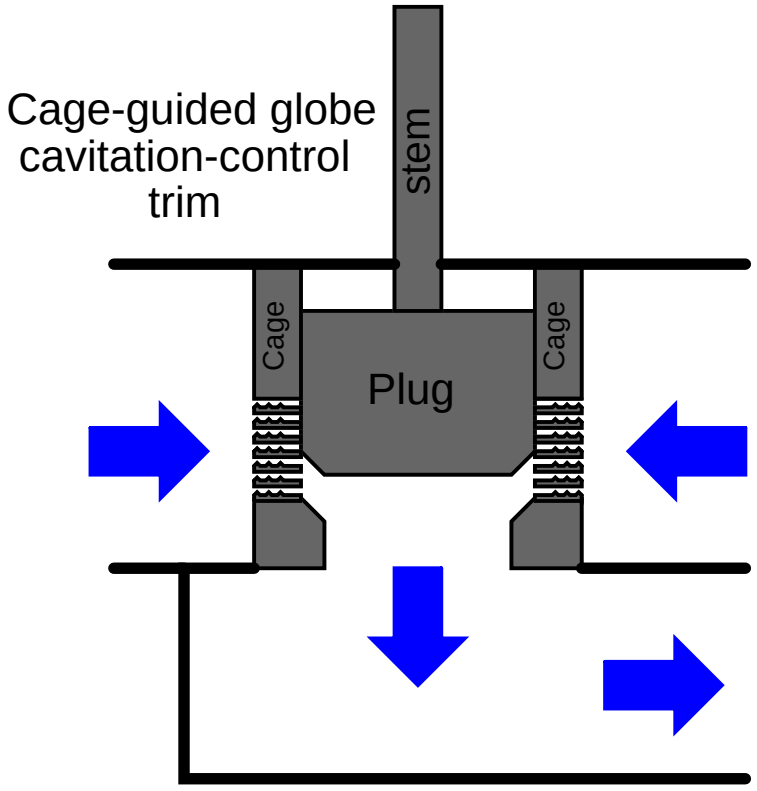
The last suggestion in this list deserves further exploration. Valve trim may be specially designed for cavitation abatement by providing multiple stages of pressure drop for the fluid as it passes through the trim. The following is a pressure versus location graph for a cavitating control valve. The liquid's vapor pressure is shown here as a dashed line marked P_{vapor}:



A valve equipped with cavitation-control trim will have a different pressure profile, with multiple *vena contracta* points where the fluid passes through a series of constrictions within the trim itself:



This way, the same final permanent pressure drop ($P_1 - P_2$) may be achieved without the lowest pressure ever falling below the liquid's vapor pressure limit. An example of cavitation-control design applied to cage-guided globe valve trim is shown in the following illustration:



Ball-style control valves, with their relatively high pressure recovery (low pressure recovery factor F_L values) are more prone to cavitation than globe valves, all other factors being equal. Special ball trim designed to help distribute pressure drops over a longer flow path is available, an example of this shown in the next photograph:

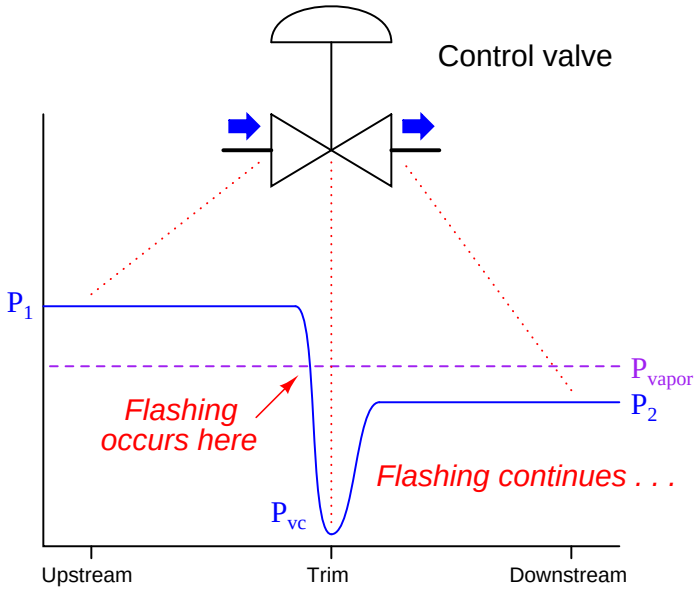


The round (ball-shaped) portion of the trim is on the far side of this piece, with the cavitation-controlling structure visible in the foreground. Fluid flow passing through the gap between the ball's edge and the valve seat spills into this multi-chambered structure where turbulence helps develop pressure drops at several locations. In a normal ball valve, there is only one location for any substantial pressure drop to develop, and that is at the narrow gap between the ball's edge and the seat. Here, multiple regions of pressure drop exist, with the intent of avoiding the liquid's vapor pressure limit at any one location, thus eliminating flashing and consequently eliminating cavitation.

Cavitation abatement method #2 is practical only in some process applications, where a non-reacting gas may be injected into the liquid stream to provide some "cushioning" within the cavitating region. The presence of non-condensible gas bubbles in the liquid stream disturbs the microjets' pathways, helping to dissipate their energy before striking the valve body walls.

Cavitation abatement method #3 involves a strategy opposite that of method #1. If, for whatever reason, we cannot avoid falling below the vapor pressure of the liquid as the flow stream moves through the valve, we may have the option of ensuring the downstream liquid pressure never rises above the liquid's vapor pressure, at least until the fluid clears past the valuable control valve and into an area of the system where cavitation damage will not be so expensive. This avoids cavitation at the cost of guaranteed flashing within the control valve, which is generally not as destructive as cavitation.

A pressure diagram shows how this method works:



Of course, flashing is not good for a control valve either. Not only does it damage the valve over time, but it also causes problems with flow capacity, as we will explore next.

27.14.4 Choked flow

Both gas and liquid control valves may experience what is generally known as *choked flow*. Simply put, "choked flow" is a condition where the rate of flow through a valve does not change substantially as downstream pressure is reduced.

Ideally, turbulent fluid flow rate through a control valve is a simple function of valve flow capacity (C_v) and differential pressure drop ($P_1 - P_2$), as described by the basic valve flow equation:

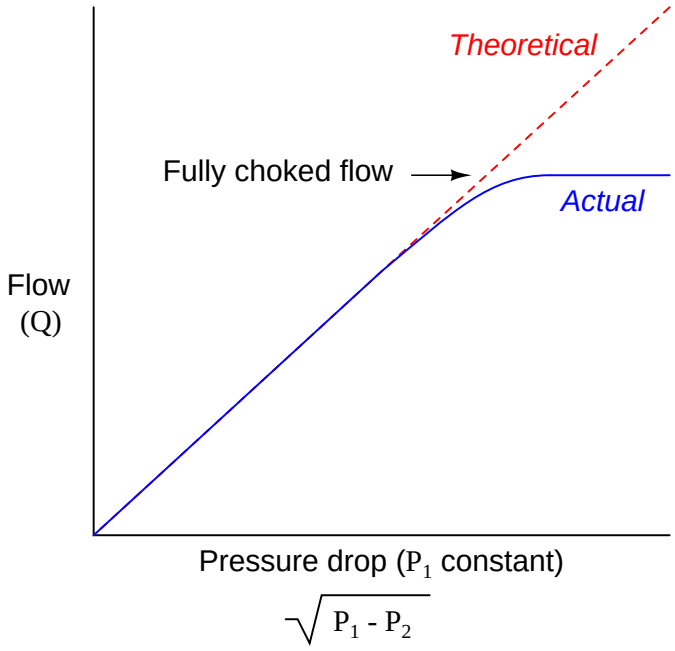
$$Q = C_v \sqrt{\frac{P_1 - P_2}{G_f}}$$

This equation simply does not apply for choked-flow conditions.

In a gas control valve, choking occurs when the velocity of the gas reaches the speed of sound for that gas. This is often referred to as *critical* or *sonic* flow. In a liquid control valve, choking occurs with the onset of flashing[52]. The reason sonic velocity is relevant to flow capacity for a control valve has to do with the propagation of pressure changes in fluids. Pascal's principle tells us that changes in pressure within a closed fluid system will manifest at all points in the fluid system, but this never happens instantaneously. Instead, pressure changes propagate through any fluid at the speed of sound within that fluid. If a fluid stream happens to move at or above the speed of sound, pressure changes downstream are simply not able to overcome the stream's velocity to affect anything upstream, which explains why the flow rate through a control valve experiencing sonic (critical) flow velocities does not change with changes in downstream pressure: those downstream pressure changes cannot propagate upstream against the fast-moving flow, and so will have no effect on the flow as it accelerates to sonic velocity at the point(s) of constriction.

Choked flow conditions become readily apparent if the flow-versus-pressure function of a control valve at any fixed opening value is graphed. The basic valve flow equation predicts a perfectly straight line at constant slope with flow rate (Q) as the vertical variable and the square root of pressure drop ($\sqrt{P_1 - P_2}$) as the horizontal variable. However, if we actually test a control valve by holding its upstream liquid pressure (P_1) constant and varying its downstream pressure (P_2) while maintaining a fixed stem position, we notice a point where flow reaches a maximum limit value:

[52] The *Control Valve Sourcebook – Power & Severe Service* on page 6-3 and the *ISA Handbook of Control Valves* on page 211 both suggest that the mechanism for choking in liquid service may be related to the speed of sound just as it is for choked flow in gas services. Normally, liquids have higher sonic velocities than gases due to their far greater bulk moduli (incompressibility). This makes choking due to sonic velocity very unlikely in liquid flowstreams. However, when a liquid flashes into vapor, the speed of sound *for that two-phase mixture of liquid and vapor* will be much less than it is for the liquid itself, opening up the possibility of sonic velocity choking.



In a choked flow condition, further reductions in downstream pressure achieve no greater flow of liquid through the valve. This is not to say that the valve has reached a maximum flow – we may still increase flow rate through a choked valve by increasing its upstream pressure. We simply cannot coax any more flow through a choked valve by decreasing its *downstream* pressure.

An approximate predictor of choked flow conditions for gas valve service is the upstream-to-minimum absolute pressure ratio. When the vena contracta pressure is less than one-half the upstream pressure, both measured in absolute pressure units, choked flow is virtually guaranteed. One should bear in mind that this is merely an approximation and not a precise prediction for choked flow. Much more information is needed about the valve design, the particular process gas, and other factors in order to reliably predict the presence of choking.

Choked flow in liquid services is predicted when the vena contracta pressure equals the liquid's vapor pressure, since choking is a function of flashing for liquid flowstreams.

No attempt will be made in this book to explain sizing procedures for control valves in choked-flow service, due to the complexity of the subject.

An interesting and useful application of choked flow in gases is a device called a *critical velocity nozzle*. This is a nozzle designed to allow a fixed flow rate of gas through it given a known upstream pressure, and a downstream pressure that is sufficiently low to ensure sonic velocities in the nozzle throat. One practical use for critical velocity nozzles is in the flow testing of compressed air systems. One or more of these nozzles are connected to the main header line of an air compressor system and allowed to vent to atmosphere. So long as the compressor(s) are able to maintain constant header pressure, the flow rate of air through the nozzles(s) is guaranteed to be fixed, allowing a technician to monitor compressor parameters under precisely known load conditions.

27.14.5 Valve noise

A troublesome phenomenon in severe services is the audible noise produced by turbulence as the fluid moves through a control valve. Noise output is worse for gas services experiencing sonic (critical) flow and for liquid services experiencing cavitation, although it is possible for a control valve to produce substantial noise even when avoiding these operating conditions. Noise produced by a control valve also translates into vibration imposed on the piping, which may cause problems such as loosening of threaded fasteners over time.

One way to reduce noise output is to use special valve trim resembling the trim used to mitigate cavitation. A common cage-guided globe valve trim design for noise reduction uses a special cage designed with numerous, small holes for process gas to flow through. One of the ways in which these small holes reduce audible noise is by shifting the *frequency* of that noise upward. This increase in frequency places the sound outside the range where the human ear is most sensitive to noise, and it also helps to reduce noise coupling to the piping, confining most of the noise "power" to the internal volume of the process fluid rather than radiating outward into the air.

Fisher manufactures a series of noise-abatement trim for process gas service called *Whisper* trim. A "Whisper" plug and cage set is shown in this next photograph:



In some versions, the holes are merely straight through the cage wall. In more sophisticated versions of Whisper trim (particularly the "WhisperFlo"), the small holes lead to a labyrinth of passages designed to dissipate energy by forcing the fluid to take several sharp turns as it passes through the wall of the cage. This allows relatively large pressure drops to develop without high fluid velocities, which is the primary causal factor for noise in control valves.

27.14.6 Erosion

A problem common to control valves used in *slurry* service (where the process fluid is a liquid containing a substantial quantity of hard, solid particles) is *erosion*, where the valve trim and body are worn by the passage of solid particles. Erosion produces some of the most striking examples of valve damage, as shown by the following photographs[53]:





Here we see large holes worn in a globe valve plug, and substantial damage done to the seat as well. The process service in this case was water with "coke fines" (small particles of coke, a solid petroleum product). Even ceramic valve seat components are not immune to damage from slurry service, as revealed by this photograph of a valve seat with a notch worn by slurry flow:



There really is no good way to reduce the effects of erosion damage from slurry flows, other than to use exceptionally hard valve trim materials. Even then, the control valve must be considered a fast-wearing component (along with pumps and any other components in harm's way of the slurry flowstream), rebuilt or replaced at regular intervals.

[53] A colleague of mine humorously refers to these valve trim samples as "shock and awe," because they so dramatically reveal the damaging nature of certain process fluid services.

Another cause of erosion in control valves is *wet steam*, where steam contains droplets of liquid water propelled at high velocity by the steam flow. A dramatic example of wet steam damage appears in this next photograph, where the cage from a Fisher valve has been literally cut in half from the flow:



Steam may also "cut" other parts of a valve if allowed to leak past. Here, we see a valve bonnet with considerable damage caused by steam leaking past the *outside* of the packing, between the packing rings and the bonnet's bore:



Any fluid with sufficient velocity may cause extensive damage to valve components. Small holes developing in the body of a valve may become large holes over time, as fluid rushes through the hole. The initial cause of the hole may be a manufacturing defect (such as *porosity* in the metal casting) or damage inflicted by the user (e.g. a crack in the valve body caused by some traumatic event). An example of such a hole in a valve body becoming worse over time appears in this next photograph, taken of a control valve removed after *40 years* of continuous service:



A close-up photograph of this same hole shows the leak path worn larger by passing fluid, allowing fluid to flow by the seat even with the valve in the fully-closed position:



In more severe process services, such holes rapidly grow in size. This next photograph shows a rather extreme example of a hole near the seat of a valve body, enlarged to the point where the valve is hardly capable of restricting fluid flow at all because the hole provides a bypass route for flow around the valve plug and seat:



27.14.7 Chemical attack

Corrosive chemicals may attack the metal components of control valves if those components are not carefully selected for the proper service. A close-up photograph of a chemically-pitted valve plug shows pitting characteristic of chemical attack:



As mentioned previously in this chapter, the effects of corrosion are multiplied when combined with the effects of cavitation. Most metals develop what is known as a *passivation layer* in response to chemical attack. The outer layer of metal corrodes, but the byproduct of that corrosion is a relatively inert compound acting to shield the rest of the metal from further attack. Rust on steel, or aluminum oxide on aluminum, are both common examples of passivation layers in response to oxidation of the metal. When cavitation happens inside a valve, however, the extremely high pressures caused by the liquid microjets will blast away any protection afforded by the passivation layer, allowing chemical attack to begin anew. The result is rapid degradation of the valve components.

27.15 Review of fundamental principles

Shown here is a partial listing of principles applied in the subject matter of this chapter, given for the purpose of expanding the reader's view of this chapter's concepts and of their general interrelationships with concepts elsewhere in the book. Your abilities as a problem-solver and as a life-long learner will be greatly enhanced by mastering the applications of these principles to a wide variety of topics, the more varied the better.

- **Definition of pressure**: $P = \frac{F}{A}$ (pressure is the amount of force applied over a specified area by a fluid. Relevant to fluid-powered valve actuators: the amount of force developed by an actuator is proportional to the fluid pressure applied to it and the area of its piston or diaphragm.
- **Pascal's principle**: changes in fluid pressure are transmitted evenly throughout an enclosed fluid volume. Relevant to fluid-powered valve actuators.
- **Fluid seals**: accomplished by maintaining tight contact between solid surfaces. Relevant to control valve plug and seat design, where even seating force is critical to maintaining tight shut-off. Also relevant to valve packing design.
- **Conservation of energy**: energy cannot be created or destroyed, only converted between different forms. Relevant to fluid velocities and pressures inside of a control valve.
- **Bernoulli's equation**: $z_1 \rho g + \frac{v_1^2 \rho}{2} + P_1 = z_2 \rho g + \frac{v_2^2 \rho}{2} + P_2$, which is an application of the Law of Energy Conservation, stating that the sum of all forms of energy in a moving fluid stream (height, kinetic, and pressure) must remain the same. Relevant to calculations of pressure drop and pressure recovery across control valve trim.
- **Conservation of mass**: mass is an intrinsic property of matter, and as such cannot be created or destroyed. Relevant to the Continuity Principle for moving fluids, where the mass flow rate of a fluid entering a control valve must equal the mass flow rate exiting the valve.
- **Linear equations**: any function represented by a straight line on a graph may be represented symbolically by the slope-intercept formula $y = mx + b$. Relevant to control valve positioners and split-ranging.
- **Zero shift**: any shift in the offset of an instrument is fundamentally additive, being represented by the "intercept" (b) variable of the slope-intercept linear formula $y = mx + b$. Relevant to control valve calibration: adjusting the "zero" of a valve positioner always adds to or subtracts from the valve stem position.
- **Span shift**: any shift in the gain of an instrument is fundamentally multiplicative, being represented by the "slope" (m) variable of the slope-intercept linear formula $y = mx + b$. Relevant to control valve calibration: adjusting the "span" of a valve positioner always multiplies or divides the stem stroke (compared to signal span).
- **Deadband and hysteresis**: the difference in response with the independent variable increasing versus decreasing. Usually caused by friction in a mechanism. Relevant to control valve response: a control valve will not go to the same position with its command signal at

some value increasing as it does at that same value decreasing, due to friction in the trim and packing.

- **Inverse mathematical functions**: an inverse function, when applied to the result of its counterpart function, "un-does" the operation and leaves you with the original quantity. Relevant to control valve characterization, where the inherently nonlinear response of a control valve installed in a process with variable pressure drop is made to behave more linearly by skewing the shape of the valve trim in an inverse manner.

- **Negative feedback**: when the output of a system is degeneratively fed back to the input of that same system, the result is decreased (overall) gain and greater stability. Relevant to control valve positioners as well as self-actuated regulators.

- **Self-balancing pneumatic mechanisms**: all self-balancing pneumatic instruments work on the principle of negative feedback maintaining a nearly constant baffle-nozzle gap. Force-balance mechanisms maintain this constant gap by balancing force against force with negligible motion, like a tug-of-war. Motion-balance mechanisms maintain this constant gap by balancing one motion with another motion, like two dancers moving in unison.

References

Baumann, Hans D., *Control Valve Primer, A User's Guide*, Second Edition, Instrument Society of America, Research Triangle Park, NC, 1994.

Brestel, Ronald; Hutchens, Wilbur; Wood, Charles, *Control Valve Packing Systems*, Technical Monograph 38, Fisher Controls International Inc., Marshalltown, IA, 1992.

"Cavitation in Control Valves", document L351 EN, Samson AG, Frankfurt, Germany.

Control Valve Handbook, Third Edition, Fisher Controls International, Inc., Marshalltown, IA, 1999.

Control Valve Sourcebook – Power & Severe Service, Fisher Controls International, Inc., Marshalltown, IA, 1988.

"Design ED, EAD, and EDR Sliding-Stem Control Valves", Product Bulletin 51.1:ED, Fisher, Marshalltown, IA, 2006.

Grumstrup, Bruce, *Considerations in the Design and Selection of Bellows Seal Equipment Valves*, Technical Monograph 37, Fisher Controls International Inc., Marshalltown, IA, 1991.

Hutchison, J.W., *ISA Handbook of Control Valves*, Second Edition, Instrument Society of America, Research Triangle Park, NC, 1976.

Jury, Floyd D., *Fundamentals of Aerodynamic Noise in Control Valves*, Technical Monograph 43, Fisher Controls International Inc., Marshalltown, IA, 1999.

Lipták, Béla G. et al., *Instrument Engineers' Handbook – Process Control Volume II*, Third Edition, CRC Press, Boca Raton, FL, 1999.

"Micro Trims for Globe and Angle Valve Applications", Product Bulletin 80.4:010, Emerson Process Management, Marshalltown, IA, 2005.

"Packing Selection Guidelines for Sliding-Stem Valves", Product Bulletin 59.1:062, Emerson Process Management, Marshalltown, IA, 2007.

"Pipeline Accident Report – Pipeline Rupture and Subsequent Fire in Bellingham, Washington June 10, 1999", NTSB/PAR-02/02, PB2002-916502, Notation 7264A, National Transportation Safety Board, Washington DC, 2002.

Richardson, Jonathan W., *Primary Seat Shutoff*, Technical Monograph 47, Fisher Controls International LLC, Marshalltown, IA, 2005.

Riveland, Marc, *Fundamentals of Valve Sizing for Liquids*, Technical Monograph 30, Fisher Controls International Inc., Marshalltown, IA, 1985.

Schafbuch, Paul, *Fundamentals of Flow Characterization*, Technical Monograph 29, Fisher Controls International Inc., Marshalltown, IA, 1985.

Warnett, Chris, *Using Valve Actuators as Predictive Maintenance Tools for MOVs*, Rotork Controls, Inc., Rochester, NY, 2000.

"Valve Sizing Technical Bulletin", document MS-06-84-E, revision 3, Swagelok Company, MI, 2002.

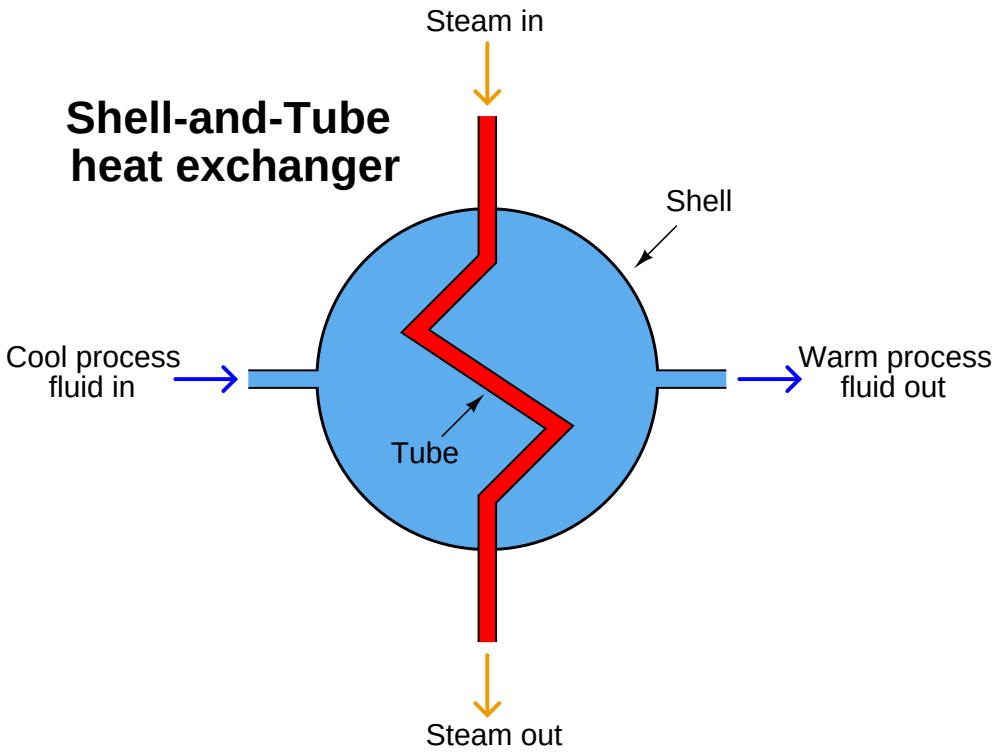
Chapter 29

Closed-loop control

Instrumentation is the science of automated measurement and control. Applications of this science abound in modern research, industry, and everyday living. From automobile engine control systems to home thermostats to aircraft autopilots to the manufacture of pharmaceutical drugs, automation surrounds us. This chapter explains some of the fundamental principles of automatic process control.

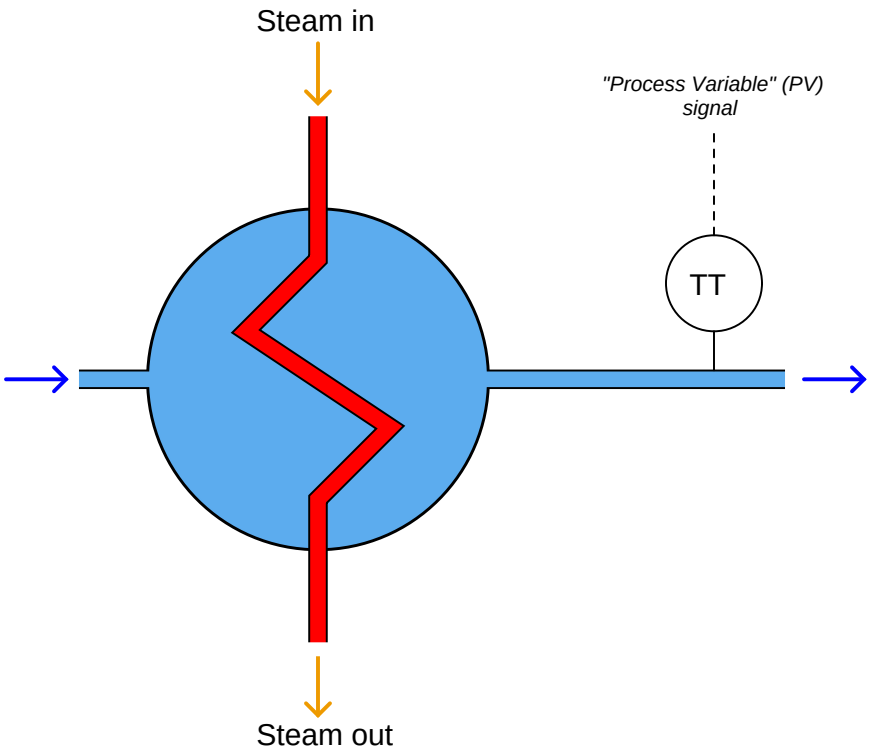
29.1 Basic feedback control principles

Before we begin our discussion on process control, we must define a few key terms. First, we have what is known as the *process*: the physical system we wish to monitor and control. For the sake of illustration, consider a heat exchanger that uses high-temperature steam to transfer heat to a lower-temperature liquid. Heat exchangers are used frequently in the chemical industries to maintain the necessary temperature of a chemical solution, so the desired blending, separation, or reactions can occur. A very common design of heat exchanger is the "shell-and-tube" style, where a metal shell serves as a conduit for the chemical solution to flow through, while a network of smaller tubes runs through the interior of the shell, carrying steam or some other heat-transfer fluid. The hotter steam flowing through the tubes transfers heat energy to the cooler process fluid surrounding the tubes, inside the shell of the heat exchanger:



In this case, the *process* is the entire heating system, consisting of the fluid we wish to heat, the heat exchanger, and the steam delivering the required heat energy. In order to maintain steady control of the process fluid's exiting temperature, we must find a way to measure it and represent that measurement in signal form so it may be interpreted by other instruments taking some form of control action. In instrumentation terms, the measuring device is known as a *transmitter*, because it *transmits* the process measurement in the form of a signal.

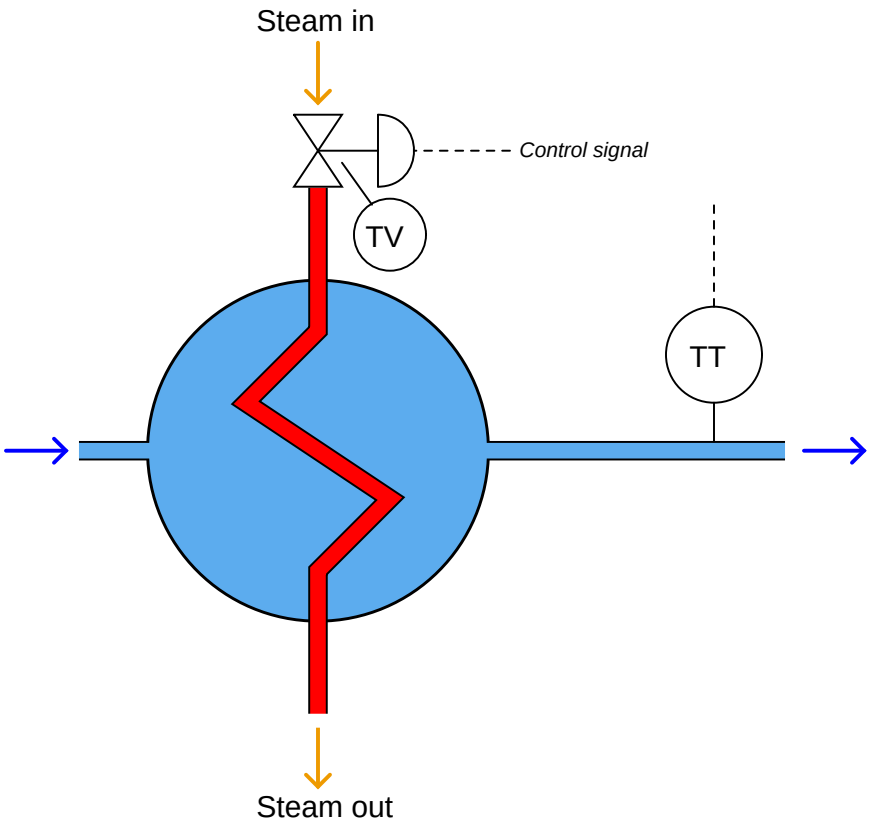
Transmitters are represented in process diagrams by small circles with identifying letters inside, in this case, "TT," which stands for **T**emperature **T**ransmitter:



The signal output by the transmitter (represented by the "PV" dashed line), representing the heated fluid's exiting temperature, is called the *process variable*. Like a variable in a mathematical equation that represents some story-problem quantity, this signal represents the measured quantity we wish to control in the process.

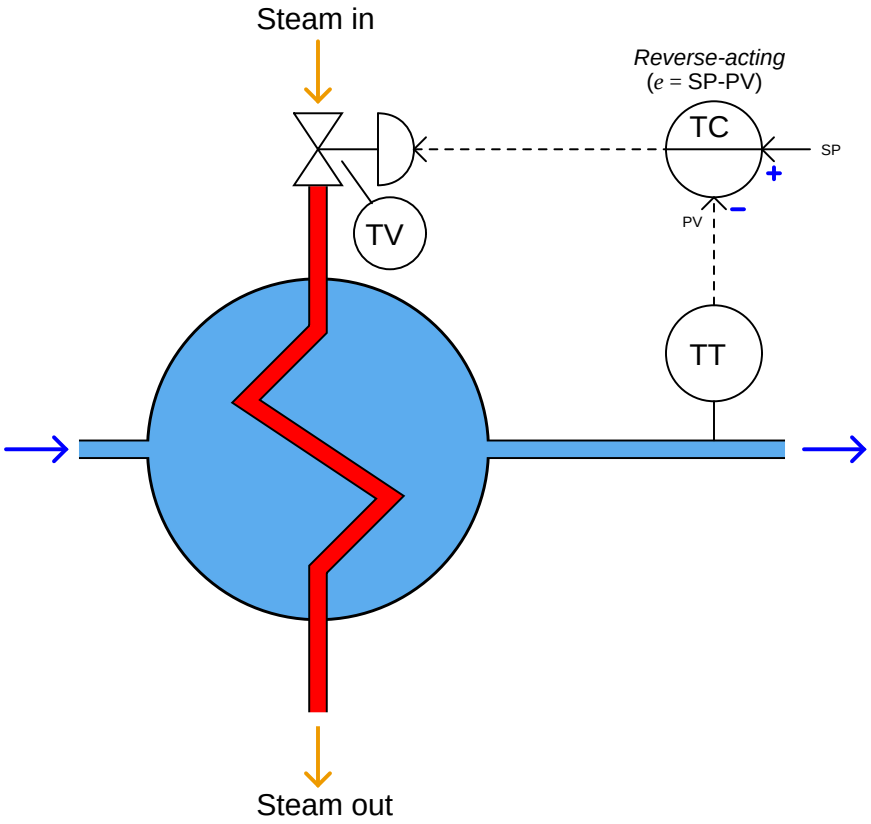
In order to exert control over the process variable, we must have some way of altering fluid flow through the heat exchanger, either of the process fluid, the steam, or both. Generally, it makes more sense to alter the flow of the heating medium (the steam), and let the process fluid flow rate be dictated by the demands of the larger process. If this heat exchanger were part of an oil refinery unit, for example, it would be far better to throttle steam flow to control oil temperature rather than to throttle the oil flow itself, since altering the oil's flow will also affect other process variables upstream and downstream of the exchanger. Ideally, the heat exchanger temperature control system would provide consistent temperature of the exiting oil, for any given incoming oil temperature and flow-rate of oil through it.

One convenient way to throttle steam flow into the heat exchanger is to use a control valve (labeled "TV" because it is a **T**emperature **V**alve). In general terms, a control valve is known as a *final control element*. Other types of final control elements exist (servo motors, variable-flow pumps, and other mechanical devices used to vary some physical quantity at will), but valves are the most common, and probably the simplest to understand. With a final control element in place, the steam flow becomes known as the *manipulated variable*, because it is the quantity we will manipulate in order to gain control over the process variable:



Valves come in a wide variety of sizes and styles. Some valves are hand-operated: that is, they have a "wheel" or other form of manual control that may be moved to "pinch off" or "open up" the flow passage through the pipe. Other valves come equipped with signal receivers and positioner devices, which move the valve mechanism to various positions at the command of a signal (usually an electrical signal, like the type output by transmitter instruments). This feature allows for remote control, so a human operator or computer device may exert control over the manipulated variable from a distance. In the previous illustration, the steam control valve is equipped with such an electrical signal input, represented by the "control signal" dashed line.

This brings us to the final component of the heat exchanger temperature control system: the *controller*. This is a device designed to interpret the transmitter's process variable signal and decide how far open the control valve needs to be in order to maintain that process variable at the desired value.



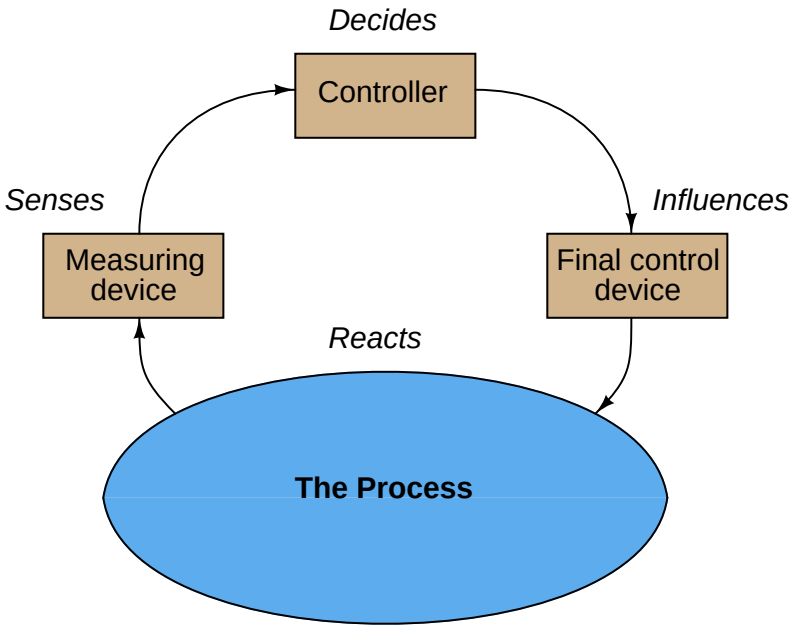
Here, the circle with the letters "TC" in the center represents the controller. Those letters stand for **T**emperature **C**ontroller, since the process variable being controlled is the process fluid's *temperature*. Usually, the controller consists of a computer making automatic decisions to open and close the valve as necessary to stabilize the process variable at some predetermined *setpoint*.

Note that the controller's circle has a solid line going through the center of it, while the transmitter and control valve circles are open. An open circle represents a field-mounted device according to the ISA standard for instrumentation symbols, and a single solid line through the middle of a circle tells us the device is located on the front of a control panel in a main control room location. So, even though the diagram might appear as though these three instruments are located close to one another, they in fact may be quite far apart. Both the transmitter and the valve must be located near the heat exchanger (out in the "field" area rather than inside a building), but the controller may be located a long distance away where human operators can adjust the setpoint from inside a safe and secure control room.

These elements comprise the essentials of a *feedback control system*: the *process* (the system

to be controlled), the *process variable* (the specific quantity to be measured and controlled), the *transmitter* (the device used to measure the process variable and output a corresponding signal), the *controller* (the device that decides what to do to bring the process variable as close to setpoint as possible), the *final control element* (the device that directly exerts control over the process), and the *manipulated variable* (the quantity to be directly altered to effect control over the process variable).

Feedback control may be viewed as a sort of information "loop," from the transmitter (measuring the process variable), to the controller, to the final control element, and through the process itself, back to the transmitter. Ideally, a process control "loop" not only holds the process variable at a steady level (the setpoint), but also maintains control over the process variable given changes in setpoint, and even changes in other variables of the process:



Specifically, the type of feedback we are employing here to control the process is *negative* or *degenerative* feedback. The term "negative" refers to the direction of action the control system takes in response to any measured change in the process variable. If something happens to drive the process variable up, the control system will automatically respond in such a way as to bring the process variable back down where it belongs. If the process variable happens to sag below setpoint, the control system will automatically act to drive the process variable back up to setpoint. Whatever the process variable does in relation to setpoint, the control system takes the opposite (inverse, or negative) action in an attempt to stabilize it at setpoint.

For example, if the unheated process fluid flow rate were to suddenly increase, the heat exchanger outlet temperature would fall due to the physics of heat transfer, but once this drop was detected by the transmitter and reported to the controller, the controller would automatically call for additional steam flow to compensate for the temperature drop, thus bringing the process variable back in agreement with the setpoint. Ideally, a well-designed and well-tuned control loop will sense and

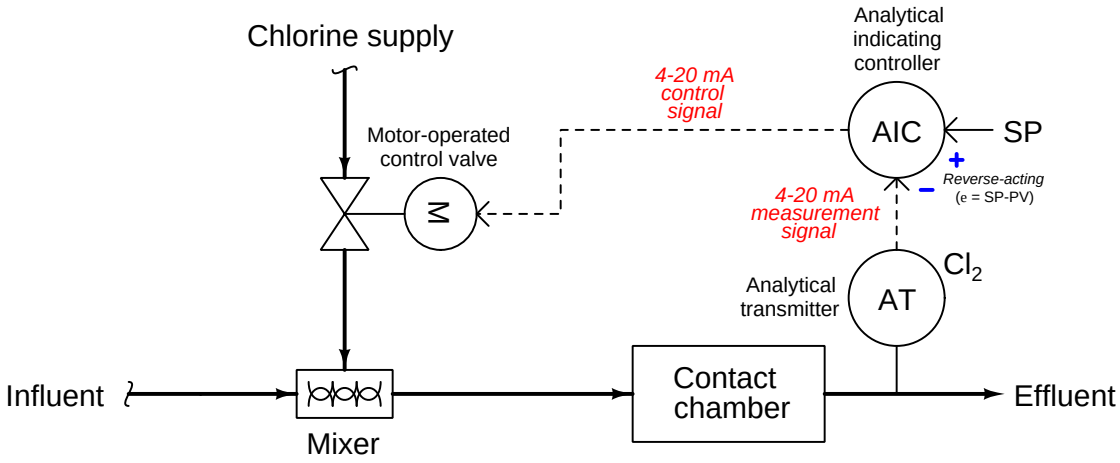
compensate for *any* change in the process or in the setpoint, the end result being a process variable value that always holds steady at the setpoint value.

The unheated fluid flow rate is an example of an uncontrolled, or *wild*, variable because our control system here has no ability to influence it. This flow is also referred to as a *load* because it "loads" or affects the process variable we are trying to stabilize. Loads are present in nearly every controlled system, and indeed are the primary factor necessitating a control system at all. Referring back to our heat exchanger process again, we could adequately control the operating temperature of it with just a manually-set steam control valve if only none of the other factors (steam temperature, fluid flow rate, incoming fluid temperature, etc.) ever changed!

Many types of processes lend themselves to feedback control. Consider an aircraft autopilot system, keeping an airplane on a steady course heading despite the effects of loads such as side-winds: reading the plane's heading (process variable) from an electronic compass and using the rudder as a final control element to change the plane's "yaw." An automobile's "cruise control" is another example of a feedback control system, with the process variable being the car's velocity, and the final control element being the engine's throttle. The purpose of a cruise control is to maintain constant driving speed despite the influence of loads such as hills, head-winds, tail-winds, and road roughness. Steam boilers with automatic pressure controls, electrical generators with automatic voltage and frequency controls, and water pumping systems with automatic flow controls are further examples of how feedback may be used to maintain control over certain process variables.

Modern technology makes it possible to control nearly anything that may be measured in an industrial process. This extends beyond the pale of simple pressure, level, temperature, and flow variables to include even certain chemical properties.

In municipal water and wastewater treatment systems, for example, numerous chemical quantities must be measured and controlled automatically to ensure maximum health and minimum environmental impact. Take for instance the chlorination of treated wastewater, before it leaves the wastewater treatment facility into a large body of water such as a river, bay, or ocean. Chlorine is added to the water to kill any residual bacteria so they do not consume oxygen in the body of water they are released to. Too little chlorine added, and not enough bacteria are killed, resulting in a high *biological oxygen demand* or *BOD* in the water which will asphyxiate the fish swimming in it. Too much chlorine added, and the chlorine itself poses a hazard to marine life. Thus, the chlorine content must be carefully controlled at a particular setpoint, and the control system must take aggressive action if the dissolved chlorine concentration strays too low or too high:

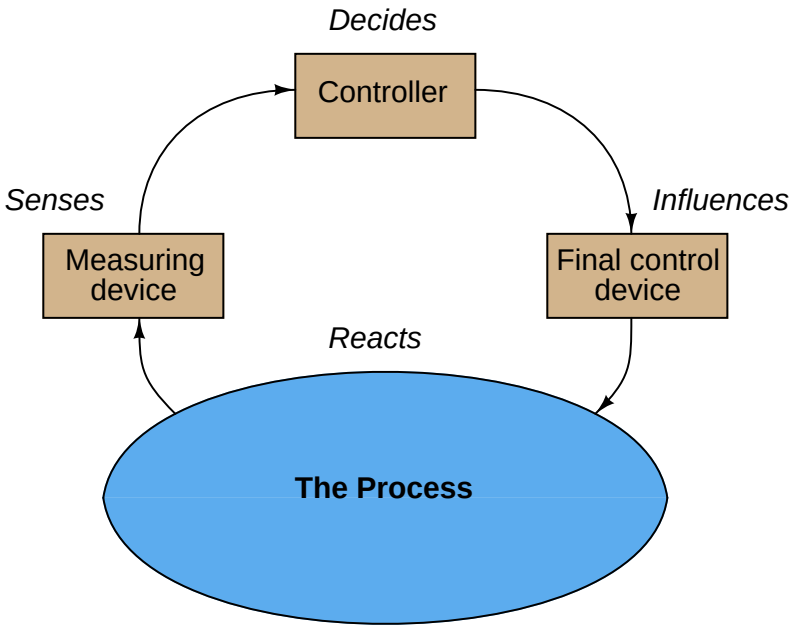


Now that we have seen the basic elements of a feedback control system, we will concentrate on the *algorithms* used in the controller to maintain a process variable at setpoint. For the scope of this topic, an "algorithm" is a mathematical relationship between the process variable and setpoint inputs of a controller, and the output (manipulated variable). Control algorithms determine *how* the manipulated variable quantity is deduced from PV and SP inputs, and range from the elementary to the very complex. In the most common form of control algorithm, the so-called "PID" algorithm, calculus is used to determine the proper final control element action for any combination of input signals.

29.2 Diagnosing feedback control problems

Negative feedback systems, in general, tend to cause much confusion for those first learning their fundamental principles and behaviors. The closed-cycle "loop" formed by the interaction of sensing element, controller, final control element, and process means essentially that *everything affects everything else*. This is especially problematic when the feedback control system in question contains a fault and must be diagnosed. For example, if an operator happens to notice that the process variable (as indicated by a manual measurement or by some trusted indicating instrument) is not holding to setpoint, it could be the result of a fault in *any* portion of the system (sensor, controller, FCE, or even the process itself).

Recall that every feedback control loop consists of four basic elements: an element that *senses* the process variable (e.g. primary sensing element, transmitter), an element that *decides* what how to regulate this process variable (e.g. a PID controller), an element that *influences* the process variable (e.g. a control valve, motor drive, or some other final control device), and finally the process itself which *reacts* to the final control device's actions:



One of the basic diagnostic strategies for any instrumentation system is to assess whether the *input value(s)* and *output value(s)* correspond for each instrument. We may apply this same strategy to each of the four elements of a feedback control "loop" to identify where the problem might exist. If you encounter one of these four system portions whose output does not correspond with its input, you know that portion of the system is faulted.

You can check each element of your feedback control loop by comparing its input with its output to see if each element is doing what it should. I recommend beginning with the controller (the decision-making element) because typically those values are the most easily monitored:

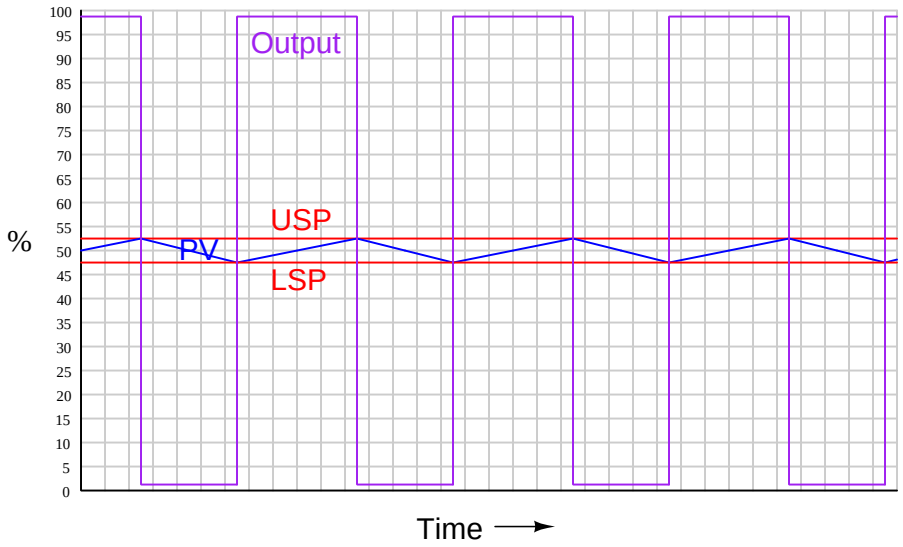
- **Decision-making:** Carefully examine the controller faceplate, looking at the values of PV, SP, and Output. Is the controller taking appropriate action to force PV equal to SP? In other words, is the Output signal at a value you would expect if the controller were functioning properly to regulate the process variable at setpoint? If so, then the controller's action and tuning are most likely not at fault. If not, then the problem definitely lies with the controller.
- **Sensing:** Compare the controller's displayed value for PV with the actual process variable value as indicated by local gauges, by feel, or by any other means of detection. If there is good correspondence between the controller's PV display and the real process variable, then there probably isn't anything wrong with the measurement portion of the control loop (e.g. transmitter, impulse lines, PV signal wiring, analog input of controller, etc.). If the displayed PV disagrees with the actual process variable value, then something is definitely wrong here.
- **Influencing:** Compare the controller's displayed value for Output with the actual status of the final control element. If there is good correspondence between the controller's Output display and the FCE's status, then there probably isn't anything wrong with the output portion of the control loop (e.g. FCE, output signal wiring, analog output of controller, etc.). If the controller Output value differs from the FCE's state, then something is definitely wrong here.
- **Reacting:** Compare the process variable value with the final control element's state. Is the process doing what you would expect it to? If so, the problem is most likely not within the process (e.g. manual valves, relief valves, pumps, compressors, motors, and other process equipment). If, however, the process is not reacting the way you would expect it to given the final control element's state, then something is definitely awry with the process itself.

29.3 On/off control

Once while working as an instrument technician in an aluminum foundry, a mechanic asked me what it was that I did. I began to explain my job, which was essentially to calibrate, maintain, troubleshoot, document, and modify (as needed) all automatic control systems in the facility. The mechanic seemed puzzled as I explained the task of "tuning" loop controllers, especially those controllers used to maintain the temperature of large, gas-fired industrial furnaces holding many tons of molten metal. "Why does a controller have to be 'tuned'?" he asked. "All a controller does is turn the burner on when the metal's too cold, and turn it off when it becomes too hot!"

In its most basic form, the mechanic's assessment of the control system was correct: to turn the burner on when the process variable (molten metal temperature) drops below setpoint, and turn it off when it rises above setpoint. However, the actual algorithm is much more complex than that, finely adjusting the burner intensity according to the amount of *error* between PV and SP, the amount of time the error has accumulated, and the rate-of-change of the error over time. In his casual observation of the furnace controllers, though, he had noticed nothing more than the full-on/full-off action of the controller.

The technical term for a control algorithm that merely checks for the process variable exceeding or falling below setpoint is *on/off control*. In colloquial terms, it is known as *bang-bang* control, since the manipulated variable output of the controller rapidly switches between fully "on" and fully "off" with no intermediate state. Control systems this crude usually provide very imprecise control of the process variable. Consider our example of the shell-and-tube heat exchanger, if we were to implement simple on/off control[1]:



As you can see, the degree of control is rather poor. The process variable "cycles" between the upper and lower setpoints (USP and LSP) without ever stabilizing at the setpoint, because that

[1]To be precise, this form of on/off control is known as *differential gap* because there are two setpoints with a gap in between. While on/off control is possible with a single setpoint (FCE on when below setpoint and off when above), it is usually not practical due to the frequent cycling of the final control element.

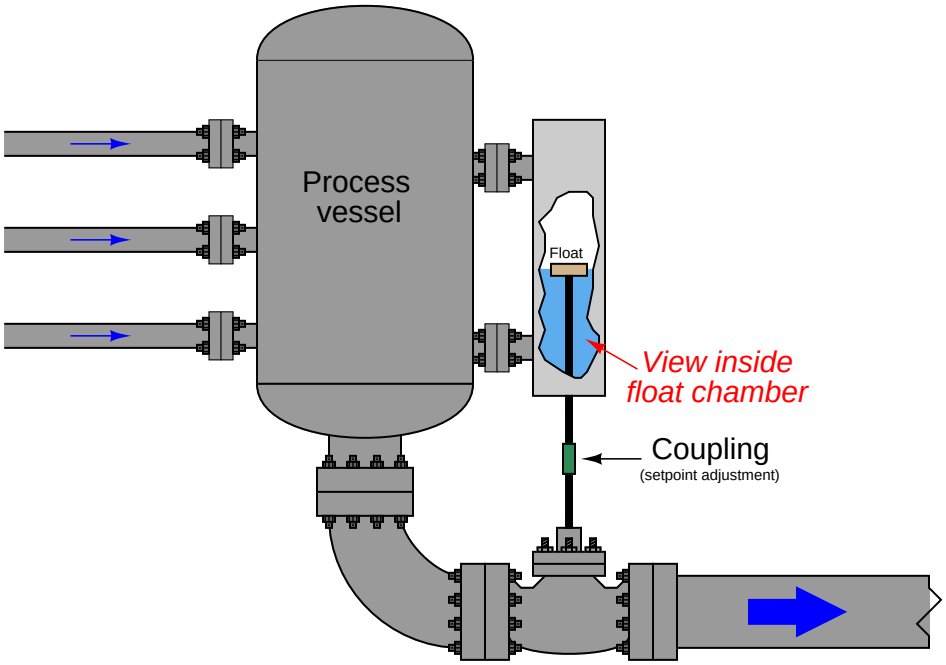
would require the steam valve to be position somewhere *between* fully closed and fully open.

This simple control algorithm may be adequate for temperature control in a house, but not for a sensitive chemical process! Can you imagine what it would be like if an automobile's cruise control system relied on this algorithm? Not only is the lack of precision a problem, but the frequent cycling of the final control element may contribute to premature failure due to mechanical wear. In the heat exchanger scenario, thermal cycling (hot-cold-hot-cold) will cause metal fatigue in the tubes, resulting in a shortened service life. Furthermore, every excursion of the process variable above setpoint is wasted energy, because the process fluid is being heated to a greater temperature than what is necessary.

Clearly, the only practical answer to this dilemma is a control algorithm able to *proportion* the final control element rather than just operate it at zero or full effect (the control valve fully closed or fully open). This, in its simplest form, is called *proportional control*.

29.4 Proportional-only control

Imagine a liquid-level control system for a vessel, where the position of a level-sensing float directly sets the stem position of a control valve. As the liquid level rises, the valve opens up proportionally:



Despite its crude mechanical nature, this *proportional* control system would in fact help regulate the level of liquid inside the process vessel. If an operator wished to change the "setpoint" value of this level control system, he or she would have to adjust the coupling between the float and valve stems for more or less distance between the two. Increasing this distance (lengthening the connection) would effectively raise the level setpoint, while decreasing this distance (shortening the connection) would lower the setpoint.

We may generalize the proportional action of this mechanism to describe *any* form of controller where the output is a direct function of process variable (PV) and setpoint (SP):

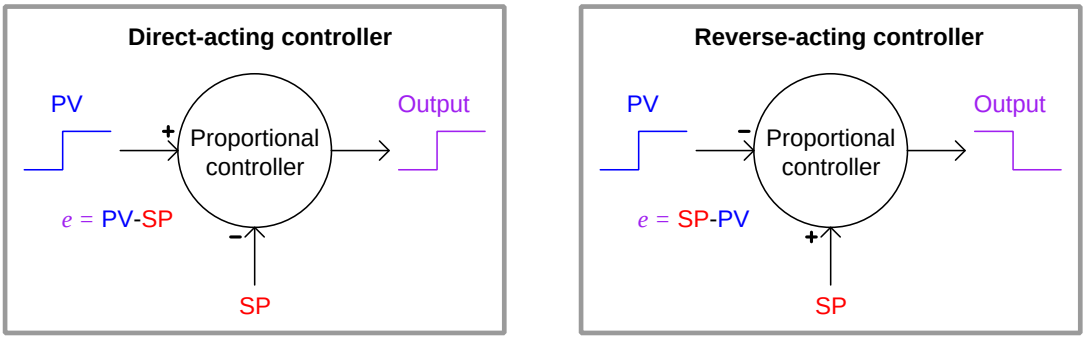
$$m = K_p e + b$$

Where,
m = Controller output
e = Error (difference between PV and SP)
K_p = Proportional gain
b = Bias

A new term introduced with this formula is e, the "error" or difference between process variable and setpoint. Error may be calculated as SP−PV or as PV−SP, depending on whether or not the controller must produce an *increasing* output signal in response to an increase in the process variable ("direct" acting), or output a *decreasing* signal in response to an increase in the process variable ("reverse" acting):

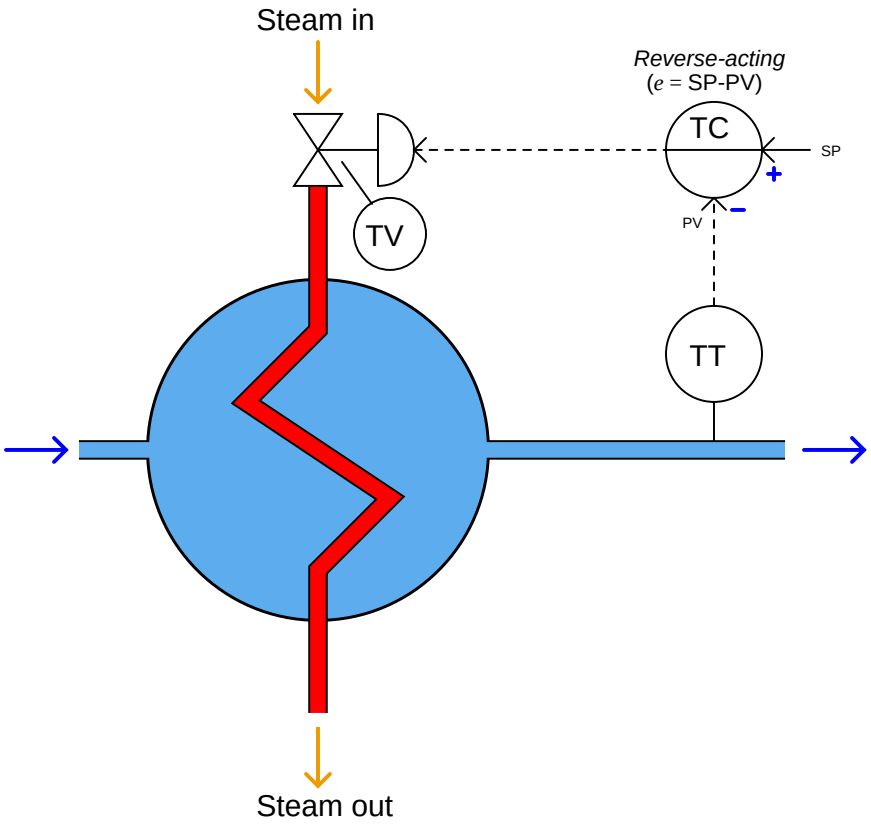
$$m = K_p(\text{PV} - \text{SP}) + b \qquad \text{(Direct-acting proportional controller)}$$

$$m = K_p(\text{SP} - \text{PV}) + b \qquad \text{(Reverse-acting proportional controller)}$$



The optional "+" and "−" symbols clarify the effect each input has on the controller output: a "−" symbol representing an *inverting* effect and a "+" symbol representing a *noninverting* effect. When we say that a controller is "direct-acting" or "reverse-acting" we are referring to it reaction to the PV signal, therefore the output signal from a "direct-acting" controller goes in the same direction as the PV signal and the output from a "reverse-acting" controller goes in the opposite direction of its PV signal. It is important to note, however, that the response to a change in setpoint (SP) will yield the *opposite* response as does a change in process variable (PV): a rising SP will drive the output of a direct-acting controller *down* while a rising SP drives the output of a reverse-acting controller *up*. "+" and "−" symbols explicitly show the effect both inputs have on the controller output, helping to avoid confusion when analyzing the effects of PV changes versus the effects of SP changes.

The direction of action required of the controller is determined by the nature of the process, transmitter, and final control element. In the case of the crude mechanical level controller, the action needs to be *direct* so that a greater liquid level will result in a further-open control valve to drain the vessel faster. In the case of the automated heat exchanger shown earlier, we are assuming that an increasing output signal sent to the control valve results in increased steam flow, and consequently higher temperature, so our controller will need to be reverse-acting (i.e. an increase in measured temperature results in a decrease in output signal; error calculated as SP−PV):



After the error has been calculated, the controller then multiplies the error signal by a constant value called the *gain*, which is programmed into the controller. The resulting figure, plus a "bias" quantity, becomes the output signal sent to the valve to proportion it. The "gain" value is exactly what it seems to be for anyone familiar with electronic amplifier circuits: a ratio of output to input. In this case, the gain of a proportional controller is the ratio of output signal change to input signal change, or how *aggressive* the controller reacts to changes in input (PV or SP).

To give a numerical example, a loop controller set to have a gain of 4 will change its output signal by 40% if it sees an input change of 10%: the ratio of output change to input change will be 4:1. Whether the input change comes in the form of a setpoint adjustment, a drift in the process variable, or some combination of the two does not matter to the magnitude of the output change.

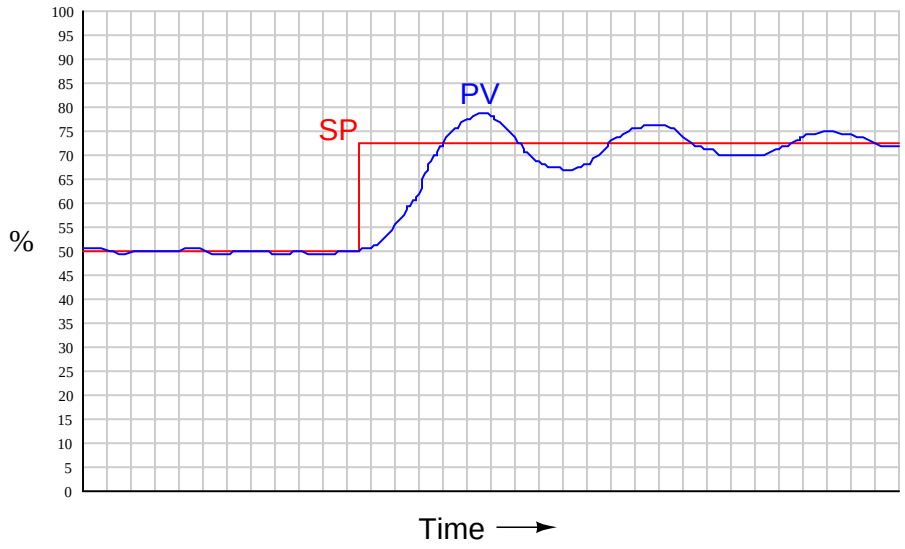
The bias value of a proportional controller is simply the value of its output whenever process

variable happens to be equal to setpoint (i.e. a condition of zero *error*). Without a bias term in the proportional control formula, the valve would always return to a fully shut (0%) condition if ever the process variable reached the setpoint value. The bias term allows the final control element to achieve a non-zero state at setpoint.

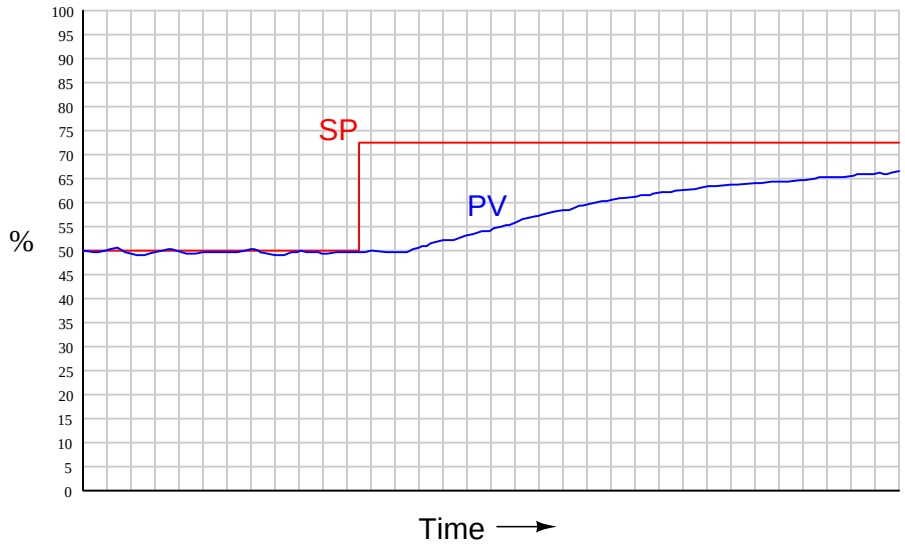
If the controller could be configured for infinite gain, its response would duplicate on/off control. That is, *any* amount of error will result in the output signal becoming "saturated" at either 0% or 100%, and the final control element will simply turn on fully when the process variable drops below setpoint and turn off fully when the process variable rises above setpoint. Conversely, if the controller is set for zero gain, it will become completely unresponsive to changes in either process variable *or* setpoint: the valve will hold its position at the bias point no matter what happens to the process.

Obviously, then, we must set the gain somewhere between infinity and zero in order for this algorithm to function any better than on/off control. Just how much gain a controller needs to have depends on the process and all the other instruments in the control loop.

If the gain is set too high, there will be oscillations as the PV converges on a new setpoint value:

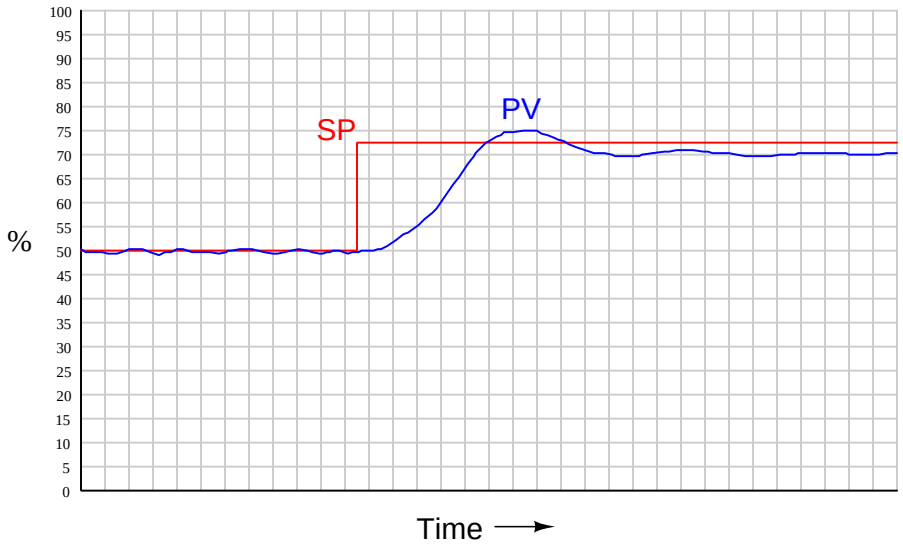


If the gain is set too low, the process response will be stable under steady-state conditions but relatively slow to respond to changes in setpoint, as shown in the following trend recording:



A characteristic deficiency of proportional control action, exacerbated with low controller gain values, is a phenomenon known as *proportional-only offset* where the PV never fully reaches SP. A full explanation of proportional-only offset is too lengthy for this discussion and will be presented in a subsequent section of the book, but may be summarized here simply by drawing attention to the proportional controller equation which tells us the output always returns to the bias value when PV reaches SP (i.e. $m = b$ when PV = SP). If anything changes in the process to require a different output value than the bias (b) to stabilize the PV, an error between PV and SP *must* develop to drive the controller output to that necessary output value. This means it is only by chance that the PV will settle precisely at the SP value – most of the time, the PV will deviate from SP in order to generate an output value sufficient to stabilize the PV and prevent it from drifting. This persistent error, or offset, worsens as the controller gain is reduced. Increasing controller gain causes this offset to decrease, but at the expense of oscillations.

With proportional-only control, the choice of gain values is really a compromise between excessive oscillations and excessive offset. A well-tuned proportional controller response is shown here:



An unnecessarily confusing aspect of proportional control is the existence of two completely different ways to express controller proportionality. In the proportional-only equation shown earlier, the degree of proportional action was specified by the constant K_p, called *gain*. However, there is another way to express the sensitivity of proportional action, and that is to state the percentage of error change necessary to make the output (m) change by 100%. Mathematically, this is the inverse of gain, and it is called *proportional band* (PB):

$$K_p = \frac{1}{\text{PB}} \qquad \text{PB} = \frac{1}{K_p}$$

Gain is always specified as a unitless value[2], whereas proportional band is always specified as a percentage. For example, a gain value of 2.5 is equivalent to a proportional band value of 40%, because the error input to this controller must change by 40% in order to make the output change a full 100%.

[2] In electronics, the unit of *decibels* is commonly used to express gains. Thankfully, the world of process control was spared the introduction of decibels as a unit of measurement for controller gain. The last thing we need is a *third* way to express the degree of proportional action in a controller!

Due to the existence of these two completely opposite conventions for specifying proportional action, you may see the proportional term of the control equation written differently depending on whether the author assumes the use of gain or the use of proportional band:

$$K_p = \text{gain} \qquad \text{PB} = \text{proportional band}$$

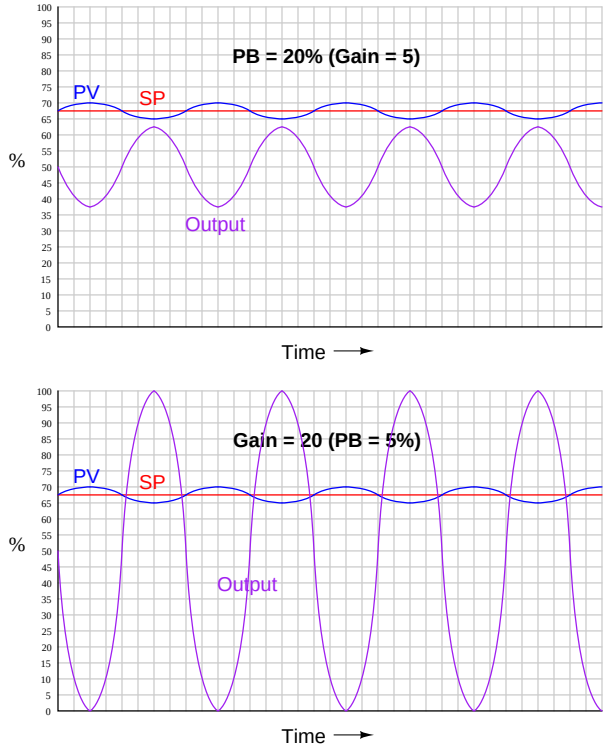
$$K_p e \qquad\qquad \frac{1}{\text{PB}} e$$

Many modern digital electronic controllers allow the user to conveniently select the unit they wish to use for proportional action. However, even with this ability, anyone tasked with adjusting a controller's "tuning" values may be required to translate between gain and proportional band, especially if certain values are documented in a way that does not match the unit configured for the controller.

When you communicate the proportional action setting of a process controller, you should always be careful to specify either "gain" or "proportional band" to avoid ambiguity. *Never* simply say something like, "The proportional setting is twenty," for this could mean either:

- Proportional band = 20%; Gain = 5 . . . *or* . . .
- Gain = 20; Proportional band = 5%

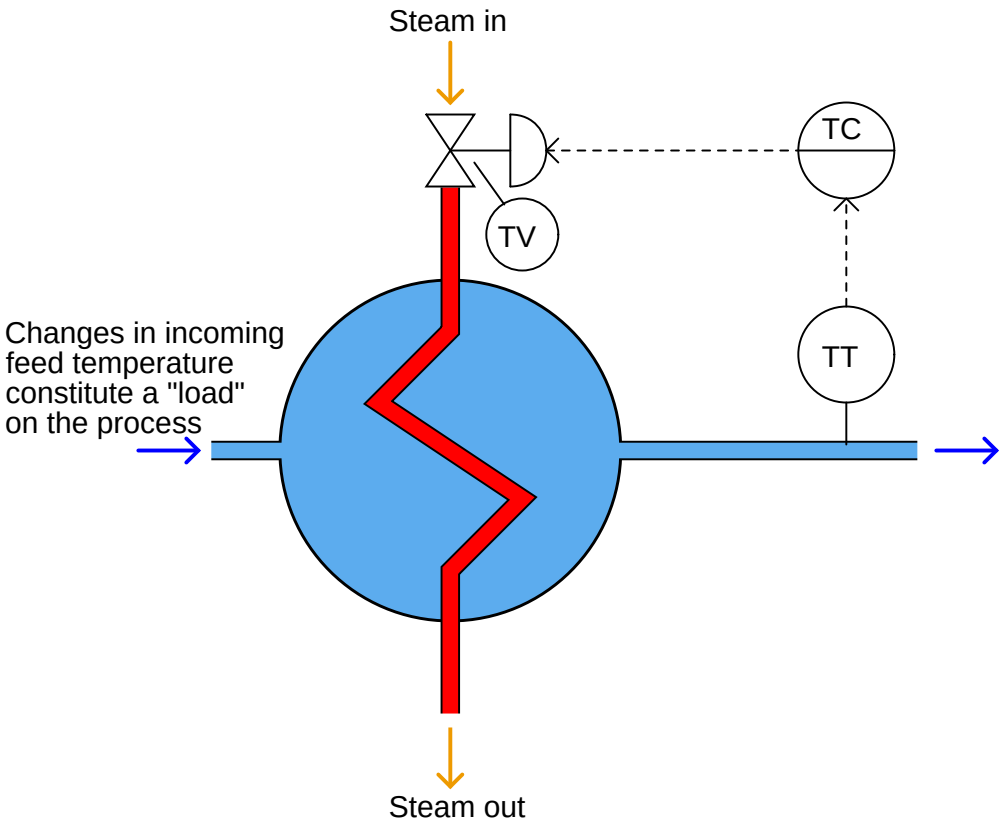
As you can see here, the real-life difference in controller response to an input disturbance (wave) depending on whether it has a proportional band of 20% or a gain of 20 is quite dramatic:



29.5 Proportional-only offset

A fundamental limitation of proportional control has to do with its response to changes in setpoint and changes in process *load*. A "load" in a controlled process is any variable not controlled by the loop controller which nevertheless affects the process variable the controller is trying to regulate. In other words, a "load" is any factor the loop controller must compensate for while maintaining the process variable at setpoint.

In our hypothetical heat exchanger system, the temperature of the incoming process fluid is an example of a load:



If the incoming fluid temperature were to suddenly decrease, the immediate effect this would have on the process would be to decrease the outlet temperature (which is the temperature we are trying to maintain at a steady value). It should make intuitive sense that a colder incoming fluid will require more heat input to raise it to the same outlet temperature as before. If the heat input remains the same (at least in the immediate future), this colder incoming flow must make the outlet flow colder than it was before. Thus, incoming feed temperature has an impact on the outlet temperature whether we like it or not, and the control system must compensate for these unforeseen and uncontrolled changes. This is precisely the definition of a "load": a burden[3] on the control

[3] One could argue that the presence of loads actually *justifies* a control system, for if there were no loads, there

system.

Of course, it is the job of the controller to counteract any tendency for the outlet temperature to stray from setpoint, but as we shall soon see this cannot be perfectly achieved with proportional control alone.

Let us perform a "thought experiment" to demonstrate this phenomenon of proportional-only offset. Imagine the controller has been controlling outlet temperature exactly at setpoint (PV = SP), and then suddenly the inlet feed temperature drops and remains colder than before. Recall that the equation for a reverse-acting proportional controller is as follows:

$$m = K_p(\text{SP} - \text{PV}) + b$$

Where,

m = Controller output
K_p = Proportional gain
SP = Setpoint
PV = Process variable
b = Bias

The introduction of colder feed fluid to the heat exchanger makes the outlet temperature (PV) begin to fall. As the PV falls, the controller calculates a positive error (SP − PV). This positive error, when multiplied by the controller's gain value, drives the output to a greater value. This opens up the steam valve, adding more heat to the exchanger.

As more heat is added, the rate of temperature drop slows down. The further the PV drops, the more the steam valve opens, until enough additional heat is being added to the heat exchanger to maintain a constant outlet temperature. However, this new stable PV value will be less than it was prior to the introduction of colder feed (i.e. less than the SP). In fact, the controller's automatic action can *never* return the PV to its original (SP) value so long as the feed remains colder than before. The reason for this is that a greater flow of steam is necessary to balance a colder feed coming in, and the only way a proportional controller is ever going to automatically drive the steam valve to this greater-flow position is if an error develops between PV and SP. Thus, an *offset* inevitably develops between PV and SP due to the load (colder feed).

We may prove the inevitability of this offset another way: imagine somehow that the PV did actually return to the SP value despite the colder feed fluid (remaining colder). If this happened, the steam valve would also return to its former throttling position where it was before the feed temperature dropped. However, we know that this former position will not allow enough steam through to the exchanger to overcome the colder feed – if it did, the PV never would have decreased to begin with! A further-open valve is precisely what we need to stabilize the PV given this colder feed, yet the only way the proportional-only controller can achieve this is if the PV actually falls below SP.

To summarize: the only way a proportional-only controller can automatically generate a new output value (m) is if the PV deviates from SP. Therefore, load changes (requiring new output values to compensate) force the PV to deviate from SP.

---

would be nothing to compensate for, and therefore no need for an automatic control system at all! In the total absence of loads, a manually-set final control element would be enough to hold most process variables at setpoint.

Another "thought experiment" may be helpful to illustrate the phenomenon of proportional-only offset. Imagine building your own cruise control system for your automobile based on the proportional-only equation: the engine's throttle position is a function of the difference between PV (road speed) and SP (the desired "target" speed). Let us further suppose that you carefully adjust the bias value of your cruise control system to achieve PV = SP on level ground at a speed of 70 miles per hour (70% on a 0 to 100 MPH speedometer scale), with the throttle at a position of 40%, and a gain (K_p) of 2:

$$m = K_p(\text{SP} - \text{PV}) + b$$

$$40\% = 2(70 - 70) + 40\%$$

Imagine now that after cruising precisely at setpoint (70% = 70 MPH), the road begins to incline uphill for several miles. This, obviously, is a load on the cruise control system. With the cruise control disengaged, the automobile would slow down because the same throttle position (40%) sufficient to maintain setpoint (70 MPH) on level ground is not enough power to maintain that same setpoint on an incline.

With the cruise control engaged, the engine throttle will automatically open further as speed drops. At a speed of 69 MPH, the throttle opens up to 42%. At a speed of 68 MPH, the throttle opens up to 44%. Every drop in speed of 1 MPH results in a 2% further-open throttle to send more power to the wheels.

Suppose the demands of this particular inclined road require a 50% throttle position for this automobile to maintain a constant speed. In order for your proportional-only cruise control system to deliver this necessary 50% throttle position, the speed will have to "droop" by 5 MPH below setpoint:

$$m = K_p(\text{SP} - \text{PV}) + b$$

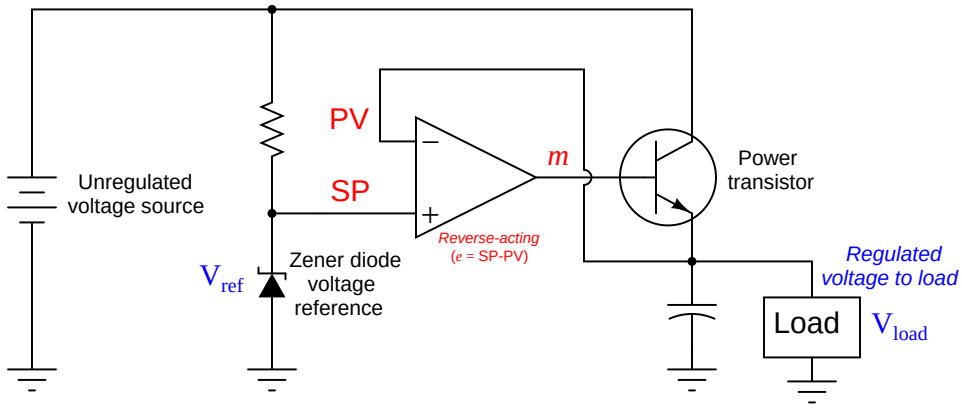
$$50\% = 2(70 - 65) + 40\%$$

There is simply no other way for your proportional-only controller to automatically achieve the requisite 50% throttle position aside from letting the speed sag below setpoint by 5% (5 MPH). Given this fact, the only way the proportional-only cruise control will ever return the speed to setpoint (70 MPH) is if and when the load conditions change to allow for a lesser throttle position of 40%. So long as the load demands a different throttle position than the bias value, the speed *must* deviate from the setpoint value of 70 MPH.

This necessary error developing between PV and SP is called *proportional-only offset*, sometimes called *droop*. The amount of droop depends on how severe the load change is, and how aggressive the controller responds (i.e. how much gain it has). The term "droop" is very misleading, as it is possible for the error to develop the other way (i.e. the PV might rise above SP due to a load change!). Imagine the opposite load-change scenario in our steam heat exchanger process, where the incoming feed temperature suddenly *rises* instead of falls. If the controller was controlling exactly at setpoint before this upset, the final result will be an outlet temperature that settles at some point *above* setpoint, enough so the controller is able to pinch the steam valve far enough closed to stop any further rise in temperature.

Proportional-only offset also occurs as a result of setpoint changes. We could easily imagine the same sort of effect following an operator's increase of setpoint for the temperature controller on the heat exchanger. After increasing the setpoint, the controller immediately increases the output signal, sending more steam to the heat exchanger. As temperature rises, though, the proportional algorithm causes the output signal to decrease. When the rate of heat energy input by the steam equals the rate of heat energy carried away from the heat exchanger by the heated fluid (a condition of *energy balance*), the temperature stops rising. This new equilibrium temperature will not be at setpoint, assuming the temperature was holding at setpoint prior to the human operator's setpoint increase. The new equilibrium temperature indeed *cannot* ever achieve any setpoint value higher than the one it did in the past, for if the error ever returned to zero (PV = SP), the steam valve would return to its old position, which we know would be insufficient to raise the temperature of the heated fluid to a new value.

An example of proportional-only control in the context of electronic power supply circuits is the following opamp voltage regulator, used to stabilize voltage to a load with power supplied by an unregulated voltage source:



In this circuit, a zener diode establishes a "reference" voltage (which may be thought of as a "setpoint" for the controlling opamp to follow). The operational amplifier acts as the proportional-only controller, sensing voltage at the load (PV), and sending a driving output voltage to the base of the power transistor to keep load voltage constant despite changes in the supply voltage or changes in load current (both "loads" in the process-control sense of the word, since they tend to influence voltage at the load circuit without being under the control of the opamp).

If everything functions properly in this voltage regulator circuit, the load's voltage will be stable over a wide range of supply voltages and load currents. However, the load voltage cannot ever *precisely* equal the reference voltage established by the zener diode, even if the operational amplifier (the "controller") is without defect. The reason for this incapacity to perfectly maintain "setpoint" is the simple fact that in order for the opamp to generate any output signal at all, there *absolutely must be* a differential voltage between the two input terminals for the amplifier to amplify. Operational amplifiers (ideally) generate an output voltage equal to the enormously high gain value (A_V) multiplied by the difference in input voltages (in this case, $V_{ref} - V_{load}$). If V_{load} (the "process

variable") were to ever achieve equality with V_{ref} (the "setpoint"), the operational amplifier would experience absolutely no differential input voltage to amplify, and its output signal driving the power transistor would fall to zero. Therefore, there must always exist some *offset* between V_{load} and V_{ref} (between process variable and setpoint) in order to give the amplifier some input voltage to amplify.

The amount of offset is ridiculously small in such a circuit, owing to the enormous gain of the operational amplifier. If we take the opamp's transfer function to be $V_{out} = A_V(V_{(+)} - V_{(-)})$, then we may set up an equation predicting the load voltage as a function of reference voltage (assuming a constant 0.7 volt drop between the base and emitter terminals of the transistor):

$$V_{out} = A_V(V_{(+)} - V_{(-)})$$

$$V_{out} = A_V(V_{ref} - V_{load})$$

$$V_{load} + 0.7 = A_V(V_{ref} - V_{load})$$

$$V_{load} + 0.7 = A_V V_{ref} - A_V V_{load}$$

$$V_{load} + A_V V_{load} = A_V V_{ref} - 0.7$$

$$(A_V + 1)V_{load} = A_V V_{ref} - 0.7$$

$$V_{load} = \frac{A_V V_{ref} - 0.7}{A_V + 1}$$

If, for example, our zener diode produced a reference voltage of 5.00000 volts and the operational amplifier had an open-loop voltage gain of 250000, the load voltage would settle at a theoretical value of 4.9999772 volts: just barely below the reference voltage value. If the opamp's open-loop voltage gain were much less – say only 100 – the load voltage would only be 4.94356 volts. This still is quite close to the reference voltage, but definitely not as close as it would be with a greater opamp gain!

Clearly, then, we can minimize proportional-only offset by increasing the gain of the process controller gain (i.e. decreasing its proportional band). This makes the controller more "aggressive" so it will move the control valve further for any given change in PV or SP. Thus, not as much error needs to develop between PV and SP to move the valve to any new position it needs to go. However, too much controller gain makes the control system unstable: at best it will exhibit residual oscillations after setpoint and load changes, and at worst it will oscillate out of control altogether. Extremely high gains work well to minimize offset in operational amplifier circuits, only because time delays are negligible between output and input. In applications where large physical processes are being controlled (e.g. furnace temperatures, tank levels, gas pressures, etc.) rather than voltages across small electronic loads, such high controller gains would be met with debilitating oscillations.

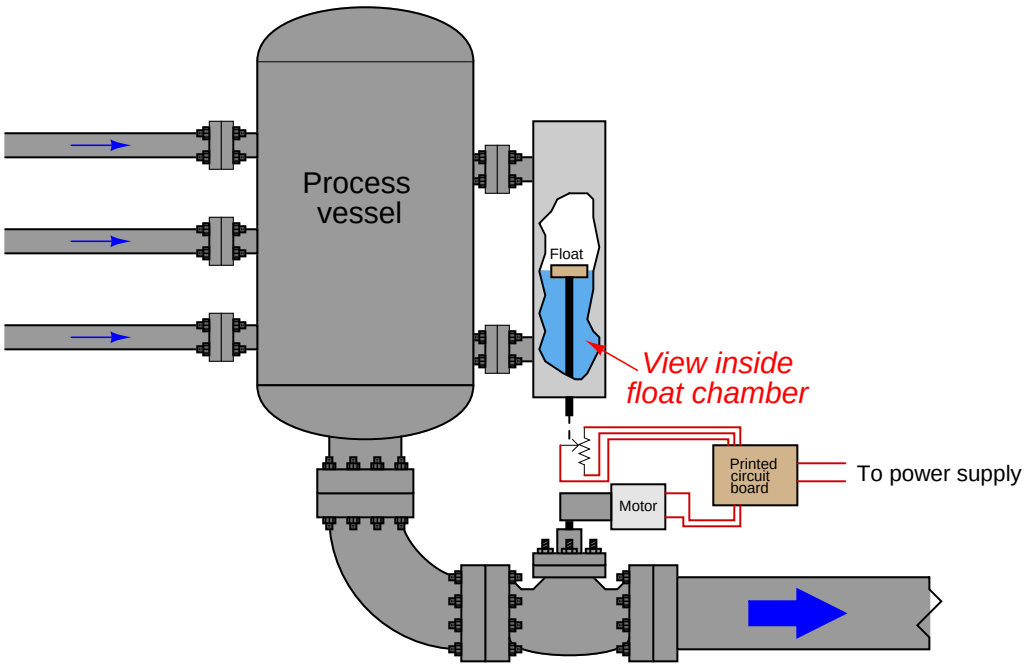
If we are limited in how much gain we can program in to the controller, how do we minimize this offset? One way is for a human operator to periodically place the controller in manual mode and move the control valve just a little bit more so the PV once again reaches SP, then place the controller back

into automatic mode. In essence this technique adjusts the "Bias" term of the controller equation. The disadvantage of this technique is rather obvious: it requires human intervention. What is the point of having an automation system requiring periodic human intervention to maintain setpoint?

A more sophisticated method for eliminating proportional-only offset is to add a different control action to the controller: one that takes action based on the amount of error between PV and SP and the amount of time that error has existed. We call this control mode *integral*, or *reset*.

29.6 Integral (reset) control

Imagine a liquid-level control system for a vessel, where the position of a level-sensing float sets the position of a potentiometer, which then sets the *speed* of a motor-actuated control valve. If the liquid level is above setpoint, the valve continually opens up; if below setpoint, the valve continually closes off:



Unlike the *proportional* control system where valve position was a direct function of float position, this control system sets the *speed* of the motor-driven valve according to the float position. The further away from setpoint the liquid level is, the *faster* the valve moves open or closed. In fact, the only time the valve will ever halt its motion is when the liquid level is precisely at setpoint; otherwise, the control valve will be in constant motion.

This control system does its job in a very different manner than the all-mechanical float-based proportional control system illustrated previously. Both systems are capable of regulating liquid level inside the vessel, but they take very different approaches to doing so. One of the most significant differences in control behavior is how the proportional system would inevitably suffer from *offset* (a persistent error between PV and SP), whereas this control system actively works at all times to eliminate offset. The motor-driven control valve literally does not rest until all error has been eliminated!

Instead of characterizing this control system as *proportional*, we call it *integral*[4] in honor of the calculus principle ("integration") whereby small quantities are accumulated over some span to form a total. Don't let the word "calculus" scare you! You are probably already familiar with the concept of numerical integration even though you may have never heard of the term before.

Calculus is a form of mathematics dealing with *changing* variables, and how rates of change relate between different variables. When we "integrate" a variable with respect to time, what we are doing is *accumulating* that variable's value as time progresses. Perhaps the simplest example of this is a vehicle odometer, accumulating the total distance traveled by the vehicle over a certain time period. This stands in contrast to a speedometer, indicating the rate of distance traveled *per* unit of time.

Imagine a car moving along at exactly 30 miles per hour. How far will this vehicle travel after 1 hour of driving this speed? Obviously, it will travel 30 miles. Now, how far will this vehicle travel if it continues for another 2 hours at the exact same speed? Obviously, it will travel 60 more miles, for a total distance of 90 miles since it began moving. If the car's speed is a constant, calculating total distance traveled is a simple matter of multiplying that speed by the travel time.

The odometer mechanism that keeps track of the mileage traveled by the car may be thought of as *integrating* the speed of the car with respect to time. In essence, it is multiplying speed times time continuously to keep a running total of how far the car has gone. When the car is traveling at a high speed, the odometer "integrates" at a faster rate. When the car is traveling slowly, the odometer "integrates" slowly.

If the car travels in reverse, the odometer will decrement (count down) rather than increment (count up) because it sees a negative quantity for speed[5]. The rate at which the odometer decrements depends on how fast the car travels in reverse. When the car is stopped (zero speed), the odometer holds its reading and neither increments nor decrements.

Now let us return to the context of an automated process to see how this calculus principle works inside a process controller. Integration is provided either by a pneumatic mechanism, an electronic opamp circuit, or by a microprocessor executing a digital integration algorithm. The variable being integrated is *error* (the difference between PV and SP) over time. Thus the integral mode of the controller ramps the output either up or down over time in response to the amount of error existing between PV and SP, and the sign of that error. We saw this "ramping" action in the behavior of the liquid level control system using a motor-driven control valve commanded by a float-positioned potentiometer: the valve stem continuously moves so long as the liquid level deviates from setpoint. The reason for this ramping action is to increase or decrease the output *as far as it is necessary* in order to completely eliminate any error and force the process variable to precisely equal setpoint. Unlike proportional action, which simply moves the output an amount proportional to any change in PV or SP, integral control action never stops moving the output until all error is eliminated.

[4] An older term for this mode of control is *floating*, which I happen to think is particularly descriptive. With a "floating" controller, the final control element continually "floats" to whatever value it must in order to completely eliminate offset.

[5] At least the old-fashioned mechanical odometers would. Modern cars use a pulse detector on the driveshaft which cannot tell the difference between forward and reverse, and therefore their odometers always increment. Shades of the movie *Ferris Bueller's Day Off*.

If proportional action is defined by the error telling the output how *far* to move, integral action is defined by the error telling the output how *fast* to move. One might think of integral as being how "impatient" the controller is, with integral action constantly ramping the output as far as it needs to go in order to eliminate error. Once the error is zero (PV = SP), of course, the integral action stops ramping, leaving the controller output (valve position) at its last value just like a stopped car's odometer holds a constant value.

If we add an integral term to the controller equation, we get something that looks like this[6]:

$$m = K_p e + \frac{1}{\tau_i} \int e \, dt + b$$

Where,

m = Controller output
e = Error (difference between PV and SP)
K_p = Proportional gain
τ_i = Integral time constant (minutes)
t = Time
b = Bias

The most confusing portion of this equation for those new to calculus is the part that says "$\int e \, dt$". The integration symbol (looks like an elongated letter "S") tells us the controller will accumulate ("sum") multiple products of error (e) over tiny slices of time (dt). Quite literally, the controller multiplies error by time (for very short segments of time, dt) and continuously adds up all those products to contribute to the output signal which then drives the control valve (or other final control element). The integral time constant (τ_i) is a value set by the technician or engineer configuring the controller, proportioning this cumulative action to make it more or less aggressive over time.

To see how this works in a practical sense, let's imagine how a proportional + integral controller would respond to the scenario of a heat exchanger whose inlet temperature suddenly dropped. As we saw with proportional-only control, an inevitable offset occurs between PV and SP with changes in load, because an error *must* develop if the controller is to generate the different output signal value necessary to halt further change in PV. We called this effect *proportional-only offset*.

Once this error develops, though, integral action begins to work. Over time, a larger and larger quantity accumulates in the integral mechanism (or register) of the controller due to the persistent error between PV and SP. That accumulated value adds to the controller's output, driving the steam control valve further and further open. This, of course, adds heat at a faster rate to the heat exchanger, which causes the outlet temperature to rise. As the temperature re-approaches setpoint, the error becomes smaller and thus the integral action proceeds at a slower rate (like a car's odometer incrementing at a slower rate as the car's speed decreases). So long as the PV is below SP (the outlet temperature is still too cool), the controller will continue to integrate upwards, driving the control valve further and further open. Only when the PV rises to exactly meet SP does

[6]The equation for a proportional + integral controller is often written without the bias term (b), because the presence of integral action makes it unnecessary. In fact, if we let the integral term completely replace the bias term, we may consider the integral term to be a self-*resetting* bias. This, in fact, is the meaning of the word "reset" in the context of PID controller action: the "reset" term of the controller acts to eliminate offset by continuously adjusting (resetting) the bias as necessary.

integral action finally rest, holding the valve at a steady position. Integral action tirelessly works to eliminate any offset between PV and SP, thus neatly eliminating the offset problem experienced with proportional-only control action.

As with proportional action, there are (unfortunately) two completely opposite ways to specify the degree of integral action offered by a controller. One way is to specify integral action in terms of *minutes* or *minutes per repeat*. A large value of "minutes" for a controller's integral action means a less aggressive integral action over time, just as a large value for proportional band means a less aggressive proportional action. The other way to specify integral action is the inverse: how many *repeats per minute*, equivalent to specifying proportional action in terms of gain (large value means aggressive action). For this reason, you will sometimes see the integral term of a PID equation written differently:

$$\tau_i = \text{minutes per repeat} \qquad K_i = \text{repeats per minute}$$

$$\frac{1}{\tau_i} \int e\, dt \qquad K_i \int e\, dt$$

Many modern digital electronic controllers allow the user to select the unit they wish to use for integral action, just as they allow a choice between specifying proportional action as gain or as proportional band.

Integral is a highly effective mode of process control. In fact, some processes respond so well to integral controller action that it is possible to operate the control loop on integral action alone, without proportional. Typically, though, process controllers implement some form of proportional plus integral ("PI") control.

Just as too much proportional gain will cause a process control system to oscillate, too much integral action (i.e. an integral time constant that is too short) will also cause oscillation. If the integration happens at too fast a rate, the controller's output will "saturate" either high or low before the process variable can make it back to setpoint. Once this happens, the only condition that will "unwind" the accumulated integral quantity is for an error to develop of the opposite sign, and remain that way long enough for a canceling quantity to accumulate. Thus, the PV must cross over the SP, guaranteeing at least another half-cycle of oscillation.

A similar problem called *reset windup* (or *integral windup*) happens when external conditions make it impossible for the controller to achieve setpoint. Imagine what would happen in the heat exchanger system if the steam boiler suddenly stopped producing steam. As outlet temperature dropped, the controller's proportional action would open up the control valve in a futile effort to raise temperature. If and when steam service is restored, proportional action would just move the valve back to its original position as the process variable returned to its original value (before the boiler died). This is how a proportional-only controller would respond to a steam "outage": nice and predictably. If the controller had integral action, however, a much worse condition would result. All the time spent with the outlet temperature below setpoint causes the controller's integral term to "wind up" in a futile attempt to admit more steam to the heat exchanger. This accumulated quantity can only be un-done by the process variable rising above setpoint for an equal error-time product[7], which means when the steam supply resumes, the temperature will rise well above setpoint

[7] Since integration is fundamentally a process of multiplication followed by addition, the units of measurement are always the product (multiplication) of the function's variables. In the case of reset (integral) control, we are multiplying

until the integral action finally "unwinds" and brings the control valve back to a same position again.

Various techniques exist to manage integral windup. Controllers may be built with limits to restrict how far the integral term can accumulate under adverse conditions. In some controllers, integral action may be turned off completely if the error exceeds a certain value. The surest fix for integral windup is human operator intervention, by placing the controller in manual mode. This typically resets the integral accumulator to a value of zero and loads a new value into the bias term of the equation to set the valve position wherever the operator decides. Operators usually wait until the process variable has returned at or near setpoint before releasing the controller into automatic mode again.

While it might appear that operator intervention is again a problem to be avoided (as it was in the case of having to correct for proportional-only offset), it is noteworthy to consider that the conditions leading to integral windup usually occur only during shut-down conditions. It is customary for human operators to run the process manually anyway during a shutdown, and so the switch to manual mode is something they would do anyway and the potential problem of windup often never manifests itself.

Integral control action has the unfortunate tendency to create loop oscillations ("cycling") if the final control element exhibits hysteresis, such as the case with a "sticky" control valve. Imagine for a moment our steam-heated heat exchanger system where the steam control valve possesses excessive packing friction and therefore refuses to move until the applied air pressure changes far enough to overcome that friction, at which point the valve "jumps" to a new position and then "sticks" in that new position. If the valve happens to stick at a stem position resulting in the product temperature settling slightly below setpoint, the controller's integral action will continually increase the output signal going to the valve in an effort to correct this error (as it should). However, when that output signal has risen far enough to overcome valve friction and move the stem further open, it is very likely the stem will once again "stick" but this time do so at a position making the product temperature settle *above* setpoint. The controller's integral action will then ramp downward in an effort to correct this new error, but due to the valve's friction making precise positioning impossible, the controller can never achieve setpoint and therefore it cyclically "hunts" above and below setpoint.

The best solution to this "reset cycling" phenomenon, of course, is to correct the hysteresis in the final control element. Eliminating friction in the control valve will permit precise positioning and allow the controller's integral action to achieve setpoint as designed. Since it is practically impossible to eliminate *all* friction from a control valve, however, other solutions to this problem exist. One of them is to program the controller to stop integrating whenever the error is less than some pre-configured value (sometimes referred to as the "integral deadband" or "reset deadband" of the controller). By activating reset control action only for significant error values, the controller ignores small errors rather than "compulsively" trying to correct for any detected error no matter how small.

---

controller error (the difference between PV and SP, usually expressed in percent) by time (usually expressed in minutes or seconds). Therefore the result will be an "error-time" product. In order for an integral controller to self-recover following windup, the error must switch signs and the error-time product accumulate to a sufficient value to cancel out the error-time product accumulated during the windup period.

29.7 Derivative (rate) control

The final element of PID control is the "D" term, which stands for *derivative*. This is a calculus concept like integral, except most people consider it easier to understand. Simply put, derivative is the expression of a variable's *rate-of-change* with respect to another variable. Finding the derivative of a function (differentiation) is the inverse operation of integration. With integration, we calculated accumulated value of some variable's product with time. With derivative, we calculate the ratio of a variable's change per unit of time. Whereas integration is fundamentally a multiplicative operation (products), differentiation always involves division (ratios).

A controller with derivative (or *rate*) action looks at how fast the process variable changes per unit of time, and takes action proportional to that rate of change. In contrast to integral (reset) action which represents the "impatience" of the controller, derivative (rate) action represents the "caution" of the controller.

If the process variable starts to change at a high rate of speed, the job of derivative action is to move the final control element in such a direction as to counteract this rapid change, and thereby moderate the speed at which the process variable changes. In simple terms, derivative action works to limit how fast the error can change.

What this will do is make the controller "cautious" with regard to rapid changes in process variable. If the process variable is headed toward the setpoint value at a rapid rate, the derivative term of the equation will diminish the output signal, thus tempering the controller's response and slowing the process variable's approach toward setpoint. This is analogous to a truck driver preemptively applying the brakes to slow the approach to an intersection, knowing that the heavy truck doesn't "stop on a dime." The heavier the truck's load, the sooner a cautious driver will apply the brakes, to avoid "overshoot" beyond the stop sign and into the intersection. For this reason, derivative control action is also called *pre-act* in addition to being called *rate*, because it acts "ahead of time" to avoid overshoot.

If we modify the controller equation to incorporate differentiation, it will look something like this:

$$m = K_p e + \frac{1}{\tau_i} \int e \, dt + \tau_d \frac{de}{dt} + b$$

Where,

m = Controller output
e = Error (difference between PV and SP)
K_p = Proportional gain
τ_i = Integral time constant (minutes)
τ_d = Derivative time constant (minutes)
t = Time
b = Bias

The $\frac{de}{dt}$ term of the equation expresses the rate of change of error (e) over time (t). The lower-case letter "d" symbols represent the calculus concept of *differentials* which may be thought of in this context as very tiny increments of the following variables. In other words, $\frac{de}{dt}$ refers to the ratio of a very small change in error (de) over a very small increment of time (dt). On a graph, this is interpreted as the slope of a curve at a specific point (slope being defined as *rise over run*).

It is also possible to build a controller with proportional and derivative actions, but lacking integral action. These are most commonly used in applications prone to wind-up[8], and where the elimination of offset is not critical:

$$m = K_p e + \tau_d \frac{de}{dt} + b$$

Many PID controllers offer the option of calculating derivative response based on rates of change for the process variable (PV) only, rather than the error (PV − SP or SP − PV). This avoids huge "spikes" in the output of the controller if ever a human operator makes a sudden change in setpoint[9]. The mathematical expression for such a controller would look like this[10]:

$$m = K_p e + \frac{1}{\tau_i} \int e \, dt + \tau_d \frac{d\text{PV}}{dt} + b$$

Even when derivative control action is calculated on PV alone (rather than on error), it is still useful for controlling processes dominated by large lag times. The presence of derivative control action in a PID controller generally means the proportional (P) and integral (I) terms may be adjusted more aggressively than before, since derivative (D) will act to limit overshoot. In other words, the judicious presence of derivative action in a PID controller lets us "get away" with using a bit more P and I action than we ordinarily could, resulting in faster approach to setpoint with minimal overshoot.

It should be mentioned that derivative mode should be used with caution. Since it acts on rates of change, derivative action will "go crazy" if it sees substantial noise in the PV signal. Even small amounts of noise possess extremely large rates of change (defined as percent PV change per minute of time) owing to the relatively high frequency of noise compared to the timescale of physical process changes.

Ziegler and Nichols, the engineers who wrote the ground-breaking paper entitled "Optimum Settings for Automatic Controllers" had these words to say regarding "pre-act" control (page 762 of the November 1942 *Transactions of the A.S.M.E.*):

> The latest control effect made its appearance under the trade name "Pre-Act." On some control applications, the addition of pre-act response made such a remarkable improvement that it appeared to be in embodiment of mythical "anticipatory" controllers. On other applications it appeared to be worse than useless. Only the difficulty of predicting the usefulness and adjustment of this response has kept it from being more widely used.

[8] An example of such an application is where the output of a loop controller may be "de-selected" or otherwise "over-ridden" by some other control function. This sort of control strategy is often used in energy-conserving controls, where multiple controllers monitoring different process variables selectively command a single FCE.

[9] It should not be assumed that such spikes are always undesirable. In processes characterized by long lag times, such a response may be quite helpful in overcoming that lag for the purpose of rapidly achieving new setpoint values. Slave (secondary) controllers in cascaded systems – where the controller receives its setpoint signal from the output of another (primary, or master) controller – may similarly benefit from derivative action calculated on error instead of just PV. As usual, the specific needs of the application dictate the ideal controller configuration.

[10] The expression shown is valid for a direct-acting controller. A reverse-acting controller with derivative action on PV rather than error must *subtract* the derivative term rather than add it to the output value.

29.8 Summary of PID control terms

PID control can be a confusing concept to understand. Here, a brief summary of each term within PID (P. I, and D) is presented for your learning benefit.

29.8.1 Proportional control mode (P)

Proportional – sometimes called *gain* or *sensitivity* – is a control action reproducing changes in input as changes in output. Proportional controller action responds to present changes in input by generating immediate and commensurate changes in output. When you think of "proportional action" (P), think *prompt*: this control action works immediately (never too soon or too late) to match changes in the input signal.

Mathematically defined, proportional action is the ratio of output change to input change. This may be expressed as a quotient of differences, or as a derivative (a rate of change, using calculus notation):

$$\text{Gain value} = \frac{\Delta\text{Output}}{\Delta\text{Input}}$$

$$\text{Gain value} = \frac{d\text{Output}}{d\text{Input}} = \frac{dm}{de}$$

For example, if the PV input of a proportional-only process controller with a gain of 2 suddenly changes ("steps") by 5 percent, and the output will immediately jump by 10 percent ($\Delta\text{Output} = \text{Gain} \times \Delta\text{Input}$). The direction of this output jump in relation to the direction of the input jump depends on whether the controller is configured for direct or reverse action.

A legacy term used to express this same concept is *proportional band*: the mathematical reciprocal of gain. "Proportional band" is defined as the amount of input change necessary to evoke full-scale (100%) output change in a proportional controller. Incidentally, it is always expressed as a percentage, never as fraction or as a per unit value:

$$\text{Proportional Band value} = \frac{\Delta\text{Input}}{\Delta\text{Output}}$$

$$\text{Proportional Band value} = \frac{d\text{Input}}{d\text{Output}} = \frac{de}{dm}$$

Using the same example of a proportional controller exhibiting an output "step" of 10% in response to a PV "step" of 5%, the proportional band would be 50%: the reciprocal of its gain ($\frac{1}{2} = 50\%$). Another way of saying this is that a 50% input "step" would be required to change the output of this controller by a full 100%, since its gain is set to a value of 2.

29.8.2 Integral control mode (I)

Integral – sometimes called *reset* or *floating control* – is a control action causing the output signal to change over time at a rate proportional to the amount of error (the difference between PV and SP values). Integral controller action responds to error accumulated over time, ramping the output signal are far as it needs to go to completely eliminate error. If proportional (P) action tells the output how *far* to move when an error appears, integral (I) action tells the output how *fast* to move when an error appears. If proportional (P) action acts on the *present*, integral (I) action acts on the *past*. Thus, how far the output signal gets driven by integral action depends on the *history* of the error over time: how much error existed, and for how long. When you think of "integral action" (I), think *impatience*: this control action drives the output further and further the longer PV fails to match SP.

Mathematically defined, integral action is the ratio of output *velocity* to input error:

$$\text{Integral value (repeats per minute)} = \frac{\text{Output velocity}}{\text{Input error}}$$

$$\text{Integral value (repeats per minute)} = \frac{\frac{dm}{dt}}{e}$$

An alternate way to express integral action is to use the reciprocal unit of "minutes per repeat." If we define integral action in these terms, the defining equations must be reciprocated:

$$\text{Integral time constant (minutes per repeat)} = \tau_i = \frac{\text{Input error}}{\text{Output velocity}}$$

$$\text{Integral time constant (minutes per repeat)} = \tau_i = \frac{e}{\frac{dm}{dt}}$$

For example, if an error of 5% appears between PV and SP on an integral-only process controller with an integral value of 3 repeats per minute (i.e. an integral time constant of 0.333 minutes per repeat), the output will begin ramping at a rate of 15% per minute ($\frac{dm}{dt}$ = Integral_value $\times$ e, or $\frac{dm}{dt} = \frac{e}{\tau_i}$). In most PI and PID controllers, integral response is also multiplied by proportional gain, so the same conditions applied to a PI controller that happened to also have a gain of 2 would result in an output ramping rate of 30% per minute ($\frac{dm}{dt}$ = Gain_value $\times$ Integral_value $\times$ e, or $\frac{dm}{dt}$ = Gain_value $\times$ $\frac{e}{\tau_i}$). The direction of this ramping in relation to the direction (sign) of the error depends on whether the controller is configured for direct or reverse action.

29.8.3 Derivative control mode (D)

Derivative – sometimes called *rate* or *pre-act* – is a control action causing the output signal to be offset by an amount proportional to the rate at which the input is changing. Derivative controller action responds to how quickly the input changes over time, biasing the output signal commensurate with that rate of input change. If proportional (P) action tells the output how *far* to move when an error appears, derivative (D) action tells the output how far to move when the input *ramps*. If proportional (P) action acts on the *present* and integral (I) action acts on the *past*, derivative (D) action acts on the *future*: it effectively "anticipates" overshoot by tempering the output response according to how fast the process variable is rising or falling. When you think of "derivative action" (D), think *discretion*: this control action is cautious and prudent, working against change.

Mathematically defined, derivative action is the ratio of output offset to input *velocity*:

$$\text{Derivative time constant (minutes)} = \tau_d = \frac{\text{Output offset}}{\text{Input velocity}}$$

$$\text{Derivative time constant (minutes)} = \tau_d = \frac{\Delta\text{Output}}{\frac{de}{dt}}$$

For example, if the PV signal begins to ramp at a rate of 5% per minute on a process controller with a derivative time constant of 4 minutes, the output will immediately become offset by 20% (ΔOutput = Derivative_value $\times$ $\frac{de}{dt}$). In most PD and PID controllers, derivative response is also multiplied by proportional gain, so the same conditions applied to a PD controller that happened to also have a gain of 2 would result in an immediate offset of 40% (ΔOutput = Gain_value $\times$ Derivative_value $\times$ $\frac{de}{dt}$). The direction (sign) of this offset in relation to the direction of the input ramping depends on whether the controller is configured for direct or reverse action.

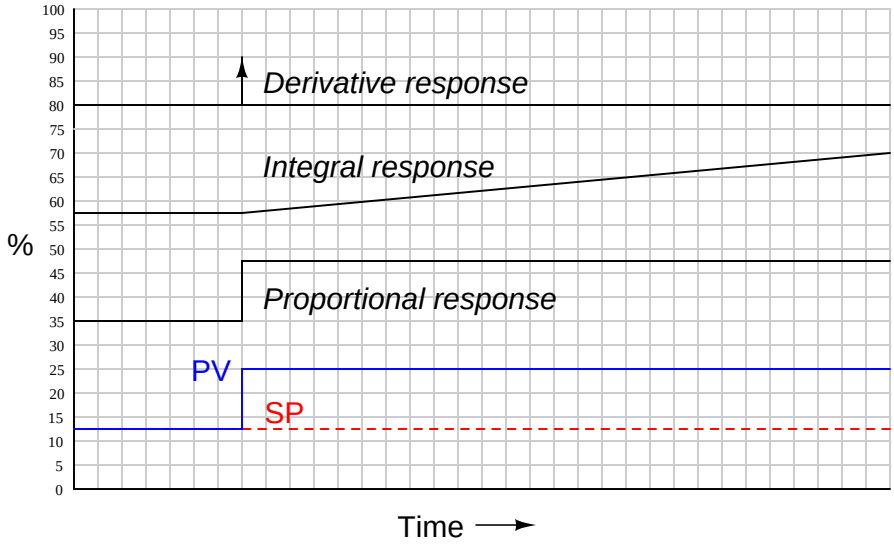
29.9 P, I, and D responses graphed

A very helpful method for understanding the operation of proportional, integral, and derivative control terms is to analyze their respective responses to the same input conditions over time. This section is divided into subsections showing P, I, and D responses for several different input conditions, in the form of graphs. In each graph, the controller is assumed to be *direct-acting* (i.e. an increase in process variable results in an increase in output).

It should be noted that these graphic illustrations are all qualitative, not quantitative. There is too little information given in each case to plot exact responses. The illustrations of P, I, and D actions focus only on the *shapes* of the responses, not their exact numerical values.

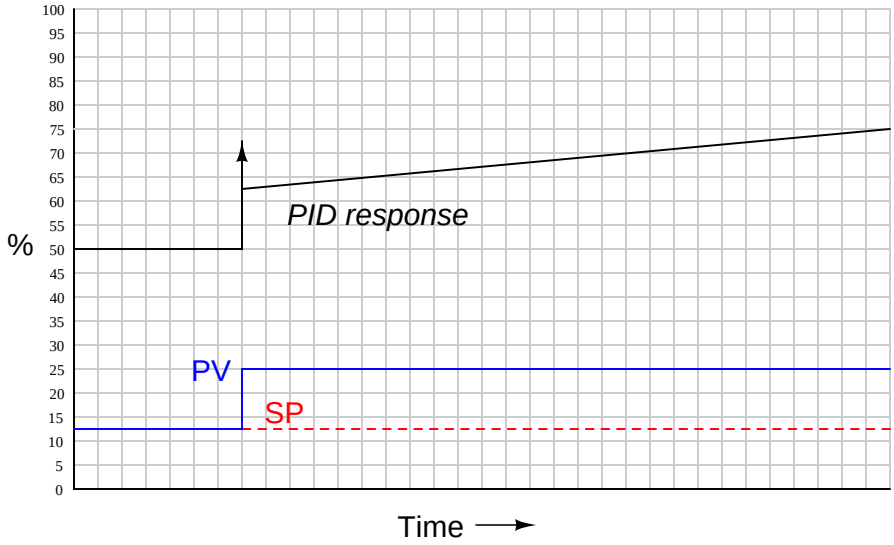
In order to *quantitatively* predict PID controller responses, one would have to know the values of all PID settings, as well as the original starting value of the output before an input change occurred and a time index of when the change(s) occurred.

29.9.1 Responses to a single step-change



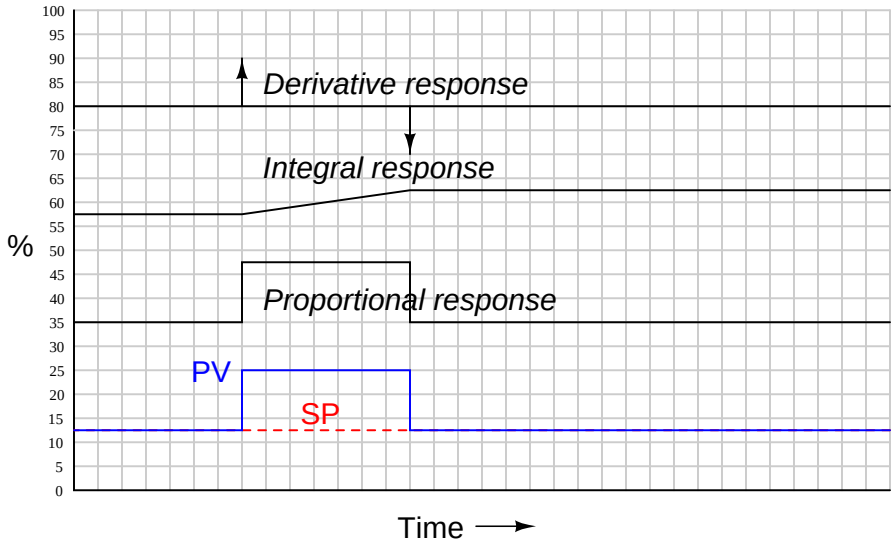
Proportional action directly mimics the shape of the input change (a step). Integral action ramps at a rate proportional to the magnitude of the input step. Since the input step holds a constant value, the integral action ramps at a constant rate (a constant *slope*). Derivative action interprets the step as an *infinite* rate of change, and so generates a "spike[11]" driving the output to saturation.

When combined into one PID output, the three actions produce this response:



[11] This is the meaning of the vertical-pointing arrowheads shown on the trend graph: momentary saturation of the output all the way up to 100%.

29.9.2 Responses to a momentary step-and-return

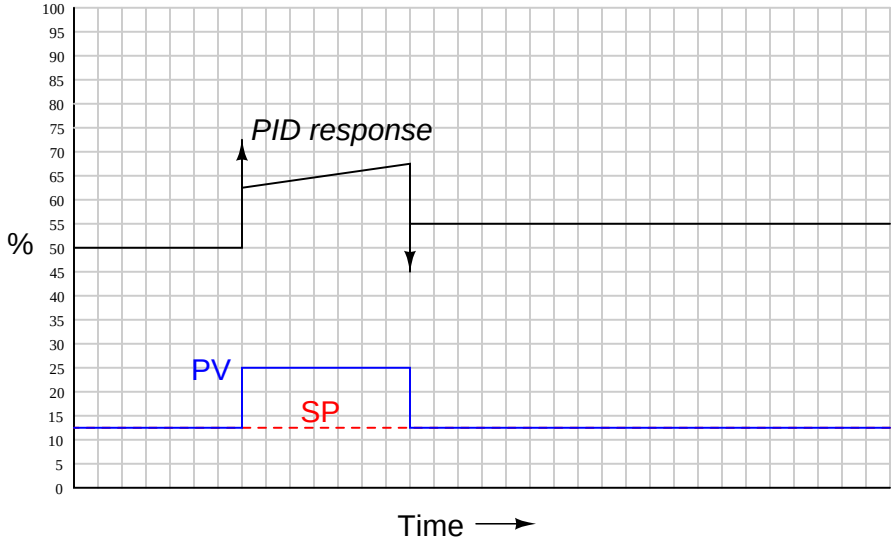


Proportional action directly mimics the shape of the input change (an up-and-down step). Integral action ramps at a rate proportional to the magnitude of the input step, for as long as the PV is unequal to the SP. Once PV = SP again, integral action stops ramping and simply holds the last value[12]. Derivative action interprets both steps as *infinite* rates of change, and so generates "spikes[13]" at the leading and at the trailing edges of the step. Note how the leading (rising) edge causes derivative action to saturate high, while the trailing (falling) edge causes it to saturate low.

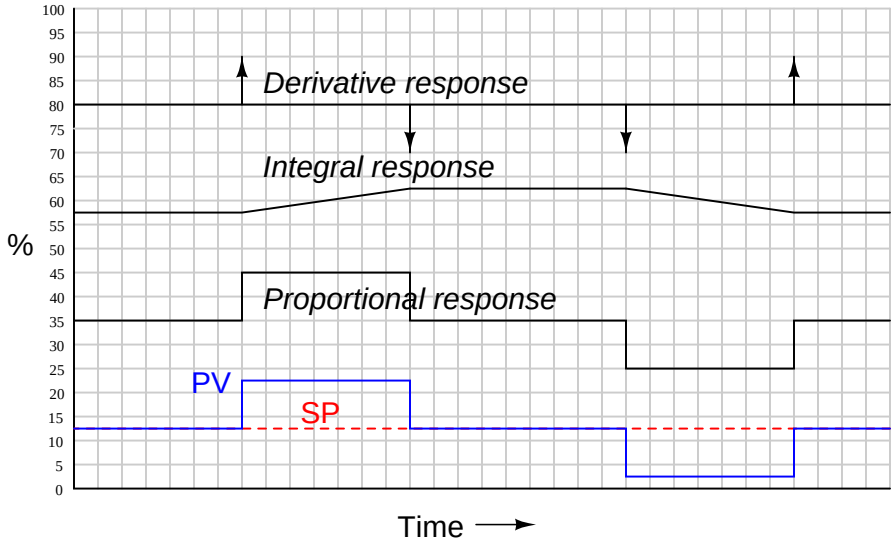
[12] This is a good example of how integral controller action represents the *history* of the PV − SP error. The continued offset of integral action from its starting point "remembers" the area accumulated under the rectangular "step" between PV and SP. This offset will go away only if a *negative* error appears having the same percent-minute product (area) as the positive error step.

[13] This is the meaning of the vertical-pointing arrowheads shown on the trend graph: momentary saturation of the output all the way up to 100% (or down to 0%).

When combined into one PID output, the three actions produce this response:

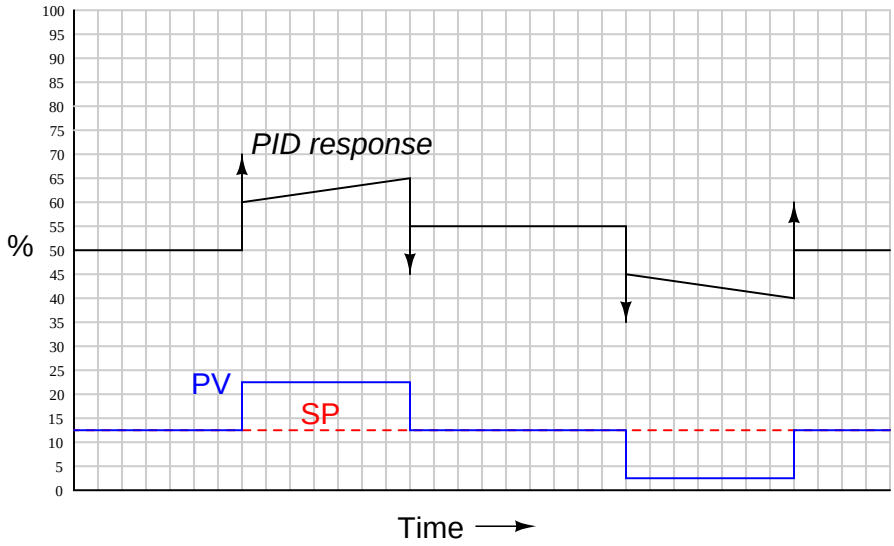


29.9.3 Responses to two momentary steps-and-returns

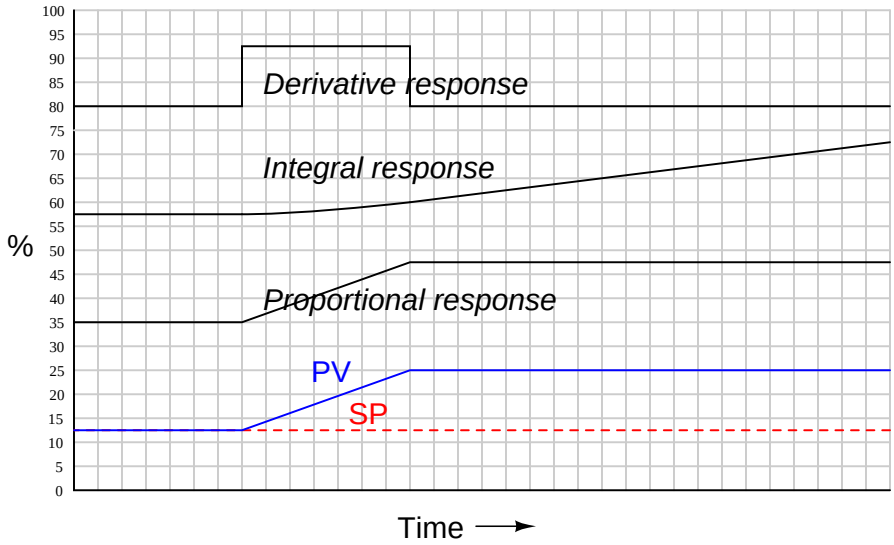


Proportional action directly mimics the shape of all input changes. Integral action ramps at a rate proportional to the magnitude of the input step, for as long as the PV is unequal to the SP. Once PV = SP again, integral action stops ramping and simply holds the last value. Derivative action interprets each step as an *infinite* rate of change, and so generates a "spike" at the leading and at the trailing edges of each step. Note how a leading (rising) edge causes derivative action to saturate high, while a trailing (falling) edge causes it to saturate low.

When combined into one PID output, the three actions produce this response:

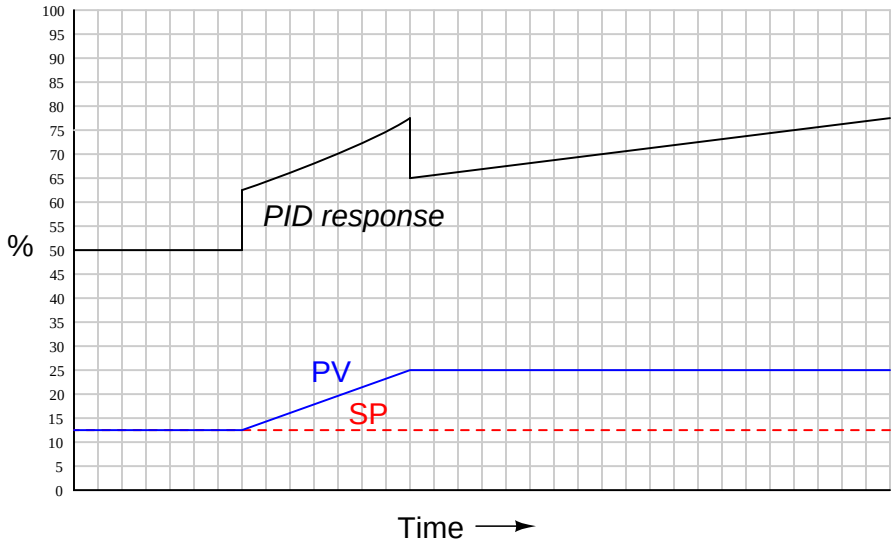


29.9.4 Responses to a ramp-and-hold

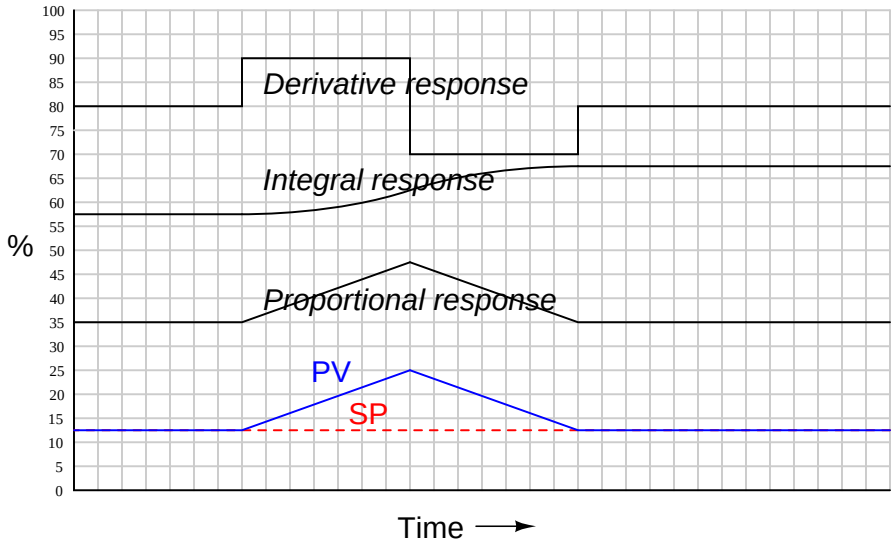


Proportional action directly mimics the ramp-and-hold shape of the input. Integral action ramps slowly at first (when the error is small) but increases ramping rate as error increases. When error stabilizes, integral rate likewise stabilizes. Derivative action offsets the output according to the input's ramping rate.

When combined into one PID output, the three actions produce this response:

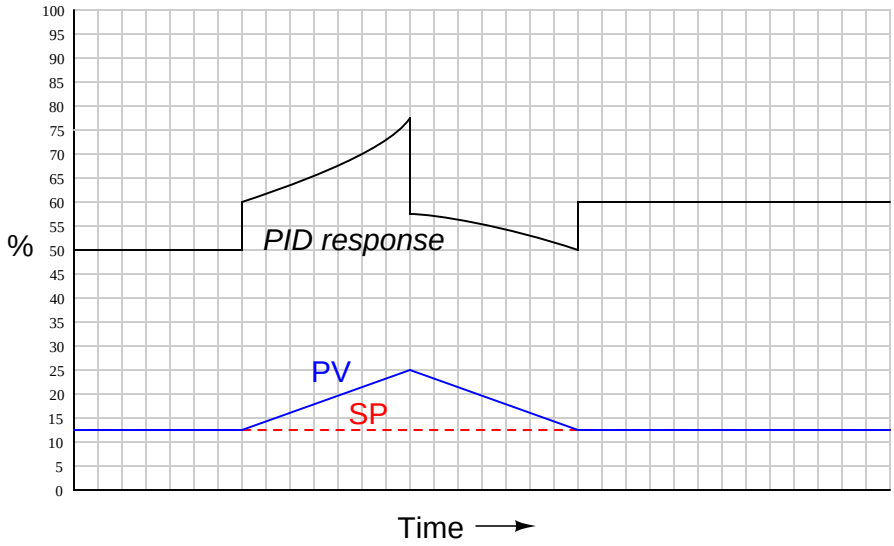


29.9.5 Responses to an up-and-down ramp

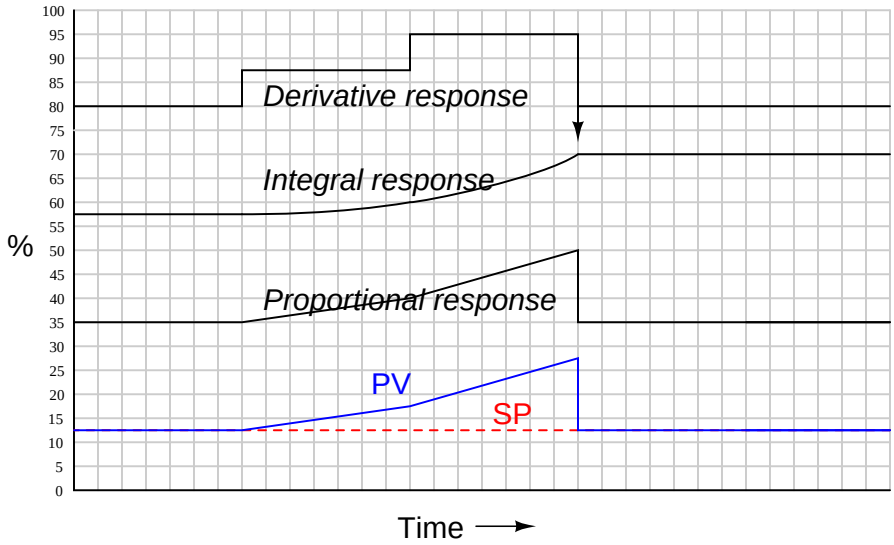


Proportional action directly mimics the up-and-down ramp shape of the input. Integral action ramps slowly at first (when the error is small) but increases ramping rate as error increases, then ramps slower as error decreases back to zero. Once PV = SP again, integral action stops ramping and simply holds the last value. Derivative action offsets the output according to the input's ramping rate: first positive then negative.

When combined into one PID output, the three actions produce this response:

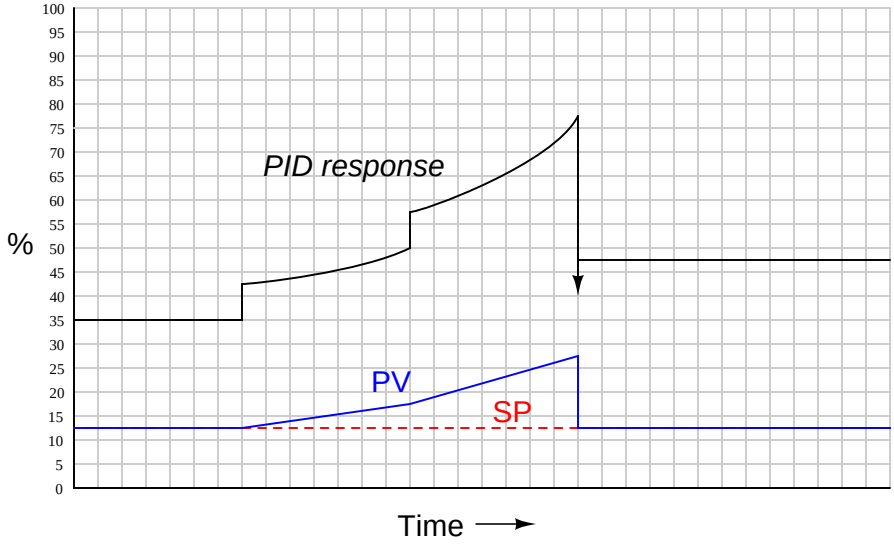


29.9.6 Responses to a multi-slope ramp

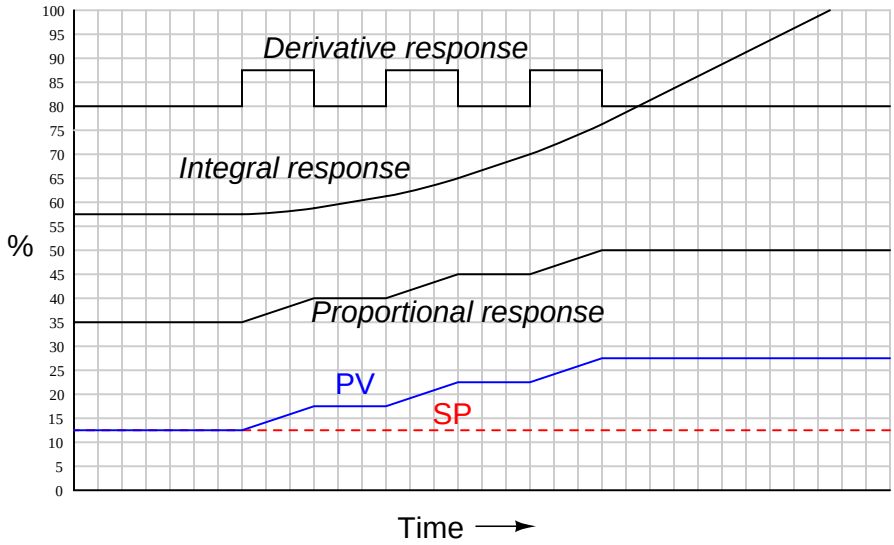


Proportional action directly mimics the ramp shape of the input. Integral action ramps slowly at first (when the error is small) but increases ramping rate as error increases, then accelerates its increase as the PV ramps even steeper. Once PV = SP again, integral action stops ramping and simply holds the last value. Derivative action offsets the output according to the input's ramping rate: first positive, then more positive, then it spikes negative when the PV suddenly returns to SP.

When combined into one PID output, the three actions produce this response:

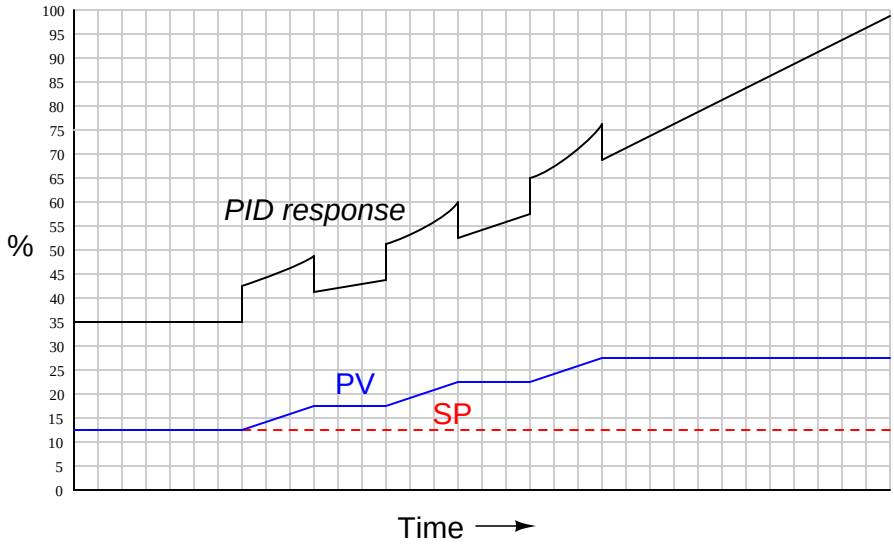


29.9.7 Responses to a multiple ramps and steps

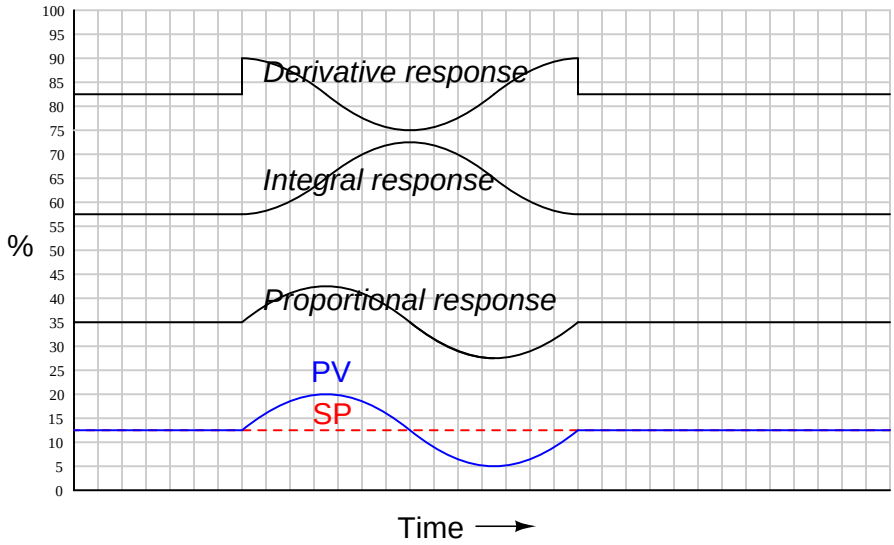


Proportional action directly mimics the ramp-and-step shape of the input. Integral action ramps slowly at first (when the error is small) but increases ramping rate as error increases. Which each higher ramp-and-step in PV, integral action winds up at an ever-increasing rate. Since PV never equals SP again, integral action never stops ramping upward. Derivative action steps with each ramp of the PV.

When combined into one PID output, the three actions produce this response:



29.9.8 Responses to a sine wavelet



As always, proportional action directly mimics the shape of the input. The 90° phase shift seen in the integral and derivative responses, compared to the PV wavelet, is no accident or coincidence. The derivative of a sinusoidal function is *always* a cosine function, which is mathematically identical to a sine function with the angle advanced by 90°:

$$\frac{d}{dx}(\sin x) = \cos x = \sin(x + 90^o)$$

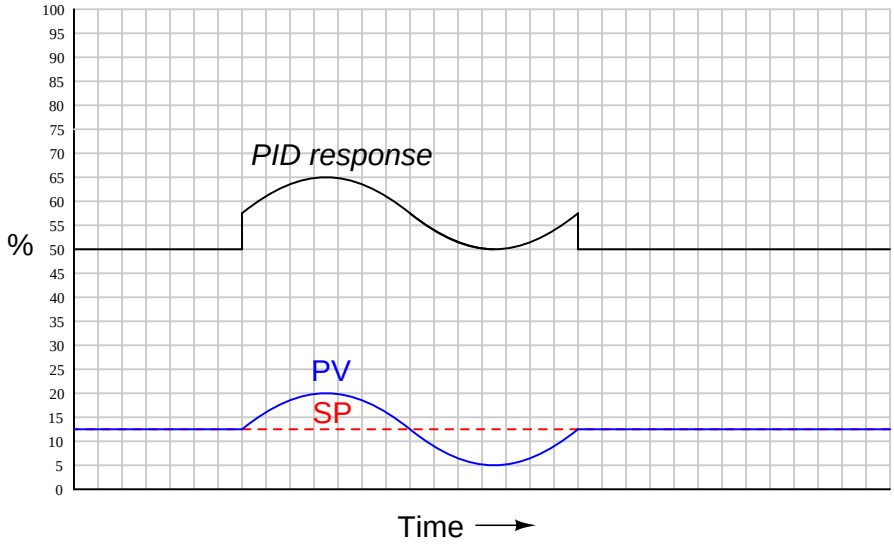
Conversely, the integral of a sine function is *always* a negative cosine function[14], which is mathematically identical to a sine function with the angle retarded by 90°:

$$\int \sin x \, dx = -\cos x = \sin(x - 90^o)$$

In summary, the derivative operation always adds a positive (leading) phase shift to a sinusoidal input waveform, while the integral operation always adds a negative (lagging) phase shift to a sinusoidal input waveform.

[14] In this example, I have omitted the constant of integration (C) to keep things simple. The actual integral is as such: $\int \sin x \, dx = -\cos x + C = \sin(x - 90^o) + C$. This constant value is essential to explaining why the integral response does not immediately "step" like the derivative response does at the beginning of the PV sine wavelet.

When combined into one PID output, these particular integral and derivative actions mostly cancel, since they happen to be sinusoidal wavelets of equal amplitude and opposite phase. Thus, the only way that the final (PID) output differs from proportional-only action in this particular case is the "steps" caused by derivative action responding to the input's sudden rise at the beginning and end of the wavelet:



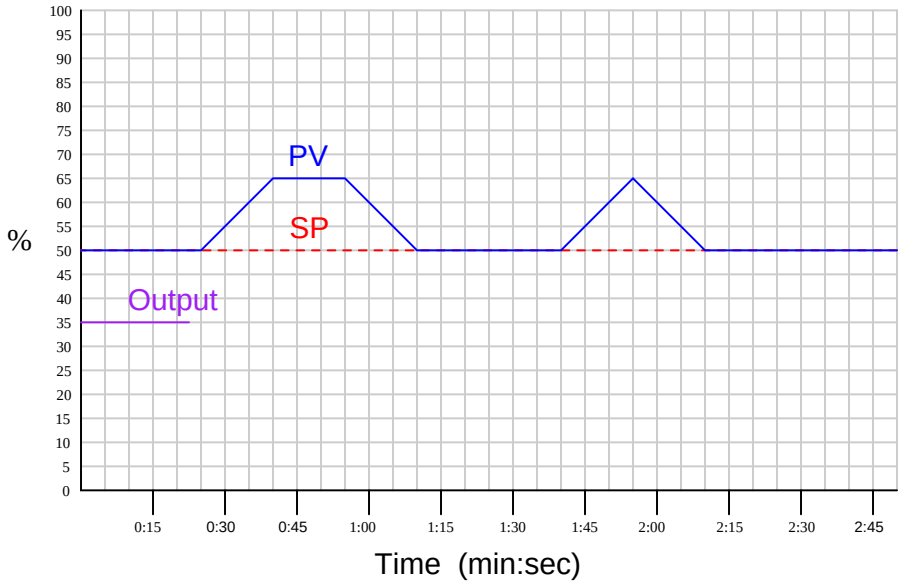
If the I and D tuning parameters were such that the integral and derivative responses were *not* equal in amplitude, their effects would not completely cancel. Rather, the resultant of P, I, and D actions would be a sine wavelet having a phase shift somewhere between -90° and $+90^o$ exclusive, depending on the relative strengths of the P, I, and D actions.

The 90 degree phase shifts associated with the integral and derivative operations are useful to understand when tuning PID controllers. If one is familiar with these phase shift relationships, it is relatively easy to analyze the response of a PID controller to a sinusoidal input (such as when a process oscillates following a sudden load or setpoint change) to determine if the controller's response is dominated by any one of the three actions. This may be helpful in "de-tuning" an over-tuned (overly aggressive) PID controller, if an excess of P, I, or D action may be identified from a phase comparison of PV and output waveforms.

29.9.9 Note to students regarding quantitative graphing

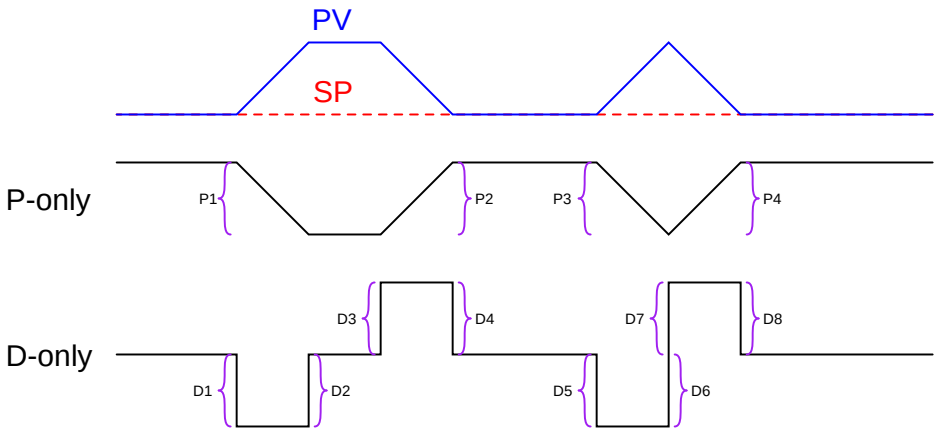
A common exercise for students learning the function of PID controllers is to practice graphing a controller's output given input (PV and SP) conditions, either qualitatively or quantitatively. This can be a frustrating experience for some students, as they struggle to accurately combine the effects of P, I, and/or D responses into a single output trend. Here, I will present a way to ease the pain.

Suppose for example you were tasked with graphing the response of a PD (proportional + derivative) controller to the following PV and SP inputs over time. You are told the controller has a gain of 1, a derivative time constant of 0.3 minutes, and is reverse-acting:

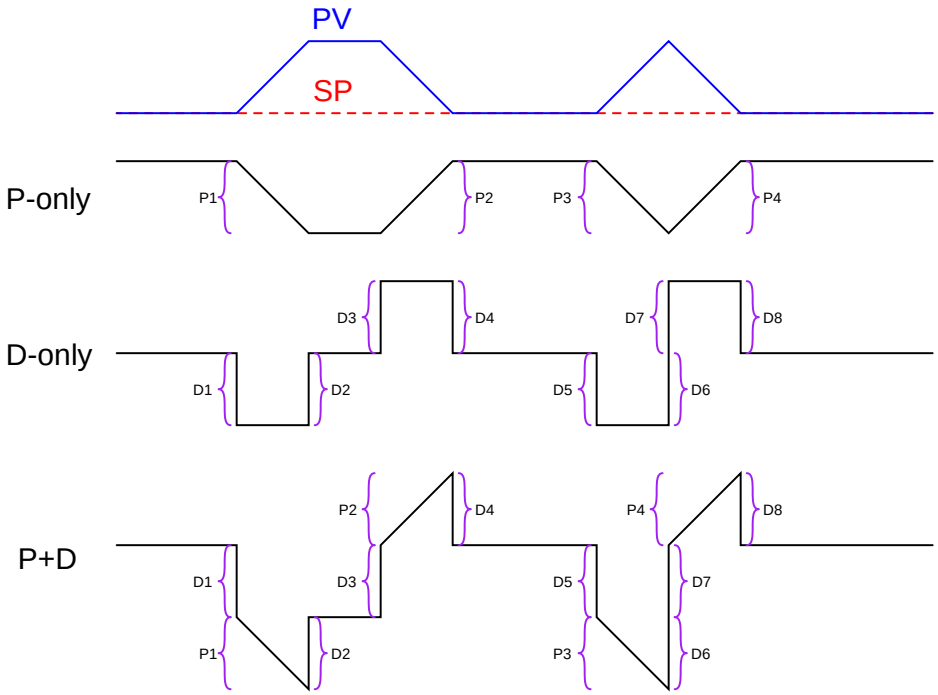


My first recommendation is to *qualitatively* sketch the individual P and D responses. Simply draw two different trends, each one right above or below the given PV/SP trends, showing the shapes of each response over time. You might even find it easier to do if you re-draw the original PV and SP trends on a piece of non-graph paper with the qualitative P and D trends also sketched on the same piece of non-graph paper. The purpose of the qualitative sketches is to separate the task of determining shapes from the task of determining numerical values, in order to simplify the process.

After sketching the separate P and D trends, label each one of the "features" (changes either up or down) in these qualitative trends. This will allow you to more easily combine the effects into one output trend later:



Now, you may qualitatively sketch an output trend combining each of these "features" into one graph. Be sure to label each ramp or step originating with the separate P or D trends, so you know where each "feature" of the combined output graph originates from:

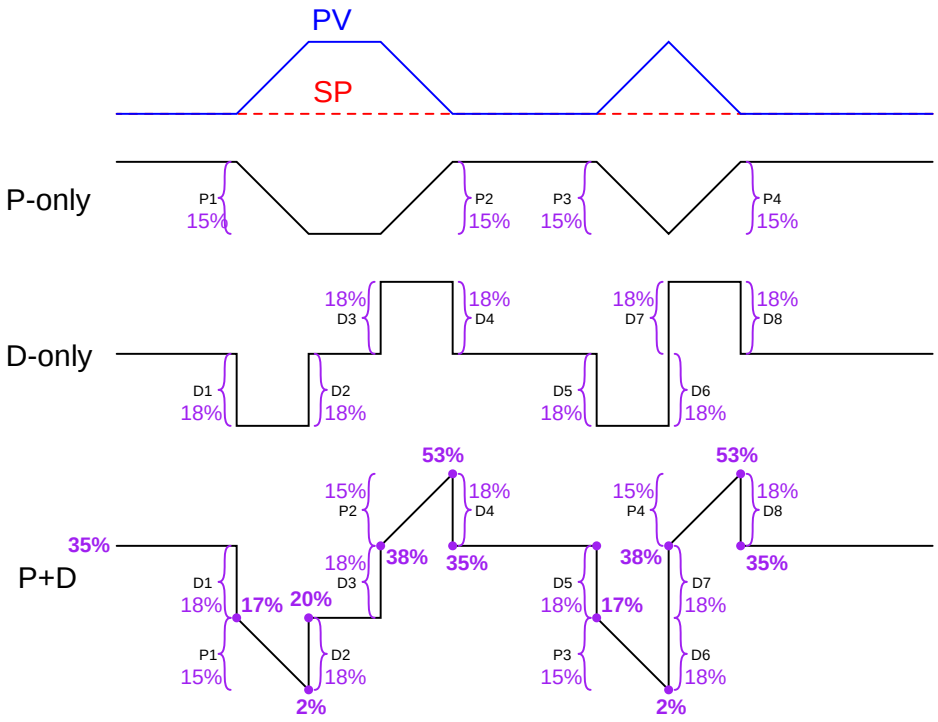


Once the general shape of the output has been qualitatively determined, you may go back to the separate P and D trends to calculate numerical values for each of the labeled "features."

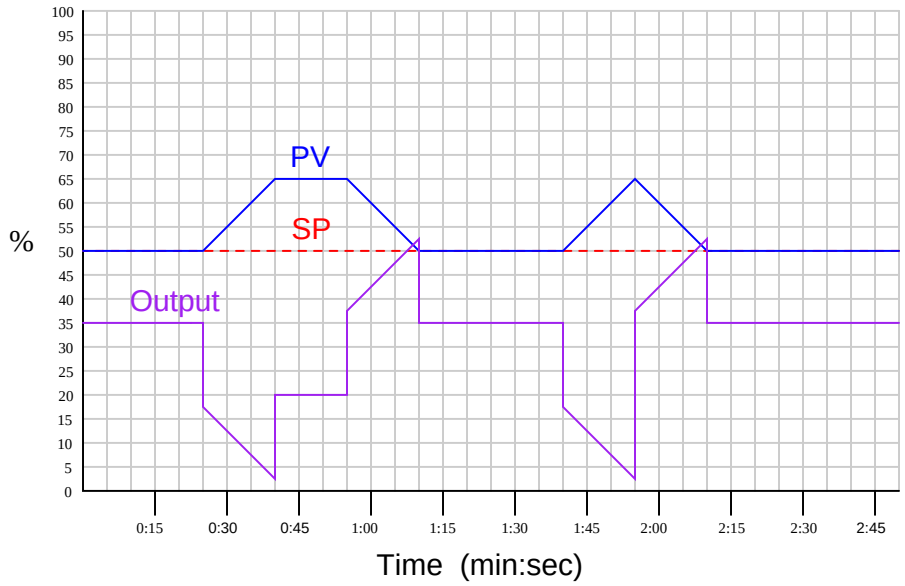
Note that each of the PV ramps is 15% in height, over a time of 15 seconds (one-quarter of a minute). With a controller gain of 1, the proportional response to each of these ramps will also be a ramp that is 15% in height.

Taking our given derivative time constant of 0.3 minutes and multiplying that by the PV's rate-of-change ($\frac{d\text{PV}}{dt}$) during each of its ramping periods (15% per one-quarter minute, or 60% per minute) yields a derivative response of 18% during each of the ramping periods. Thus, each derivative response "step" will be 18% in height.

Going back to the qualitative sketches of P and D actions, and to the combined (qualitative) output sketch, we may apply the calculated values of 15% for each proportional ramp and 18% for each derivative step to the labeled "features." We may also label the starting value of the output trend as given in the original problem (35%), to calculate actual output values at different points in time. Calculating output values at specific points in the graph becomes as easy as cumulatively adding and subtracting the P and D "feature" values to the starting output value:



Now that we know the output values at all the critical points, we may quantitatively sketch the output trend on the original graph:



29.10 Different PID equations

For better or worse, there are no fewer than *three* different forms of PID equations implemented in modern PID controllers: the *parallel*, *ideal*, and *series*. Some controllers offer the choice of more than one equation, while others implement just one. It should be noted that more variations of PID equation exist than these three, but that these are the three major variations.

29.10.1 Parallel PID equation

The equation used to describe PID control so far in this chapter is the simplest form, sometimes called the *parallel* equation, because each action (P, I, and D) occurs in separate terms of the equation, with the combined effect being a simple sum:

$$m = K_p e + \frac{1}{\tau_i} \int e\, dt + \tau_d \frac{de}{dt} + b \qquad \textbf{Parallel PID equation}$$

In the parallel equation, each action parameter (K_p, τ_i, τ_d) is independent of the others. At first, this may seem to be an advantage, for it means each adjustment made to the controller should only affect one aspect of its action. However, there are times when it is better to have the gain parameter affect all three control actions (P, I, and D)[15].

We may show the independence of the three actions mathematically, by breaking the equation up into three different parts, each one describing its contribution to the output (Δm):

$$\Delta m = K_p \Delta e \qquad \text{Proportional action}$$

$$\Delta m = \frac{1}{\tau_i} \int e\, dt \qquad \text{Integral action}$$

$$\Delta m = \tau_d \frac{de}{dt} \qquad \text{Derivative action}$$

As you can see, the three portions of this PID equation are completely separate, with each tuning parameter (K_p, τ_i, and τ_d) acting independently within its own term of the equation.

[15] An example of a case where it is better for gain (K_p) to influence all three control modes is when a technician re-ranges a transmitter to have a larger or smaller span than before, and must re-tune the controller to maintain the same loop gain as before. If the controller's PID equation takes the parallel form, the technician must adjust the P, I, *and* D tuning parameters proportionately. If the controller's PID equation uses K_p as a factor in all three modes, the technician need only adjust K_p to re-stabilize the loop.

29.10.2 Ideal PID equation

An alternate version of the PID equation designed such that the gain (K_p) affects all three actions is called the *Ideal* or *ISA* equation:

$$m = K_p \left(e + \frac{1}{\tau_i} \int e \, dt + \tau_d \frac{de}{dt} \right) + b \qquad \textbf{Ideal or ISA PID equation}$$

Here, the gain constant (K_p) is distributed to all terms within the parentheses, equally affecting all three control actions. Increasing K_p in this style of PID controller makes the P, the I, *and* the D actions equally more aggressive.

We may show this mathematically, by breaking the "ideal" equation up into three different parts, each one describing its contribution to the output (Δm):

$$\Delta m = K_p \Delta e \qquad \text{Proportional action}$$

$$\Delta m = \frac{K_p}{\tau_i} \int e \, dt \qquad \text{Integral action}$$

$$\Delta m = K_p \tau_d \frac{de}{dt} \qquad \text{Derivative action}$$

As you can see, all three portions of this PID equation are influenced by the gain (K_p) owing to algebraic distribution, but the integral and derivative tuning parameters (τ_i and τ_d) act independently within their own terms of the equation.

29.10.3 Series PID equation

A third version, with origins in the peculiarities of pneumatic controller mechanisms and analog electronic circuits, is called the *Series* or *Interacting* equation:

$$m = K_p \left[\left(\frac{\tau_d}{\tau_i} + 1 \right) e + \frac{1}{\tau_i} \int e \, dt + \tau_d \frac{de}{dt} \right] + b \qquad \textbf{Series or Interacting PID equation}$$

Here, the gain constant (K_p) affects all three actions (P, I, and D) just as with the "ideal" equation. The difference, though, is the fact that both the integral and derivative constants have an effect on proportional action as well! That is to say, adjusting either τ_i or τ_d does not merely adjust those actions, but also influences the aggressiveness of proportional action[16].

We may show this mathematically, by breaking the "series" equation up into three different parts, each one describing its contribution to the output (Δm):

$$\Delta m = K_p \left(\frac{\tau_d}{\tau_i} + 1 \right) \Delta e \qquad \text{Proportional action}$$

$$\Delta m = \frac{K_p}{\tau_i} \int e \, dt \qquad \text{Integral action}$$

$$\Delta m = K_p \tau_d \frac{de}{dt} \qquad \text{Derivative action}$$

As you can see, all three portions of this PID equation are influenced by the gain (K_p) owing to algebraic distribution. However, the proportional term is also affected by the values of the integral and derivative tuning parameters (τ_i and τ_d). Therefore, adjusting τ_i affects both the I and P actions, adjusting τ_d affects both the D and P actions, and adjusting K_p affects all three actions.

This "interacting" equation is an artifact of certain pneumatic and electronic controller designs. Back when these were the dominant technologies, and PID controllers were modularly designed such that integral and derivative actions were separate hardware modules included in a controller at additional cost beyond proportional-only action, the easiest way to implement the integral and derivative actions was in a way that just happened to have an interactive effect on controller gain. In other words, this odd equation form was a sort of compromise made for the purpose of simplifying the physical design of the controller.

Interestingly enough, many digital PID controllers are programmed to implement the "interacting" PID equation even though it is no longer an artifact of controller hardware. The rationale for this programming is to have the digital controller behave identically to the legacy analog electronic or pneumatic controller it is replacing. This way, the proven tuning parameters of the old controller may be plugged into the new digital controller, yielding the same results. In essence, this is a form of "backward compatibility" between digital PID control and analog (electronic or pneumatic) PID control.

[16] This becomes especially apparent when using derivative action with low values of τ_i (aggressive integral action). The error-multiplying term $\frac{\tau_d}{\tau_i} + 1$ may become quite large if τ_i is small, even with modest τ_d values.

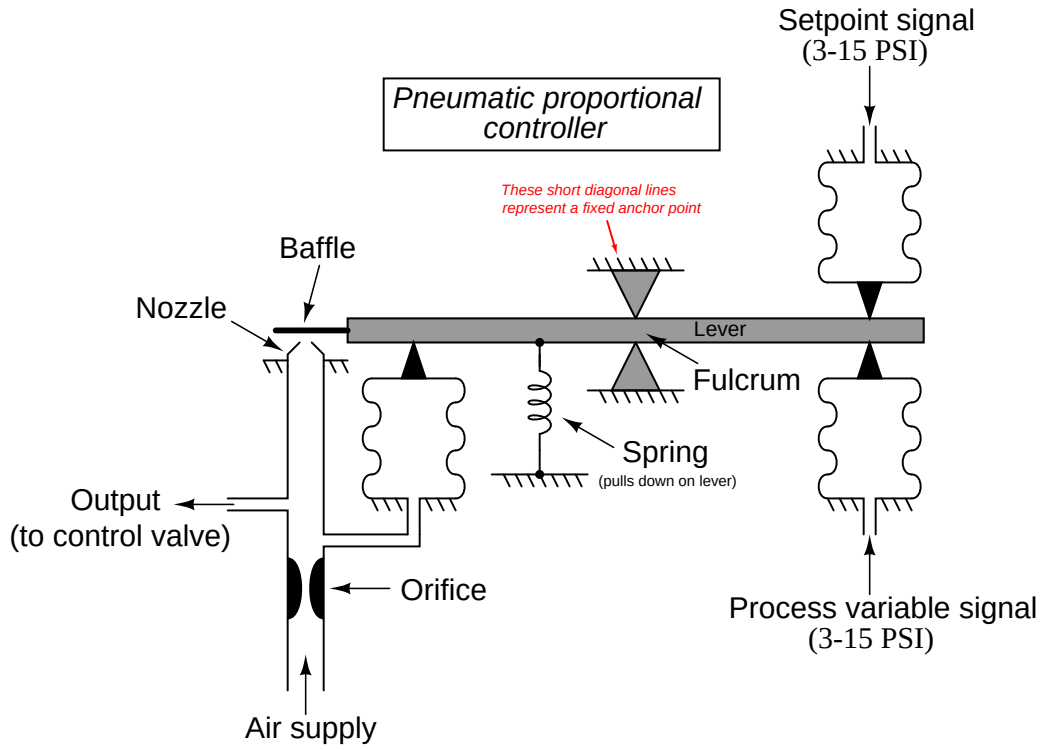
29.11 Pneumatic PID controllers

A *pneumatic* controller receives a process variable (PV) signal as a variable air pressure, compares that signal against a desired setpoint (SP) value, and then mechanically generates another air pressure signal as the output, driving a final control element.

Throughout this section I will make reference to a pneumatic controller mechanism of my own design. This mechanism does not directly correspond to any particular manufacturer or model of pneumatic controller, but shares characteristics common to many. This design is shown here for the purpose of illustrating the development of P, I, and D control actions in as simple a context as possible.

29.11.1 Proportional control action

Many pneumatic PID controllers use the *force-balance* principle. One or more input signals (in the form of pneumatic pressures) exert a force on a beam by acting through diaphragms, bellows, and/or bourdon tubes, which is then counter-acted by the force exerted on the same beam by an output air pressure acting through a diaphragm, bellows, or bourdon tube. The self-balancing mechanical system "tries" to keep the beam motionless through an exact balancing of forces, the beam's position precisely detected by a nozzle/baffle mechanism:

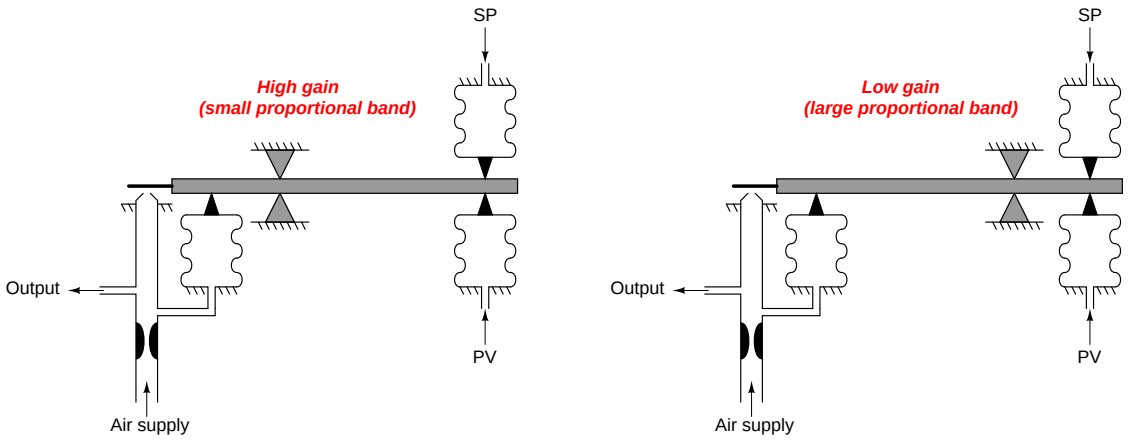


The action of this particular controller is *direct*, since an increase in process variable signal (pressure) results in an increase in output signal (pressure). Increasing process variable (PV) pressure attempts to push the right-hand end of the beam up, causing the baffle to approach the nozzle. This blockage of the nozzle causes the nozzle's pneumatic backpressure to increase, thus increasing the amount of force applied by the output feedback bellows on the left-hand end of the beam and returning the flapper (very nearly) to its original position. If we wished to reverse the controller's action, all we would need to do is swap the pneumatic signal connections between the input bellows, so that the PV pressure was applied to the upper bellows and the SP pressure to the lower bellows.

Any factor influencing the ratio of input pressure(s) to output pressure may be exploited as a gain (proportional band) adjustment in this mechanism. Changing bellows area (either both the PV and SP bellows equally, or the output bellows by itself) would influence this ratio, as would a change in output bellows position (such that it pressed against the beam at some difference distance

from the fulcrum point). Moving the fulcrum left or right is also an option for gain control, and in fact is usually the most convenient to engineer.

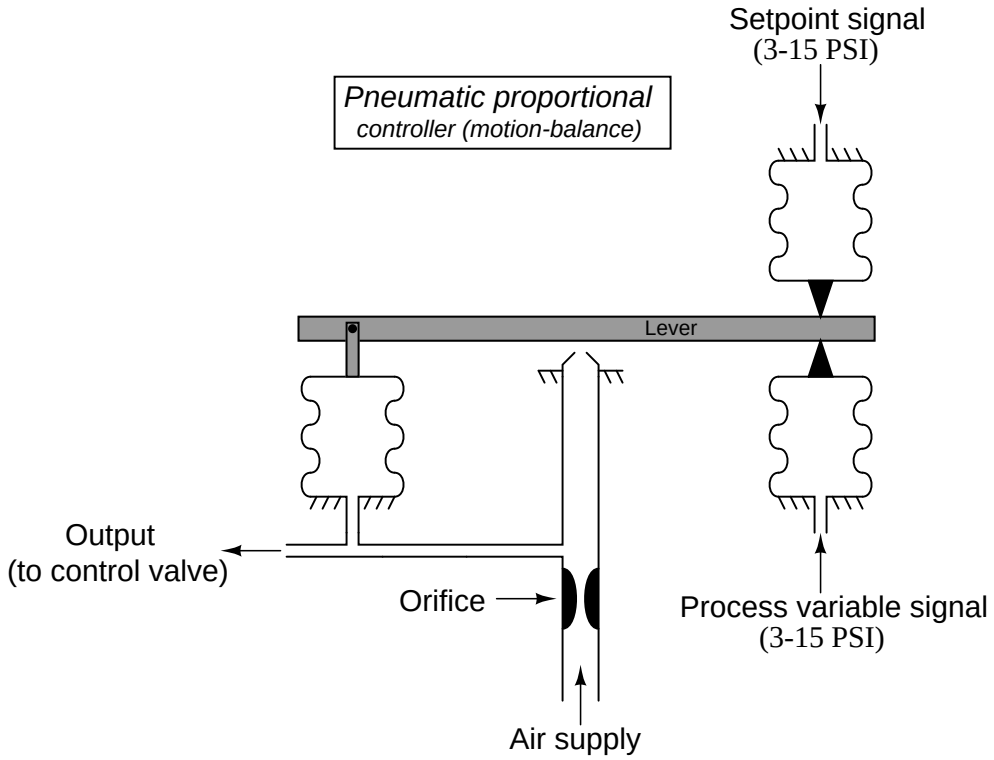
In this illustration the fulcrum is shown moved to two different positions, to effect a change in gain:



Moving the fulcrum closer to the output bellows places that bellows at a mechanical disadvantage for generating torque (leverage) on the beam. This means any given change in input (PV or SP) force is more difficult for the output bellows to counterbalance. The output pressure, therefore, must change to a greater degree in order for this force-balance mechanism to achieve balance. A greater change in output pressure for a given change in input pressure is the definition of a gain *increase*.

Conversely, moving the fulcrum farther away from the output bellows increases that bellows' mechanical advantage. This additional leverage makes it easier for the output bellows to counteract changes in input force, resulting in less output pressure change required to balance any given input pressure change. A lesser change in output pressure for a given change in input pressure is characteristic of a gain *decrease*.

Some pneumatic controllers employ the *motion-balance* principle instead of the force-balance principle in their operation. In contrast to a force-balance system where opposing forces cancel each other to restrain motion of the mechanism, a motion-balance system freely moves as the signal pressures traverse their working ranges. A simple motion-balance proportional controller design appears here:

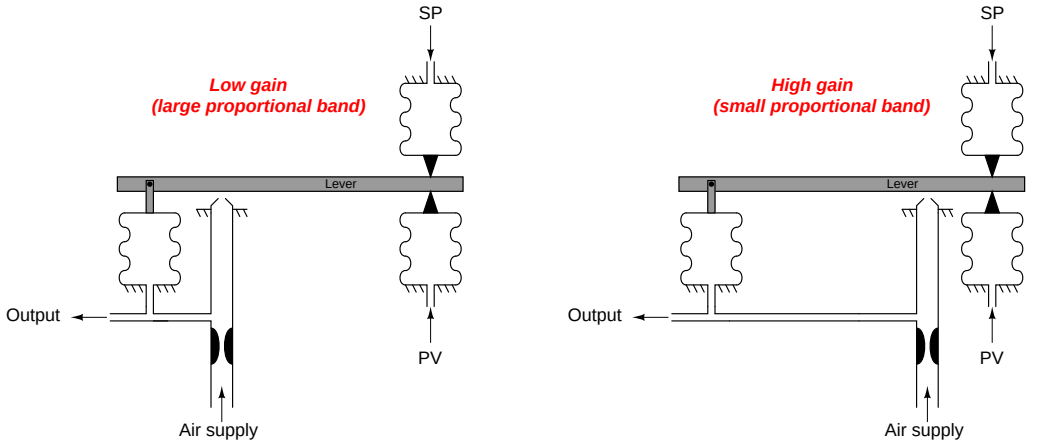


As the process variable signal increases, the right-hand end of the lever is forced up. This motion draws the lever away from the nozzle, resulting in decreased nozzle backpressure. The decreased backpressure causes the output bellows to collapse[17], moving the left-hand end of the lever down and returning the nozzle/lever gap to (approximately) where it was before the PV signal change. This behavior identifies this controller as *reverse-acting*. If direct action were desired, all we would need to do is swap the process variable and setpoint input pressure connections.

Unlike the force-balance controller mechanism where the lever is maintained in an essentially stationary position by equal and opposite forces, the lever in this motion-balance system is free to tilt. In fact, tilting is precisely how a (nearly) constant nozzle gap is maintained: as one end of the lever moves (either up or down), the other end moves in the opposite direction to keep the nozzle/lever gap constant in the middle.

[17] Being a motion-balance mechanism, these bellows must act as spring elements in order to produce consistent pressure/motion behavior. Some pneumatic controllers employ coil springs inside the brass bellows assembly to provide the necessary "stiffness" and repeatability.

The gain of such a mechanism may be changed by moving the position of the nozzle along the lever's length. However, it must be understood that this position change will have the opposite effect on gain compared with the fulcrum position change described for the force-balance mechanism. Here in the motion-balance system, it is the relative *travel* of each bellows that matters for gain, not the relative *leverage* (torque):

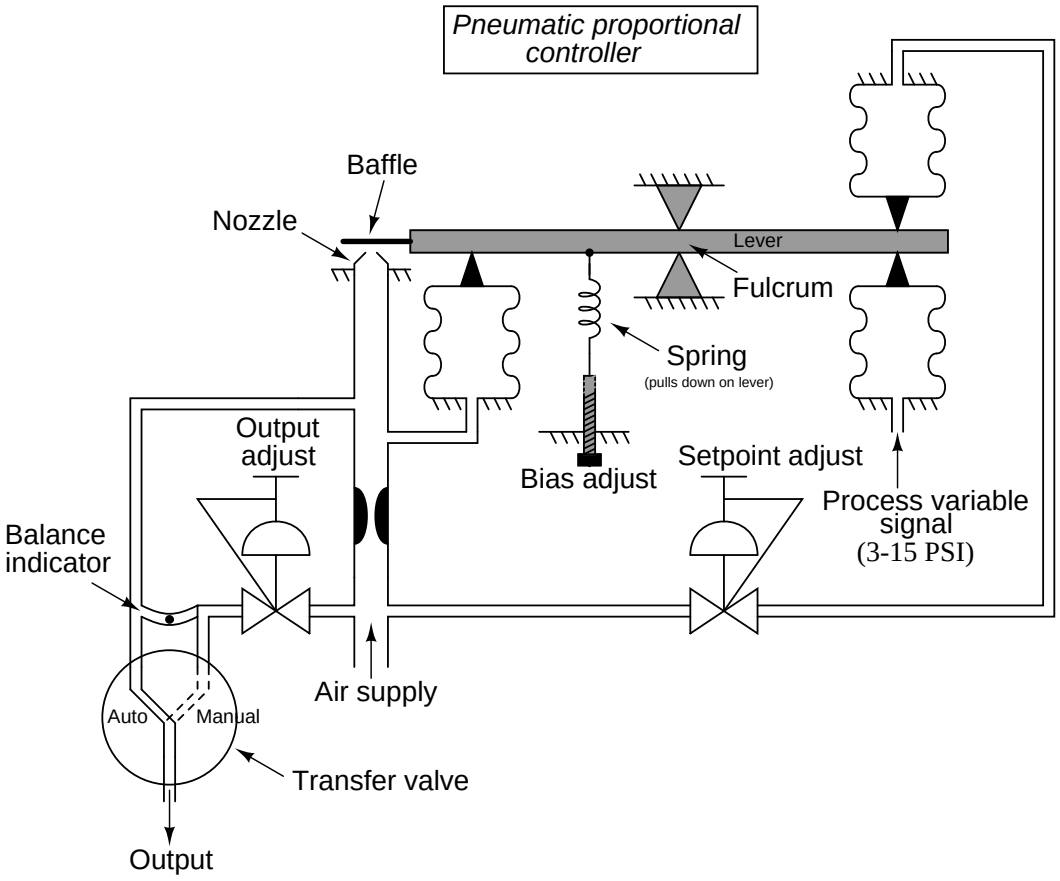


With the nozzle positioned closer to the output bellows, that bellows need not stretch or collapse as much in order to maintain the nozzle gap constant even with a large motion at the input (right-hand) end of the lever. The output pressure in this case will change only slightly for large changes in PV or SP pressures: characteristic of a low gain.

Moving the nozzle closer to the input (PV and SP) bellows gives those bellows more influence over the nozzle/lever gap. The output bellows must expand and contract quite a bit more than the input bellows in order to maintain a constant nozzle gap for any motion at the input side. This requires a greater change in output pressure for a given change in input pressure: the definition of increased gain.

29.11.2 Automatic and manual modes

A more practical pneumatic proportional controller mechanism is shown in the next illustration, complete with setpoint and bias adjustments, and a manual control mode:



"Bumpless" transfer between automatic and manual modes is a very important feature for any loop controller because it allows human operators to change the mode of the controller without introducing an unnecessary disturbance to the process being controlled. Without provision for bumpless transfer, the output signal of the controller may suddenly change whenever the mode is switched between automatic and manual. This sudden signal change will cause the final control element to suddenly "step" to some new level of effect on the process.

In this particular pneumatic controller, bumpless auto/manual transfer is accomplished by the operator paying attention to the *balance indicator* revealing any air pressure difference between the output bellows and the output adjust pressure regulator. When in automatic mode, a switch to manual mode involves adjusting the output regulator until the balance indicator registers zero pressure difference, then switching the transfer valve to the "manual" position. The controller output is then at the direct command of the output adjust pressure regulator, and will not respond

to changes in either PV or SP. "Bumplessly" switching back to automatic mode requires that the setpoint pressure regulator be adjusted until the balance indicator once again registers zero pressure difference, then switching the transfer valve to the "auto" position. The controller output will once again respond to changes in PV and SP.

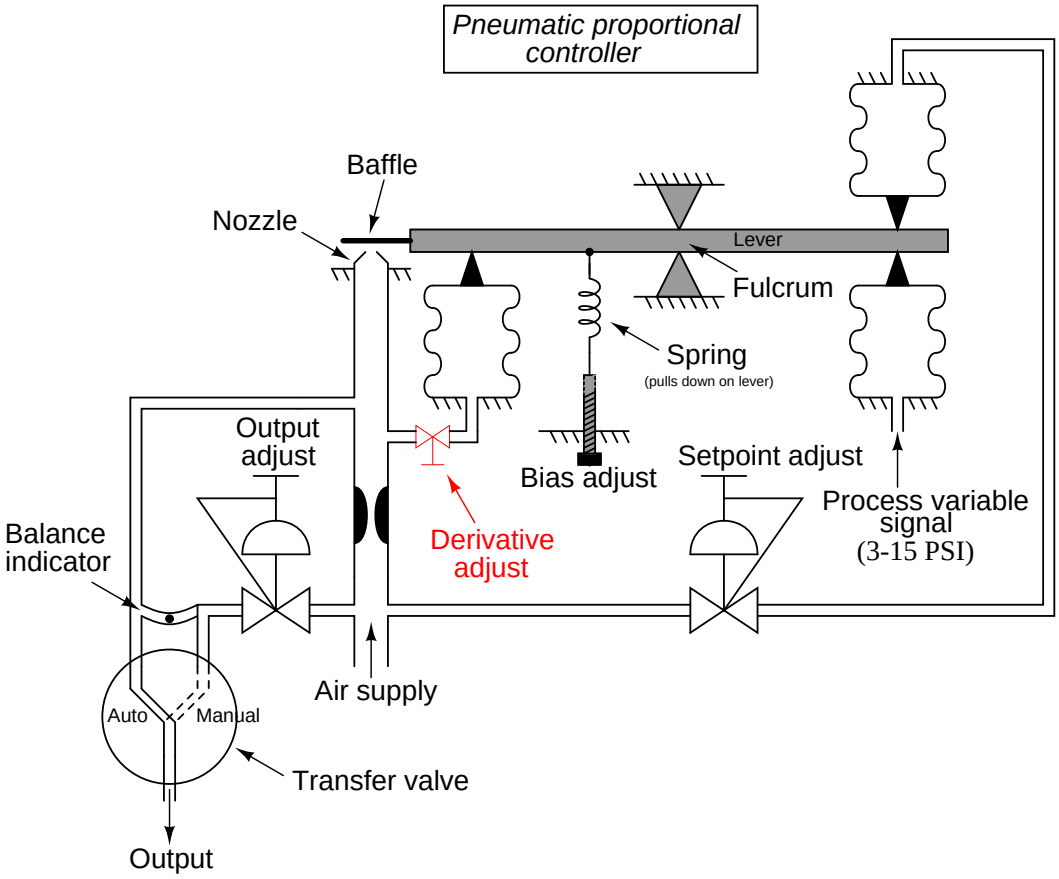
A photograph showing a Foxboro model 43AP pneumatic controller manual/auto transfer switch and balance indicator appears here:



The metal ball within the curved plastic tube indicates equal pressures between automatic and manual modes when centered in the tube. To achieve bumpless transfer between automatic and manual modes, one must never switch the auto/manual valve unless that ball is centered. To center the ball while in automatic mode, the manual output pressure must be adjusted to achieve balance with the automatic-mode output pressure. To center the ball while in manual mode, the automatic-mode output pressure must be adjusted to achieve balance with the manual-mode output pressure – a condition attained by adjusting the *setpoint* knob.

29.11.3 Derivative control action

Derivative (rate) control action is relatively easy to add to this pneumatic controller mechanism. All we need to do is place a restrictor valve between the nozzle tube and the output feedback bellows, causing the bellows to delay filling or emptying its air pressure over time:

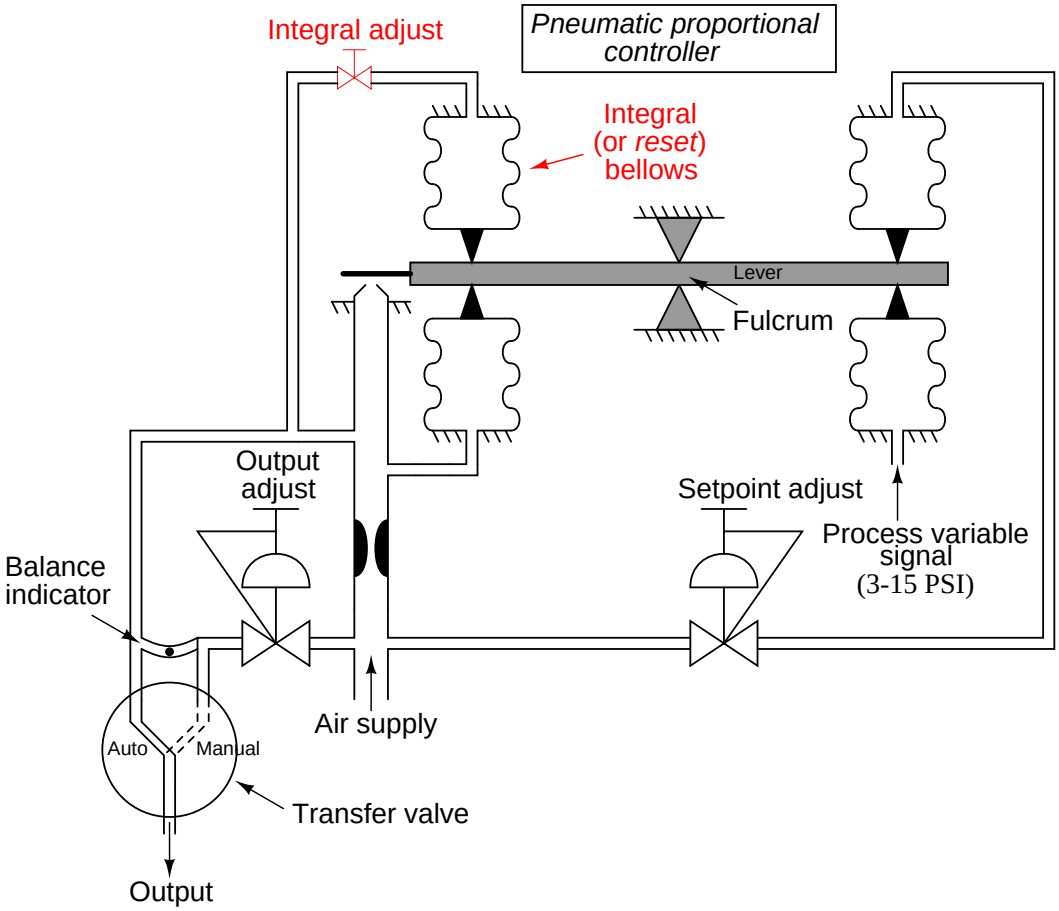


If any sudden change occurs in PV or SP, the output pressure will saturate before the output bellows has the opportunity to equalize in pressure with the output signal tube. Thus, the output pressure "spikes" with any sudden "step change" in input: exactly what we would expect with derivative control action.

If either the PV or the SP ramps over time, the output signal will ramp in direct proportion (proportional action), but there will *also* be an added offset of pressure at the output signal in order to keep air flowing either in or out of the output bellows at a constant rate to generate the force necessary to balance the changing input signal. Thus, derivative action causes the output pressure to shift either up or down (depending on the direction of input change) more than it would with just proportional action alone in response to a ramping input: exactly what we would expect from a controller with both proportional and derivative control actions.

29.11.4 Integral control action

Adding integral action to our hypothetical pneumatic controller mechanism requires the placement of a second bellows (a "reset" bellows) opposite the output feedback bellows, and another restrictor valve to the mechanism[18]:

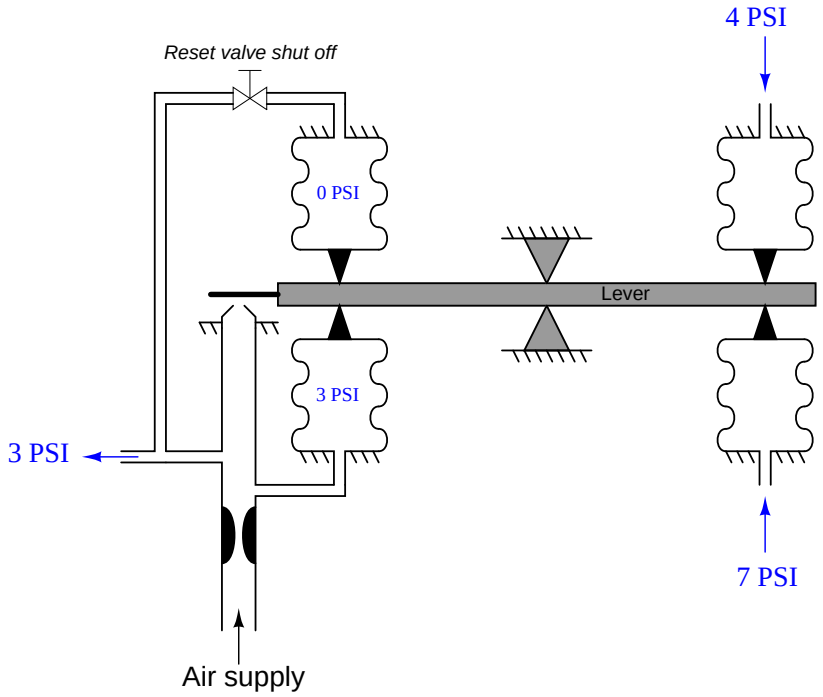


This second bellows takes air pressure from the output line and translates it into force that opposes the original feedback bellows. At first, this may seem counter-productive, for it nullifies the ability of this mechanism to continuously balance the force generated by the PV and SP bellows. Indeed, it would render the force-balance system completely ineffectual if this new "reset" bellows were allowed to inflate and deflate with no time lag. However, with a time lag provided by the

[18] Practical integral action also requires the elimination of the bias spring and adjustment, which formerly provided a constant downward force on the left-hand side of the beam to give the output signal the positive offset necessary to avoid saturation at 0 PSI. Not only is a bias adjustment completely unnecessary with the addition of integral action, but it would actually cause problems by making the integral action "think" an error existed between PV and SP when there was none.

restriction of the integral adjustment valve and the volume of the bellows (a sort of pneumatic "RC time constant"), the nullifying force of this bellows becomes delayed over time. As this bellows slowly fills (or empties) with pressurized air from the nozzle, the change in force on the beam causes the regular output bellows to have to "stay ahead" of the reset bellows action by constantly filling (or emptying) at some rate over time.

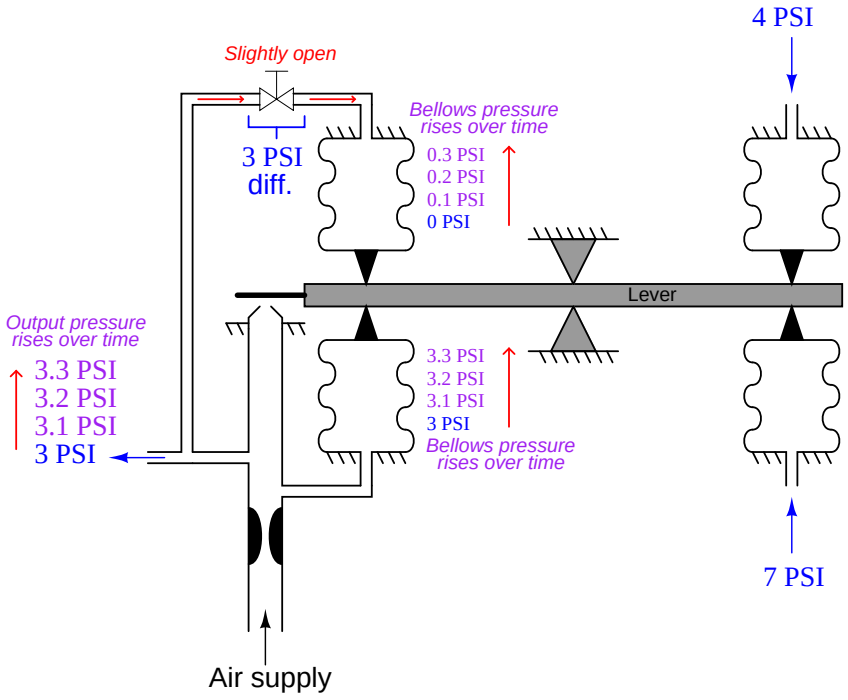
To better understand this integrating action, let us perform a "thought experiment" on a simplified version of the controller. The following mechanism has been stripped of all unnecessary complexity so that we may focus on just the proportional and integral actions. Here, we envision the PV and SP air pressure signals differing by 3 PSI, causing the force-balance mechanism to instantly respond with a 3 PSI output pressure to the feedback bellows (assuming a central fulcrum location, giving a controller gain of 1). The reset (integral) valve has been completely shut off at the start of this thought experiment:



With 0 PSI of air pressure in the reset bellows, it is as though the reset bellows does not exist at all. The mechanism is a simple proportional-only pneumatic controller.

Now, imagine opening up the reset valve just a little bit, so that the output air pressure of 3 PSI begins to slowly fill the reset bellows. As the reset bellows fills with pressurized air, it begins to push down on the left-hand end of the force beam. This forces the baffle closer to the nozzle, causing the output pressure to rise. The regular output bellows has no restrictor valve to impede its filling, and so it *immediately* applies more upward force on the beam with the rising output pressure. With this greater output pressure, the reset bellows has an even greater "final" pressure to achieve, and so its rate of filling continues.

The result of these two bellows' opposing forces (one instantaneous, one time-delayed) is that the lower bellows' pressure must always *lead 3 PSI ahead of the upper bellows' pressure* in order to maintain a pressure difference of 3 PSI necessary to balance force with the PV and SP bellows (whose pressures differ by 3 PSI). This creates a constant 3 PSI differential pressure across the reset restriction valve, resulting in a constant flow of air into the reset bellows at a rate determined by that pressure drop and the opening of the restrictor valve. Eventually this will cause the output pressure to saturate at maximum, but until then the practical importance of this rising pressure action is that the mechanism now exhibits *integral control response* to the constant error between PV and SP:

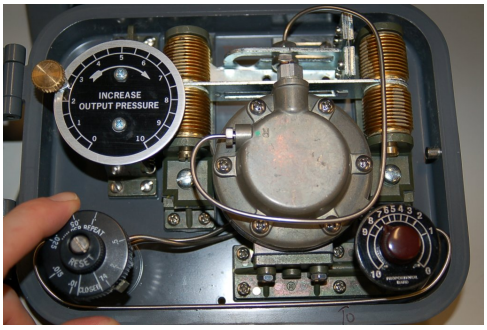


The greater the difference in pressures between PV and SP (i.e. the greater the *error*), the more pressure drop will develop across the reset restriction valve, causing the reset bellows to fill (or empty, depending on the sign of the error) with compressed air at a faster rate[19], causing the output pressure to change at a faster rate. Thus, we see in this mechanism the defining nature of integral control action: that the magnitude of the error determines the *velocity* of the output signal (its rate of change over time, or $\frac{dm}{dt}$). The rate of integration may be finely adjusted by changing the opening of the restrictor valve, or adjusted in large steps by connecting *capacity tanks* to the reset bellows to greatly increase its effective volume.

[19]These restrictor valves are designed to encourage laminar air flow, making the relationship between volumetric flow rate and differential pressure drop *linear* rather than quadratic as it is for large control valves. Thus, a doubling of pressure drop across the restrictor valve results in a doubling of flow rate into (or out of) the reset bellows, and a consequent doubling of integration rate. This is precisely what we desire and expect from a controller with integral action.

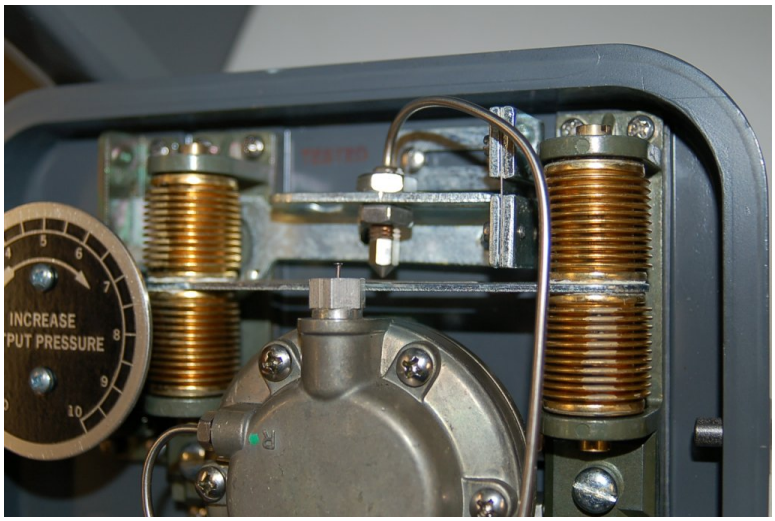
29.11.5 Fisher MultiTrol

Front (left) and rear (right) photographs of a real pneumatic controller (a Fisher "MultiTrol" unit) appear here:





The mechanism is remarkably similar to the one used throughout the explanatory discussion, with the important distinction of being *motion-balance* instead of force balance. Proportional and integral control modes are implemented through the actions of four brass bellows pushing as opposing pairs at either end of a beam:



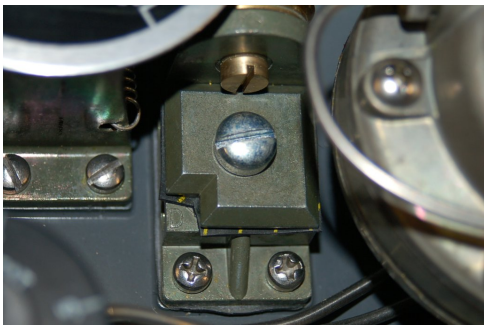
The nozzle may be seen facing down at the middle of the beam, with the center of the beam acting as a baffle. Setpoint control is achieved by moving the position of the nozzle up and down with respect to the beam. A setpoint dial (labeled "Increase Output Pressure") turns a cam which moves the nozzle closer to or farther away from the beam. This being a motion-balance system, an offset in nozzle position equates to a biasing of the output signal, causing the controller to seek a new process variable value.

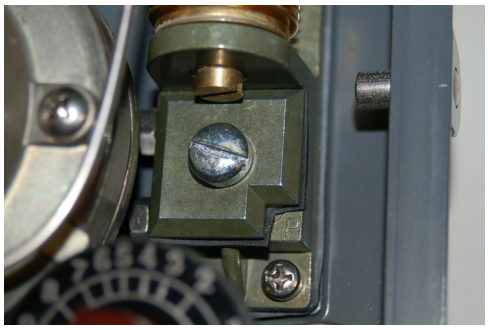
Instead of altering the position of a fulcrum to alter the gain (proportional band) of this controller, gain control is effected through the use of a "pressure divider" valve proportioning the amount of output air pressure sent to the feedback bellows. Integral rate control is implemented exactly the same way as in the hypothetical controller mechanism illustrated in the discussion: by adjusting a valve restricting air flow to and from the reset bellows. Both valves are actuated by rotary knobs with calibrated scales. The reset knob is actually calibrated in units of minutes per repeat, while the proportional band knob is labeled with a scale of arbitrary numbers:



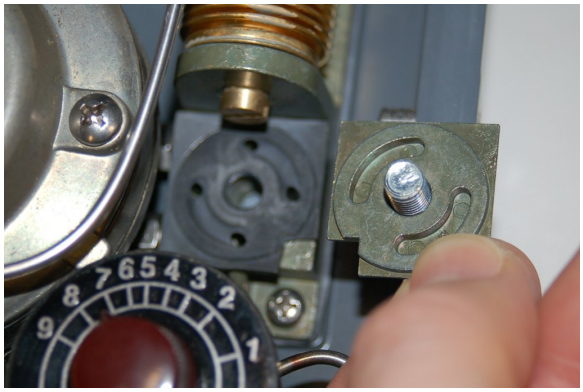


Selection of direct versus reverse action is accomplished in the same way as selection between proportional and snap-action (on-off) control: by movable manifolds re-directing air pressure signals to different bellows in the mechanism. The direct/reverse manifold appears in the left-hand photograph (the letter "D" stands for *direct* action) while the proportional/snap manifold appears in the right-hand photograph (the letter "P" stands for *proportional* control):





Either setting is made by removing the screw holding the manifold plate to the controller body, rotating the plate one-quarter turn, and re-attaching. The following photograph shows one of the manifold plates removed and turned upside-down for inspection of the air passages:

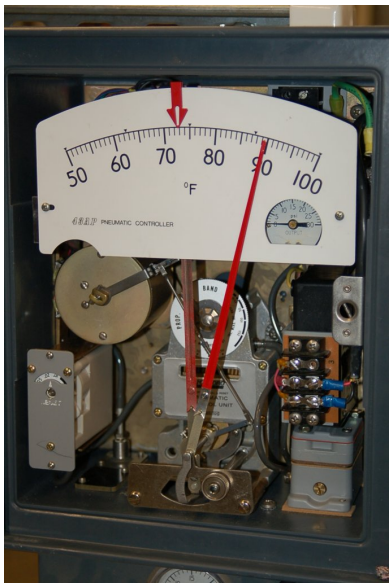


The two quarter-circumference slots seen in the manifold plate connect adjacent air ports together. Rotating the plate 90 degrees connects the four air ports together as two different pairs.

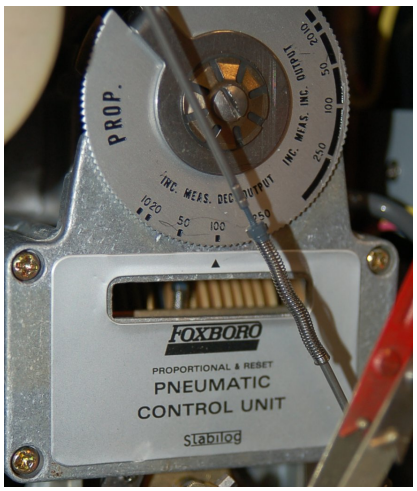
29.11.6 Foxboro model 43AP

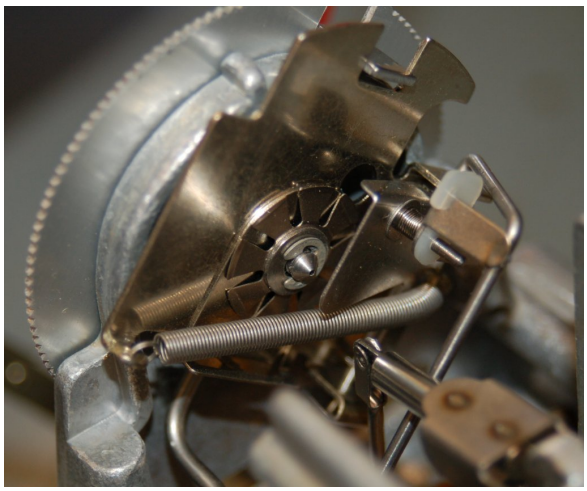
The Fisher MultiTrol pneumatic controller is a very simple device, intended for field-mounting near the pneumatic transmitter and control valve to form a control loop for non-precision applications. A more sophisticated field-mounted pneumatic controller is the Foxboro model 43AP, sporting actual PV and SP indicating pointers, plus more precise tuning controls. The following photographs show one of these controllers, with the access door closed (left) and open (right):





At the heart of this controller is a motion-balance "pneumatic control unit" mechanism. A dial for setting proportional band (and direct/reverse action) appears on the front of the mechanism:





Note the simple way in which direct and reverse actions are described on this dial: either *increasing measurement, decreasing output* (reverse action) or *increasing measurement, increasing output* (direct action).

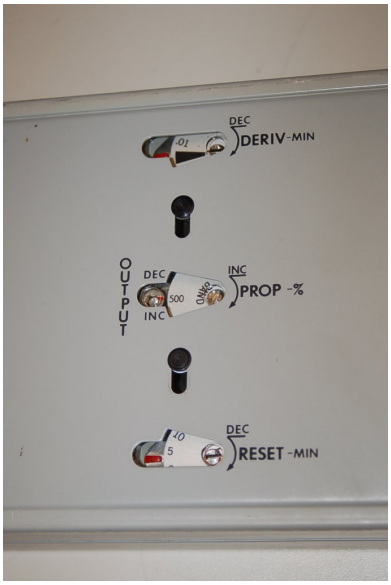
29.11.7 Foxboro model 130

Foxboro also manufactured panel-mounted pneumatic controllers, the model 130 series, for larger-scale applications where multiple controllers needed to be located in one compact space. A bank of four Foxboro model 130 pneumatic controllers appears in the next photograph:

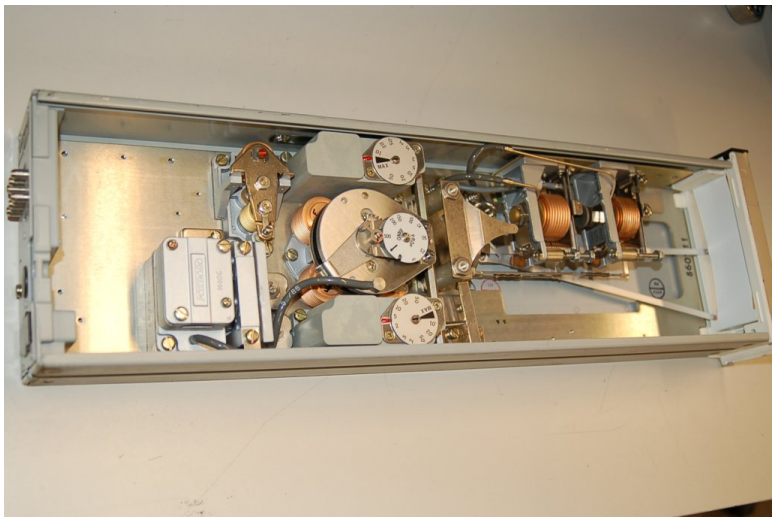


Each controller may be partially removed (slid out) from its slot in the rack, the P, I, and D settings adjustable on the left side panel with a screwdriver:

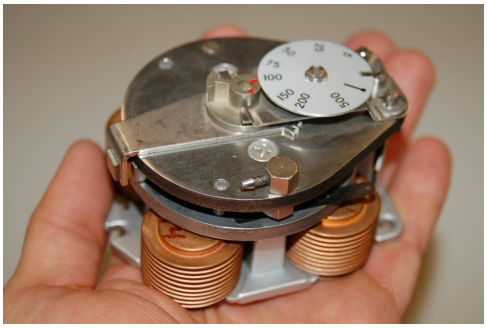


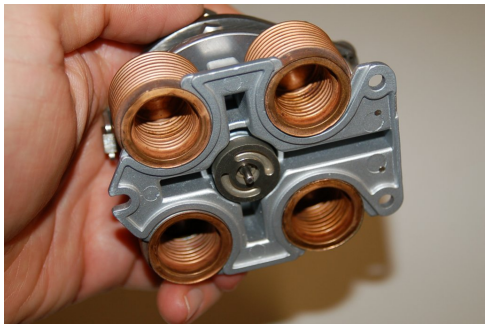


With the side panel removed, the entire mechanism is open to viewing:

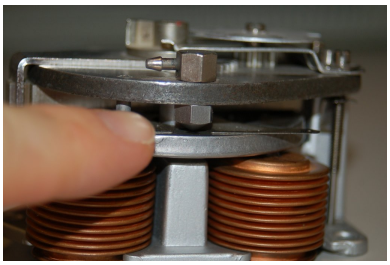


The heart of the model 130 controller is a four-bellows force-balance mechanism, identical in principle to the hypothetical force-balance PID controller mechanism used throughout the explanatory discussion. Instead of the four bellows acting against a straight beam, however, these bellows push against a circular disk:

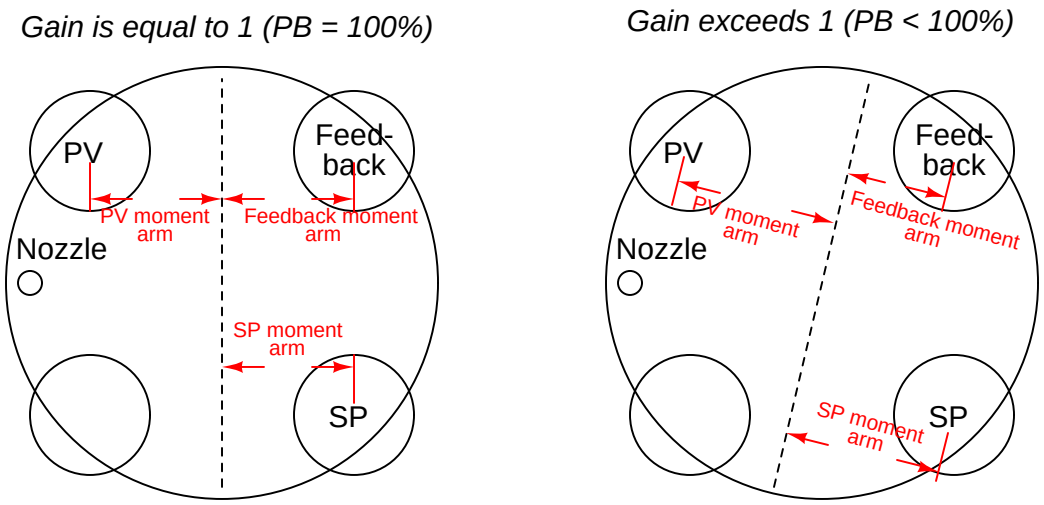




A nozzle (shown in the next photograph) detects if the disk is out of position (unbalanced), sending a back-pressure signal to an amplifying relay which then drives the feedback bellows:



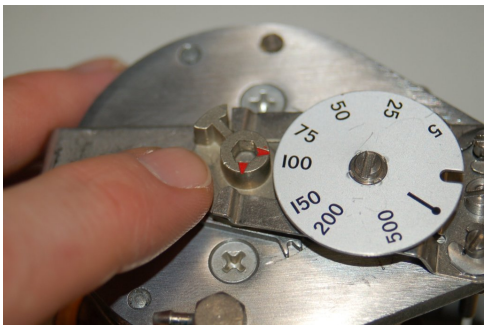
The disk rocks along an axis established by a movable bar. As this bar is rotated at different angles relative to the face of the disk, the fulcrum shifts with respect to the four bellows, providing a simple and effective gain adjustment:

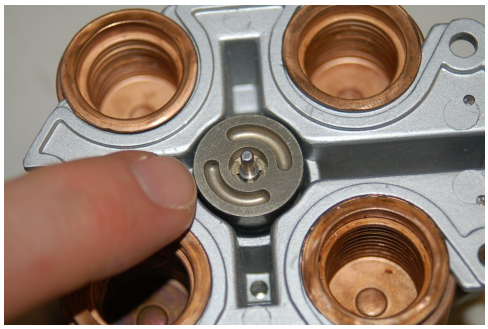


If the moment arms (lever lengths) between the input (PV and SP) bellows and the feedback bellows are equal, both sets of bellows will have equal leverage, and the gain will be one (a proportional band setting of 100%). However, if the fulcrum bar is rotated to give the input bellows more leverage and the feedback bellows less leverage, the feedback bellows will have to "work harder" (exert more force) to counteract any imbalance of force created by the input (PV and SP) bellows, thus creating a greater gain: more output pressure for the same amount of input pressure.

The fourth (lower-left) bellows acting on the disk provides an optional reset (integral) function. Its moment arm (lever length) of course is always equal to that of the feedback bellows, just as the PV and SP bellows' moment arm lengths are always equal, being positioned opposite the fulcrum line.

Selection between direct and reverse action works on the exact same principle as in the Fisher MultiTrol controller – by connecting four air ports in one of two paired configurations. A selector (movable with a hex wrench) turns an air signal port "switch" on the bottom of the four-bellows unit, effectively switching the PV and SP bellows:



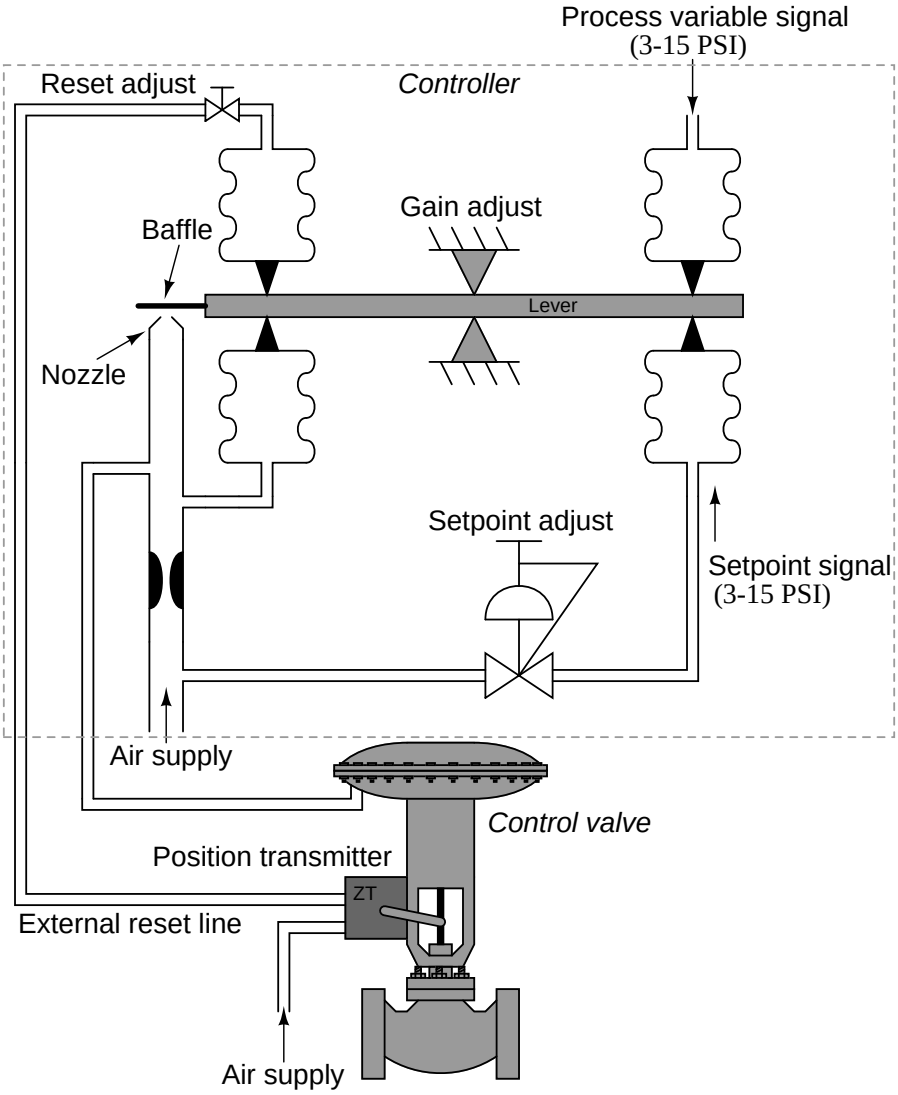


An interesting characteristic of most pneumatic controllers is modularity of function: it is possible to order a pneumatic controller that is proportional-only (P), proportional plus integral (P+I), or full PID. Since each control mode requires additional components to implement, a P-only pneumatic controller costs less than a P+I pneumatic controller, which in turn costs less than a full PID pneumatic controller. This explains the relative scarcity of full PID pneumatic controllers in industry: why pay for additional functionality if less will suffice for the task at hand?

29.11.8 External reset (integral) feedback

Some pneumatic controllers come equipped with an option for *external reset*: a feature useful in control systems to avoid integral windup if and when the process stops responding to changes in controller output. Instead of receiving a pneumatic signal directly from the output line of the controller, the reset bellows receives its signal through another pneumatic line, connected to a location in the control system where the final *effect* of the output signal (m) is seen. If for some reason the final control element cannot achieve the state called for by the controller, the controller will sense this through the external reset signal, and will cease integration to avoid "wind-up."

In the following illustration[20], the external reset signal comes from a pneumatic *position transmitter* (ZT) mounted to the sliding stem of the control valve, sending back a 3-15 PSI signal representing valve stem position:



If something happens to the control valve causing it to freeze position when the controller commands it to move – suppose the stem encounters a mechanical "stop" limiting travel, or a piece of solid material jams the valve trim so it cannot close further – the pneumatic pressure signal sent from the position transmitter to the controller's reset bellows will similarly freeze. After the

[20] In case you are wondering, this controller happens to be *reverse-acting* instead of direct. This is of no consequence to the feature of external reset.

pneumatic lag caused by the reset restrictor valve and bellows passes, the reset bellows force will remain fixed. This halts the controller's integral action, which was formerly based on a "race" between the output feedback bellows and the reset bellows, causing the feedback bellows to "lead" the reset bellows pressure by an amount proportional to the error between PV and SP. This "race" caused the output pressure to wind either up or down depending on the sign of the error. Now that the reset bellows pressure is frozen due to the control valve stem position being frozen, however, the "race" comes to an end and the controller exhibits only proportional action. Thus, the dreaded effect of integral windup – where the integral action of a controller continues to act even though the change in output is of no effect on the process – is averted.

29.12 Analog electronic PID controllers

Although analog electronic process controllers are considered a newer technology than pneumatic process controllers, they are actually "more obsolete" than pneumatic controllers. Panel-mounted (inside a control room environment) analog electronic controllers were a great improvement over panel-mounted pneumatic controllers when they were first introduced to industry, but they were superseded by digital controller technology later on. Field-mounted pneumatic controllers were either replaced by panel-mounted electronic controllers (either analog or digital) or left alone. Applications still exist for field-mounted pneumatic controllers, even now at the beginning of the 21st century, but very few applications exist for analog electronic controllers in any location.

Analog electronic controllers enjoy two inherent advantages over digital electronic controllers: greater reliability[21] and faster response. However, these advantages have been diminishing as digital control technology has advanced. Today's digital electronic technology is far more reliable than the digital technology available during the heyday of analog electronic controllers. Now that digital controls have achieved very high levels of reliability, the first advantage of analog control is largely academic[22], leaving only the second advantage for practical consideration. The advantage of faster speed may be fruitful in applications such as motion control, but for most industrial processes even the slowest digital controller is fast enough[23]. Furthermore, the numerous advantages offered by digital technology (data recording, networking capability, self-diagnostics, flexible configuration, function blocks for implementing different control strategies) severely weaken the relative importance of reliability and speed.

Most analog electronic PID controllers utilize *operational amplifiers* in their designs. It is relatively easy to construct circuits performing amplification (gain), integration, differentiation, summation, and other useful control functions with just a few opamps, resistors, and capacitors.

[21] The reason for this is the low component count compared to a comparable digital control circuit. For any given technology, a simpler device will tend to be more reliable than a complex device if only due to there being fewer components to fail. This also suggests a third advantage of analog controllers over digital controllers, and that is the possibility of easily designing and constructing your own for some custom application such as a hobby project. A digital controller is not outside the reach of a serious hobbyist to design and build, but it is definitely more challenging due to the requirement of programming expertise in addition to electronic hardware expertise.

[22] It is noteworthy that analog control systems are completely immune from "cyber-attacks" (malicious attempts to foil the integrity of a control system by remote access), due to the simple fact that their algorithms are fixed by physical laws and properties of electronic components rather than by code which may be edited. This new threat constitutes an inherent weakness of digital technology, and has spurred some thinkers in the field to reconsider analog controls for the most critical applications.

[23] The real problem with digital controller speed is that the time delay between successive "scans" translates into dead time for the control loop. Dead time is the single greatest impediment to feedback control.

29.12.1 Proportional control action

The basic proportional-only control algorithm follows this formula:

$$m = K_p e + b$$

Where,
m = Controller output
e = Error (difference between PV and SP)
K_p = Proportional gain
b = Bias

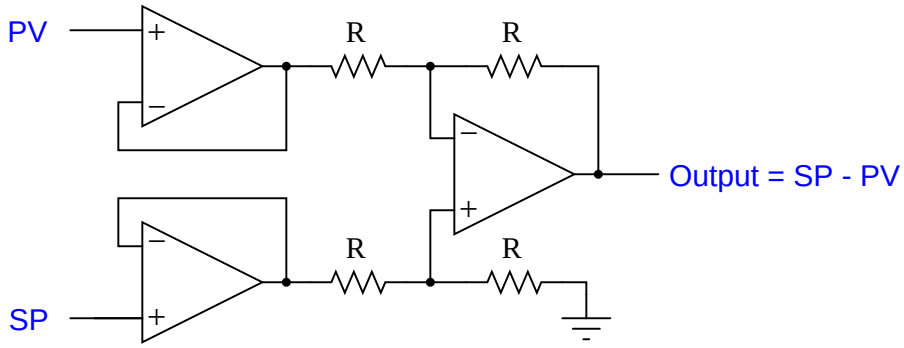
The "error" variable (e) is the mathematical difference between process variable and setpoint. If the controller is direct-acting, $e = \text{PV} - \text{SP}$. If the controller is reverse-acting, $e = \text{SP} - \text{PV}$. Thus,

$$m = K_p(\text{PV} - \text{SP}) + b \qquad \text{Direct-acting}$$

$$m = K_p(\text{SP} - \text{PV}) + b \qquad \text{Reverse-acting}$$

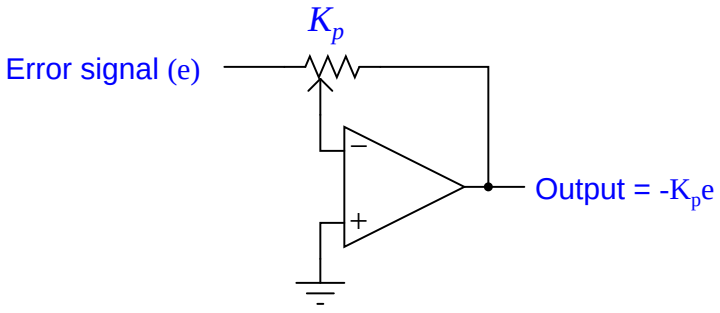
Mathematical operations such as subtraction, multiplication by a constant, and addition are quite easy to perform using analog electronic (operational amplifier) circuitry. Prior to the advent of reliable digital electronics for industrial applications, it was natural to use analog electronic circuitry to perform proportional control for process control loops.

For example, the subtraction function necessary to calculate error (e) from process variable and setpoint signals may be performed with a three-amplifier "subtractor" circuit:



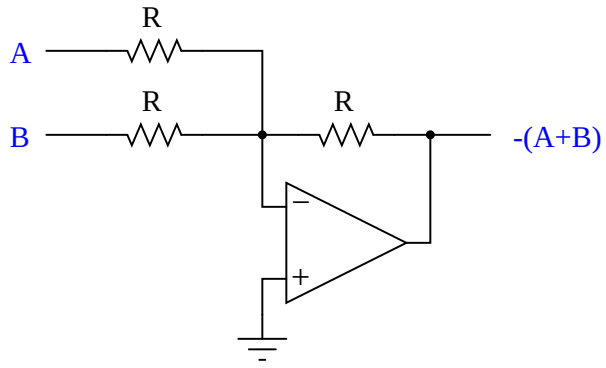
This particular subtractor circuit calculates error for a reverse-acting controller. As the PV signal increases, the error signal decreases (becomes more negative). It could be modified for direct action simply by swapping the two inputs: SP on top and PV on bottom such that the Output becomes PV − SP.

Gain is really nothing more than multiplication by a constant, in this case the constant being K_p. A very simple one-amplifier analog circuit for performing this multiplication is the *inverting*[24] *amplifier* circuit:



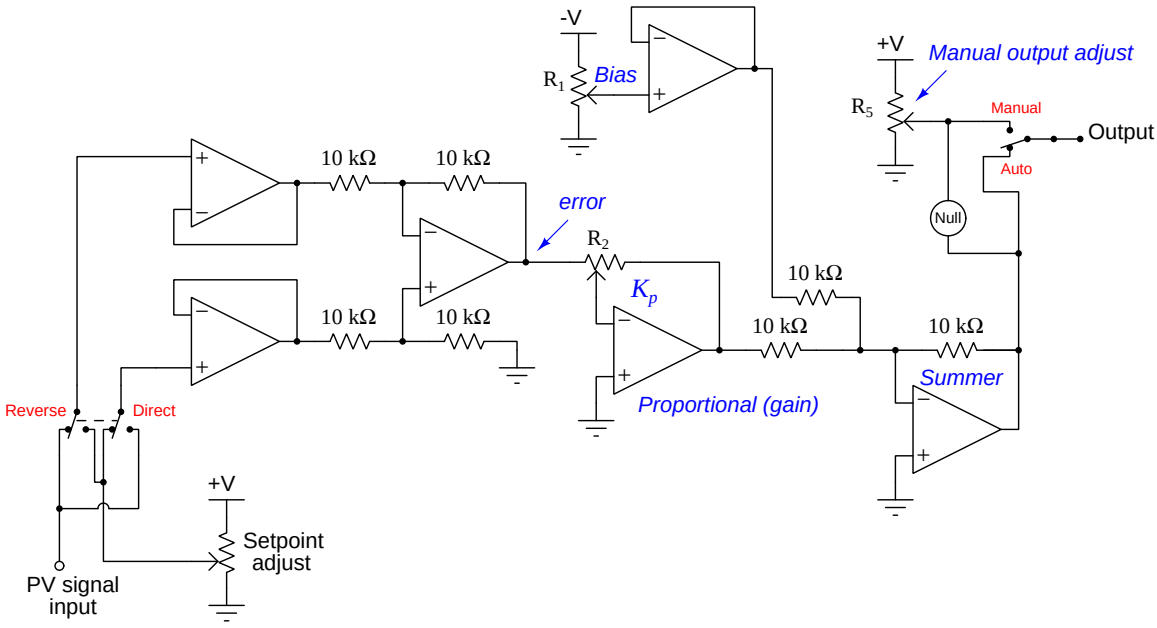
With the potentiometer's wiper in mid-position, the voltage gain of this circuit will be 1 (with an inverted polarity which we shall ignore for now). Moving the wiper toward the left-hand side of the potentiometer increases the circuit's gain past unity, while moving the wiper toward the right-hand side of the potentiometer decreases the gain toward zero.

In order to add the bias (b) term in the proportional control equation, we need an analog circuit capable of summing two voltage signals. This need is nicely met in the *inverting summer* circuit, shown here:



[24] This circuit configuration is called "inverting" because the mathematical sign of the output is always opposite that of the input. This sign inversion is not an intentional circuit feature, but rather a consequence of the input signal facing the opamp's inverting input. Non-inverting multiplier circuits also exist, but are more complicated when built to achieve multiplication factors less than one.

Combining all these analog functions together into one circuit, and adding a few extra features such as direct/reverse action selection, bias adjustment, and manual control with a null voltmeter to facilitate bumpless mode transfer, gives us this complete analog electronic proportional controller:



29.12.2 Derivative and integral control actions

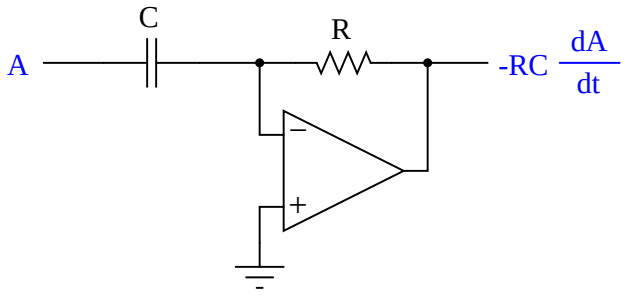
Differentiating and integrating live voltage signals with respect to time is quite simple using operational amplifier circuits. Instead of using all resistors in the negative feedback network, we may implement these calculus functions by using a combination of *capacitors* and resistors, exploiting the capacitor's natural derivative relationship between voltage and current:

$$I = C\frac{dV}{dt}$$

Where,

I = Current through the capacitor (amperes)
C = Capacitance of capacitor (farads)
V = Voltage across the capacitor (volts)
$\frac{dV}{dt}$ = Rate-of-change of voltage across the capacitor (volts per second)

If we build an operational amplifier with a resistor providing negative feedback current through a capacitor, we create a *differentiator* circuit where the output voltage is proportional to the rate-of-change of the input voltage:



Since the inverting input of the operational amplifier is held to ground potential by feedback (a "virtual ground"), the capacitor experiences the full input voltage of signal A. So, as A varies over time, the current through that capacitor will directly represent the signal A's rate of change over time ($I = C\frac{dA}{dt}$). This current passes through the feedback resistor, creating a voltage drop at the output of the amplifier directly proportional to signal A's rate of change over time. Thus, the output voltage of this circuit reflects the input voltage's instantaneous rate of change, albeit with an inverted polarity. The mathematical term RC is the *time constant* of this circuit. For a differentiator circuit such as this, we typically symbolize its time constant as τ_d (the "derivative" time constant).

For example, if the input voltage to this differentiator circuit were to ramp at a constant rate of +4.3 volts per second (rising) with a resistor value of 10 kΩ and a capacitor value of 33 μF (i.e. τ_d = 0.33 seconds), the output voltage would be a constant −1.419 volts:

$$V_{out} = -RC\frac{dV_{in}}{dt}$$

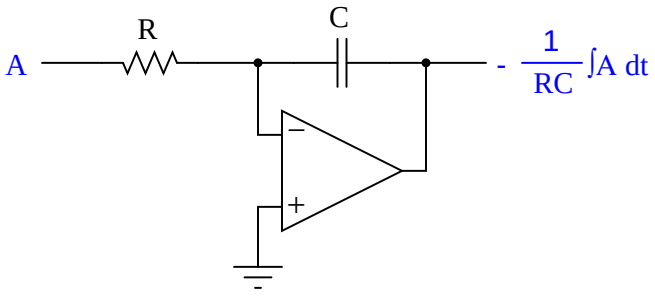
$$V_{out} = -(10000\ \Omega)(33 \times 10^{-6}\ \text{F})\left(\frac{4.3\ \text{V}}{\text{s}}\right)$$

$$V_{out} = -(0.33\ \text{s})\left(\frac{4.3\ \text{V}}{\text{s}}\right)$$

$$V_{out} = -1.419\ \text{V}$$

Recall that the purpose of derivative action in a PID controller is to react to sudden changes in either the error (e) or the process variable (PV). This circuit fulfills that function, by generating an output proportional to the input voltage's rate of change.

If we simply swap[25] the locations of the resistor and capacitor in the feedback network of this operational amplifier circuit, we create an *integrator* circuit where the output voltage rate-of-change is proportional to the input voltage:



This integrator circuit provides the exact inverse function of the differentiator. Rather than a changing input signal generating an output signal proportional to the input's rate of change, an input signal in this circuit controls the rate at which the output signal changes.

The way it works is by acting as a *current source*, pumping current into the capacitor at a value determined by the input voltage and the resistor value. Just as in the previous (differentiator) circuit where the inverting terminal of the amplifier was a "virtual ground" point, the input voltage in this circuit is impressed across the resistor R. This creates a current which must go through capacitor C on its way either to or from the amplifier's output terminal. As we have seen in the capacitor's equation ($I = C\frac{dV}{dt}$), a current forced through a capacitor causes the capacitor's voltage to change over time. This changing voltage becomes the output signal of the integrator circuit. As in the case of the differentiator circuit, the mathematical term RC is the *time constant* of this circuit as well. Being an integrator, we customarily represent this "integral" time constant as τ_i.

Any amount of change in output voltage (ΔV_{out}) occurring between some initial time (t_0) and a finishing time (t_f) may be calculated by the following integral:

$$\Delta V_{out} = -\frac{1}{RC}\int_{t_0}^{t_f} V_{in}\, dt$$

If we wish to know the absolute output voltage at the end of that time interval, all we need to do is add the circuit's initial output voltage (V_0, i.e. the voltage stored in the capacitor at the initial time t_0) to the calculated change:

$$V_{out} = -\frac{1}{RC}\int_{t_0}^{t_f} V_{in}\, dt + V_0$$

[25] This inversion of function caused by the swapping of input and feedback components in an operational amplifier circuit points to a fundamental principle of negative feedback networks: namely, that placing a mathematical element within the feedback loop causes the amplifier to exhibit the inverse of that element's intrinsic function. This is why voltage dividers placed within the feedback loop cause an opamp to have a multiplicative gain (division → multiplication). A circuit element exhibiting a logarithmic response, when placed within a negative feedback loop, will cause the amplifier to exhibit an exponential response (logarithm → exponent). Here, an element having a time-differentiating response, when placed inside the feedback loop, causes the amplifier to time-integrate (differentiation → integration). Since the opamp's output voltage must assume any value possible to maintain (nearly) zero differential voltage at the input terminals, placing a mathematical function in the feedback loop forces the output to assume the inverse of that function in order to "cancel out" its effects and achieve balance at the input terminals.

For example, if we were to input a constant DC voltage of +1.7 volts to this circuit with a resistor value of 81 kΩ and a capacitor value of 47 μF (i.e. $\tau_i = 3.807$ seconds), the output voltage would ramp at a constant rate of −0.447 volts per second[26]. If the output voltage were to begin at −3.0 volts and be allowed to ramp for exactly 12 seconds at this rate, it would reach a value of −8.359 volts at the conclusion of that time interval:

$$V_{out} = -\frac{1}{RC}\int_{t_0}^{t_f} V_{in}\,dt + V_0$$

$$V_{out} = -\left(\frac{1}{(81000\,\Omega)(47 \times 10^{-6}\text{ F})}\right)\left(\int_0^{12} 1.7\text{ V}\,dt\right) - 3\text{ V}$$

$$V_{out} = -\left(\frac{1}{3.807\text{ s}}\right)(20.4\text{ V}\cdot\text{s}) - 3\text{ V}$$

$$V_{out} = -5.359\text{ V} - 3\text{ V}$$

$$V_{out} = -8.359\text{ V}$$

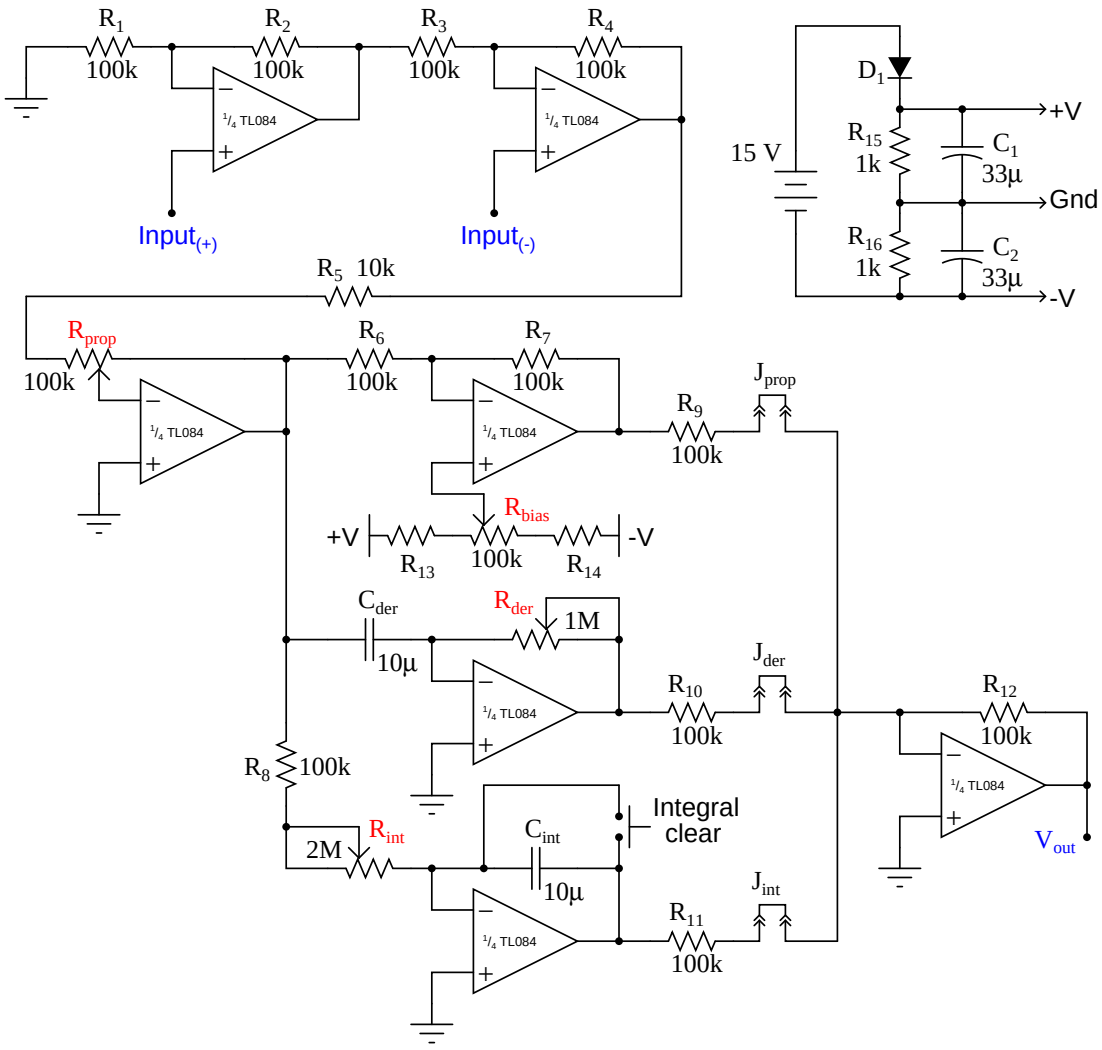
If, after ramping for some amount of time, the input voltage of this integrator circuit is brought to zero, the integrating action will cease. The circuit's output will simply hold at its last value until another non-zero input signal voltage appears.

Recall that the purpose of integral action in a PID controller is to eliminate offset between process variable and setpoint by calculating the error-time product (how far PV deviates from SP, and for how long). This circuit will fulfills that function if the input voltage is the error signal, and the output voltage contributes to the output signal of the controller.

[26] If this is not apparent, imagine a scenario where the +1.7 volt input existed for precisely one second's worth of time. However much the output voltage ramps in that amount of time must therefore be its rate of change in volts per second (assuming a linear ramp). Since we know the area accumulated under a constant value of 1.7 (high) over a time of 1 second (wide) must be 1.7 volt-seconds, and τ_i is equal to 3.807 seconds, the integrator circuit's output voltage must ramp 0.447 volts during that interval of time. If the input voltage is positive and we know this is an inverting opamp circuit, the direction of the output voltage's ramping must be negative, thus a ramping rate of *−0.447 volts per second*.

29.12.3 Full-PID circuit design

The following schematic diagram shows a full PID controller implemented using eight operational amplifiers, designed to input and output voltage signals representing PV, SP, and Output[27]:



It is somewhat stunning to realize that such a controller, fully capable of controlling many industrial process types, may be constructed using only two integrated circuit "chips" (two "quad" operational amplifiers) and a handful of passive electronic components. The only significant engineering challenge in this simple circuit design is achieving slow enough time constants (in the

[27] The two input terminals shown, $\text{Input}_{(+)}$ and $\text{Input}_{(-)}$ are used as PV and SP signal inputs, the correlation of each depending on whether one desires direct or reverse controller action.

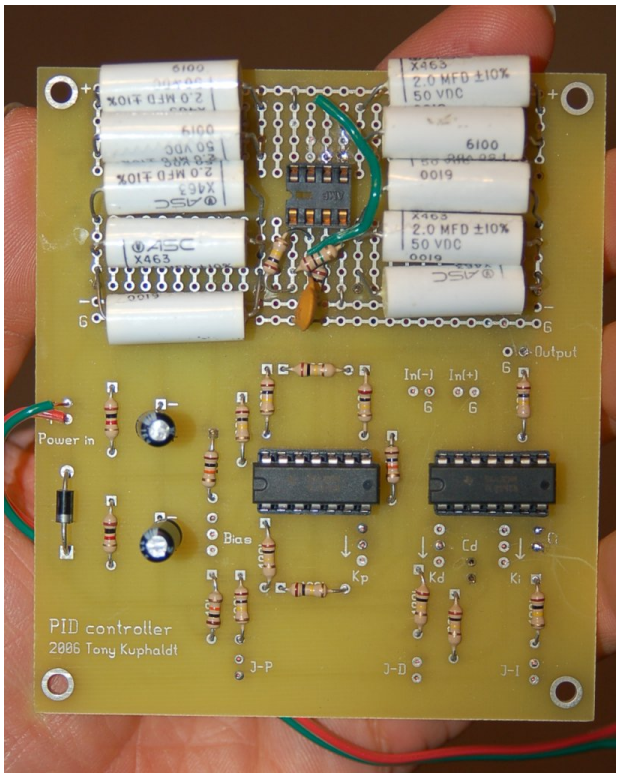
range of minutes rather than seconds) in the integrator and differentiator functions using non-polarized capacitors[28].

This controller implements the so-called *ideal* PID algorithm, with the proportional (gain) value distributing to the integral and derivative terms:

$$m = K_p \left(e + \frac{1}{\tau_i} \int e \, dt + \tau_d \frac{de}{dt} \right) \qquad \textbf{Ideal PID equation}$$

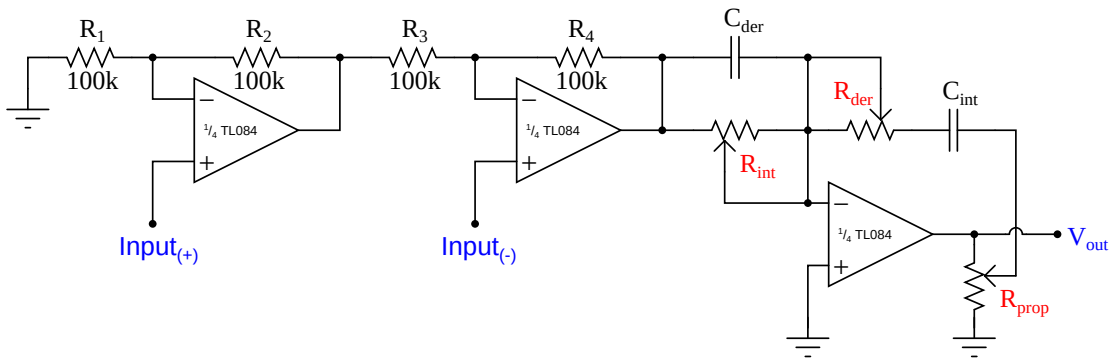
We may determine this from the schematic diagram by noting that the I and D functions each receive their input signals from the output of the proportional amplifier (the one with the R_{prop} potentiometer). Adjusting R_{prop} affects not only the controller's proportional gain, but also the sensitivity of τ_i and τ_d.

An actual implementation of this PID controller in printed circuit board form appears here:



[28] This particular design has integral and derivative time value limits of 10 seconds, maximum. These relatively "quick" tuning values are the result of having to use non-polarized capacitors in the integrator and differentiator stages. The practical limits of cost and size restrict the maximum value of on-board capacitance to around 10 μF each.

It is possible to construct an analog PID controller with fewer components. An example is shown here:



As you can see, a *single* operational amplifier does all the work of calculating proportional, integral, *and* derivative responses. The first two amplifiers do nothing but buffer the input signals and calculate error (PV − SP, or SP − PV, depending on the direction of action).

One of the consequences of consolidating all three control terms in a single amplifier is that those control terms interact with each other. The mathematical expression of this control action is shown here, called the *series* or *interacting* PID equation:

$$m = K_p \left[\left(\frac{\tau_d}{\tau_i} + 1 \right) e + \frac{1}{\tau_i} \int e\, dt + \tau_d \frac{de}{dt} \right] \qquad \textbf{Series or Interacting PID equation}$$

Not only does a change in gain (K_p) alter the relative responses of integral and derivative in the series equation (as it also does in the ideal equation), but changes in either integral or derivative time constants also have an effect on proportional response! This is especially noticeable when the integral time constant is set to some very small value, which is typically the case on fast-responding, self-regulating processes such as liquid flow or liquid pressure control.

It should be apparent that an analog controller implementing the series equation is simpler in construction than one implementing either the parallel or ideal PID equation. This also happens to be true for pneumatic PID controller mechanisms: the simplest analog controller designs all implement the series PID equation[29].

[29] An interesting example of engineering tradition is found in electronic PID controller designs. While it is not too terribly difficult to build an analog electronic controller implementing either the parallel or ideal PID equation (just a few more parts are needed), it is quite challenging to do the same in a pneumatic mechanism. When analog electronic controllers were first introduced to industry, they were often destined to replace old pneumatic controllers. In order to ease the transition from pneumatic to electronic control, manufacturers built their new electronic controllers to behave exactly the same as the old pneumatic controllers they would be replacing. The same legacy followed the advent of digital electronic controllers: many digital controllers were programmed to behave in the same manner as the old pneumatic controllers, for the sake of operational familiarity, not because it was easier to design a digital controller that way.

29.12.4 Single-loop analog controllers

One popular analog electronic controller was the Foxboro model 62H, shown in the following photographs. Like the model 130 pneumatic controller, this electronic controller was designed to fit into a rack next to several other controllers. Tuning parameters were adjustable by moving potentiometer knobs under a side-panel accessible by partially removing the controller from its rack:





The Fisher corporation manufactured a series of analog electronic controllers called the AC^2, which were similar in construction to the Foxboro model 62H, but very narrow in width so that many could be fit into a compact panel space. Here we see a pair of Fisher AC^2 controllers mounted side-by-side in the same rack, used to control liquid level in a pulping process:



Like the pneumatic panel-mounted controllers preceding, and digital panel-mount controllers to follow, the tuning parameters for a panel-mounted analog electronic controller were typically

accessed on the controller's side. The controller could be slid partially out of the panel to reveal the P, I, and D adjustment knobs (as well as direct/reverse action switches and other configuration controls).

Indicators on the front of an analog electronic controller served to display the process variable (PV), setpoint (SP), and manipulated variable (MV, or output) for operator information. Many analog electronic controllers did not have separate meter indications for PV and SP, but rather used a single meter movement to display the *error signal*, or difference between PV and SP. On the Foxboro model 62H, a hand-adjustable knob provided both indication and control over SP, while a small edge-reading meter movement displayed the error. A negative meter indication showed that the PV was below setpoint, and a positive meter indication showed that the PV was above setpoint.

The Fisher AC^2 analog electronic controller used the same basic technique, cleverly applied in such a way that the PV was displayed in real engineering units. The setpoint adjustment was a large wheel, mounted so the edge faced the operator. Along the circumference of this wheel was a scale showing the process variable range, from the LRV at one extreme of the wheel's travel to the URV at the other extreme of the wheel's travel. The actual setpoint value was the middle of the wheel from the operator's view of the wheel edge. A single meter movement needle traced an arc along the circumference of the wheel along this same viewable range. If the error was zero (PV = SP), the needle would be positioned in the middle of this viewing range, pointed at the same value along the scale as the setpoint. If the error was positive, the needle would rise up to point to a larger (higher) value on the scale, and if the error was negative the needle would point to a smaller (lower) value on the scale. For any fixed value of PV, this error needle would therefore move in exact step with the wheel as it was rotated by the operator's hand. Thus, a single adjustment and a single meter movement displayed both SP and PV in very clear and unambiguous form.

Taylor manufactured a line of analog panel-mounted controllers that worked much the same way, with the SP adjustment being a graduated tape reeled to and fro by the SP adjustment knob. The middle of the viewable section of tape (as seen through a plastic window) was the setpoint value, and a single meter movement needle pointed to the PV value as a function of error. If the error happened to be zero (PV = SP), the needle would point to the middle of this viewable section of tape, which was the SP value.

Another popular panel-mounted analog electronic controller was the Moore Syncro, which featured plug-in modules for implementing different control algorithms (different PID equations, nonlinear signal conditioning, etc.). These plug-in function modules were a hardware precursor to the software "function blocks" appearing in later generations of digital controllers: a simple way of organizing controller functionality so that technicians unfamiliar with computer programming could easily configure a controller to do different types of control functions. Later models of the Syncro featured fluorescent bargraph displays of PV and SP for easy viewing in low-light conditions.

Analog single-loop controllers are largely a thing of the past, with the exception of some low-cost or specialty applications. An example of the former is shown here, a simple analog temperature controller small enough to fit in the palm of my hand:



This particular controller happened to be part of a sulfur dioxide analyzer system, controlling the internal temperature of a gas regulator panel to prevent vapors in the sample stream from condensing in low spots of the tubing and regulator system. The accuracy of such a temperature control application was not critical – if temperature was regulated to $\pm$ 5 degrees Fahrenheit it would be more than adequate. This is an application where an analog controller makes perfect sense: it is very compact, simple, extremely reliable, and inexpensive. None of the features associated with digital PID controllers (programmability, networking, precision) would have any merit in this application.

29.12.5 Multi-loop analog control systems

In contrast to single-loop analog controllers, *multi-loop* systems control dozens or even hundreds of process loops at a time. Prior to the advent of reliable digital technology, the only electronic process control systems capable of handling the numerous loops within large industrial installations such as power generating plants, oil refineries, and chemical processing facilities were analog systems, and several manufacturers produced multi-loop analog systems just for these large-scale control applications.

One of the most technologically advanced analog electronic products manufactured for industrial control applications was the Foxboro SPEC 200 system[30]. Although the SPEC 200 system used panel-mounted indicators, recorders, and other interface components resembling panel-mounted control systems, the actual control functions were implemented in a separate equipment rack which Foxboro called a *nest*[31]. Printed circuit boards plugged into each "nest" provided all the control functions (PID controllers, alarm units, integrators, signal selectors, etc.) necessary, with analog signal wires connecting the various functions together with panel-mounted displays and with field instruments to form a working system.

Analog field instrument signals (4-20 mA, or in some cases 10-50 mA) were all converted to a 0-10 VDC range for signal processing within the SPEC 200 nest. Operational amplifiers (mostly the model LM301) formed the "building blocks" of the control functions, with a $\pm$ 15 VDC power supply providing DC power for everything to operate.

[30] Although the SPEC 200 system – like most analog electronic control systems – is considered "mature" (Foxboro officially declared the SPEC 200 and SPEC 200 Micro systems as such in March 2007), working installations may still be found at the time of this writing (2010). A report published by the Electric Power Research Institute (see References at the end of this chapter) in 2001 documents a SPEC 200 analog control system installed in a nuclear power plant in the United States as recently as 1992, and another as recently as 2001 in a Korean nuclear power plant.

[31] Foxboro provided the option of a self-contained, panel-mounted SPEC 200 controller unit with all electronics contained in a single module, but the split architecture of the display/nest areas was preferred for large installations where many dozens of loops (especially cascade, feedforward, ratio, and other multi-component control strategies) would be serviced by the same system.

The following photographs show a model 2AX+A4 proportional-integral (P+I) controller card for a SPEC 200 system inserted into a metal frame (called a "module" by Foxboro). This module was designed to fit into a slot in a SPEC 200 "nest" where it would reside alongside many other similar cards, each card performing its own control function:

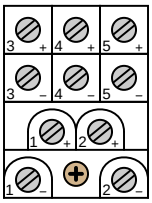




Tuning and alarm adjustments may be seen in the right-hand photograph. This particular controller is set to a proportional band value of approximately 170%, and an integral time constant of just over 0.01 minutes per repeat. A two-position rotary switch near the bottom of the card selected either reverse ("Dec") or direct ("Inc") control action.

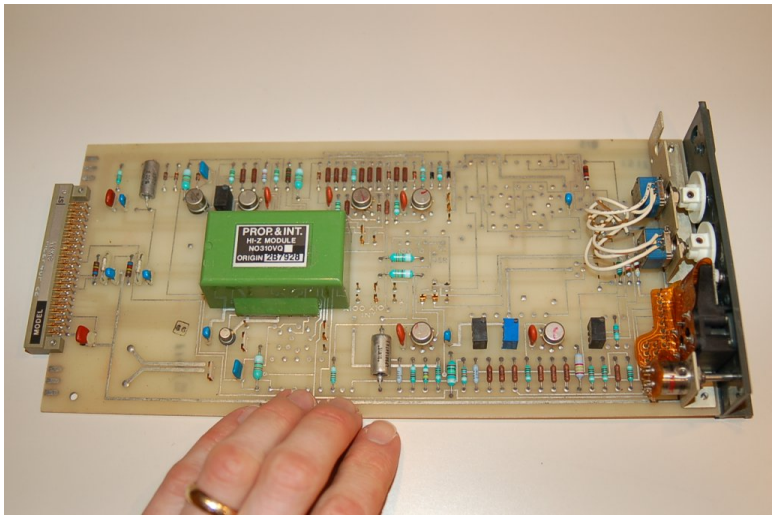
The array of copper pins at the top of the module form the male half of a cable connector, providing connection between the control card and the front-panel instrument accessible to operations personnel. Since the tuning controls appear on the face of this controller card (making it a "card tuned" controller), they were not accessible to operators but rather only to the technical personnel with access to the nest area. Other versions of controller cards ("control station tuned") had blank places where the P and I potentiometer adjustments appear on this model, with tuning adjustments provided on the panel-mounted instrument displays for easier access to operators.

The set of ten screw terminals at the bottom of the module provided connection points for the input and output voltage signals. The following list gives the general descriptions of each terminal pair, with the descriptions for this particular P + I controller written in *italic* type:

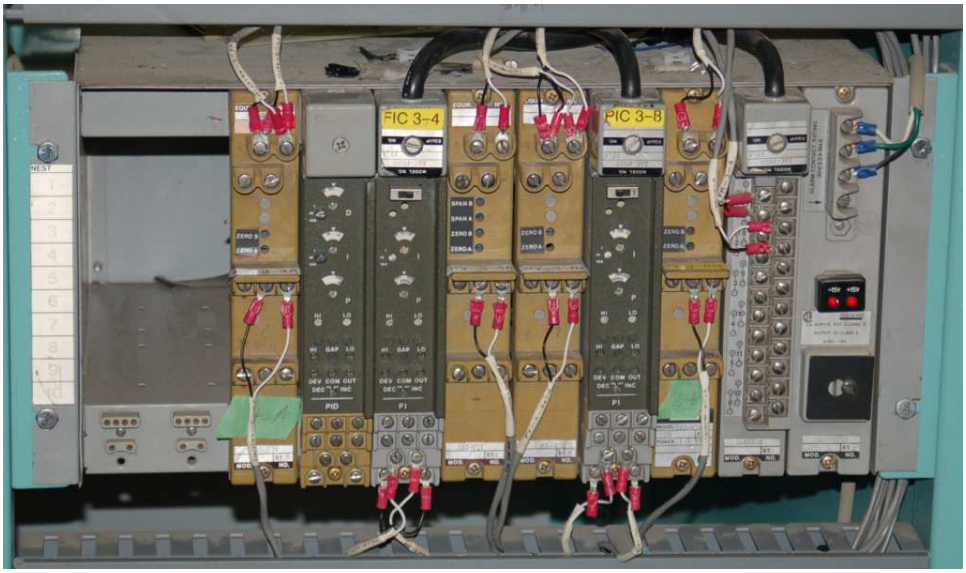


- Terminals (1+) and (1−): Input signal #1 (*Process variable input*)
- Terminals (2+) and (2−): Output signal #1 (*Manipulated variable output*)
- Terminals (3+) and (3−): Input #2, Output #4, or Option #1 (*Remote setpoint*)
- Terminals (4+) and (4−): Input #3, Output #3, or Option #2 (*Optional alarm*)
- Terminals (5+) and (5−): Input #4, Output #2, or Option #3 (*Optional 24 VAC*)

A photograph of the printed circuit board (card) removed from the metal module clearly shows the analog electronic components:



The following photograph shows a set of functioning Foxboro SPEC 200 controller modules residing in a "nest," used to control a flow loop and a pressure loop in a wood pulping process. Both active controller modules are P + I units. An unused PID module resides just to the left of the flow controller module:



Foxboro engineers went to great lengths in their design process to maximize reliability of the SPEC 200 system, already an inherently reliable technology by virtue of its simple, analog nature. As a result, the reliability of SPEC 200 control systems is the stuff of legend[32].

29.13 Digital PID controllers

The vast majority of PID controllers in service today are digital in nature. Microprocessors executing PID algorithms provide many advantages over any form of analog PID control (pneumatic or electronic), not the least of which being the ability to network with personal computer workstations and other controllers over wired or wireless (radio) networks.

[32] I once encountered an engineer who joked that the number "200" in "SPEC 200" represented the number of years the system was designed to continuously operate. At another facility, I encountered instrument technicians who were a bit afraid of a SPEC 200 system running a section of their plant: the system had *never suffered a failure of any kind* since it was installed decades ago, and as a result no one in the shop had any experience troubleshooting it. As it turns out, the entire facility was eventually shut down and sold, with the SPEC 200 nest running faithfully until the day its power was turned off! The functioning SPEC 200 controllers shown in the photograph were in continuous use at British Columbia Institute of Technology at the time of the photograph, taken in December of 2014.

29.13.1 Stand-alone digital controllers

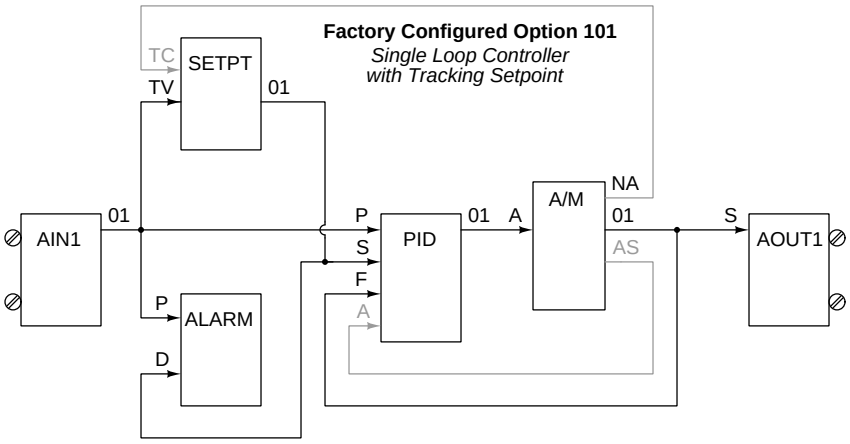
If the internal components of a panel-mounted pneumatic or analog electronic controller (such as the Foxboro models 130 or 62, respectively) were completely removed and replaced by all-digital electronic componentry, the result would be a *stand-alone digital PID controller*. From the outside, such a digital controller looks very similar its technological ancestors, but its capabilities are far greater.

An example of a popular panel-mounted digital controller is the Siemens model 353 (formerly the Moore Products model 353):

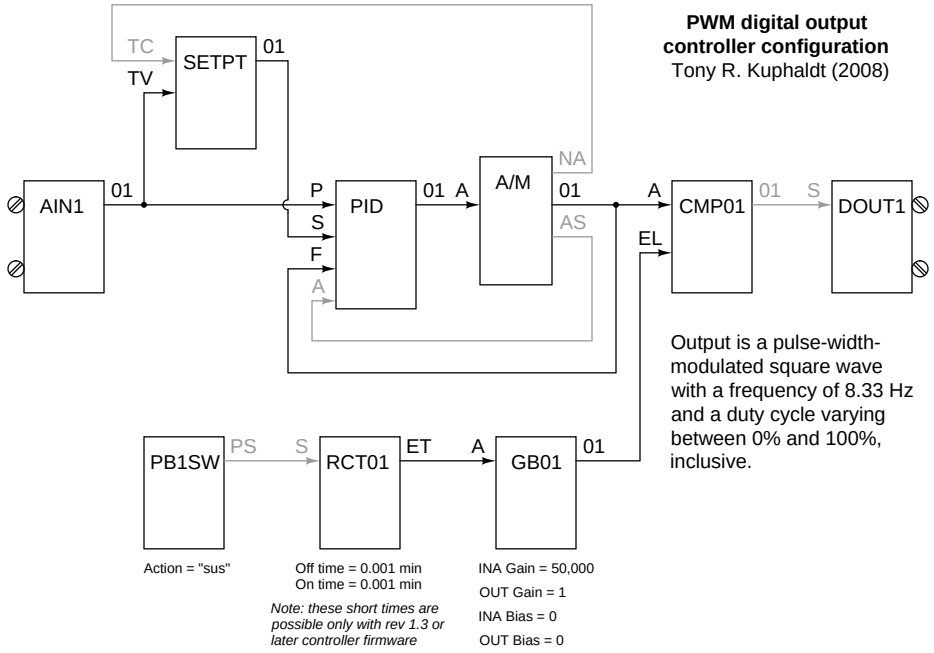


This particular controller, like many high-end digital controllers and larger digital control systems, is programmed in a function block language. Each function block in the controller is a software subroutine performing a specific function on input signals, generating at least one output signal. Each function block has a set of configuration parameters telling it how to behave. For example, the PID function block in a digital controller would have parameters specifying direct or reverse action, gain (K_p), integral time constant (τ_i), derivative time constant (τ_d), output limits, etc.

Even the "stock" configuration for simple, single-loop PID control is a collection of function blocks linked together:



The beauty of function block programming is that the same blocks may be easily re-linked to implement custom control strategies. Take for instance the following function block program written for a Siemens model 353 controller to provide a pulse-width-modulation (PWM, or time-proportioned) output signal instead of the customary 4-20 mA DC analog output signal. The application is for an electric oven temperature control system, where the oven's heating element could only be turned on and off fully rather than continuously varied:



In order to specify links between function blocks, each of the used lettered block inputs is mapped to the output channel of another block. In the case of the time-proportioned function block program, for example, the "P" (process variable) input of the PID function block is set to get its signal from the "01" output channel of the AIN1 (analog input 1) function block. The "TV" (tracking value) input of the SETPT (setpoint) function block is also set to the "01" output channel of the AIN1 function block, so that the setpoint value generator has access to the process variable value in order to implement setpoint tracking. Any function block output may drive an unlimited number of function block inputs (fan-out), but each function block input may receive a signal from *only one* function block output. This is a rule followed within all function block languages to prevent multiple block output signals from conflicting (attempting to insert different signal values into the same input).

In the Siemens controllers, function block programming may be done by entering configuration data using the front-panel keypad, or by using graphical software running on a personal computer networked with the controller.

For applications not requiring so much capability, and/or requiring a smaller form factor, other panel-mounted digital controllers exist. The Honeywell model UDC3000 is a popular example of a 1/4 DIN (96 mm $\times$ 96 mm) size digital controller:



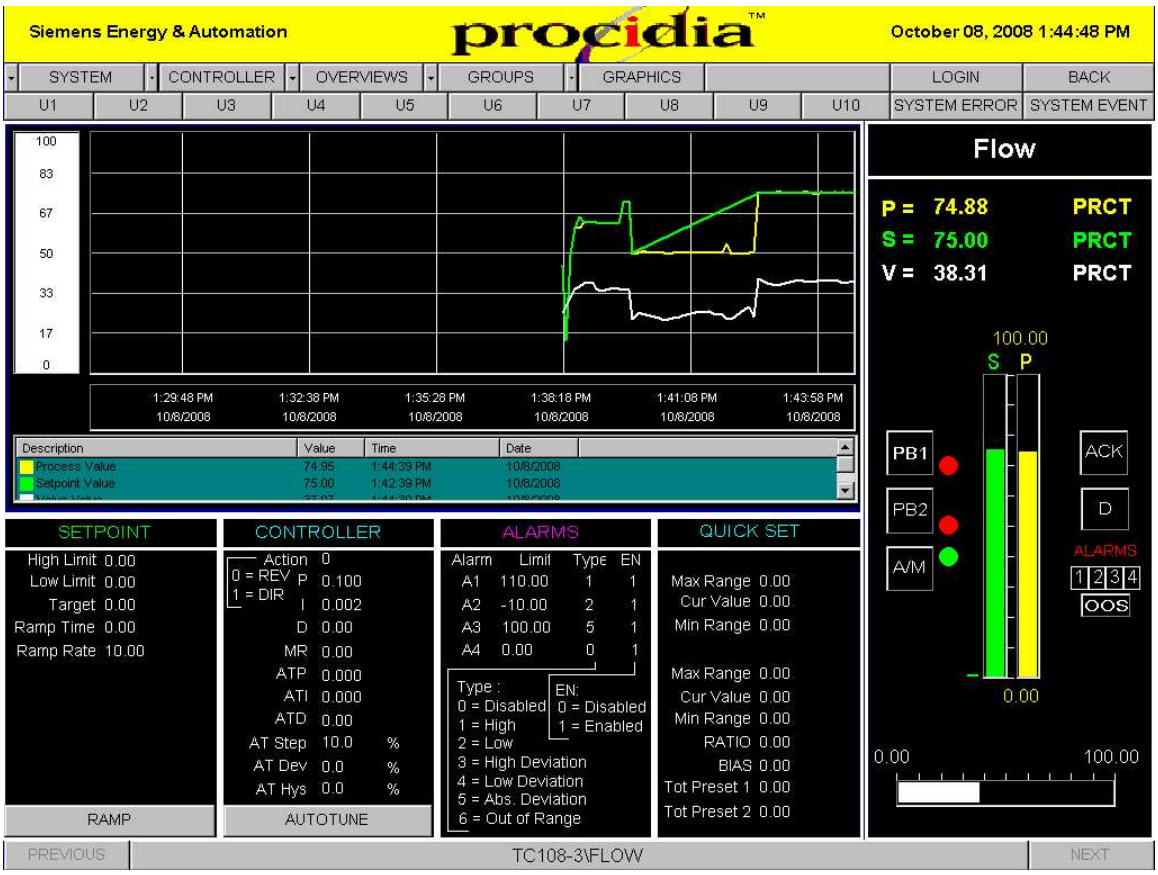
Even smaller panel-mounted controllers are produced by a wide array of manufacturers for applications requiring minimum functionality: 1/8 DIN (96 mm $\times$ 48 mm), 1/16 DIN (48 mm $\times$ 48 mm), and even 1/32 DIN (48 mm $\times$ 24 mm) sizes are available.

One of the advantageous capabilities of modern stand-alone controllers is the ability to exchange data over digital networks. This provides operations and maintenance personnel alike the ability to remotely monitor and even control (adjust setpoints, switch modes, change tuning parameters, etc.) the process controller from a computer workstation. The Siemens model 353 controller (with appropriate options) has the ability to digitally network over Ethernet, a very common and robust digital network standard. The following photographs show three such controllers connected to a network through a common 4-port Ethernet "hub" device:





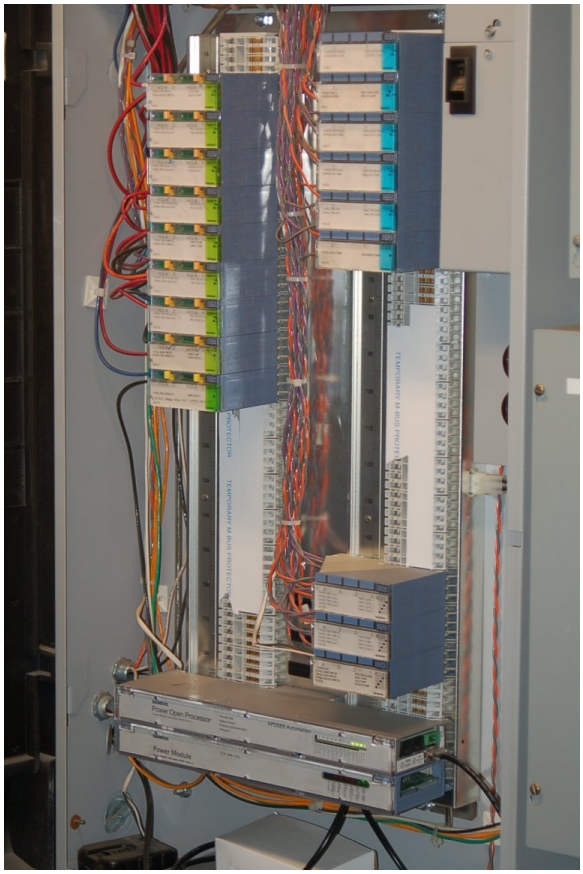
Special software (in this case, Siemens *Procidia*) running on a computer workstation connected to the same Ethernet network acquires data from and sends data to the networked controllers. Screenshots of this software show typical displays allowing complete control over the function of the process controllers:



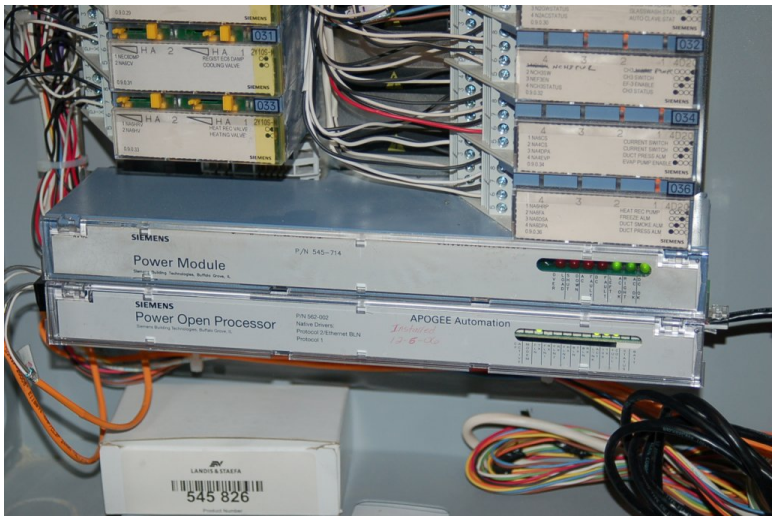
29.13.2 Direct digital control (DDC)

A microprocessor operating at sufficient clock speed is able to execute more than one PID control algorithm for a process loop, by "time-sharing" its calculating power: devoting slices of time to the evaluation of each PID equation in rapid succession. This not only makes multiple-loop digital control possible for a single microprocessor, but also makes it very attractive given the microprocessor's natural ability to manage data archival, transfer, and networking. A single computer is able to execute PID control for multiple loops, and also make that loop control data accessible between loops (for purposes of cascade, ratio, feedforward, and other control strategies) and accessible on networks for human operators and technicians to easily access.

Such *direct digital* control (DDC) has been applied with great success to the problem of building automation, where temperature and humidity controls for large structures benefit from large-scale data integration. The following photograph shows a Siemens APOGEE building automation system with multiple I/O (input/output) cards providing interface between analog instrument signals and the microprocessor's digital functions:

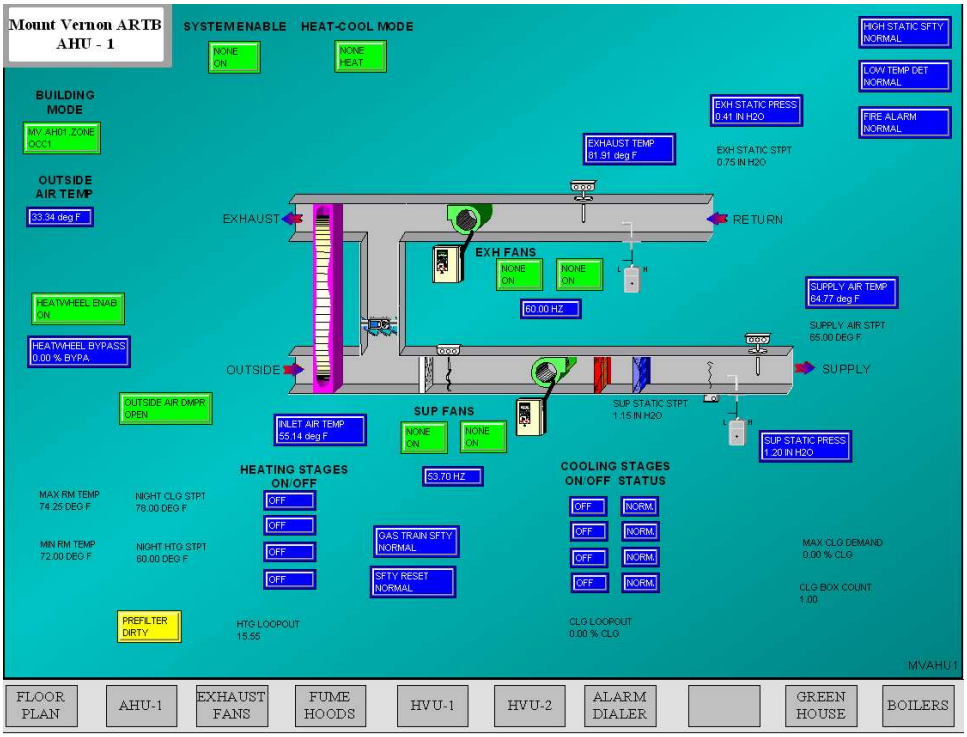


A close-up view of an APOGEE processor shows the device handling all mathematical calculations for the PID control:



Other than a few LEDs, there is no visual indication in this panel of what the system is doing at any particular time. Operators, engineers, and technicians alike must use software running on a networked personal computer to access data in this control system.

An example of the HMI (Human-Machine Interface) software one might see used in conjunction with a DDC controller is shown here, also from a Siemens APOGEE building control system:



This particular screenshot shows monitored and controlled variables for a heat exchanger ("heat wheel") used to exchange heat between outgoing and incoming air for the building.

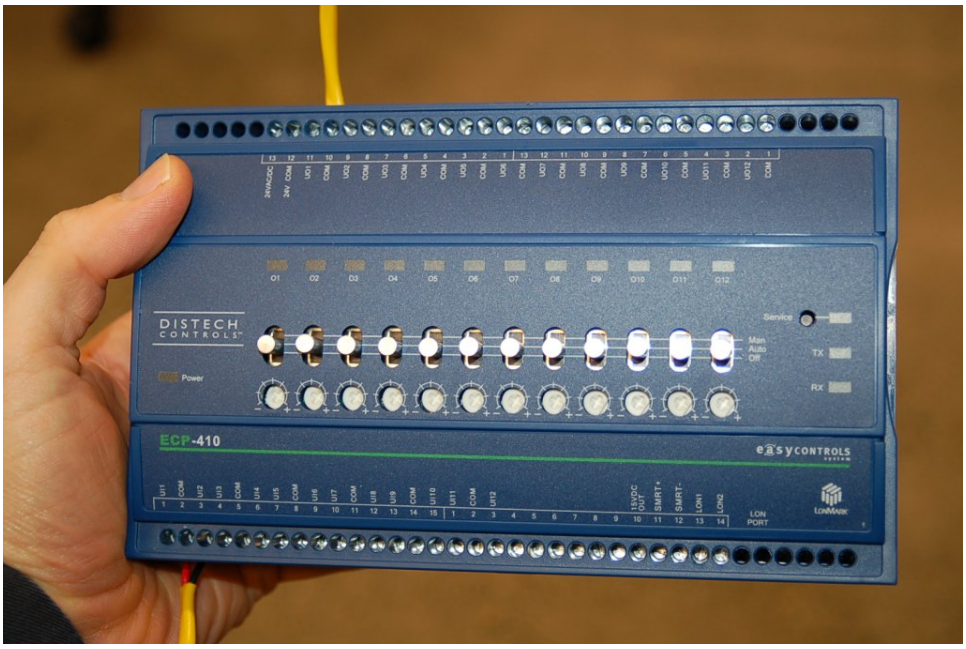
A smaller-scale example of a DDC system is the Delta model DSC-1280 controller, an example shown in the following photograph:



This system does not have plug-in I/O cards like the Siemens APOGEE, but instead is monolithic in design, with all inputs and outputs part of one large "motherboard" PCB. The model DSC-1280 controller has 12 input channels and 8 output channels (hence the model number "1280"). An Ethernet cable (RJ-45 plug) is seen in the upper-left corner of this unit, through which a remotely-located personal computer communicates with the DDC using a high-level protocol called BACnet. In many ways, BACnet is similar to Modbus, residing at layer 7 of the OSI Reference Model (the so-called *Application Layer*), unconcerned with the details of data communication at the Physical or Data Link layers. This means, like Modbus, BACnet commands may be sent and received over a variety of lower-level network standards, with Ethernet being the preferred[33] option at the time of this writing.

[33] Thanks to the explosion of network growth accompanying personal computers in the workplace, Ethernet is ubiquitous. The relatively high speed and low cost of Ethernet communications equipment makes it an attractive network standard over which a great many high-level industrial protocols communicate.

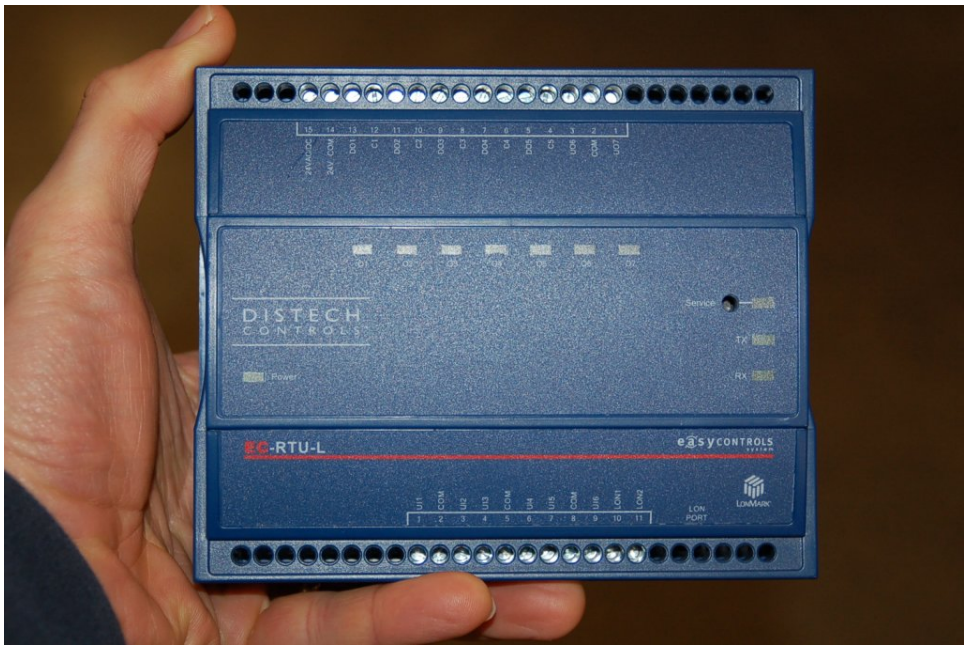
Another example of a small-scale DDC is this Distech model ECP-410 unit:



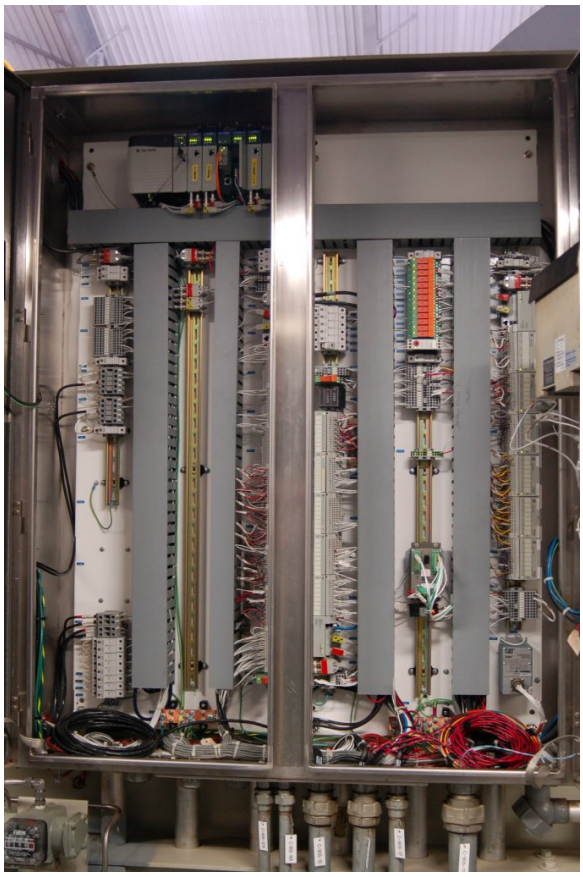
This controller uses *LonWorks* as its communication protocol rather than BACnet. Like BACnet, LonWorks is an upper-layer protocol and may be transported over Ethernet as well as simpler serial communication formats. The model ECP-410 DDC controller has a two-wire connection for its LonWorks network.

Programming of DDC controllers ranges from text-based languages (similar to BASIC) to function-block programming. The Delta DSC-1280 is an example of a controller programmed in text, while the Distech ECP-410 supports function blocks in addition to text-based programming.

The application-specific nature of DDC (environmental controls for the interior of large buildings and other facilities) lends itself to controller units pre-programmed to perform well-defined tasks rather than general-purpose controllers designed to be user-programmable for any task. An example of such a "fixed" program controller is the Distech model EC-RTU-L, designed to control a "rooftop unit" for air handling:



A more common application of industrial DDC is the use of programmable logic controllers (PLCs) to control multiple loops. PLCs were originally invented for on/off (discrete) process control functions, but have subsequently grown in speed and capability to execute analog PID control functions as well. This next photograph shows an Allen-Bradley (Rockwell) ControlLogix PLC used to control the operation of a gas turbine engine. The PLC may be seen in the upper-left corner of the enclosure, with the rest of the enclosure devoted to terminal blocks and accessory components:



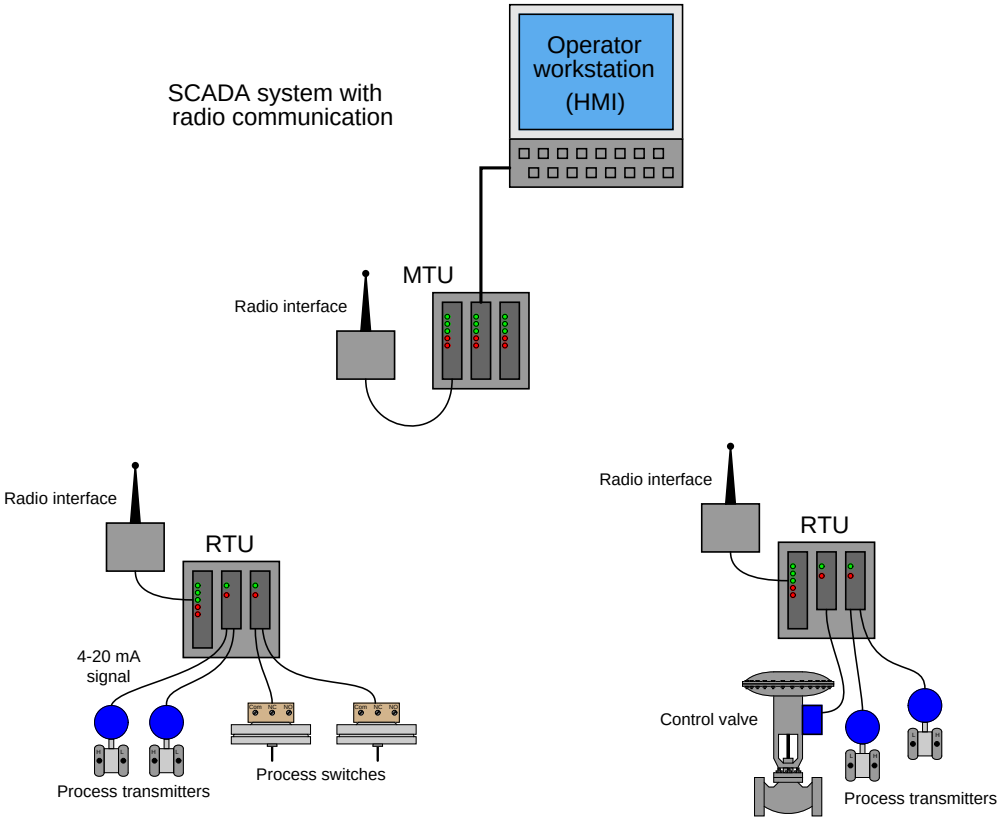
A strong advantage of using PLCs for analog loop control is the ability to easily integrate discrete controls with the analog controls. It is quite easy, for example, to coordinate the sequential start-up and shut-down functions necessary for intermittent operation with the analog PID controls necessary for continuous operation, all within one programmable logic controller. It should be noted, however, that many early PLC implementations of PID algorithms were crude at best, lacking the finesse of stand-alone PID controllers. Even some modern PLC analog functions are mediocre[34] at the time of this writing (2008).

[34] An aspect common to many PLC implementations of PID control is the use of the "parallel" PID algorithm instead of the superior "ISA" or "non-interacting" algorithm. The choice of algorithm may have a profound effect on tuning, and on tuning procedures, especially when tuning parameters must be re-adjusted to accommodate changes in transmitter range.

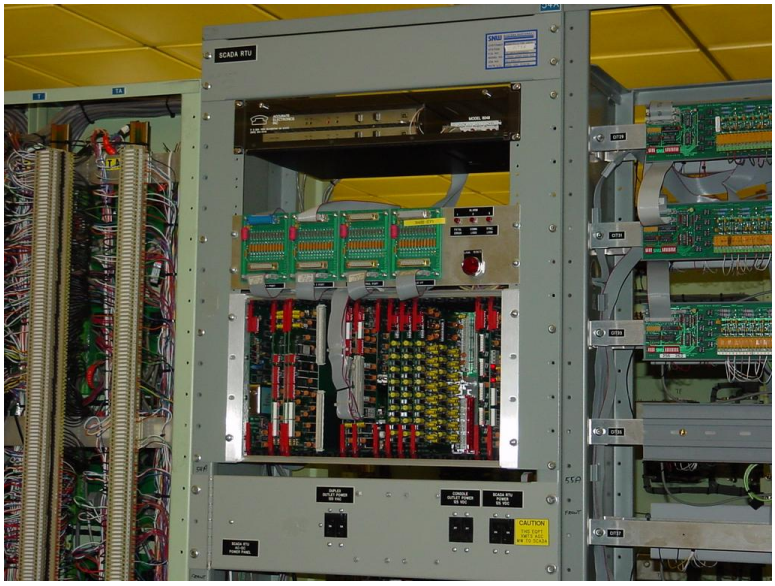
29.13.3 SCADA and telemetry systems

A similar control system architecture to Direct Digital Control (DDC) – assigning a single microprocessor to the task of managing multiple control functions, with digital communication between the microprocessor units – is used for the management of systems which are by their very nature spread over wide geographical regions. Such systems are generally referred to as *SCADA*, which is an acronym standing for *Supervisory Control And Data Acquisition*.

The typical SCADA system consists of multiple *Remote Terminal Unit* (RTU) devices connected to process transmitters and final control elements, implementing basic control functions such as motor start/stop and PID loop control. These RTU devices communicate digitally to a *Master Terminal Unit* (MTU) device at a central location where human operators may monitor the process and issue commands.



A photograph of an RTU "rack" operating at a large electric power substation is shown here:



Some RTU hardware, such as the substation monitoring system shown above, is custom-manufactured for the application. Other RTU hardware is more general in purpose, intended for the monitoring and control of natural gas and oil production wells, but applicable to other applications as well.

The Fisher ROC 800 – shown in the photograph below – is an example of an RTU designed to operate with a minimum of electrical power, so that a single solar panel and battery will be sufficient for year-round operation in remote environments. The particular unit shown is installed in a natural gas metering station, where is monitors gas pressure, temperature, and flow rate, and also controls the injection of an "odorizing" compound into the gas to give it a bad smell:

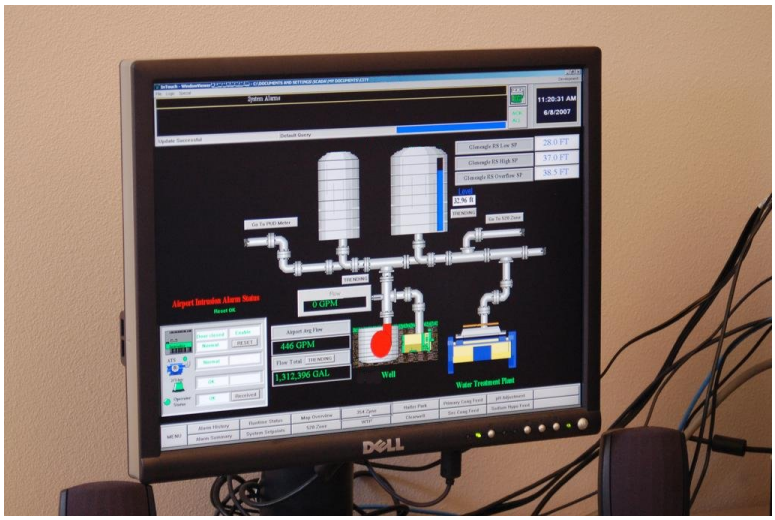


Standard programmable logic controllers (PLCs) are ideal candidates for use as RTU devices. Modern PLCs have all the I/O, networking, and control algorithm capability necessary to function as remote terminal units. Commercially available Human-Machine Interface (HMI) software allowing personal computers to display PLC variable values potentially turns every PC into a Master Terminal Unit (MTU) where operators can view process variables, change setpoints, and issue other commands for controlling the process.

A photograph of such HMI software used to monitor a SCADA system for a set of natural gas compressors is shown here:



Another photograph of a similar system used to monitor and control drinking water reservoirs for a city is shown here:



A concept closely related to SCADA is that of *telemetry*, the word literally meaning "distance measuring" (i.e. measuring something over a distance). The acronym SCADA, by containing the word "control," implies two-way communication (measurement and control) between the master location and the remote location. In applications where the flow of information is strictly one-way (simplex) from the remote location to the master location, "telemetry" is a more apt description.

Telemetry systems find wide application in scientific research. Seismographs, river and stream flowmeters, weather stations, and other remotely-located measurement instruments connected (usually by radio links) to some centralized data collection center are all examples of telemetry. Any industrial measurement (-only) application spanning a large distance could likewise be classified as a telemetry system, although you will sometimes find the term "SCADA" applied even when the communication is simplex in nature.

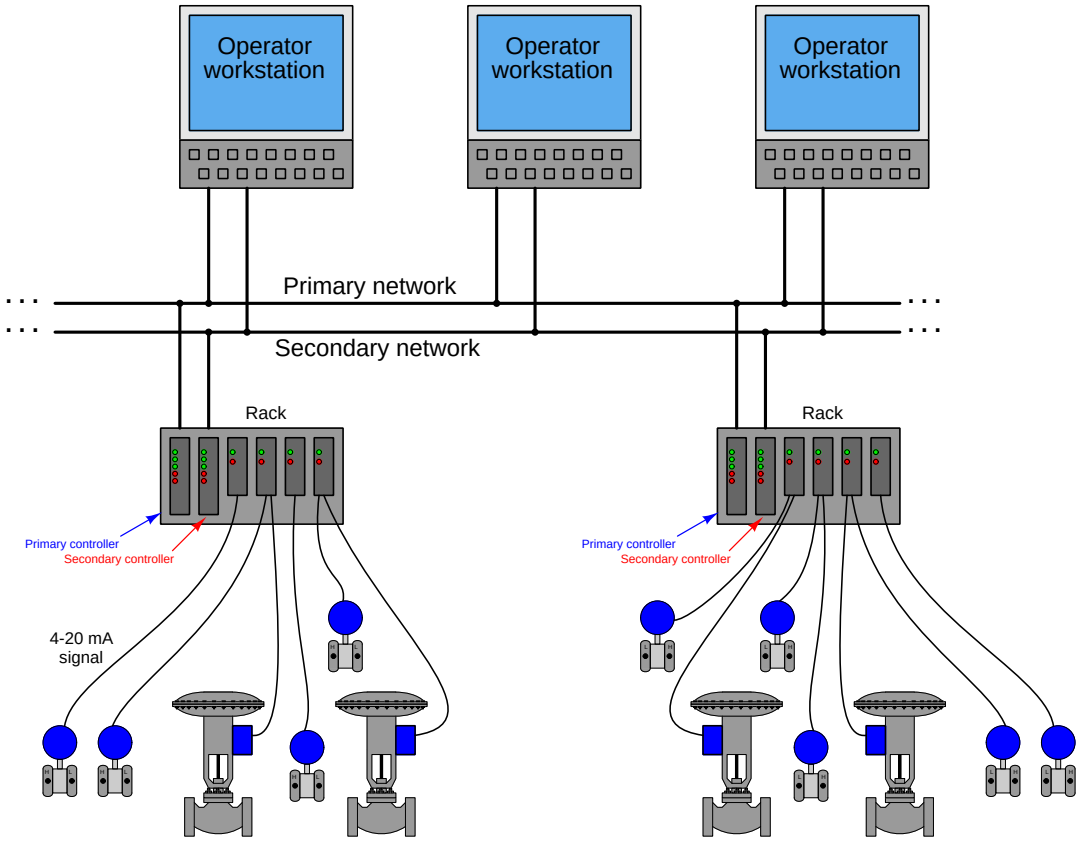
29.13.4 Distributed Control Systems (DCS)

A radically new concept appeared in the world of industrial control in the mid-1970's: the notion of *distributed* digital control. Direct digital control during that era[35] suffered a substantial problem: the potential for catastrophic failure if the single digital computer executing *multiple* PID control functions were to ever halt. Digital control brings many advantages, but it isn't worth the risk if the entire operation will shut down (or catastrophically fail!) following a hardware or software failure within that one computer.

Distributed control directly addressed this concern by having multiple control computers – each one responsible for only a handful of PID loops – distributed throughout the facility and networked together to share information with each other and with operator display consoles. With individual process control "nodes" scattered throughout the campus, each one dedicated to controlling just a few loops, there would be less concentration of liability as there would be with a single-computer DDC system. Such distribution of computing hardware also shortened the analog signal wiring, because now the hundreds or thousands of analog field instrument cables only had to reach as far as the distributed nodes, not all the way to a centralized control room. Only the networking cable had to reach that far, representing a drastic reduction in wiring needs. Furthermore, distributed control introduced the concept of *redundancy* to industrial control systems: where digital signal acquisition and processing hardware units were equipped with "spare" units designed to automatically take over all critical functions in the event of a primary failure.

[35] Modern DDC systems of the type used for building automation (heating, cooling, security, etc.) almost always consist of networked control nodes, each node tasked with monitoring and control of a limited area. The same may be said for modern PLC technology, which not only exhibits advanced networking capability (fieldbus I/O networks, Ethernet, Modbus, wireless communications), but is often also capable of redundancy in both processing and I/O. As technology becomes more sophisticated, the distinction between a DDC (or a networked PLC system) and a DCS becomes more ambiguous.

The following illustration shows a typical distributed control system (DCS) architecture:



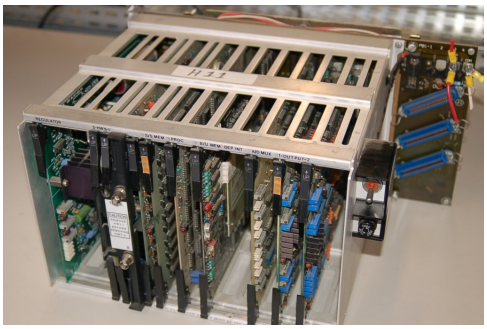
Each "rack" contains a microprocessor to implement all necessary control functions, with individual I/O (input/output) "cards" for converting analog field instrument signals into digital format, and vice-versa. Redundant processors, redundant network cables, and even redundant I/O cards address the possibility of component failure. DCS processors are usually programmed to perform routine self-checks[36] on redundant system components to ensure availability of the spare components in the event of a failure.

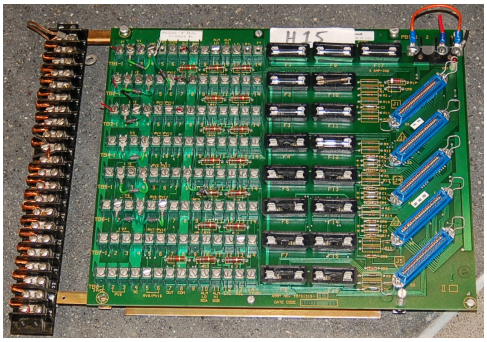
If there ever was a total failure in one of the "control racks" where the redundancy proved insufficient for the fault(s), the only PID loops faulted will be those resident in that rack, not any of the other loops throughout the system. Likewise, if ever the network cables become severed or otherwise faulted, only the information flow between those two points will suffer; the rest of the system will continue to communicate data normally. Thus, one of the "hallmark" features of a DCS is its tolerance to serious faults: even in the event of severe hardware or software faults, the impact to process control is minimized by design.

[36] An example of such a self-check is scheduled switching of the networks: if the system has been operating on network cable "A" for the past four hours, it might switch to cable "B" for the next four hours, then back again after another four hours to continually ensure both cables are functioning properly.

One of the very first distributed control systems in the world was the Honeywell TDC2000 system[37], introduced in 1975. By today's standards, the technology was crude[38], but the concept was revolutionary.

Each rack (called a "box" by Honeywell) consisted of an aluminum frame holding several large printed circuit boards with card-edge connectors. A "basic controller" box appears in the left-hand photograph. The right-hand photograph shows the termination board where the field wiring (4-20 mA) connections were made. A thick cable connected each termination board to its respective controller box:





Controller redundancy in the TDC2000 DCS took the form of a "spare" controller box serving as a backup for up to eight other controller boxes. Thick cables routed all analog signals to this spare controller, so that it would have access to them in the event it needed to take over for a failed controller. The spare controller would become active on the event of *any* fault in any of the other controllers, including failures in the I/O cards. Thus, this redundancy system provided for processor failures as well as I/O failures. All TDC2000 controllers communicated digitally by means of a dual coaxial cable network known as the "Data Hiway." The dual cables provided redundancy in network communications.

[37] To be fair, the Yokogawa Electric Corporation of Japan introduced their CENTUM distributed control system the same year as Honeywell. Unfortunately, while I have personal experience maintaining and using the Honeywell TDC2000 system, I have zero personal experience with the Yokogawa CENTUM system, and neither have I been able to obtain technical documentation for the original incarnation of this DCS (Yokogawa's latest DCS offering goes by the same name). Consequently, I can do little in this chapter but mention its existence, despite the fact that it deserves just as much recognition as the Honeywell TDC2000 system.

[38] Just to give some perspective, the original TDC2000 system used whole-board processors rather than microprocessor chips, and magnetic core memory rather than static or dynamic RAM circuits! Communication between controller nodes and operator stations occurred over thick coaxial cables, implementing master/slave arbitration with a separate device (a "Hiway Traffic Director" or HTD) coordinating all communications between nodes. Like Bob Metcalfe's original version of Ethernet, these coaxial cables were terminated at their end-points by termination resistors, with coaxial "tee" connectors providing branch points for multiple nodes to connect along the network.

A typical TDC2000 operator workstation appears in the next photograph:



Over the years following its 1975 introduction, the Honeywell system grew in sophistication with faster networks (the "Local Control Network" or LCN), more capable controller racks (the "Process Manager" or PM series), and better operator workstations. Many of these improvements were incremental, consisting of add-on components that could work with existing TDC2000 components so that the entire system need not be replaced to accept the new upgrades.

Other control equipment manufacturers responded to the DCS revolution started by Honeywell and Yokogawa by offering their own distributed control systems. The Bailey Network 90 (Net90) DCS, Bailey Infi90 DCS, and the Fisher Provox systems are examples. Foxboro, already an established leader in the control system field with their SPEC 200 analog system, first augmented the SPEC 200 with digital capabilities (the VIDEOSPEC workstation consoles, FOX I/A computer, INTERSPEC and FOXNET data networks), then developed an entirely digital distributed control system, the SPECTRUM.

Some modern distributed control systems offered at the time of this writing (2008) include:

- ABB *800xA*
- Emerson *DeltaV* and *Ovation*
- Foxboro (Invensys) *I/A*
- Honeywell *Experion PKS*
- Yokogawa *CENTUM VP* and *CENTUM CS*

For a visual comparison with the Honeywell TDC2000 DCS, examine the following photograph of an Emerson DeltaV DCS rack, with processor and multiple I/O modules:



A photograph of an Emerson Ovation DCS rack shows a vertically-oriented backplane accepting multiple I/O modules:



Many modern distributed control systems such as the Emerson DeltaV use regular personal computers rather than proprietary hardware as operator workstations. This cost-saving measure leverages existing computer and display technologies without sacrificing control-level reliability (since the control hardware and software is still industrial-grade):



As previously mentioned in the Direct Digital Control (DDC) subsection, programmable logic controllers (PLCs) are becoming more and more popular as PID control platforms due to their ever-expanding speed, functionality, and relatively low cost. It is now possible with modern PLC hardware and networking capabilities to build a truly distributed control system with individual PLCs as the processing nodes, and with redundancy built into each of those nodes so that any single failure does not interrupt critical control functions. Such a system may be purchased at a fraction of the up-front cost of a fully-fledged DCS.

However, what is currently lacking in the PLC world is the same level of hardware and software integration necessary to build a functional distributed control system that comes as ready-to-use as a system pre-built by a DCS manufacturer. In other words, if an enterprise chooses to build their own distributed control system using programmable logic controllers, they must be prepared to do a *lot* of programming work in order to emulate the same level of functionality and power as a pre-engineered DCS[39]. Any engineer or technician who has experienced the power of a modern DCS – with its self-diagnostic, "smart" instrument management, event auditing, advanced control strategy, pre-engineered redundancy, data collection and analysis, and alarm management capabilities – realizes these features are neither luxuries nor are they trivial to engineer. Woe to anyone who thinks these critical features may be created by incumbent staff at a lesser cost!

[39] I know of a major industrial manufacturing facility (which shall remain nameless) where a PLC vendor promised the same technical capability as a full DCS at approximately one-tenth the installed cost. Several years and several tens of thousands of man-hours later, the sad realization was this "bargain" did not live up to its promise, and the decision was made to remove the PLCs and go with a complete DCS from another manufacturer. *Caveat emptor!*

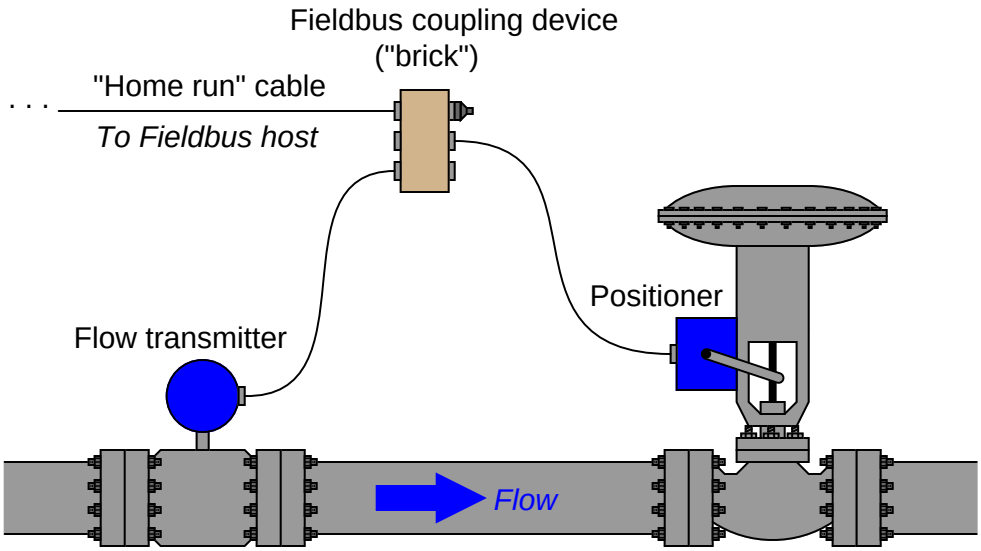
29.13.5 Fieldbus control

The DCS revolution started in the mid-1970's was fundamentally a moving of control system "intelligence" from a centralized location to distributed locations. Rather than have a single computer (or a panel full of single-loop controllers) located in a central control room implement PID control for a multitude of process loops, many (smaller) computers located closer to the process areas would execute the PID and other control functions, with network cables shuttling data between those distributed locations and the central control room.

Beginning in the late 1980's, the next logical step in this evolution of control architecture saw the relocation of control "intelligence" to the field instruments themselves. In other words, the new idea was to equip individual transmitters and control valve positioners with the necessary computational power to implement PID control all on their own, using digital networks to carry process data between the field instruments and any location desired. This is the fundamental concept of *fieldbus*.

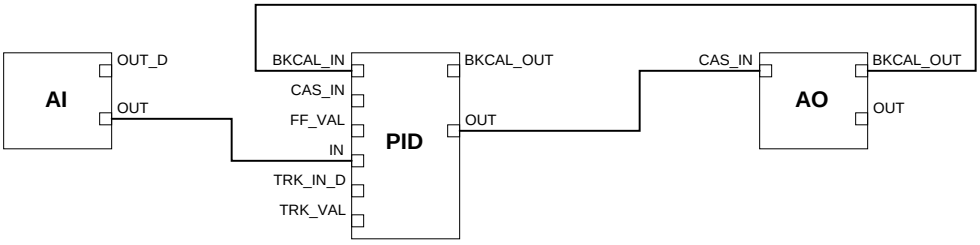
"Fieldbus" as a technical term has multiple definitions. Many manufacturers use the word "fieldbus" to describe any digital network used to transport data to and from field instruments. In this subsection, I use the word "fieldbus" to describe a design philosophy where field instruments possess all the necessary "intelligence" to control the process, with no need for separate centralized (or even distributed) control hardware. *FOUNDATION Fieldbus* is the first standard to embody this fully-distributed control concept, the technical details of this open standard maintained and promoted by the *Fieldbus Foundation*. The aim of this Foundation is to establish an open, technical standard for *any* manufacturer to follow in the design of their fieldbus instruments. This means a FOUNDATION Fieldbus (FF) transmitter manufactured by Smar will work seamlessly with a FF control valve positioner manufactured by Fisher, communicating effortlessly with a FF-aware host system manufactured by ABB, and so on. This may be thought of in terms of being the digital equivalent of the 3-15 PSI pneumatic signal standard or the 4-20 mA analog electronic signal standard: so long as all instruments "talk" according to the same standard, brands and models may be freely interchanged to build any control system desired.

To illustrate the general fieldbus concept, consider this flow control system:



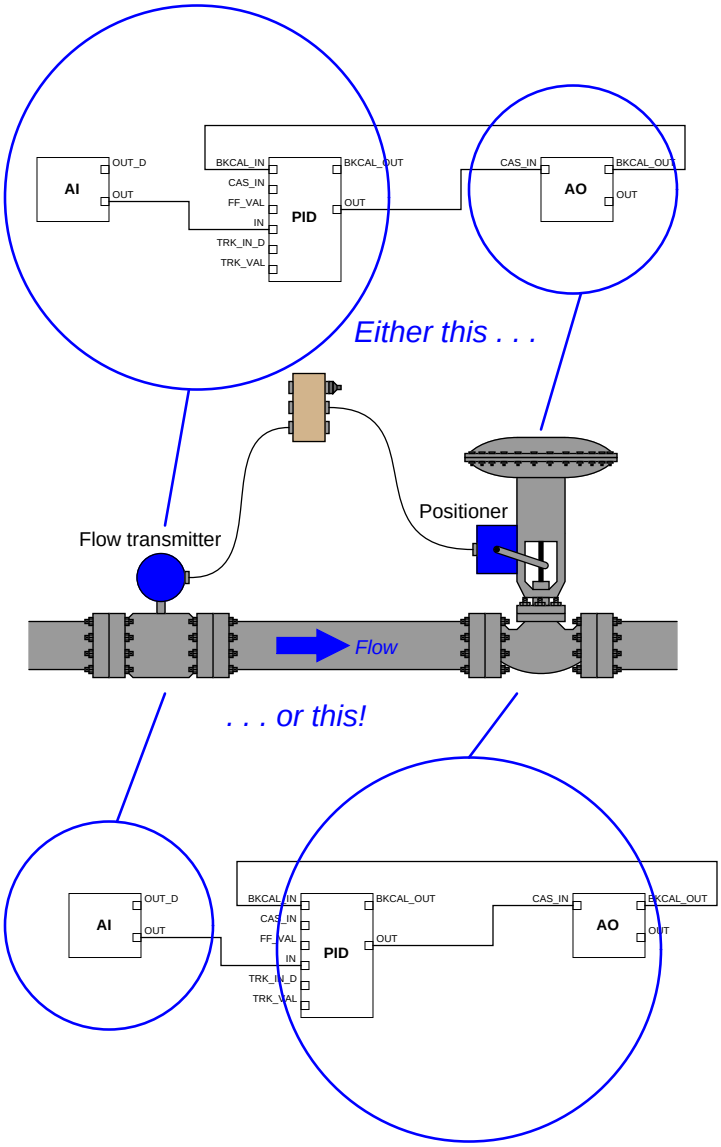
Here, a fieldbus *coupling device* provides a convenient junction point for cables coming from the transmitter, valve positioner, and host system. FOUNDATION Fieldbus devices both receive DC power and communicate digitally over the same twisted-pair cables. In this case, the host system provides DC power for the transmitter and positioner to function, while communication of process data occurs primarily between the transmitter and positioner (with little necessary involvement of the host system[40]).

As with distributed control systems, FOUNDATION Fieldbus instruments are programmed using a function block language. In this case, we must have an analog input (for the transmitter's measurement), a PID function block, and an analog output (for the valve positioner) to make a complete flow control system:



[40] Although it is customary for the host system to be configured as the *Link Active Scheduler* (LAS) device to schedule and coordinate all fieldbus device communications, this is not absolutely necessary. Any suitable field instrument may also serve as the LAS, which means a host system is not even necessary except to provide DC power to the instruments, and serve as a point of interface for human operators, engineers, and technicians.

The analog input (AI) block must reside in the transmitter, and the analog output (AO) block must reside in the valve positioner, since those blocks necessarily relate to the measured and controlled variables, respectively. However, the PID block may reside in *either* field device:



Practical reasons do exist for choosing one location of the PID function block over the other, most notably the difference in communication loading between the two options[41]. However, there

[41] With the PID function block programmed in the flow transmitter, there will be twice as many scheduled communication events per macrocycle than if the function block is programmed into the valve positioner. This is evident by the number of signal lines connecting circled block(s) to circled block(s) in the above illustration.

is no *conceptual* limitation to the location of the PID function block. In a fieldbus control system where the control "intelligence" is distributed all the way to the field instrument devices themselves, there are no limits to system flexibility.

29.14 Practical PID controller features

In order for any PID controller to be practical, it must be able to do more than just implement the PID equation. This section identifies and explains some of the basic features found on most (but not all!) modern PID controllers:

- Manual versus Automatic mode
- Output tracking
- Setpoint tracking
- Alarming
- PV characterization and damping
- Setpoint limits
- Output limits
- PID tuning security

29.14.1 Manual and automatic modes

When a controller continually calculates output values based on PV and SP values over time, it is said to be operating in *automatic* mode. This mode, of course, is what is necessary to regulate any process. There are times, however, when it is desirable to allow a human operator to manually "override" the automatic action of the PID controller. Applicable instances include process start-up and shut-down events, emergencies, and maintenance procedures. A controller that is being "overridden" by a human being is said to be in *manual mode*.

A very common application of manual mode is during maintenance of the sensing element or transmitter. If an instrument technician needs to disconnect a process transmitter for calibration or replacement, the controller receiving that transmitter's signal cannot be left in automatic mode. If it is, then the controller may[42] take sudden corrective action the moment the transmitter's signal goes dead. If the controller is first placed in manual mode before the technician disconnects the transmitter, however, the controller will ignore any changes in the PV signal, letting its output signal be adjusted at will by the human operator. If there is another indicator of the same process variable as the one formerly reported by the disconnected transmitter, the human operator may elect to read that other indicator and play the part of a PID controller, manually adjusting the final control element to maintain the alternate indicator at setpoint while the technician completes the transmitter's maintenance.

An extension of this "mode" concept applies to controllers configured to receive a setpoint from another device (called a *remote* or *cascaded* setpoint). In addition to an automatic and a manual mode selection, a third selection called *cascade* exists to switch the controller's setpoint from human operator control to remote (or "cascade") control.

[42] The only reason I say "may" instead of "will" is because some modern digital controllers are designed to automatically switch to manual-mode operation in the event of a sensor or transmitter signal loss. Any controller not "smart" enough to shed its operating mode to manual in the event of PV signal loss will react dramatically when that PV signal dies, and this is not a good thing for an operating loop!

29.14.2 Output and setpoint tracking

The provision of manual and automatic operating modes creates a set of potential problems for the PID controller. If, for example, a PID controller is switched from automatic to manual mode by a human operator, and then the output is manually adjusted to some new value, what will the output value do when the controller is switched *back* to automatic mode? In some crude PID controller designs, the result would be an immediate "jump" back to the output value calculated by the PID equation while the controller was in manual. In other words, some controllers never stop evaluating the PID equation – even while in manual mode – and will default to that automatically-calculated output value when the operating mode is switched from manual to automatic.

This can be very frustrating to the human operator, who may wish to use the controller's manual mode as a way to change the controller's bias value. Imagine, for example, that a PD controller (no integral action) is operating in automatic mode at some low output value, which happens to be too low to achieve the desired setpoint. The operator switches the controller to manual mode and then raises the output value, allowing the process variable to approach setpoint. When PV nearly equals SP, the operator switches the controller's mode back to automatic, expecting the PID equation to start working again from this new starting point. In a crude controller, however, the output would jump back to some lower value, right where the PD equation would have placed it for these PV and SP conditions.

A feature designed to overcome this problem – which is so convenient that I consider it an essential feature of any controller with a manual mode – is called *output tracking*. With output tracking, the bias value of the controller shifts every time the controller is placed into manual mode and the output value manually changed. Thus, when the controller is switched from manual mode to automatic mode, the output does *not* immediately jump to some previously-calculated value, but rather "picks up" from the last manually-set value and begins to control from that point as dictated by the PID equation. In other words, output tracking allows a human operator to arbitrarily offset the output of a PID controller by switching to manual mode, adjusting the output value, and then switching back to automatic mode. The output will continue its automatic action from this new starting point instead of the old starting point.

A very important application of output tracking is in the manual correction of integral wind-up (sometimes called *reset windup* or just *windup*). This is what happens to a controller with integral action if for some reason the process variable *cannot* achieve setpoint no matter how far the output signal value is driven by integral action. An example might be on a temperature controller where the source of heat for the process is a steam system. If the steam system shuts down, the temperature controller *cannot* warm the process up to the temperature setpoint value no matter how far open the steam valve is driven by integral action. If the steam system is shut down for too long, the result will be a controller output saturated at maximum value in a futile attempt to warm the process. If and when the steam system starts back up, the controller's saturated output will now send *too much heating steam* to the process, causing the process temperature to overshoot setpoint until integral action drives the output signal back down to some reasonable level. This situation may be averted, however, if the operator switches the temperature controller to manual mode as soon as the steam system shuts down. Even if this preventive step is not taken, the problem of overshoot may be averted upon steam system start-up if the operator uses output tracking by quickly switching the controller into manual mode, adjusting the output down to a reasonable level, and then switching back into automatic mode so that the controller's output value is no longer "wound up" at a high

level[43].

A similar feature to output tracking – also designed for the convenience of a human operator switching a PID controller between automatic and manual modes – is called *setpoint tracking*. The purpose of setpoint tracking is to equalize SP and PV while the controller is in manual mode, so that when the controller gets switched back into automatic mode, it will begin its automatic operation with no error (PV = SP).

This feature is most useful during system start-ups, where the controller may have difficulty controlling the process in automatic mode under unusual conditions. Operators often prefer to run certain control loops in manual mode from the time of initial start-up until such time that the process is near normal operating conditions. At that point, when the operator is content with the stability of the process, the controller is assigned the responsibility of maintaining the process at setpoint. With setpoint tracking present in the controller, the controller's SP value will be held equal to the PV value (whatever that value happens to be) for the entire time the controller is in manual mode. Once the operator decides it is proper to switch the controller into automatic mode, the SP value freezes at that last manual-mode PV value, and the controller will continue to control the PV at that SP value. Of course, the operator is free to adjust the SP value to any new value while the controller is in automatic mode, but this is at the operator's discretion.

Without setpoint tracking, the operator would *have to* make a setpoint adjustment either before or after switching the controller from manual mode to automatic mode, in order to ensure the controller was properly set up to maintain the process variable at the desired value. With setpoint tracking, the setpoint value will default to the process variable value when the controller was last in manual mode, which (it is assumed) will be close enough to the desired value to suffice for continued operation.

Unlike output tracking, for which there is virtually no reason not to have the feature present in a PID controller, there may very well be applications where we do not wish to have setpoint tracking. For some processes[44], the setpoint value *should* remain fixed at all times, and as such it would be undesirable to have the setpoint value drift around with the process variable value every time the controller was placed into manual mode.

[43] I once had the misfortune of working on an analog PID controller for a chlorine-based wastewater disinfection system that lacked output tracking. The chlorine sensor on this system would occasionally fail due to sample system plugging by algae in the wastewater. When this happened, the PV signal would fail low (indicating abnormally low levels of chlorine gas dissolved in the wastewater) even though the actual dissolved chlorine gas concentration was adequate. The controller, thinking the PV was well below SP, would ramp the chlorine gas control valve further and further open over time, as integral action attempted to reduce the error between PV and SP. The error never went away, of course, because the chlorine sensor was plugged with algae and simply could not detect the actual chlorine gas concentration in the wastewater. By the time I arrived to address the "low chlorine" alarm, the controller output was already wound up to 100%. After cleaning the sensor, and seeing the PV value jump up to some outrageously high level, the controller would take a long time to "wind down" its output because its integral action was very slow. I could not use manual mode to "unwind" the output signal, because this controller lacked the feature of output tracking. My "work-around" solution to this problem was to re-tune the integral term of the controller to some really fast time constant, watch the output "wind down" in fast-motion until it reached a reasonable value, then adjust the integral time constant back to its previous value for continued automatic operation.

[44] Boiler steam drum water level control, for example, is a process where the setpoint really should be left at a 50% value at all times, even if there maybe legitimate reasons for occasionally switching the controller into manual mode.

29.14.3 Alarm capabilities

A common feature on many instrument systems is the ability to alert personnel to the onset of abnormal process conditions. The general term for this function is *alarm*. Process alarms may be triggered by process switches directly sensing abnormal conditions (e.g. high-temperature switches, low-level alarms, low-flow alarms, etc.), in which case they are called *hard alarms*. A *soft alarm*, by contrast, is an alarm triggered by some continuous measurement (i.e. a signal from a process transmitter rather than a process switch) exceeding a pre-programmed alarm limit value.

Since PID controllers are designed to input continuous process measurements, it makes sense that a controller could be equipped with programmable alarm limit values as well, to provide "soft" alarm capability without adding additional instruments to the loop[45]. Not only is PV alarming easy to implement in most PID controllers, but *deviation* alarming is easy to implement as well. A "deviation alarm" is a soft alarm triggered by excessive deviation (error) between PV and SP. Such an event indicates control problems, since a properly-operating feedback loop should be able to maintain reasonable agreement between PV and SP at all times.

Alarm capabilities find their highest level of refinement in modern distributed control systems (DCS), where the networked digital controllers of a DCS provide convenient access and advanced management of hard and soft alarms alike. Not only can alarms be accessed from virtually any location in a facility in a DCS, but they are usually time-stamped and archived for later analysis, which is an *extremely* important feature for the analysis of emergency events, and the continual improvement of process safety.

29.14.4 Output and setpoint limiting

In some process applications, it may not be desirable to allow the controller to automatically manipulate the final control element (control valve, variable-speed motor, heater) over its full 0% - 100% range. In such applications, a useful controller feature is an *output limit*. For example, a PID flow controller may be configured to have a minimum output limit of 5%, so that it is not able to close the control valve any further than the 5% open position in order to maintain "minimum flow" through a pump. The valve may still be fully closed (0% stem position) in manual mode, but just not in automatic mode[46].

Similarly, setpoint values may be internally limited in some PID controllers, such that an operator cannot adjust the setpoint above some limiting value or below some other limiting value. In the event that the process variable *must* be driven outside these limits, the controller may be placed in manual mode and the process "manually" guided to the desired state by an operator.

[45] It is very important to note that soft alarms are not a replacement for hard alarms. There is much wisdom in maintaining both hard and soft alarms for a process, so there will be redundant, non-interactive levels of alarming. Hard and soft alarms should complement each other in any critical process.

[46] Some PID controllers limit manual-mode output values as well, so be sure to check the manufacturer's documentation for output limiting on your particular PID controller!

29.14.5 Security

There is justifiable reason to prevent certain personnel from having access to certain parameters and configurations on PID controllers. Certainly, operations personnel need access to setpoint adjustments and automatic/manual mode controls, but it may be unwise to grant those same operators unlimited access to PID tuning constants and output limits. Similarly, instrument technicians may require access to a PID controller's tuning parameters, but perhaps should be restricted from editing configuration programs maintained by the engineering staff.

Most digital PID controllers have some form of security access control, allowing for different levels of permission in altering PID controller parameters and configurations. Security may be crude (a hidden switch located on a printed circuit board, which only the maintenance personnel should know about), sophisticated (login names and passwords, like a multi-user computer system), or anything in between, depending on the level of development invested in the feature by the controller's manufacturer.

An interesting solution to the problem of security in the days of analog control systems was the architecture of Foxboro's SPEC 200 analog electronic control system. The controller displays, setpoint adjustments, and auto/manual mode controls were located on the control room panel where anyone could access them. All other adjustments (PID settings, alarm settings, limit settings) could be located in the *nest* area where all the analog circuit control cards resided. Since the "nest" racks could be physically located in a room separate from the control room, personnel access to the nest room served as access security to these system parameters.

At first, the concept of controller parameter security may seen distrustful and perhaps even insulting to those denied access, especially when the denied persons possess the necessary knowledge to understand the functions and consequences of those parameters. It is not uncommon for soft alarm values to be "locked out" from operator access despite the fact that operators understand very well the purpose and functions of these alarms. At some facilities, PID tuning is the exclusive domain of process engineers, with instrument technicians and operators alike barred from altering PID tuning constants even though some operators and many technicians may well understand PID controller tuning.

When considering security access, there is more to regard than just knowledge or ability. At a fundamental level, security is a task of limiting access commensurate with *responsibility*. In other words, security restrictions exist to exclude those not charged with particular responsibilities. Knowledge and ability are necessary conditions of responsibility (i.e. one cannot reasonably be held responsible for something beyond their knowledge or control), but they are not *sufficient* conditions of responsibility (i.e. knowing how to, and being able to perform a task does not confer responsibility for that task getting completed). An operator may very well understand how and why a soft alarm on a controller works, but the responsibility for altering the alarm value may reside with someone else whose job description it is to ensure the alarm values correspond to plant-wide policies.

29.15 Digital PID algorithms

Instrument technicians should not have to concern themselves over the programming details internal to digital PID controllers. Ideally, a digital PID controller should simply perform the task of executing PID control with all the necessary features (setpoint tracking, output limiting, etc.) without the end-user having to know anything about those details. However, in my years of experience I have seen enough examples of poor PID implementation to warrant an explanatory section in this book, both so instrumentation professionals may recognize poor PID implementation when they see it, and also so those with the responsibility of designing PID algorithms may avoid some common mistakes.

29.15.1 Introduction to pseudocode

In order to show digital algorithms, I will use a form of notation called *pseudocode*: a text-based language instructing a digital computing device to implement step-by-step procedures. "Pseudocode" is written to be easily read and understood by human beings, yet similar enough in syntax and structure to real computer programming languages for a human programmer to be able to easily translate to a high-level programming language such as BASIC, C++, or Java. Since pseudocode is not a formal computer language, we may use it to very efficiently describe certain algorithms (procedures) without having to abide by strict "grammatical" rules as we would if writing in a formal language such as BASIC, C++, or Java.

Program loops

Each line of text in the following listing represents a command for the digital computer to follow, one by one, in order from top to bottom. The `LOOP` and `ENDLOOP` markers represent the boundaries of a program *loop*, where the same set of encapsulated commands are executed over and over again in cyclic fashion:

Pseudocode listing [47]

```
LOOP
  PRINT "Hello World!"   // This line prints text to the screen
  OUTPUT audible beep on the speaker   // This line beeps the speaker
ENDLOOP
```

In this particular case, the result of this program's execution is a continuous printing of the words "Hello World!" to the computer's display with a single "beep" tone accompanying each printed line. The words following a double-slash (//) are called *comments*, and exist only to provide explanatory text for the human reader, not the computer. Admittedly, this example program would be both impractical and annoying to actually run in a computer, but it does serve to illustrate the basic concept of a program "loop" shown in pseudocode.

[47] I have used a typesetting convention to help make my pseudocode easier for human beings to read: all formal commands appear in bold-faced blue type, while all comments appear in italicized red type. All other text appears as normal-faced black type. One should remember that the computer running any program cares not for how the text is typeset: all it cares is that the commands are properly used (i.e. no "grammatical" or "syntactical" errors).

Assigning values

For another example of pseudocode, consider the following program. This code causes a variable (x) in the computer's memory to alternate between two values of 0 and 2 indefinitely:

Pseudocode listing

```
DECLARE x to be an integer variable
SET x = 2  // Initializing the value of x

LOOP
  // This SET command alternates the value of x with each pass
  SET x = 2 - x
ENDLOOP
```

The first instruction in this listing declares the type of variable x will be. In this case, x will be an *integer* variable, which means it may only represent whole-number quantities and their negative counterparts – no other values (e.g. fractions, decimals) are possible. If we wished to limit the scope of x even further to represent just 0 or 1 (i.e. a single bit), we would have to declare it as a *Boolean* variable. If we required x to be able to represent fractional values as well, we would have to declare it as a *floating-point* variable. Variable declarations are important in computer programming because it instructs the computer how much space in its random-access memory to allocate to each variable, which necessarily limits the range of numbers each variable may represent.

The next instruction initializes x to a value of two. Like the declaration, this instruction need only happen once at the beginning of the program's execution, and never again so long as the program continues to run. The single `SET` statement located between the `LOOP` and `ENDLOOP` markers, however, repeatedly executes as fast as the computer's processor allows, causing x to rapidly alternate between the values of two and zero.

It should be noted that the "equals" sign (=) in computer programming often has a different meaning from that commonly implied in ordinary mathematics. When used in conjunction with the `SET` command, an "equals" sign *assigns* the value of the right-hand quantity to the left-hand variable. For example, the command `SET x = 2 - x` tells the computer to first calculate the quantity $2 - x$ and then set the variable x to this new value. It definitely does *not* mean to imply x is actually equal in value to $2 - x$, which would be a mathematical contradiction. Thus, you should interpret the `SET` command to mean "set equal to . . ."

Testing values (conditional statements)

If we mean to simply test for an equality between two quantities, we may use the same symbol (=) in the context of a different command, such as "IF":

Pseudocode listing

```
DECLARE x to be an integer variable

LOOP

  // (other code manipulating the value of x goes here)

  IF x = 5 THEN
    PRINT "The value of the number is 5"
    OUTPUT audible beep on the speaker
  ENDIF
ENDLOOP
```

This program repeatedly tests whether or not the variable x is equal to 5, printing a line of text and producing a "beep" on the computer's speaker if that test evaluates as true. Here, the context of the IF command tells us the equals sign is a test for equality rather than a command to assign a new value to x. If the condition is met ($x = 5$) then all commands contained within the IF/ENDIF set are executed.

Some programming languages draw a more explicit distinction between the operations of equality test versus assignment by using different symbol combinations. In C and C++, for example, a single equals sign (=) represents assignment while a double set of equals signs (==) represents a test for equality. In Structured Text (ST) PLC programming, a single equals sign (=) represents a test for equality, while a colon plus equals sign (:=) represents assignment. The combination of an exclamation point and an equals sign (!=) represents "not equal to," used as a test condition to check for *inequality* between two quantities.

Branching and functions

A very important feature of any programming language is the ability for the path of execution to change (i.e. the program "flow" to *branch* in another direction) rather than take the exact same path every time. We saw shades of this with the `IF` statement in our previous example program: the computer would print some text and output a beep sound if the variable x happened to be equal to 5, but would completely skip the `PRINT` and `OUTPUT` commands if x happened to be any other value.

An elegant way to modularize a program into separate pieces involves writing portions of the program as separate *functions* which may be "called" as needed by the main program. Let us examine how to apply this concept to the following conditional program:

Pseudocode listing

```
DECLARE x to be an integer variable

LOOP

  // (other code manipulating the value of x goes here)

  IF x = 5 THEN
    PRINT "The value of the number is 5"
    OUTPUT audible beep on the speaker
  ELSEIF x = 7 THEN
    PRINT "The value of the number is 7"
    OUTPUT audible beep on the speaker
  ELSEIF x = 11 THEN
    PRINT "The value of the number is 11"
    OUTPUT audible beep on the speaker
  ENDIF
ENDLOOP
```

This program takes action (printing and outputting beeps) if ever the variable x equals either 5, 7, or 11, but not for any other values of x. The actions taken with each condition are quite similar: print the numerical value of x and output a single beep. In fact, one might argue this code is ugly because we have to keep repeating one of the commands verbatim: the `OUTPUT` command for each condition where we wish to computer to output a beep sound.

We may streamline this program by placing the `PRINT` and `OUTPUT` commands into their own separate "function" written outside the main loop, and then *call* that function whenever we need it. The boundaries of this function's code are marked by the `BEGIN` and `END` labels shown near the bottom of the listing:

Pseudocode listing

```
DECLARE n to be an integer variable
DECLARE x to be an integer variable
DECLARE PrintAndBeep to be a function

LOOP

  // (other code manipulating the value of x goes here)

  IF x = 5 OR x = 7 OR x = 11 THEN
    CALL PrintAndBeep(x)
  ENDIF
ENDLOOP


BEGIN PrintAndBeep (n)
  PRINT "The value of the number is" (n) "!"
  OUTPUT audible beep on the speaker
  RETURN
END PrintAndBeep
```

The main program loop is much shorter than before because the repetitive tasks of printing the value of x and outputting beep sounds has been moved to a separate function. In older computer languages, this was known as a *subroutine*, the concept being that flow through the main program (the "routine") would branch to a separate sub-program (a "subroutine") to do some specialized task and then return back to the main program when the sub-program was done with its task.

Note that the program execution flow never reaches the PrintAndBeep function unless x happens to equal 5, 7, or 11. If the value of x never matches any of those specific conditions, the program simply keeps looping between the `LOOP` and `ENDLOOP` markers.

Note also how the value of x gets *passed* on to the PrintAndBeep function, then read inside that function under another variable name, n. This was not strictly necessary for the purpose of printing the value of x, since x is the only variable in the main program. However, the use of a separate ("local") variable within the PrintAndBeep function enables us at some later date to use that function to act on other variables within the main program while avoiding conflict. Take this program for example:

Pseudocode listing

```
DECLARE n to be an integer variable
DECLARE x to be an integer variable
DECLARE y to be an integer variable
DECLARE PrintAndBeep to be a function

LOOP

  // (other code manipulating the value of x and y goes here)

  IF x = 5 OR x = 7 OR x = 11 THEN
    CALL PrintAndBeep(x)
  ENDIF
  IF y = 0 OR y = 2 THEN
    CALL PrintAndBeep(y)
  ENDIF
ENDLOOP


BEGIN PrintAndBeep (n)
  PRINT "The value of the number is" (n) "!"
  OUTPUT audible beep on the speaker
  RETURN
END PrintAndBeep
```

Here, the PrintAndBeep function gets used to print certain values of x, then re-used to print certain values of y. If we had used x within the PrintAndBeep function instead of its own variable (n), the function would only be useful for printing the value of x. Being able to pass values to functions makes those functions more useful.

A final note on branching and functions: most computer languages allow a function to call itself if necessary. This concept is known as *recursion* in computer science.

29.15.2 Position versus velocity algorithms

The canonical "ideal" or "ISA" variety of PID equation takes the following form:

$$m = K_p \left(e + \frac{1}{\tau_i} \int e \, dt + \tau_d \frac{de}{dt} \right)$$

Where,
m = Controller output
e = Error (SP − PV or PV − SP, depending on controller action being direct or reverse)
K_p = Controller gain
τ_i = Integral (reset) time constant
τ_d = Derivative (rate) time constant

The same equation may be written in terms of "gains" rather than "time constants" for the integral and derivative terms. This re-writing exhibits the advantage of consistency from the perspective of PID tuning, where each tuning constant has the same (increasing) effect as its numerical value grows larger:

$$m = K_p \left(e + K_i \int e \, dt + K_d \frac{de}{dt} \right)$$

Where,
m = Controller output
e = Error
K_p = Controller gain
K_i = Integral (reset) gain (repeats per unit time)
K_d = Derivative (rate) gain

However the equation is written, there are two major ways in which it is commonly implemented in a digital computer. One way is the *position* algorithm, where the result of each pass through the program "loop" calculates the actual output value. If the final control element for the loop is a control valve, this value will be the position of that valve's stem, hence the name *position algorithm*. The other way is the so-called *velocity* algorithm, where the result of each pass through the program "loop" calculates the amount the output value will *change*. Assuming a control valve for the final control element once again, the value calculated by this algorithm is the distance the valve stem will travel *per scan of the program*. In other words, the magnitude of this value describes how *fast* the valve stem will travel, hence the name *velocity algorithm*.

Mathematically, the distinction between the position and velocity algorithms is a matter of differentials: the position equation solves for the output value (m) directly while the velocity equation solves for small increments (differentials) of m, or dm.

A comparison of the position and velocity equations shows both the similarities and the differences:

$$m = K_p \left(e + K_i \int e\, dt + K_d \frac{de}{dt} \right) \qquad \text{Position equation}$$

$$dm = K_p \left(de + K_i e\, dt + K_d \frac{d^2 e}{dt} \right) \qquad \text{Velocity equation}$$

Of the two approaches to implementing PID control, the position algorithm makes the most intuitive sense and is the easiest to understand.

We will begin our exploration of both algorithms by examining their application to proportional-only control. This will be a simpler and "gentler" introduction than showing how to implement full PID control. The two respective proportional-only control equations we will consider are shown here:

$$m = K_p e + \text{Bias} \qquad \text{Position equation for P-only control}$$

$$dm = K_p de \qquad \text{Velocity equation for P-only control}$$

You will notice how a "bias" term is required in the position equation to keep track of the output's "starting point" each time a new output value is calculated. No such term is required in the velocity equation, because the computer merely calculates *how far the output moves from its last value* rather than the output's value from some absolute reference.

First, we will examine a simple pseudocode program for implementing the proportional-only equation in its "position" form:

Pseudocode listing for a "position algorithm" proportional-only controller

```
DECLARE PV, SP, and Out to be floating-point variables
DECLARE K_p, Error, and Bias to be floating-point variables
DECLARE Action, and Mode to be boolean variables

LOOP
  SET PV = analog_input_channel_N  // Update PV
  SET K_p = operator_input_channel_Gain  // From operator interface

  IF Action = 1 THEN
    SET Error = SP - PV  // Calculate error assuming reverse action
  ELSE THEN
    SET Error = PV - SP  // Calculate error assuming direct action
  ENDIF

  IF Mode = 1 THEN  // Automatic mode (if Mode = 1)
    SET Out = K_p * Error + Bias
    SET SP = operator_input_channel_SP  // From operator interface
  ELSE THEN  // Manual mode (if Mode = 0)
    SET Out = operator_input_channel_Out // From operator interface
    SET SP = PV  // Setpoint tracking
    SET Bias = Out  // Output tracking
  ENDIF
ENDLOOP
```

The first SET instructions within the loop update the PV to whatever value is being measured by the computer's analog input channel (channel N in this case), and the K_p variable to whatever value is entered by the human operator through the use of a keypad, touch-screen interface, or networked computer. Next, a set of IF/THEN conditionals determines which way the error should be calculated: Error = SP − PV if the control action is "reverse" (Action = 1) and Error = PV − SP if the control action is "direct" (Action = 0).

The next set of conditional instructions determines what to do in automatic versus manual modes. In automatic mode (Mode = 1), the output value is calculated according to the position equation and the setpoint comes from a human operator's input. In manual mode (Mode = 0), the output value is no longer calculated by an equation but rather is obtained from the human operator's input, the setpoint is forced equal to the process variable, and the Bias value is continually made equal to the value of the output. Setting SP = PV provides the convenient feature of *setpoint tracking*, ensuring an initial error value of zero when the controller is switched back to automatic mode. Setting the Bias equal to the output provides the essential feature of *output tracking*, where the controller begins automatic operation at an output value precisely equal to the last manual-mode output value.

Next, we will examine a simple pseudocode program for implementing the proportional-only equation in its "velocity" form:

Pseudocode listing for a "velocity algorithm" proportional-only controller

```
DECLARE PV, SP, and Out to be floating-point variables
DECLARE K_p, Error, and last_Error to be floating-point variables
DECLARE Action, and Mode to be boolean variables

LOOP
  SET PV = analog_input_channel_N  // Update PV
  SET K_p = operator_input_channel_Gain  // From operator interface
  SET last_Error = Error

  IF Action = 1 THEN
    SET Error = SP - PV  // Calculate error assuming reverse action
  ELSE THEN
    SET Error = PV - SP  // Calculate error assuming direct action
  ENDIF

  IF Mode = 1 THEN  // Automatic mode (if Mode = 1)
    SET Out = Out + (K_p * (Error - last_Error))
    SET SP = operator_input_channel_SP  // From operator interface
  ELSE THEN  // Manual mode (if Mode = 0)
    SET Out = operator_input_channel_Out // From operator interface
    SET SP = PV  // Setpoint tracking
  ENDIF
ENDLOOP
```

The code for the velocity algorithm is mostly identical to the code for the position algorithm, with just a few minor changes. The first difference we encounter in reading the code from top to bottom is that we calculate a new variable called "last_Error" immediately prior to calculating a new value for Error. The reason for doing this is to provide a way to calculate the differential *change* in error (de) from scan to scan of the program. The variable "last_Error" remembers the value of Error during the previous scan of the program. Thus, the expression "Error − last_Error" is equal to the amount the error has changed from last scan to the present scan.

When the time comes to calculate the output value in automatic mode, we see the SET command calculating the change in output (K_p multiplied by the change in error), then adding this change in output to the existing output value to calculate a new output value. This is how the program translates calculated output increments into an actual output value to drive a final control element. The mathematical expression "K_p * (Error − last_Error)" defines the incremental change in output value, and this increment is then added to the current output value to generate a new output value.

From a human operator's point of view, the position algorithm and the velocity algorithm are identical with one exception: how each controller reacts to a sudden change in gain (K_p). To understand this difference, let us perform a "thought experiment" where we imagine a condition of

constant error between PV and SP. Suppose the controller is operating in automatic mode, with a setpoint of 60% and a (steady) process variable value of 57%. We should not be surprised that a constant error might exist for a proportional-only controller, since we should be well aware of the phenomenon of *proportional-only offset*.

How will this controller react if the gain is suddenly increased in value while operating in automatic mode? If the controller executes the position algorithm, the result of a sudden gain change will be a sudden change in its output value, since output is a direct function of error and gain. However, if the controller executes the velocity algorithm, the result of a sudden gain change will be no change to the output at all, so long as the error *remains constant*. Only when the error begins to change will there be any noticeable difference in the controller's behavior compared to how it acted before the gain change. This is because the velocity algorithm is a function of gain and *change in error*, not error directly.

Comparing the two responses, the velocity algorithm's response to changes in gain is regarded as "better-mannered" than the position algorithm's response to changes in gain. When tuning a controller, we would rather not have the controller's output suddenly jump in response to simple gain changes[48], and so the velocity algorithm is generally preferred. If we allow the gain of the algorithm to be set by another process variable[49], the need for "stable" gain-change behavior becomes even more important.

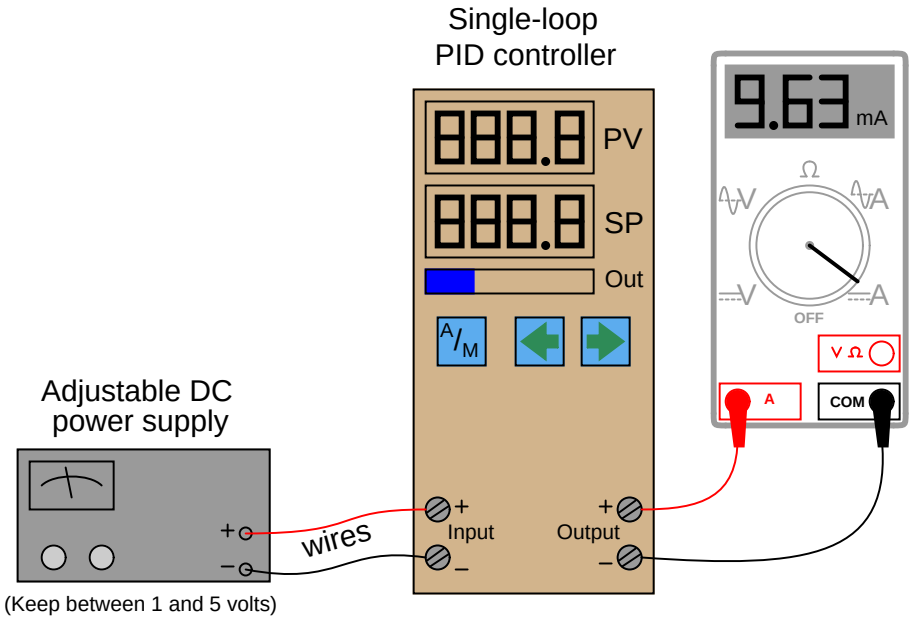
[48] It should be noted that this is precisely what happens when you change the gain in a pneumatic or an analog electronic controller, since all analog PID controllers implement the "position" equation. Although the choice between "position" and "velocity" algorithms in a digital controller is arbitrary, it is *much* easier to build an analog mechanism or circuit implementing the position algorithm than it is to build an analog "velocity" controller.

[49] We call this an *adaptive gain* control system.

29.16 Note to students

PID control can be a frustrating subject for many students, even those with previous knowledge of calculus. At times it can seem like an impossibly abstract concept to master.

Thankfully, there is a relatively simple way to make PID control more "real," and that is hands-on experience with a real PID controller. I advise you acquire an electronic single-loop PID controller[50] and set it up with an adjustable DC power supply and milliammeter as such:



Most electronic controllers input a 1 to 5 VDC signal for the process variable (often with a 250 ohm resistor connected across the input terminals to generate a 1-5 VDC drop from a 4-20 mA current signal, which you will not need here). By adjusting the DC power supply between 1 and 5 volts DC, you will simulate a transmitter signal to the controller's input between 0% and 100%.

The milliammeter reads current output by the controller, 4 mA representing a 0% output signal and 20 mA representing a 100% output signal. With the power supply and milliammeter both connected to the appropriate terminals on the controller, you are all set to simulate input conditions and watch the controller's output response.

This arrangement does not simulate a process, and so there will be no feedback for you to observe. The purpose of this setup is to simply learn how the controller is supposed to respond to different PV and SP conditions, so that you may gain an intuitive "feel" for the PID algorithm to supplement your theoretical understanding of it. Experimentation with a real process (or even a simulated process) comes later (see section 30.6 beginning on page 2493).

[50] Many instrument manufacturers sell simple, single-loop controllers for reasonable prices, comparable to the price of a college textbook. You need to get one that accepts 1-5 VDC input signals and generates 4-20 mA output signals, and has a "manual" mode of operation in addition to automatic – these features are *very important!* Avoid controllers that can only accept thermocouple inputs, and/or only have time-proportioning (PWM) outputs.

Once you have all components connected, you should check to see that everything works:

- Set power supply to 1, 3, and then 5 volts DC. The controller's PV display should read 0%, 50%, and 100%, respectively. The PV display should follow closely to the power supply voltage signal over time. If the display seems to "lag" behind the power supply adjustment, then it means the controller has damping configured for the input signal. You should keep the damping set to the minimum possible value, so the controller is as responsive as it can be.
- Put the controller in manual mode and set the output to 0%, 50%, and then 100%. The milliammeter should register 4 mA, 12 mA, and 20 mA, respectively.

After checking these basic functions, you may proceed to do the following experiments. For each experiment, I recommend setting the PV input signal to 3 volts DC (50%), and manually setting the output to 50% (12 mA on the milliammeter). When you are ready to test the P,I,D responses of the controller, place the controller into automatic mode and then observe the results.

29.16.1 Proportional-only control action

1. Set the controller PV input to 50% (3 volts) and the output value to 50% in manual mode.
2. Configure the controller for reverse action (this is typically the default setting).
3. Configure the PID settings for proportional action only. This may be done by setting the gain equal to 1 (P.B. = 100%), the integral setting to zero repeats per minute (maximum minutes per repeat), and the derivative setting to zero minutes. Some controllers have the ability to switch to a "proportional-only" algorithm – if your controller has that ability, this is the best way to get set up for this exercise.
4. Switch the controller mode to "automatic."
5. Adjust the PV signal to 75% (4 volts) and observe the output. How far does the output signal move from its starting value of 50%? How does the magnitude of this step relate to the magnitude of the PV step? Does the output signal drift or does it remain the same when you stop changing the PV signal?
6. Adjust the PV signal to 25% (2 volts) and observe the output. How far does the output signal move from its starting value of 50%? How does the magnitude of this step relate to the magnitude of the PV step? Does the output signal drift or does it remain the same when you stop changing the PV signal?
7. Change the controller's gain setting to some different value and repeat the previous two steps. How does the output step magnitude relate to the input step-changes in each case? Do you see the relationship between controller gain and how the output responds to changes in the input?
8. Smoothly vary the input signal back and forth between 0% and 100% (1 and 5 volts). How does the output respond when you do this? Try changing the gain setting again and re-checking.
9. Switch the controller's action from *reverse* to *direct*, then repeat the previous step. How does the output respond now?

29.16.2 Integral-only control action

1. Set the controller PV input to 50% (3 volts) and the output value to 50% in manual mode.
2. Configure the controller for reverse action (this is typically the default setting).
3. Configure the PID settings for integral action only. If the controller has an "I-only" mode, this is the best way to get set up for this exercise. If there is no way to completely turn off proportional action, then I recommend setting the gain value to the minimum non-zero value allowed, and setting the integral constant to an aggressive value (many repeats per minute, or fractions of a minute per repeat). If your controller does have an integral-only option, I recommend setting the integral time constant at 1 minute. Set derivative action at zero minutes.
4. Switch the controller mode to "automatic."
5. Adjust the PV signal to 75% (4 volts) and observe the output. Which way does the output signal move? Does the output signal drift or does it remain the same when you stop changing the PV signal? How does this action compare with the proportional-only test?
6. Adjust the PV signal to 25% (2 volts) and observe the output. Which way does the output signal move? Does the output signal drift or does it remain the same when you stop changing the PV signal? How does this action compare with the proportional-only test?
7. Change the controller's integral setting to some different value and repeat the previous two steps. How does the rate of output ramping relate to the input step-changes in each case? Do you see the relationship between the integral time constant and how the output responds to changes in the input?
8. Smoothly vary the input signal back and forth between 0% and 100% (1 and 5 volts). How does the output respond when you do this? Try changing the integral setting again and re-checking.
9. Where must you adjust the input signal to get the output to stop moving? When the output finally does settle, is its value consistent (i.e. does it always settle at the same value, or can it settle at different values)?
10. Switch the controller's action from *reverse* to *direct*, then repeat the previous two steps. How does the output respond now?

29.16.3 Proportional plus integral control action

1. Set the controller PV input to 50% (3 volts) and the output value to 50% in manual mode.
2. Configure the controller for reverse action (this is typically the default setting).
3. Configure the PID settings with a proportional (gain) value of 1 (P.B. = 100%) and an integral value of 1 repeat per minute (or 1 minute per repeat). Set derivative action at zero minutes.
4. Switch the controller mode to "automatic."
5. Adjust the PV signal to 75% (4 volts) and observe the output. Which way does the output signal move? Does the output signal drift or does it remain the same when you stop changing the PV signal? How does this action compare with the proportional-only test and with the integral-only test?
6. Adjust the PV signal to 25% (2 volts) and observe the output. Which way does the output signal move? Does the output signal drift or does it remain the same when you stop changing the PV signal? How does this action compare with the proportional-only test and with the integral-only test?
7. Change the controller's gain setting to some different value and repeat the previous two steps. Can you tell which aspect of the output signal's response is due to proportional action and which aspect is due to integral action?
8. Change the controller's integral setting to some different value and repeat those same two steps. Can you tell which aspect of the output signal's response is due to proportional action and which aspect is due to integral action?
9. Smoothly vary the input signal back and forth between 0% and 100% (1 and 5 volts). How does the output respond when you do this? Try changing the gain and/or integral settings again and re-checking.
10. Switch the controller's action from *reverse* to *direct*, then repeat the previous two steps. How does the output respond now?

29.16.4 Proportional plus derivative control action

1. Set the controller PV input to 50% (3 volts) and the output value to 50% in manual mode.
2. Configure the controller for reverse action (this is typically the default setting).
3. Configure the PID settings with a proportional (gain) value of 1 (P.B. = 100%) and a derivative value of 1 minute. Set integral action at zero repeats per minute (maximum number of minutes per repeat).
4. Switch the controller mode to "automatic."
5. Adjust the PV signal to 75% (4 volts) and observe the output. Which way does the output signal move? How does the output signal value compare while you are adjusting the input voltage versus after you reach 4 volts and take your hand off the adjustment knob? How does this action compare with the proportional-only test?
6. Adjust the PV signal to 25% (2 volts) and observe the output. Which way does the output signal move? How does the output signal value compare while you are adjusting the input voltage versus after you reach 4 volts and take your hand off the adjustment knob? How does this action compare with the proportional-only test?
7. Change the controller's gain setting to some different value and repeat the previous two steps. Can you tell which aspect of the output signal's response is due to proportional action and which aspect is due to derivative action?
8. Change the controller's derivative setting to some different value and repeat those same two steps. Can you tell which aspect of the output signal's response is due to proportional action and which aspect is due to derivative action?
9. Smoothly vary the input signal back and forth between 0% and 100% (1 and 5 volts). How does the output respond when you do this? Try changing the derivative setting again and re-checking.
10. Switch the controller's action from *reverse* to *direct*, then repeat the previous two steps. How does the output respond now?

29.16.5 Full PID control action

1. Set the controller PV input to 50% (3 volts) and the output value to 50% in manual mode.
2. Configure the controller for reverse action (this is typically the default setting).
3. Configure the PID settings with a proportional (gain) value of 1 (P.B. = 100%), an integral value of 1 repeat per minute (or 1 minute per repeat), and a derivative action of 1 minute.
4. Switch the controller mode to "automatic."
5. Adjust the PV signal to 75% (4 volts) and observe the output. Which way does the output signal move? Does the output signal drift or does it remain the same when you stop changing the PV signal? How does magnitude of the output signal compare while you are changing the input voltage, versus when the input signal is steady?
6. Adjust the PV signal to 25% (2 volts) and observe the output. Which way does the output signal move? Does the output signal drift or does it remain the same when you stop changing the PV signal? How does magnitude of the output signal compare while you are changing the input voltage, versus when the input signal is steady?
7. Change the controller's gain setting to some different value and repeat the previous two steps. Can you tell which aspect of the output signal's response is due to proportional action, which aspect is due to integral action, and which aspect is due to derivative action?
8. Change the controller's integral setting to some different value and repeat the same two steps. Can you tell which aspect of the output signal's response is due to proportional action, which aspect is due to integral action, and which aspect is due to derivative action?
9. Change the controller's derivative setting to some different value and repeat the same two steps. Can you tell which aspect of the output signal's response is due to proportional action, which aspect is due to integral action, and which aspect is due to derivative action?
10. Smoothly vary the input signal back and forth between 0% and 100% (1 and 5 volts). How does the output respond when you do this? Try changing the gain, integral, and/or derivative settings again and re-checking.
11. Switch the controller's action from *reverse* to *direct*, then repeat the previous two steps. How does the output respond now?

29.17 Review of fundamental principles

Shown here is a partial listing of principles applied in the subject matter of this chapter, given for the purpose of expanding the reader's view of this chapter's concepts and of their general interrelationships with concepts elsewhere in the book. Your abilities as a problem-solver and as a life-long learner will be greatly enhanced by mastering the applications of these principles to a wide variety of topics, the more varied the better.

- **Linear equations**: any function represented by a straight line on a graph may be represented symbolically by the slope-intercept formula $y = mx + b$. Relevant to proportional control algorithms.
- **Zero shift**: any shift in the offset of an instrument is fundamentally additive, being represented by the "intercept" (b) variable of the slope-intercept linear formula $y = mx + b$. Relevant to controller tuning: adjusting the "bias" of a loop controller always adds to or subtracts from its output signal.
- **Span shift**: any shift in the gain of an instrument is fundamentally multiplicative, being represented by the "slope" (m) variable of the slope-intercept linear formula $y = mx + b$. Relevant to controller tuning: adjusting the "gain" of a loop controller always multiplies or divides the response of its output for a given input change.
- **Negative feedback**: when the output of a system is degeneratively fed back to the input of that same system, the result is decreased (overall) gain and greater stability. Relevant to loop controller action: in order for a control system to be stable, the feedback must be negative.
- **Self-balancing pneumatic mechanisms**: all self-balancing pneumatic instruments work on the principle of negative feedback maintaining a nearly constant baffle-nozzle gap. Force-balance mechanisms maintain this constant gap by balancing force against force with negligible motion, like a tug-of-war. Motion-balance mechanisms maintain this constant gap by balancing one motion with another motion, like two dancers moving in unison.
- **Self-balancing opamp circuits**: all self-balancing operational amplifier circuits work on the principle of negative feedback maintaining a nearly zero differential input voltage to the opamp. Making the "simplifying assumption" that the opamp's differential input voltage is exactly zero assists in circuit analysis, as does the assumption that the input terminals draw negligible current.
- **Amplification**: the control of a relatively large signal by a relatively small signal. Relevant to the role of loop controllers exerting influence over a process variable at the command of a measurement signal. In behaving as amplifiers, loop controllers may oscillate if certain criteria are met.
- **Barkhausen criterion**: is overall loop gain is unity (1) or greater, and phase shift is 360°, the loop will sustain oscillations. Relevant to control system stability, explaining why the loop will "cycle" (oscillate) if gain is too high.
- **Time constant**: (τ), defined as the amount of time it takes a system to change 63.2% of the way from where it began to where it will eventually stabilize. The system will be within 1%

of its final value after 5 time constants' worth of time has passed (5τ). Relevant to process control loops, where natural lags contribute to time constants, usually of multiple order.

References

"FOUNDATION Fieldbus", document L454 EN, Samson AG, Frankfurt, Germany, 2000.

"Identification and Description of Instrumentation, Control, Safety, and Information Systems and Components Implemented in Nuclear Power Plants", EPRI, Palo Alto, CA: 2001. 1001503.

Kernighan, Brian W. and Ritchie, Dennis M., *The C Programming Language*, Bell Telephone Laboratories, Incorporated, Murray Hill, NJ, 1978.

Lavigne, John R., *Instrumentation Applications for the Pulp and Paper Industry*, The Foxboro Company, Foxboro, MA, 1979.

Lipták, Béla G. et al., *Instrument Engineers' Handbook – Process Control Volume II*, Third Edition, CRC Press, Boca Raton, FL, 1999.

Mollenkamp, Robert A., *Introduction to Automatic Process Control*, Instrument Society of America, Research Triangle Park, NC, 1984.

"Moore 353 Process Automation Controller User's Manual", document UM353-1, Revision 11, Siemens Energy and Automation, 2003.

Shinskey, Francis G., *Energy Conservation through Control*, Academic Press, New York, NY, 1978.

Shinskey, Francis G., *Process-Control Systems – Application / Design / Adjustment*, Second Edition, McGraw-Hill Book Company, New York, NY, 1979.

"SPEC 200 Systems", technical information document TI 200-100, Foxboro, 1980.

"SPEC 200 System Configuration", technical information document TI 200-105, Foxboro, January 1975.

"SPEC 200 System Wiring", technical information document TI 200-260, Foxboro, 1972.

Ziegler, J. G., and Nichols, N. B., "Optimum Settings for Automatic Controllers", *Transactions of the American Society of Mechanical Engineers* (ASME), Volume 64, pages 759-768, Rochester, NY, November 1942.